



资深技术专家实战经验结晶，Neo Technology首席科学家作序推荐

通过大量实践案例，全面讲解使用Neo4j设计、实现、查询图形数据库的核心概念、技术和方法

MANNING

数据库
技术丛书

Neo4j in Action

Neo4j 实战



[英] 阿列克萨·武科蒂奇 (Aleksa Vukotic)

[美] 尼基·瓦特 (Nicki Watt)

[美] 塔里克·阿贝卓布 (Tareq Abedrabbo) 著

[美] 多米尼克·福克斯 (Dominic Fox)

[美] 乔纳斯·帕特纳 (Jonas Partner)

张秉森 孔倩 张晨策 译



机械工业出版社
China Machine Press

数据库技术丛书

Neo4j实战

Neo4j in Action

（英）武科蒂奇（Vukotic, A.） 等著

张秉森 孔倩 张晨策 译

ISBN: 978-7-111-53112-8

本书纸版由机械工业出版社于2016年出版，电子版由华章分社（北京华章图文信息有限公司，北京奥维博世图书发行有限公司）全球范围内制作与发行。

版权所有，侵权必究

客服热线：+ 86-10-68995265

客服信箱：service@bbbvip.com

官方网址：www.hzmedia.com.cn

新浪微博 @华章数媒

微信公众号 华章电子书（微信号：hzebook）

目录

译者序

序言

前言

作者简介

第一部分 Neo4j概述

第1章 Neo4j数据库的一个应用案例

1.1 为什么要有Neo4j

1.2 关系数据库中的图形数据

1.3 Neo4j中的图形数据

1.4 大数据下的SQL联接操作与Neo4j图形遍历的对比

1.5 图形

1.6 Neo4j在NoSQL领域的地位

1.7 Neo4j具有与ACID兼容的数据格式

1.8 本章小结

第2章 Neo4j的数据模型

2.1 Neo4j中数据模型的类型

2.2 领域建模

2.3 更多实例

2.4 本章小结

第3章 Neo4j开发入门

3.1 图形数据结构建模

3.2 使用Neo4j API

3.3 节点标签

3.4 本章小结

第4章 强大的图形遍历功能

4.1 使用Neo4j核心Java API进行遍历

4.2 使用Neo4j的遍历API进行遍历

4.3 本章小结

第5章 数据索引

5.1 创建索引项

5.2 通过邮箱地址查找用户

5.3 对多个匹配结果的处理

5.4 对索引过的数据进行修改的处理

5.5 自动索引

5.6 索引的成本/效益权衡

5.7 本章小结

第二部分 Neo4j应用开发

第6章 Neo4j的查询语言Cypher

6.1 Cypher简介

6.2 Cypher的基本句法

6.3 用Cypher更新图形数据

6.4 高级Cypher

6.5 本章小结

第7章 事务

7.1 事务的基础知识

7.2 事务的高级功能

7.3 与其他事务管理系统的集成

7.4 事务事件

7.5 本章小结

第8章 深度遍历

8.1 遍历的顺序

8.2 扩展关系

8.3 管理唯一性

8.4 双向遍历

8.5 本章小结

第9章 Spring Data Neo4j

9.1 SDN适合做什么

9.2 用SDN建模

9.3 访问和持久化实体

9.4 对象图形映射选项

9.5 执行查询和遍历

9.6 本章小结

第三部分 Neo4j应用实例

第10章 Neo4j的嵌入式模式与服务器模式

10.1 使用模式概述

10.2 嵌入式模式

10.3 服务器模式

10.4 权衡选项

10.5 充分利用服务器模式

10.6 本章小结

第11章 Neo4j的架构与应用

11.1 高层Neo4j架构

11.2 Neo4j的高可用性

11.3 备份

11.4 本书可能没有涵盖但你可能会关注的问题

11.5 本章小结

11.6 最后的设想

附录A 安装Neo4j服务器

附录B 设置和运行示例程序

附录C 设置使用SDN的项目环境

附录D 获得更多的帮助

译者序

Neo4j对于大多数人来说，可能是比较陌生的。其实，Neo4j是一个图形数据库的名字，就像传统关系数据库中的Oracle和SQL Server一样。图形数据库是一个比较新的概念。大家所熟悉的传统关系数据库的查询使用的是结构化查询语言，即SQL（Structured Query Language）。而像Neo4j这样的图形数据库（Neo4j属于NoSQL数据库的一种）是最近几年发展起来的新技术。随着大数据时代的到来，常规的关系数据库技术已显得力不从心，因此，应运而生的是能应对海量数据的新型数据库技术，Neo4j就是其中的佼佼者。

本书是一本从基础知识到实际应用项目开发的教科书，作者均来自相关公司的开发和技术部门，有着深厚的应用和技术背景。本书以实例为依托，详细地对实例做建模分析，并给出经过测试的源代码，读者可以按照相关步骤实现实例，这是一种学习建模和编程的极好方法。

参加本书翻译的有张秉森、孔倩和张晨策。其中张秉森负责翻译第1章、第2章、第7章、第8章、第9章、第10章和第11章以及附录等，孔倩负责翻译第3章和第4章，张晨策负责翻译第5章和第6章。全书由张秉森负责统一校对和审稿。

由于本书的主要内容都是比较新的技术，而且作者完全从技术的角度编写本书，涉及的技术内容多，专业跨度大，给本书的翻译工作增加了不少的难度。尽管译者始终仔细求证与琢磨，但是难免还会有疏漏和错误，恳请广大读者不吝赐教。译者的联系方式：331006787@qq.com。

序言

数据库领域正在经历着一场巨大的变革，关系数据库的霸权地位已经动摇，受到NoSQL旗下大量新技术的挑战。其中，图形数据库在分析各领域数据方面日益成为一种重要工具。

大多数NoSQL数据库用于解决关系数据库在性能上明显的限制，关系数据库遇到以指数增长的数据量时，就会出现难以应付的局面，这在过去的几年中我们已经亲眼目睹过。但是，数据量的增加只是我们面对的诸多挑战之一。不仅仅是数据量的增加，而且数据变得越来越交叉互联，结构也越来越多样化。简而言之，数据正在变得越来越复杂，远远超出连接的概念。

为了解决性能和可扩展性问题，NoSQL已经普遍放弃了关系模型处理交叉互联数据的方式。而图形数据库开辟了互联数据的世界，其性能超出了关系数据库几个数量级。如果我们想从数据中得到许多最有趣的问题的结果，则要求我们不但能理解许多东西是相互关联的，而且也要理解这些关联中有许多不同。图形数据库为这种理解提供了功能最强大和性能最佳的方式。

关联的数据给大多数的NoSQL数据库带来了许多困难，这需要以作为不连接的聚合管理文档、列或键/值对。要使用这些技术创建任何连接的外观，我们必须找到一种既适合使数据非标准化也适合回避实质上不连接的模型中连接的方法。当我们在建立Neo4j的时候，就发现这并不是一件容易的事情！

相对于同一时间段NoSQL领域的佼佼者，Neo4j已经到了开花结果的时候了（事实上，在技术上，Neo4j已经超前了许多其他NoSQL技术好几

年)。Neo4j为高连接数据提供了类传统数据库的支持（包括事务的安全性），同时在性能上也比传统数据库高出几个数量级（从分钟级别提高到毫秒级别）。对于社会计算、推荐引擎、电信、授权和访问控制、路由和物流、产品目录、数据中心管理、职业管理、欺诈探测、治安和地理空间等各种领域，Neo4j都能适用，事实证明它是应对复杂数据的一个理想选择。

因为Neo4j是目前最流行的图形数据库，所以开发者早晚会遇到。我们知道“第一次接触”像Neo4j这样的新技术，会使人感到眼花缭乱。关于不同API、构建、查询语言和框架选择的严格性往往会令人生畏，这很容易使人放弃。

本书启发开发人员快速将Neo4j运行起来以解决这些问题。本书结合实际编程例子来讲解Neo4j的API和查询语言，提供了基于作者丰富的数据库应用经验的许多例子。为了补充开发建议，作者也讨论部署选项和解决方案的体系结构。其结果就像在全系统开发生命周期的背景下所见到的一个圆形的整体视图。

作为Neo4j的贡献者和开发者，我们非常欣慰地看到本书收货满满，实用性很强，并通过可重复的测试来描述技术点。本书的作者是来自世界各地的最有经验的Neo4j用户，我们坚信他们的威望和知识通过本书能够传递给所有的开发者。

Jim Webber

NEO TECHNOLOGY的首席科学家

Ian Robinson

前言

从很早以前开始，图形问题就是计算机编程中一些最普遍的问题。回想那时，尤其是在编程时，需要建立层次结构树、访问控制列表和映射表。当要存储图形的时候，程序员将图形转换成表格并使用关系数据库作为底层存储。我们不得不做大量的探究来保存最基本的图形数据，直到有了以Neo4j为代表的图形数据库，才有了其他的选择。

十多年前，Neo4j就开始了它的生涯，2010年正式发布了1.0版本，2.0版本是在2013年12月发布的。我们当中的大多数人一直在积极使用Neo4j，并且见证了它的发展及在不同项目上的应用。图形数据库，尤其是Neo4j，获得了越来越高的关注度，越来越多的人和公司认识到，对于复杂的、具有挑战性的相互连接的数据，Neo4j是图形数据库中唯一的健壮和坚实的解决方案。

我们非常高兴能够浓缩我们的真实工作经历及知识编写成这本实践性很强的书。本书将为你打下一个坚实的基础并创建一些帮助你尽快运行Neo4j的项目。

关于本书

Neo4j作为一个图形数据库在过去的大约十年间发生了很大的变化。从一个纯粹在基于Java领域内操作的图形数据库开始，它已经发展成为适应许多语言和框架的图形数据库。

当我们开始着手写这本书的时候，最新版本是1.9。Neo4j 2.0版本是真

正的规则颠覆者，引进了新的特性，包括非常期待的（内置）节点标签的概念。因此本书尽管还有一些与1.x相关的内容，但是已经更新到涵盖了2.0的特色，所有相关的样例代码和例子都已经专门经过了2.0版本的验证。到本书出版时无疑还会有新的版本，然而，本书所采取的深思熟虑的一步一步的方法，应该为你提供了学习、起步和运行任意的核心Neo4j 2.0+版本的基础知识和技能，当然，除了没有预见的巨大变化。

由于Java是促使Neo4j诞生的语言，因此我们决定使用Java作为主要的语言以演示本书的各种技术和方法。此外，这在以前也是唯一一种可用语言，语言的选择也使我们能够包括一些章节详细讨论如何明确地利用本机核心Neo4j API的优点来做某些工作。这当然对基于Java的客户端是具有很大好处的。然而，如果我们重新从头开始，我们可能会花更多时间来讲Cypher，因为Cypher与图形数据库交互更加容易，且是跨平台的，包括Java、Shell等其他所有平台。然而，我们将这个留作可能的第2版，因为我们仍然相信本书中有许多核心基本概念和方法需要首先传达。本书假设使用的是最新版本的JDK 7。另外，本书附带的样例代码使用Maven作为我们创建依赖工具的选择。对于那些不熟悉Maven的用户，附录B中提供了一个快速入门帮助你起步与进入正常运行。

应当指出的是，这并不意味着本书只是一本参考书。如果是那样的话，本书的篇幅将会长很多。本书的目标是给你足够的知识并理解每一个希望精通的领域，然后再继续向前。如果你想进一步探索特定的领域，书中提供了相关内容的链接，在那里你可以得到更多的相关信息。

本书章节编排

本书分为三部分。第一部分介绍并且涵盖数据模型、Neo4j入门和强大的遍历功能。第二部分讨论应用程序的开发并且涵盖Cypher和Spring。第三部分讨论产品实例。

第1章介绍图形数据库的通用概念，包括浏览Neo4j的一些重要方面和最适合的应用场景。该章介绍Neo4j在NoSQL领域中能解决的一些问题，包括与传统关系数据库的对比。

第2章考察如何及为什么在Neo4j中建模数据，包括在图形数据库中数据建模场景的通用方法。给出了来自不同领域的例子，让你了解在Neo4j中建模的灵活性。

从第3章我们真正开始动手了。该章介绍Neo4j核心Java API，通过创建一个表示用户社交网络和他们喜欢的电影的图形的所有步骤来加深理解。该章涵盖创建和连接节点以及获取这些节点的附加信息。该章也探讨区分不同类型节点的策略，包括标签的使用。

第4章建立这个社交网络的域，更深入地探讨核心API，尤其是重点探讨遍历功能（这里讲的是Neo4j遍历API），这是一个查询图形数据库的强大方法。

第5章介绍Neo4j中可用的索引策略。创建和遍历图形数据是非常棒的，但是你需要一个找到起始点或多个起始点的策略，起始点是图形中开始遍历的点。在查看从Neo4j 2.0版本往后可用的内置索引选项前，你将从查看手动（传统）索引开始。

第6章介绍Cypher，即Neo4j中人能读懂的查询语言。这一章介绍Cypher

的性质，演示对图形操作的基本句法，也涵盖在Neo4j数据库的日常开发和维护中会用到的高级特性。

第7章重点讨论在NoSQL空间中Neo4j的独特亮点之一——完全支持ACID事务，提供一些不同应用的例子以及在某些方面更深入的探究。

虽然从第4章开始接触遍历功能，但编写高效的遍历是成功查询图形数据的关键。在第8章，我们更深入地挖掘遍历API的内部方法，因此你可以学习到如何以一个有效的方式用本机API解决最复杂的图形问题。

第9章讨论面向图形对象的库Spring Data Neo4j（SDN）。尽管SDN不是Neo4j官方提供的，但是该章通过介绍SDN来重点演示Neo4j开源框架如何作为一个库去提供丰富的对象图形模型和Neo4j支持的数据之间进行可靠的、无缝的映射。该章再一次使用我们信任的用户及其所爱电影的社交网络来演示这些要点。

第10章探讨在Neo4j中使用的两种主要模式，即嵌入式模式和服务器模式，本书主要讲解嵌入式模式的核心概念。该章也介绍了服务器模式，服务器模式可以被任何的客户端使用，并稍微深入地探讨每一种模式，比较各自的优缺点，包括如果你选择的是服务器模式的话，如何充分利用服务器。

第11章介绍高级Neo4j架构。以该知识为框架，该章探讨当你准备将Neo4j投入到产品中时应该考虑哪些因素，包括扩展和其他使Neo4j高可用性的需求，以及当需要时如何备份和还原你的数据库。

四个附录指导你完成安装、设置和运行Neo4j，还包括Maven和SDN以及提供寻求更多帮助的信息。

代码约定和下载

本书的所有源代码以代码体形式给出。在许多程序中，以注释的形式指出关键的概念，在文本中有时用编号给出有关代码的附加信息。

书中给出的大部分代码能够在随带的样例源代码中以多种形式找到。样例源代码可以从华章网站（www.hzbook.com）上免费下载。

样例源代码以一组JUnit样式的测试结构给出，其目的是强调或演示讨论中的特定代码。附录B给出了如何运行这些样例源代码的说明。

致谢

本书的编写经历了相当长的一段时间，因此，首先和最应该感谢的是我们所有的家人和朋友，他们不知疲倦地与我们在一起、忍受着我们，并且在深夜给我们准备咖啡以便我们能保持旺盛的精力。谢谢你们！

首先，我们要感谢Open Credo，这是我们在写本书时工作的公司，给我们提供了能够在本书中分享经验的机会——大都是利用工作之余的时间，但是，也占用到非常宝贵的工作时间。这是非常令人感激的！

特别要感谢Neo4j的朋友们，英国的Jim Webber和Ian Robinson、德国的Michael Hunger和整个朋友圈，他们不断地为Neo4j工作，但最重要的是他们的有价值评论和反馈帮助我们深入细致恰当地修改本书。非常感谢英国的朋友在我们遇到一些“有趣”的挑战时提供的指导。

感谢我们的编辑Karen Miller，感谢她这段时间的耐心和理解，加上整个Manning团队的积极工作，使我们最终完成本书。这是一项艰苦的工作，

非常感谢他们的支持和指导，使本书顺利出版。

感谢我们的读者，他们在作者的在线论坛发布了修正和评论。感谢下面的相关人员，他们在开发期间阅读了我们的手稿并提供了宝贵的意见：

Adam Frankl、Bill LaPrise、Brian Gyss、Christoph Jasinski、Frank Uzzolino、Fred Patton、Janeen Johnson、John D.Lewis、Joshua White、Mark Watson、Philippe Lamote、Pouria Amirian、Rikke Willer、Robert Gimbel、Rod Hilton和Stephen Kitt。

还要感谢Jim Webber和Ian Robinson，他们为本书写了序言。还有Craig Taverner，是他作为技术顾问在出版前又阅读了一遍本书的手稿和例子的代码。

还有其他许多以不同的方式做出贡献的人。我们无法通过名字提及每一个人，因为那样意味着这个致谢的篇幅将非常长，但是我们想要深挚地感谢曾经给过我们帮助使得本书得以出版的每一个人。

作者简介

Aleksa Vukotic之前在Open Credo担任数据管理实践领导者，现任Noble Group的开发平台部主管。他是一个软件架构师和开发者、敏捷编程的倡导者、作者和培训师，并且是许多Neo4j项目的开发者，如用图形数据模型解决复杂的访问控制列表和推荐引擎问题。

Nicki Watt是Open Credo的首席顾问。她务实、心灵手巧，是一个问题解决专家，乐于使用“正确的工具完成工作”。Nicki一直使用Neo4j等开源工具以及框架参与各种项目，包括对基于引擎问题的探究和推荐。她也是Spring Data Neo4j项目的贡献者之一。

Tareq Abedrabbo是Open Credo的首席技术官（CTO）。他对编程语言有着强烈的兴趣，从Scala和Python到Google Go。他在几个NoSQL技术领域（包括Neo4j、MongoDB和Redis）有着非常丰富的知识，并在很早以前就开始积极从事Spring项目，是Spring Web Service项目的核心成员。

Dominic Fox是Open Credo的顾问，尤其喜欢把对编程语言理论的意见进行实践并讲解给他人。他作为专业开发者涉足了多种领域的工作，包括文档管理、电信和金融，也包括Neo4j的培训。他一直期望创建优秀的工具和库，能使每天的编程工作更加直观、可靠。

Jonas Partner是Open Credo的首席执行官（CEO），该公司是Neo Technology的服务合作伙伴，是解决复杂数据问题的专业公司。他还是《Spring Integration in Action》的合作作者。

第一部分 Neo4j概述

- 第1章 Neo4j数据库的一个应用案例
- 第2章 Neo4j的数据模型
- 第3章 Neo4j开发入门
- 第4章 强大的图形遍历功能
- 第5章 数据索引

Neo4j是什么？它适合于做什么？它是解决你所涉及问题领域的数据库吗？它能做什么事情？在本书的第一部分，我们将回答这些问题。

第1章介绍了通用图形数据库的概念，并开始探索一些Neo4j的关键内容。第2章继续探讨通常与图形相关的问题和领域，并把重点放在不同情况下的图形数据建模技术和方法上。第3章至第5章才是我们真正开始动手工作的部分。书中使用了一个社交网络的用户和他们喜欢的电影作为例子，探索Neo4j如何以核心API（core API）来实现创建和连接节点的基本功能以及识别不同类型节点的技术。图形数据的遍历也是Neo4j的一个重要特点，第4章通过探讨Neo4j的遍历应用程序接口（Traversal API）对其进行了重点介绍。

第5章介绍了Neo4j的各种“索引（indexing）”策略和提供的选项。在查看从Neo4j 2.0及以上版本的内置索引和可用的选项之前，应先以查看手动（传统）选项开始。

第1章 Neo4j数据库的一个应用案例

本章包括以下内容：

- 几个Neo4j图形数据库的应用案例
- 与传统的关系数据库相比，Neo4j有什么不同
- Neo4j在大型NoSQL领域的地位
- Neo4j的主要特点

计算机科学与数学密切相关，它的很多概念都来自于数学的哲学。算法、密码学、计算和自动化，甚至连基本的数学逻辑和布尔代数理论都是数学概念，正是这些概念将两门学科紧密地关联了起来。数学的另一个课题也经常^①在计算机科学的书籍和文章中看到：[图论](#)。在计算机科学中，图形用于表示特定的数据结构，如组织层级结构、社交网络以及处理流程。通常，在软件设计阶段，结构、流程和算法用图形流程图在白板上描述。面向对象结构的计算机系统也是以继承、组成和对象成员作为图形建模的。

尽管在软件开发的过程中广泛使用图形，但是，当涉及数据保存的时候，开发人员往往忘记了图形。我们试图将数据装进关系型表格中，并对其结构规范化和再规范化，一直到它看上去完全不是它应该表示的东西的样子。

访问控制列表就是一个例子。这是许多企业应用程序解决了一遍又一遍的问题。通常有用户表、任务表和资源表。那么就有“多对多”的表格，映射

用户对任务和任务对资源。最后，至少要有五个关系型表格来表示一个非常简单的数据结构，这其实就是一个图形。即使使用一个对象关系映射（ORM）工具来映射这些数据到对象模型，这也是一个图形。

如果能用数据的自然形式表示数据，使数据的映射更加直观生动，免去那些从数据引擎上存取数据的大量重复的数据转换过程，那样岂不是更好吗？通过图形数据库，我们得以实现。图形数据库使用图形模型将数据作为图形存储，图形数据库由顶点和边构成，顶点和边这两个图形可以构建任何的图形。

此外，具有悠久历史的图论中的所有算法都可以在图形数据库中使用，以更有效地解决与图形有关的问题，在查询用时上会比关系数据库的查询用时更少。

一旦你读了本书，你就会熟悉Neo4j，Neo4j是目前最优秀的图形数据库之一。在本书中，你将了解图形数据库Neo4j如何帮助你建模和解决图形问题，即使在大数据时也具有更优良的运行性能和更合理的方式。

1.1 为什么要有Neo4j

为什么要使用图形数据库呢？或者更具体地说，Neo4j是你的数据库的正确选择吗？正如前面所提到的，对于人们试图使用逻辑的方法，并用类似于图形的结构和概念对他们特殊问题的领域进行建模和描述那是很自然的，尽管他们最终可能并不是以图型数据库存储数据。选择正确的数据库（或者是在存储领域中已有的多种语言版本的数据库中选择多种数据库）存储数据可以使应用程序的运行速度大大加快，正像如果选错了数据库就会使程序完全崩溃一样。

对于这个问题可以使用一个例子来做出很好说明。取一个非常适合于用图形数据库解决的问题，看看怎样应用Neo4j解决，并与使用另一个不同数据库的存储做出对比。为了便于比较，我们将使用传统的关系数据库作为比较对象，因为这是在一般情况下大多数人涉及的存储对比的选择。更重要的是，这也是大多数人在目前，也可能以后仍然会在解决问题时选择的关系数据库模型。

我们要探讨的例子是一个社交网络，一组可能相互之间是朋友的用户。图1-1显示了该社交网络，用箭头连接的用户之间是朋友关系。



注意

要使语义正确，朋友的关系应该是双向的。在Neo4j中，双向性是使用两个关系建立的，以每一个关系建立一个方向（在Neo4j中，每个关系必须要明确定义一个方向，但以后会定义多个方向）。因此，你应该会看到每对朋友之间都有两个独立的朋友关系，在每个方向上有一个关系。为简单起

见，我们建立了单向直接的朋友关系。在第2章和第3章中你将会了解为什么这种数据模型在Neo4j中实际上是非常高效的。

让我们来看看关系数据库如何存储用户及他们的朋友。

1.2 关系数据库中的图形数据

在关系数据库中，通常使用两个具有关系的表格存储社交网络的数据：一个用于存储用户信息，而另一个用于存储用户之间的关系（参见图1-2）。

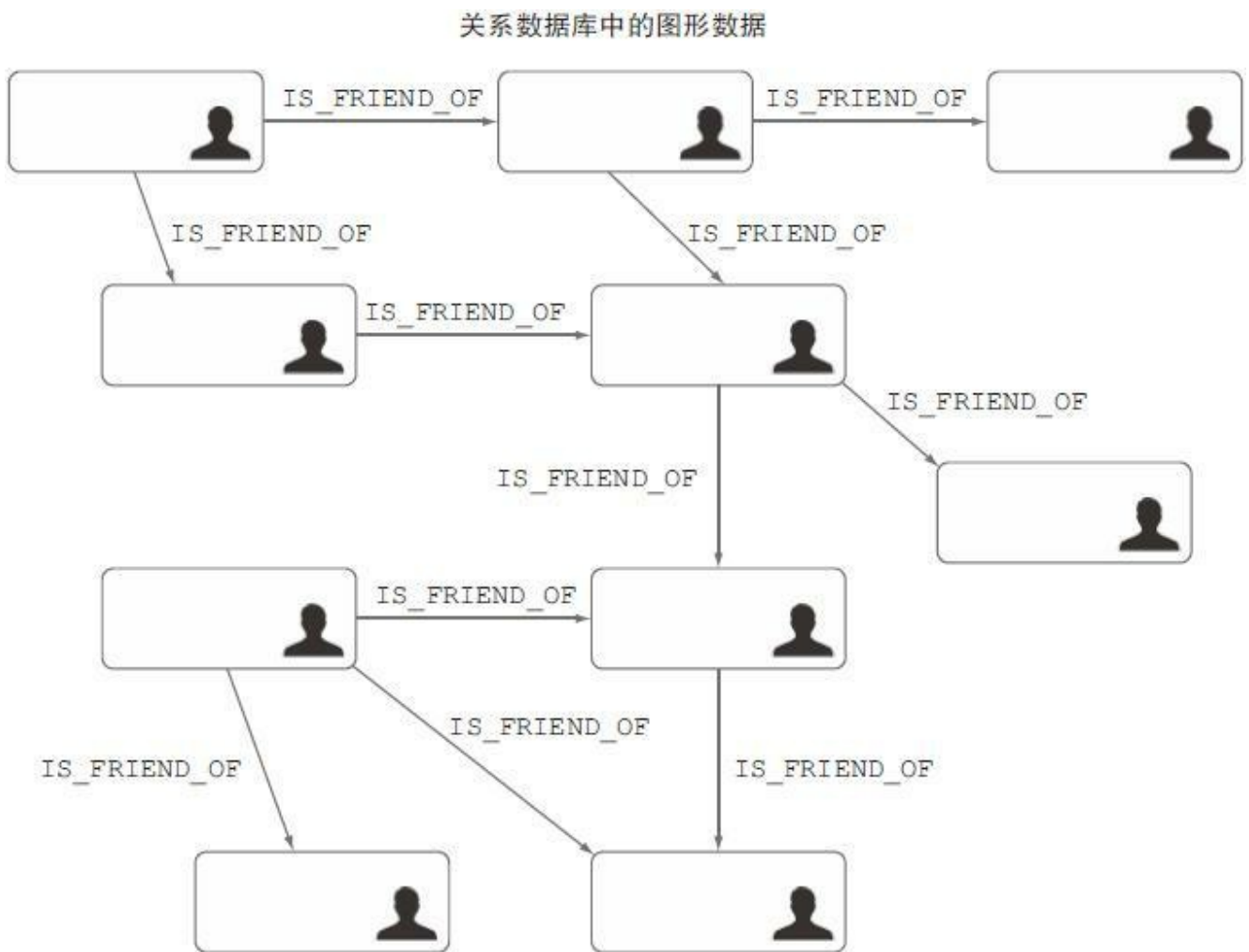


图1-1 以图形数据结构描述的用户与他们的朋友

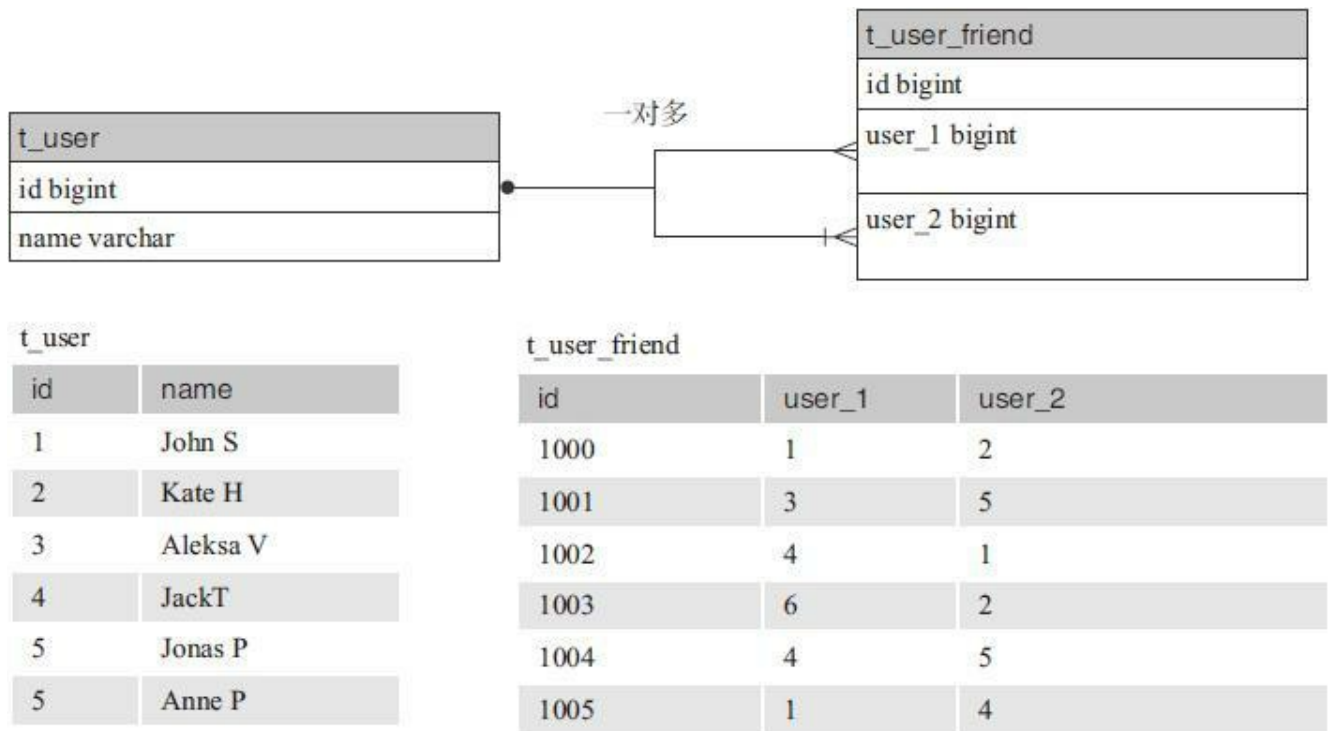


图1-2 描述用户及其朋友数据的SQL图表

程序1-1显示了在MySQL数据库中用SQL脚本语言创建的表。

【程序1-1】 SQL脚本语言定义的社交网络数据表

```

create table t_user (
  id bigint not null,
  name varchar(255) not null,
  primary key (id)
);
create table t_user_friend (
  id bigint not null,
  user_1 bigint not null,
  user_2 bigint not null,
  primary key (id)
);
alter table t_user_friend
  add index FK416055ABC6132571 (user_1),
  add constraint FK416055ABC6132571
    foreign key (user_1) references t_user (id);
alter table t_user_friend
  add index FK416055ABC6132572 (user_2),
  add constraint FK416055ABC6132572
    foreign key (user_2) references t_user (id);
  
```

定义存储用户信息表

定义存储朋友关系表

外键约束

表`t_user`包含着用户信息的列，而表`t_user_friend`只有两列，用外键关系

引用表t_user。主键和外键具有索引，以便进行快速查找操作。索引是关系数据库中使用的典型查找技术。

使用MySQL查询图形数据

你会如何去查询一个关系数据库呢？获取一个特定用户的直接朋友数是相当简单的。如下的基本select查询就可以实现这一目的：

```
select count(distinct uf.*) from t_user_friend uf where uf.user_1 = ?
```



注意

我们在所有的例子中计算所有的朋友数量，因此，不能通过加载实际的数据而使CPU或内存超载。

如何找到一个用户的朋友的所有朋友？通常，在查询前，典型的做法是将表t_user_friend与它自身联接起来。

```
select count(distinct uf2.*) from t_user_friend uf1
➡ inner join t_user_friend uf2 on uf1.user_1 = uf2.user_2
➡ where uf1.user_1 = ?
```

流行的社交网络通常具有从你的具有一定分离度或深度的朋友圈推荐潜在的朋友或联系人的功能。如果你想做某一些相似的事情以寻找某一个用户的朋友的朋友的朋友，你仅仅需要另外的一个join操作。

```
select count(distinct uf3.*) from t_user_friend uf1
➡ inner join t_user_friend uf2 on uf1.user_1 = uf2.user_2
➡ inner join t_user_friend uf3 on uf2.user_1 = uf3.user_2
➡ where uf1.user_1 = ?
```

同样，要做一个四层关系的循环，就需要四个join操作。要获得著名的六度分离问题的所有连接，将需要6个join操作。

对于这种做法，没有什么不正常的，但是有一个潜在的问题：尽管你只关心单一用户的朋友的朋友，但你必须对表t_user_friend中的所有数据做一个join操作，然后丢弃你不再感兴趣的所有行。对于一个小的数据集，这将会不会是一个大的问题。但是，如果这个社交网络不断发展壮大，你可能开始遇到一些严重的性能问题。正如你将看到的，这将会给关系型数据引擎带来巨大的压力。

为了说明这种查询的性能，在一个具有1000个用户的小数据集上运行几次朋友的朋友查询，并在每次调用和记录每次的查询结果的同时，增加搜索的深度。在一个具有1000用户的小数据集中，平均每一个用户具有50个朋友。表t_user包含了1000条记录，而表t_user_friend包含了1000×50=50000条记录。

在每个深度上运行了10次查询，这仅仅是在任何情况下都能帮助提高性能的一个开始，记录下在每一深度上的最快运行时间。除在程序1-1中用SQL脚本语言对数据库的列定义索引外，不对数据库进行其他的性能调整。表1-1给出了实验的结果。

表1-1 使用MySQL数据库引擎对一个有1000个用户的数据库使用多个join查询的运行时间

深 度	查询 1000 个用户的运行时间（秒）	计数结果
2	0.28	~900
3	0.213	~999
4	10.273	~999
5	92.613	~999

 注意

所有的实验均在Intel i7-CPU、8GB内存的商用笔记本电脑上完成，本书

的写作也是在这台笔记本电脑上完成的。



在深度为3、4、5时均返回数值999，这是由于数据集太小，数据库中的每一个用户都相互关联着。

正如你能够看到的，MySQL处理查询的深度为2和3时，其表现相当不错。关系数据库的join操作并非不常见，而是很通常的操作。因此很多数据库引擎都把这种操作作为默认的设计。使用数据库相关列上的索引也能帮助关系数据库最大化其执行join查询操作的性能。

然而，在深度为4和5时，性能显著下降：涉及深度4的查询需要10秒以上才能完成，而在深度5时，尽管计数结果没有改变，但执行的时间就更长了，超过了一分半钟。这说明了当处理图形数据时MySQL所受到的限制：深度图形需要多个join操作时，关系数据库通常表现得不太好。

SQL的join操作的低效率性

要在深度5查找一个用户的所有朋友，关系数据库引擎需要产生t_user_friend表的笛卡儿积五次。对于一个拥有50000个记录的表，所得到的数据集将有 50000^5 行（ 102.4×10^{21} ），这需要相当长的时间和计算能力来做出这些计算。最后只返回了你感兴趣的不到1000条的记录，而放弃了超过99%的计算结果！

正如你所看到的，关系数据库并不擅长多对多关系的数据模型，尤其是在大型数据集时。而相反，Neo4j正好擅长多对多的关系，因此，让我们来看看Neo4j是如何使用相同的数据集来实现的。不是使用表、列和外键，而

是通过将用户作为节点、朋友关系作为节点之间的关系来建立模型的。

1.3 Neo4j中的图形数据

Neo4j将数据作为顶点和边存储（或者用Neo4j术语，节点和关系存储）。用户被定义为节点，朋友关系代表用户节点之间的关系。如果回顾图1-1的社交网络，你会发现它无非就是一张图，以用户作为节点和朋友关系箭头作为关系。

关系数据库和Neo4j图形数据库的重要区别是数据查询。在Neo4j图形数据库中，没有表和列的概念，也没有基于SQL的select和join命令。那么Neo4j如何查询图形数据库呢？

答案并不是“写一个分布式的MapReduce函数”。像所有的图形数据库一样，Neo4j借用图论的强大数学概念，并作为一个强大和高效率的引擎用于查询数据。这个概念就是图形遍历，图形遍历是使Neo4j具有强大处理大型图形数据的主要工具之一。

图形数据遍历

遍历是在图形数据库中在由关系连接的节点之间移动访问一组节点的操作。它是图形数据库数据检索的一个基本操作，因此，它也是图形模型中所特有的。遍历的重要概念是其本身的局域化，遍历查询数据时仅仅用到所需的数据，而不需要像在关系数据库中使用join操作那样对所有的数据集实施代价昂贵的分组操作。

Neo4j提供了丰富的遍历应用程序接口（API），可以用于在图形数据库中漫游。此外，还可以使用REST API或Neo4j的查询语言遍历数据。本书将

以大量的篇幅来讲解基本原理和Neo4j遍历数据的最佳实践。

要得到一个用户的所有朋友，运行程序1-2中的代码。

【程序1-2】 Neo4j在深度2查询用户所有朋友的遍历API代码

```
TraversalDescription traversalDescription =  
    ➤ Traversal.description()  
    ➤ .relationships("IS_FRIEND_OF", Direction.OUTGOING)  
    ➤ .evaluator(Evaluators.atDepth(2))  
    ➤ .uniqueness(Uniqueness.NODE_GLOBAL);  
Iterable<Node> nodes = traversalDescription.traverse(nodeById).nodes();
```

如果你不理解程序1-2中的代码也不必着急，所有的内容都会在后续的章节中慢慢地做出透彻的讲解。图1-3是基于前面对遍历的描述给出的这个社交网站的图形遍历图示。

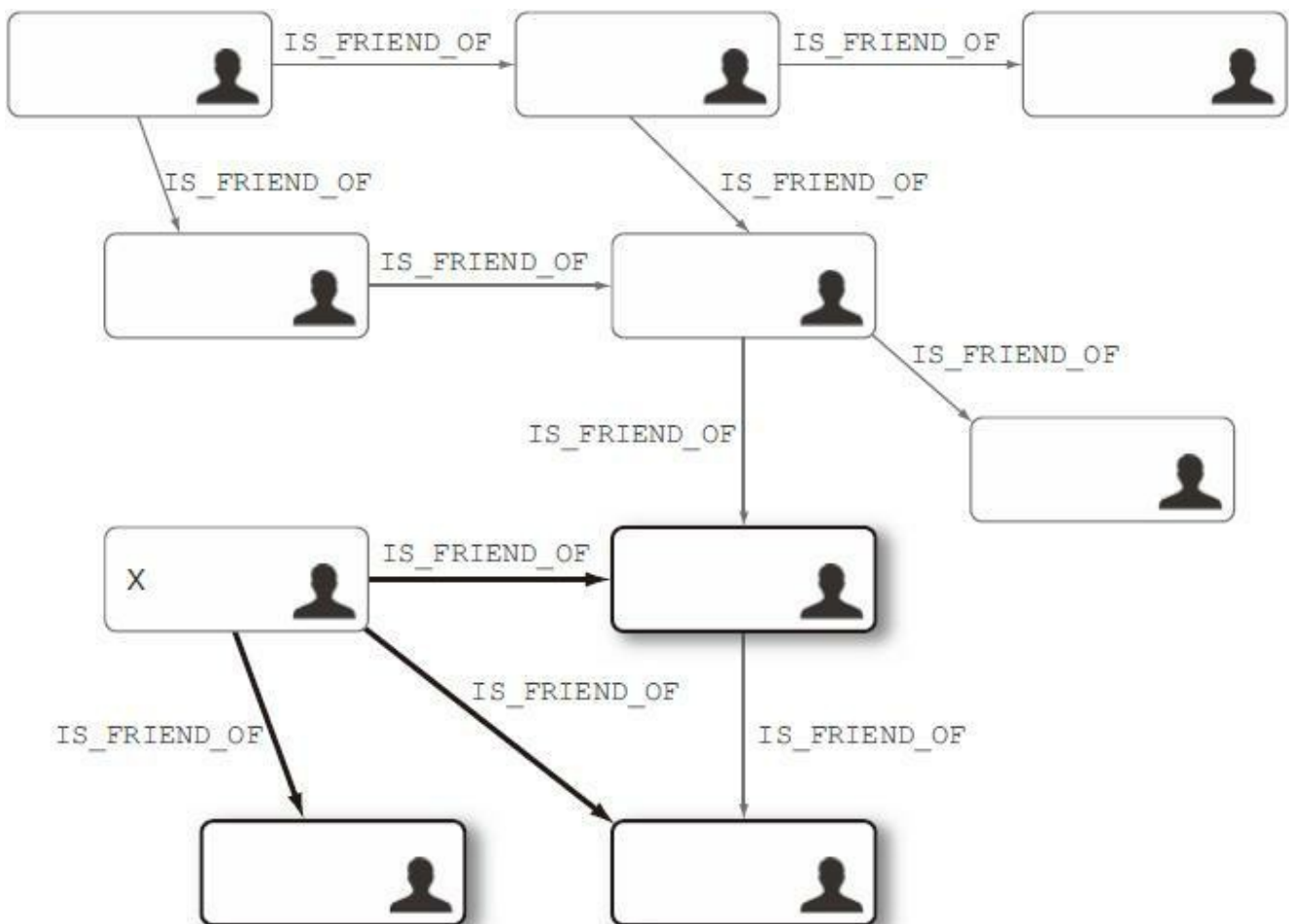


图1-3 社交网络图形数据库的遍历

在遍历开始前，首先选择遍历开始的节点（图1-3中的节点X）。然后跟随所有的朋友关系（箭头）并收集访问过的节点作为结果。遍历继续通过连接它们的关系从一个节点走到另一个节点。关系的方向并不影响遍历，你可以沿着箭头的方向往上或者往下，其效率是相同的。当不符合规则时，则停止遍历。例如，规则可能是从开始的节点只访问深度1的节点，在这种情况下，一旦深度1上的所有节点都访问完，遍历就停止（图1-3中的黑箭头表示本例子所跟随的关系）。

表1-2给出了在图形数据库中运行遍历的性能指标，该图形数据库中的数据与前面的MySQL数据库中的数据相同（这里遍历的功能与前面的数据查询功能相同，在给定的深度查询用户朋友的朋友）。这里再次使用的是一个1000个用户的数据集，平均每个用户有50个朋友。

表1-2 Neo4j对一个有1000个用户的图形数据库遍历的运行时间

深 度	查询 1000 个用户的运行时间（秒）	计数结果
2	0.04	~900
3	0.06	~999
4	0.07	~999
5	0.07	~999



注意

类似于MySQL的设置，不需要对Neo4j的例子进行额外的性能优化。Neo4j是在嵌入式并以默认配置在2048MB的Java虚拟机（JVM）堆内存下运行的。

需要注意的首要事情是，除了最简单的一种查询外，对其他所有查询，

Neo4j表现得都非常出色。只有当查询朋友的朋友（在深度2）时，MySQL的表现才能与Neo4j的遍历相媲美。在深度3的遍历查询中，朋友比MySQL快四倍。当在深度4进行遍历操作时，其对比结果高了五个数量级。在深度5，Neo4j的遍历查询速度要比MySQL的查询快上千万倍。

从表1-2可以得出另外一个结论，随着遍历深度的变化，当返回的节点数保持不变时，查询性能仅有轻微的降低。而MySQL的查询性能随着查询深度的变化而下降，这是因为所做的笛卡儿积的大部分都被丢弃。Neo4j跟踪记录访问过的节点，于是可以跳过之前访问过的节点，因此查询性能显著提高。

为了查询深度5的所有关系，MySQL将对t_user_friend表执行笛卡儿积运算5次，那将产生50000⁵个记录，其中只有1000个是有用的，其他的都被丢弃。Neo4j则只访问数据库中的节点，当没有更多的节点可访问时，它将停止遍历。这就是为什么Neo4j可以在返回的节点数目保持不变的情况下保持稳定的性能，而使用MySQL的查询将有明显的性能下降。



图形遍历明显优于等效的MySQL查询（在深度4和深度5的遍历优于等效的MySQL查询上千倍）。与此同时，遍历性能不会随深度明显降低，在深度5的遍历仅仅比深度2的遍历慢0.03秒。对于最复杂的MySQL查询的性能要比简单的查询慢10000倍以上。

但是，当遇到大数据时，这种图形遍历的表现如何？为了得到答案，让我们以一个拥有100万用户的数据集作为实例。

1.4 大数据下的SQL联接操作与Neo4j图形遍历的对比

对于这个实验，我们严格地使用与以前相同的数据结构，唯一的差别是数据的数量。

在MySQL的s_user表中，有1000000条记录，而在t_user_friend表中，大约有 $1000000 \times 50 = 50000000$ 条记录。针对该数据集，我们执行四个相同的查询语句（在深度2、3、4和5的关系）。表1-3给出了在这种情况下所得到的SQL查询的性能参数。

表1-3 使用MySQL数据库引擎对一个有100万个用户的数据集进行多重join查询的执行时间

深 度	查询 100 万个用户的运行时间 (秒)	计数结果
2	0.016	~2500
3	30.267	~125 000
4	1 543.505	~600 000
5	没有完成	—

把这些结果与MySQL对1000个用户的数据集的查询结果相比较，你可以看到，深度2的查询性能基本保持不变，这可以通过MySQL引擎有效地使用索引对表的join查询进行处理来做解释。在深度3和4（其中分别使用3个和4个join操作）时，其结果差很多，至少有两个数量级的差别。对于深度5的所有关系，在运行本段程序代码的一个小时内，SQL没有完成查询。



注意

存储这些例子的大量数据，需要一个相当大的磁盘空间。要生成样本数据和运行这些例子，需要超过10GB的可用磁盘空间。

这些结果清楚地表明，MySQL关系数据库只对单个join查询进行了优化，即使在大数据集时也是这样。针对大数据集时，其多重join查询的性能显著下降，到某种程度上，有些查询甚至不能完成（例如，在一个具有100万个用户的数据集中查询深度5的所有朋友）。

为什么关系数据库查询这么慢？

基于join操作的工作方式，表1-3的结果从某种程度上是可以预期的。正如我们前面所讨论的，每一个join操作都会产生所有潜在行组合的笛卡儿积，然后筛选出那些与where语句不匹配的项。对于一个拥有100万个用户的数据集，5个join操作的笛卡儿积（相当于对深度5的查询）蕴含着巨大数量的行记录——数以十亿计。这里没用太多的零以便阅读。过滤掉所有不匹配的查询结果的代价是巨大的，使得在深度5的SQL查询在一个合理的时间无法完成。

我们用Neo4j的遍历重复了同样的实验。用100万个节点代表用户，并有大约5000万个关系存储在Neo4j数据库中，同前面的例子一样，我们执行了同样的4个遍历，得到的运行性能数据如表1-4所示。

表1-4 Neo4j对一个有100万个用户的图形数据库遍历的运行时间

深 度	查询 100 万个用户的运行时间（秒）	计数结果
2	0.01	~2500
3	0.168	~110 000
4	1.359	~600 000
5	2.132	~800 000

正如你所看到的，数据量成千倍的增加并没有明显影响Neo4j的性能。随着深度的增加，遍历确实变慢，但最主要的原因是返回结果数量的增加。随着结果数量的增加，其性能呈线性减慢，在数据集和深度都增大的情况

下，这也是可以预期的。尽管如此，这也至少比MySQL的性能好上百倍。

Neo4j性能的可预期性的主要原因是图形遍历的局部性，不管图形中有多少个节点和关系，根据遍历规则，遍历将仅仅访问那些连接到起始节点的节点。请记住，关系数据库的join操作计算笛卡儿积并丢弃不相关的结果，随着数据集的增大，将按指数变化影响其性能。然而，Neo4j只访问与遍历相关的节点，所以能够不受总数据集大小的影响，从而保持期待的性能。要遍历访问的节点越多，遍历就会越慢，正如当增加深度时所见到的。但是，这种变慢是线性的，而且仍然不受总图形大小的影响。

Neo4j的速度秘密在哪里？

并不是Neo4j的开发者发明了一种超高速的算法，也不是基于一种具有魔幻速度的技术（其实就是在Java中实现的）。

秘密就在数据结构中——图形的局部性使得遍历的速度非常快。假设你要在一个当地的小足球场为你的球队加油。如果有人问你坐在你身边15英尺以内有多少人，你会站起来，并以最快的速度数一数你身边的人。现在设想你在国家足球场看比赛，有很多观众，你要回答的是同一个问题，即在你15英尺以内有多少人。假设这两个足球场的人数密度是一样的，那你会有大致相同的人数需要数，所用的时间也会非常接近。于是可以说，不管足球场能容纳多少人，你将以预计的时间把你身边的人数完，因为你只需要数坐在你身边15英尺以内的人，而不用去关心在球场另一边包厢里的人。

Neo4j引擎正像本例一样工作——以预期的速度访问与起始节点相连的节点。即使整个图形的节点增加（假设节点密度一样），性能仍然可以保持预期的速度。

如果对关系数据库的查询采用同样的足球比赛类比，那你需要数一数所有在球场的人数，然后删去那些不在你身边的人，这可不是给出相互关联数据的最有效方法。

这些实验证明了Neo4j图形数据库在查询图数据时比使用关系数据库的速度更快。另外，一个Neo4j实例处理了超过三个数量级的数据集而不使其性能降低。遍历的性能不受图形数据大小的影响是Neo4j的一个重要特征，这使得Neo4j成为解决图形问题的理想数据库，即使数据集非常大时也没有问题。

在接下来的章节中，我们将尽量回答什么是图形数据的问题，以及Neo4j如何帮助你像自然图形结构一样建立数据模型。

1.5 图形

图形是最普遍的自然结构。1774年瑞士数学家欧拉发表的对柯尼斯堡七桥问题的解决方案被称为图论的第一篇科学论文。七桥问题的解决代表图论的出现，现在，图论的模型和算法已经被广泛地应用于多个科学和工程领域。

现代科学中有许多图形应用的例子。例如，用图形描述化学化合物中的原子和分子的相对位置，物理学中原子的连接规则也用图形描述。在生物学中，进化树实际上也是一种形式的图形。在语言学中，用图形建模来描述语义规则和语法树。网络分析、交通建模应用、电信通信以及拓扑应用都基于图论。

在计算机科学中，很多问题以图形建模：社交网络、访问控制列表、网络拓扑结构、分类层次结构和推荐引擎等。传统上，所有这些问题都将使用图形对象模型（由开发团队实现），并以规范化的形式作为数据存储在一组以外键相关联的表中。

在前面的章节中，我们已经看到了如何将一个社交网络使用两个表和一个外键约束存储在关系模型中。使用Neo4j图形数据库，你可以保存数据的自然图形结构，而不会影响性能或存储空间，因此极大地提高了查询的性能。

当使用Neo4j图形数据库解决图形问题时，开发者的工作量将会减少，因为图形数据库的API中已经有了图形结构。数据以图形节点和关系的自然形式存储，所以，对象关系不匹配的情况很少发生。使用Neo4j的遍历API可

以使图形查询操作以高效率和高性能的方式完成。在本书中，我们将演示Neo4j在解决诸如访问控制列表和使用推荐引擎的社交网络等问题的强大功能。

如果你不熟悉图论，也不必担心。我们将在后面相关的章节中介绍一些图论的概念和算法，以及它们对图形存储引擎如Neo4j的适应性。

在接下来的章节中，我们将讨论为什么图形数据库（以及其他的NoSQL技术）开始在计算机行业中普及。我们还将讨论NoSQL系统的分类，并把重点放在它们的区别和适用性上。

1.6 Neo4j在NoSQL领域的地位

自从有了计算机软件，应用程序需要处理的数据的复杂性大大增加。数据的复杂性不仅包括它的大小，还包括它的相互关联性，结构的千变万化以及对数据的同时访问。

随着数据在这些方面的变化，人们已经认识到关系数据库从很长一段时间以来，尽管在事实上已经成为了数据存储的标准，但并不适合于解决面对数据复杂性日益剧增的所有问题。于是，多种新的存储技术应运而生，其共同的目标就是解决关系数据库不适合解决的问题。所有这些新的存储技术都归于NoSQL麾下。



注意

尽管NoSQL的名字已被人们接受，但这并没有准确反映出其代表的意义，反而给人们以错误的印象，认为它只是对SQL而言的一个概念。其实一个更好的名字，非关系数据库可能更适合于它，因为关系型/非关系范式是讨论的主题，而SQL只是关系型技术使用的一种语言。

NoSQL的诞生意味着应对数据的变化需要新的技术。Neo4j作为一个通用的图形数据库，只是NoSQL家族中的一员，它与其他许多或多或少与存储技术相关的成员一样。

随着NoSQL领域的迅速发展，它已变得越来越流行，并且具有了很多不同的解决方案和技术可供选择。任何新进入NoSQL世界的人，在选择技术时，都会面临很多选择。这就是为什么我们在这一节中将讲解NoSQL技术的

分类，并重点讲解每一类的适用性。此外，我们还将解释图形数据库的地位和Neo4j在NoSQL领域中的地位。

1.6.1 主键值存储

主键值存储代表了最简单，但却非常强大的处理大量并发访问数据的方法。缓存是一种典型的主键值技术。主键值存储允许使用非常简单的结构存储数据，这些数据常常在内存中，甚至在高并发环境下的高速访问。

这些数据存储在一个巨大的哈希表中，由主键进行访问。数据取主键值对的形式，对其操作大都局限于使用简单的put（write）和get（read）操作。其值仅支持简单的数据结构如文本或二进制内容，虽然一些较新的主键值存储支持一组有限的复杂数据类型（例如，Redis支持列表和映射作为值）。

主键值存储是最简单的NoSQL技术。所有其他的NoSQL类别都建立在简单性、高性能和可扩展性的主键值存储技术上，以更好地适合一些特定的使用场所。

1.6.2 列族存储

分布式主键值模型缩放得很好，但是，在这个模型中，必须使用某些类型的数据结构。这就是列族存储类别如何进入到NoSQL领域中的。

列族存储是将类似的值（或列）作为一组一起存储在同一列族中（例如，用户数据或有关书籍的信息）。使用这种方法，从一个单一的主键值存

储发展到一组相关值的存储。（可以把列族中存储的数据看作为一张映射图，或者将主键值存储到每一个值都是另外一张映射图的地方。）列族都存在于一个单一的文件中，以实现更好的读取和写入相关数据的性能。这种方法的主要目的是实现在大数据时的高性能和高可用性。所以一点也不奇怪，这个领域的领先技术是Google的BigTable和Cassandra，最初都是由Facebook开发的。

1.6.3 面向文档的数据库

很多实际问题（如内容管理系统、用户注册数据以及CRM数据）的需求看上去像一个文件的数据结构。面向文档的数据库正好提供了这样的地方，用以存储简单的，然而高效的，无架构的文档数据。本文档模型中使用的数据结构，可以使你添加自包含的文件和关联关系到该文档的数据中。

你可以将面向文档数据库看作主键值存储，这里的值就是一个文件。这使得面向文档数据库更容易为通用软件问题建立数据模型，但这使它的性能和可扩展性相比主键值和列族存储稍微有些降低。内置于存储系统的对象模型的便利通常是一个对所有情况很好的权衡，但不包括大量的并发使用情况（毕竟我们大多数人并不想再创建一个Google或者一个Facebook）。

1.6.4 图形数据库

图形数据库的设计遵从以下的观点，即开发人员常常在他们的应用程序中建立图形状的数据结构，但仍然将数据以一个非自然的方式存储，或者是

在表格和关系数据库的列中，或者在其他的NoSQL存储系统中。正如我们之前所提到的，像ACL列表、社交网络或任何类型的网络问题都是自然的图形问题。图形数据模型是图形数据库的核心，最终能够将代表图形的数据作为一个直接图形存储一个对象模型。

这个数据模型可以自然地表达许多非常复杂的软件需求，图形遍历查询的效率和性能是图形数据库的主要优势。

1.6.5 与NoSQL类数据库的比较

表1-5给出了每一个NoSQL类别的使用实例及所代表的技术。

表1-5 NoSQL分类概要

NoSQL 分类	典型的应用案例	最著名的技术
主键值存储	■ 缓存 ■ 快速读取访问简单域 ■ 大规模并发系统	■ Redis ■ Memcached ■ Tokyo Cabinet
列族存储	■ 大规模写入 ■ 并发数据访问（读和写）	■ Cassandra ■ Google BigTable ■ Apache HBase
面向文档数据库	■ 当域模型是一个自然文件 ■ 使用自然文件数据结构简化开发 ■ 高可扩展系统（尽管在主键值存储和列族存储的低层次上）	■ MongoDB ■ CouchDB

（续）

NoSQL 分类	典型的应用案例	最著名的技术
图形数据库	■ 拥有互联的数据 ■ 能够自然的用节点和关系表达的域 ■ 社交网络 ■ 推荐引擎 ■ 访问控制列表	■ Neo4j ■ AllegroGraph ■ OrientDB

到目前为止，在本章中你已经看到了使用Neo4j有效解决与图形相关的问题，以及如何将常见的实际问题以自然的图形模型建模的例子，并了解了

在广阔的NoSQL领域中图形数据库和Neo4j特别适合的领域。Neo4j还有一个关键的、其他NoSQL存储所没有的特点，那就是Neo4j采用了企业界一个新的极其重要的存储技术：事务行为。

1.7 Neo4j具有与ACID兼容的数据格式

自从NoSQL技术开始流行以来，事务管理一直是人们讨论的一个重点话题。对提高性能和可扩展性的事务属性的权衡一直是非关系型技术处理大型数据的通用做法。有些（例如BigTable、Cassandra和CouchDB）使用了权衡的一致性，允许客户在分布式系统（最终一致性）的某些情况下读取陈旧数据。在注重读取性能（例如内存缓存）的主键值存储中，对数据的持久性没有太高的要求。同样，在一个单一操作层面的原子性，一个单一事务中没有包含对多个数据库进行联接操作的可能性，这就是典型的面向文档的数据库。

这里提到的每一种方法都是在某一种特定的情况下（如高速缓存、高速数据读取卷、高负载和并发访问）使用才有效。当涉及将非关系数据库引进任何企业或公司环境时，缺少基于事务处理的ACID通常是第一道门槛。虽然事务属性是在很久以前为关系数据库设计的，但事务属性仍然在很多实际应用中发挥着重要和关键的作用。于是，Neo4j采取了不同的方法。

Neo4j的目标是成为一个图形数据库，并把重点放在数据库上。这意味着你将在Neo4j数据库中得到ACID的全部支持：

- **原子性**（A）——你可以在一个事务中包含对多个数据库的操作，并确保它们都是在原子性的情况下执行的，如果其中有一个操作发生问题，则整个事务将全部回滚。

- **一致性**（C）——当向Neo4j数据库中写数据时，你必须保证随后访问数据库的每个用户读取的都是最近、更新的数据。

·**隔离性 (I)** ——必须确保一个单一事务中的操作与另外的事务中的操作是隔离的，因此，一个事务中的写操作不会影响另一个事务中的读操作。

·**持久性 (D)** ——必须确保写入到Neo4j数据库中的数据保存到磁盘，以便数据库重新启动或服务器崩溃后仍然可以获取写入的数据。



注意

了解Neo4j是一个事务数据库是很重要的，你知道的在关系数据库中关于事务的一切都适应于Neo4j。我们前面提到的事务支持是Neo4j与其他NoSQL数据库之间的主要区别，NoSQL数据库不支持所有的ACID属性。

对任何使用关系数据库提供的担保并通过ACID事务支持可以确保无缝地过渡到Neo4j，ACID事务支持也提供了安全和方便的图形数据工作方式。事务支持是Neo4j的一个重要特点，这是它与其他NoSQL数据库解决方案的区别之一。这不仅对NoSQL的爱好者，就是对企业用户也是一个很好的选择。

1.8 本章小结

通过本章的学习，我们知道了相对于关系数据库，Neo4j在处理一些特殊问题时更加有效，性能更加优越，例如在社交网络实例中。我们以Neo4j为例，说明了图形数据库在处理与图形相关的问题时相对于关系数据库的性能和可扩展性的明显优势。

我们也了解了一些Neo4j的主要特点，以及其在与数据存储技术相关的NoSQL庞大队伍中的地位。

使用图形数据库存储数据会影响到你对数据的原有思维方式，这是因为数据建模变得非常重要。图形数据库中没有关系数据库中的表和列，也没有其他NoSQL技术中的键和值。你必须将思维转换为以图形代表数据上。在下一章中，我们将给你一些对图形数据建模的最佳实践指导和讲解。

第2章 Neo4j的数据模型

本章包括以下内容：

- 为什么Neo4j需要一个数据模型
- 领域建模
- 探索其他的领域

在本章中，我们将讨论如何以及为什么要在Neo4j中为数据建模，并讨论图形数据库中的建模方法。先让我们看看对给定领域建模的另一种方法，即以不同的方式利用节点、关系、属性和标签建模。我们还将给出几个来自不同领域的例子以展示Neo4j数据建模的灵活性。

我们来看看用Cypher查询语言写的查询语句。这纯粹是为演示，让你了解查询这里给出的模型是多么的简单。你不需要了解Cypher就可以开始给数据建模，后面将对Cypher查询语言做详细的介绍。

2.1 Neo4j中数据模型的类型

不同于传统的关系数据库管理系统（Relational Database Management System, RDBMS），Neo4j是一个无架构数据库。在开始添加数据之前，你并不需要定义表和关系。一个节点可以具有你喜欢的任何属性，任何节点都可以与其他任何节点建立关系。Neo4j数据库中的数据模型隐含在它存储的数据中，而不是明确地将数据模型定义为数据库本身的一个部分。它是对你想要存入数据库的数据的一个描述，而不是数据库的一系列方法来限制将要存储的内容。

由于Neo4j的数据建模是描述性的，而不是规定性的，因此可以很容易对数据库将要存储的数据进行一致性的描述，这样就可以以期望的方式构架查询，并以相似的方式来描述相似的实体。Neo4j的查询语言Cypher，以数据模式匹配的方式工作，因此，将数据模型看作一个基本模式的库存列表就是一种方法，例如：

- 一个人的居住地址。
- 地铁线路中的一个地铁站与另一个地铁站的连接。
- 智能电话套餐包含了一部智能电话和话费套餐。

这些模式适合于以下实际场景：

- Peter住在Gables，Mary住在Acacia大街13号。
- 银行站点连接Moorgate站点，银行站点和Moorgate站点都连在西北线路

上。

·话费较少的用户套餐包括一部FictoPhone智能手机和每月150分钟的移动对移动免费通话。

一旦你对将要存储在数据库中的数据进行了一致性描述，你就可以在以后使用该描述来对将要存入数据库的数据的查询进行推理。

表达这种描述的一种常见方法是图表。通过绘制表示数据的常用图形模式图，你就可以以直观的方式看到你的数据。在Neo4j的早期数据建模阶段，笔和白板、一张纸或餐巾纸往往是你需要开始讨论的数据样子。

2.1.1 图表建模的一个简单实例

假设人与人之间并不是一种朋友的关系，而你感兴趣的是人群之间的关联模型。一个组可能包含几个人，并且一个人也可能属于多个组。在传统的关系数据库中，通常会使用三个表的模型表达这种情景：一个人员表、一个组表和一个用于连接人和组的多对多关系链接表，如图2-1所示。

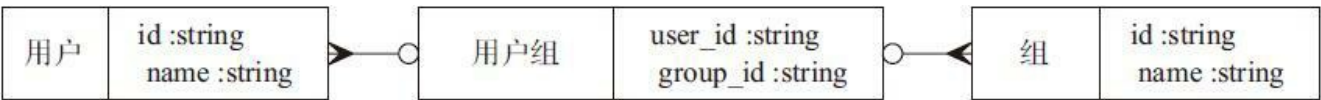


图2-1 关系数据库中用链接表连接的用户和组

在Neo4j中，你可以将用户和组以同样的模型描述为节点，将组成员描述为节点之间的关系，如图2-2所示。

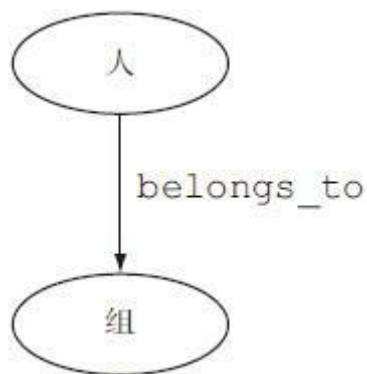


图2-2 一个人属于一个组

在这里你大致可以看到节点是图形数据库的数据模型实体（与RDBMS的术语对应），而关系毋庸置疑是节点之间的关系或连接。用纯语言表达这个图表的关系就是“一个人属于一个组”。

让我们看看有几个人和几个组的情况会是如图2-3所示的样子。

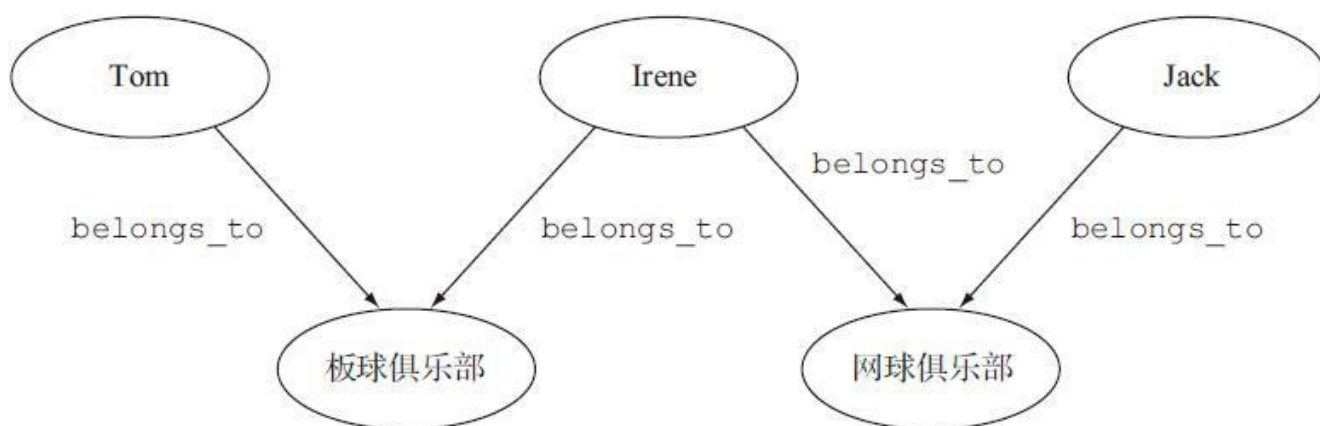
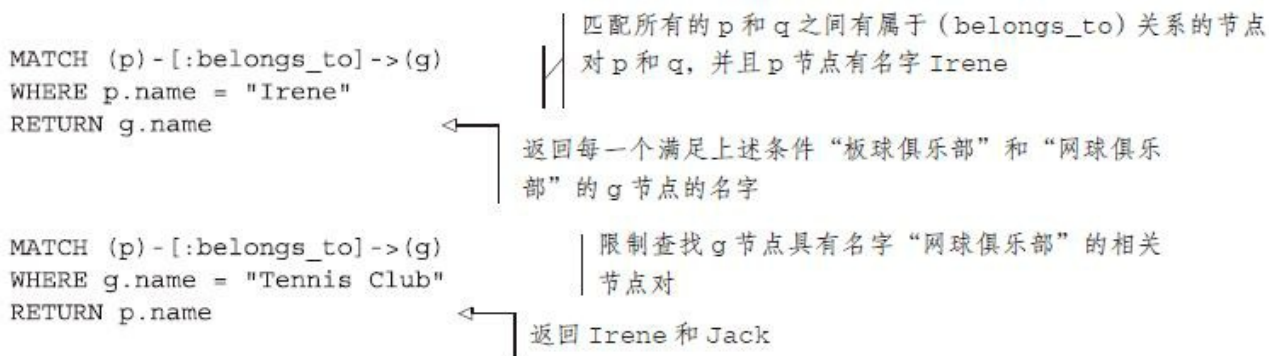


图2-3 两个组中的三个人

图2-2所示的模式出现在图2-3的几个地方。事实上，整个图2-3可以通过叠加几次图2-2得到。

当查询这个模型中的数据时，可能有两个问题需要回答。首先，对于一个给定的人，这个人属于哪个组？第二，对于一个给定的组，哪些人属于这个组？使用Neo4j的查询语言Cypher很容易实现。



看上去这种模式适合我们的实例。通过从“一个人属于一个组”的简单关系开始，我们仅仅通过重复这种关系就能生成一个复杂的图形。我们最初的图2-2实际上代表了整个图形。

对于更复杂的模型，有时按相反的方向思考更容易理解。可以首先在系统中画一张图把例子中所有不同的东西表示为节点，然后绘制箭头表示出这些节点之间的所有不同种类的关系。一旦开始看到重复的图案，就可以把它们拉出来并把它们归属为更小的图片单元。

2.1.2 图表建模的一个复杂实例

让我们多看几个不同的例子，看看它是如何分解成更小的图案的。在大型组织中，基于用户不同的角色、不同的任务和不同的组别授予其权限访问不同的系统。下面是用户可能具有的访问权限示例。

- **直接的**——你有权限访问共享文件系统中你自己文件夹中的文件。
- **基于某任务**——作为数据库的管理员，你有权限连接到数据库控制台。
- **基于组成员**——作为市场和销售团队的成员，你有权限连接到联系人

管理系统。

·基于对组所有成员指定的任务——作为一个开发团队的成员，所有的成员都有源代码控制提交的任务，都有权限从源代码库中取出代码并进行修改后返回原代码库。

图2-4是一个未连接的授予这些权限的访问控制系统图。

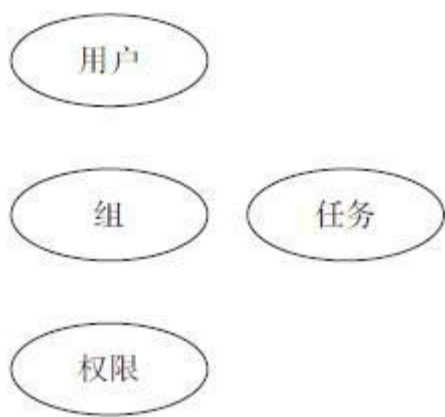


图2-4 访问控制系统中的实体

图2-5显示了所绘实体之间可能存在的关系的同一图形。

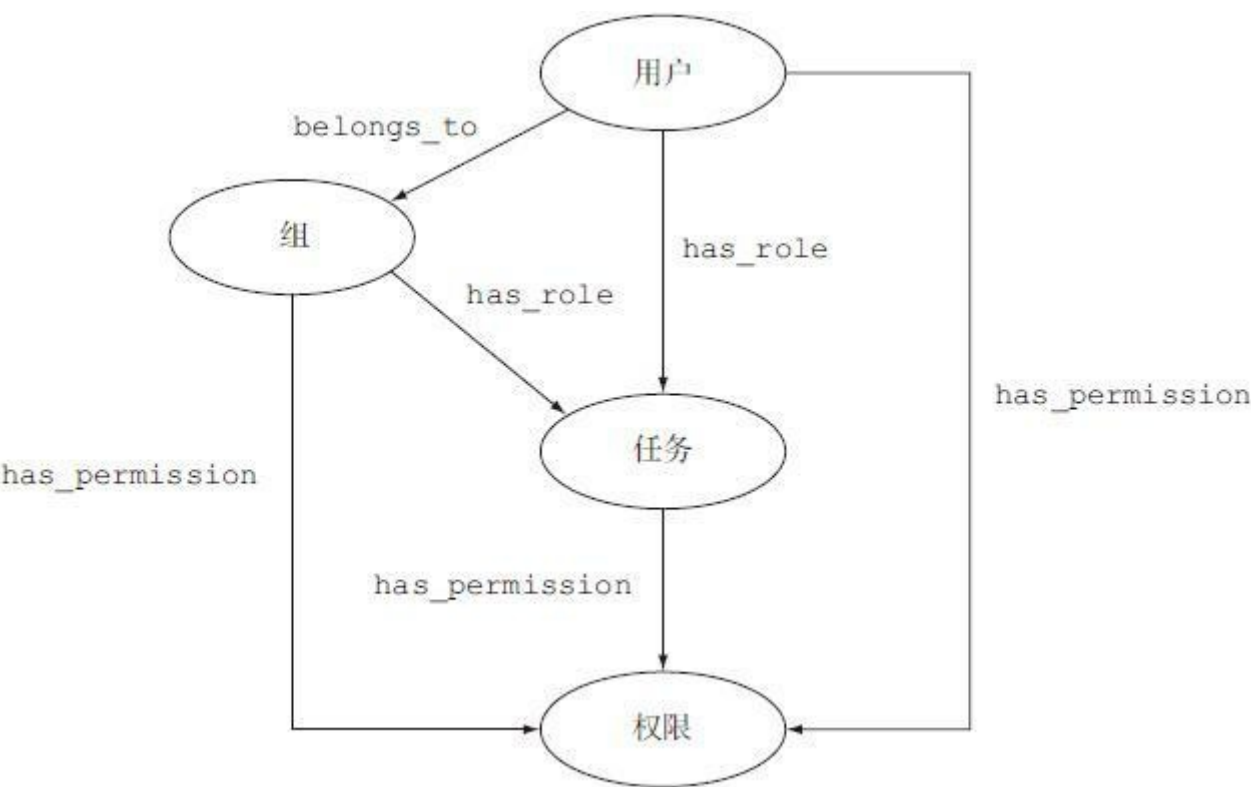


图2-5 访问控制系统中实体之间的关系

正如你所看到的，有几种从用户到权限的可能路径。为了找出一个用户所拥有的权限，你需要找出一个用户节点通过任何途径连接的所有权限节点。在传统的关系数据库中，这需要一个复杂的查询表达每一个不同路径的join操作。

- 从用户直接到权限（一表join操作）。
- 从用户到组再到权限（二表join操作）。
- 从用户到任务再到权限（二表join操作）。
- 从用户到组、到任务再到权限（三表join操作）。

使用Cypher，一个短的查询即可实现：

```
MATCH (u:User)-[*]->(p:Permission)
WHERE u.name = "User name"
RETURN DISTINCT p.name
```

这将匹配任何第一个具有用户标签且其指定的属性值为姓名（name）的节点与第二个标记为权限节点的节点对，并且在它们之间有一个可以是任意长度的路径。最后，返回所有匹配了的权限名称。

现在假设你想对这个模型添加一个新功能，以反映组织的层次结构。用户不仅可以属于组，组还可以是较大组的下级组，例如，Tester、UX Designer和Coder，可能隶属于开发组的小组。图2-6显示了在组之间具有下级组（subgroup_of）隶属关系扩展的新模型。

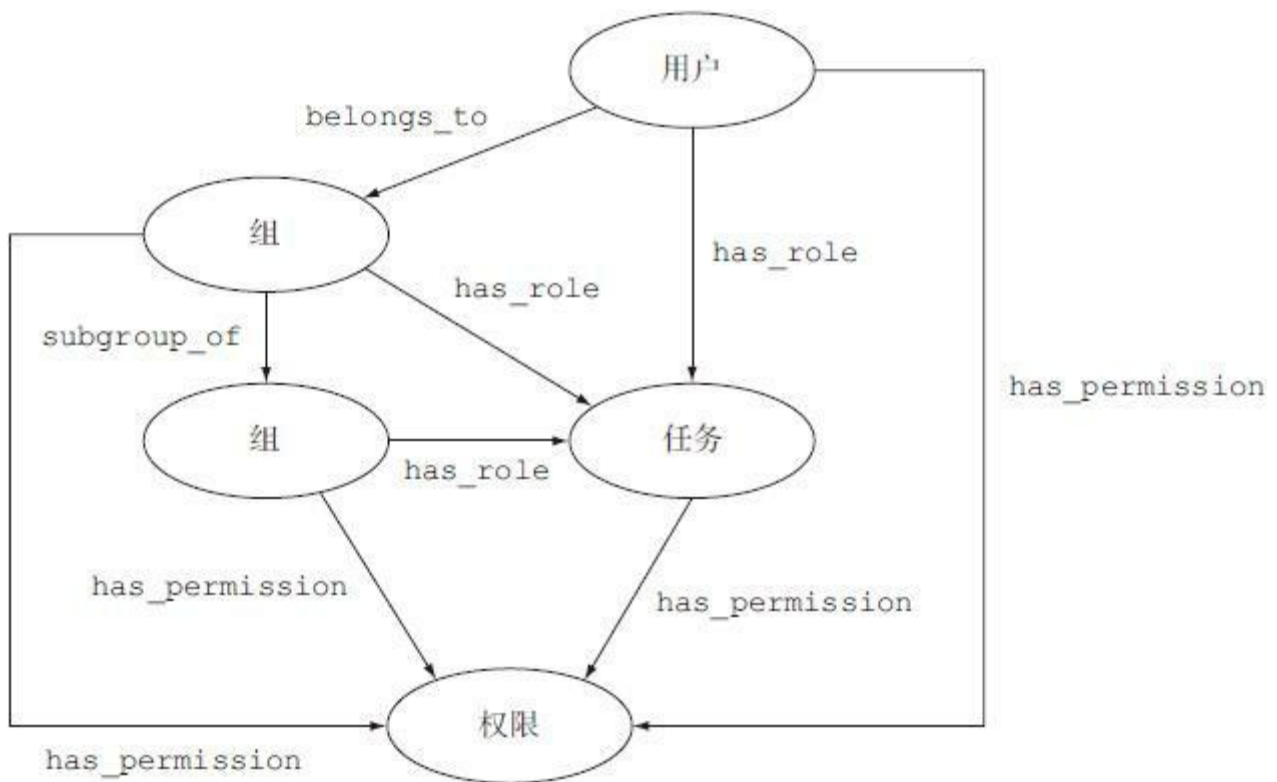


图2-6 具有支持下级组扩展的访问控制模型

要改变关系数据库的查询以支持此模型的方案可不是一件容易的事，而最初模型的Cypher查询也完全适合这一扩展的模型，因为对用户查询一个权限与查询一个关系没有任何区别。

2.2 领域建模

在2.1节讨论的模型中，节点除了自己的标识名字以外，其本身具有的额外信息太少。用图形的节点表示领域实体——用户（User）、组（Group）、任务（Role）和权限（Permission）是非常直观和清晰的。但是，并非一切事情都必须成为一个节点，例如，并没有任何节点表示用户的姓名，而这是以用户节点的属性（property）表示的。

属性是Neo4j数据模型的重要组成部分，而且这也是它不同于关系数据库模型的重要特征。在Neo4j中，节点和关系都可以具有任意数量的属性，这是非常重要的键/值对（例如，姓名/“Irene”）。它们通常用来存储节点和关系的特定属性。

让我们考虑一个**实体**（以节点建模）和一个实体的**属性**之间区别不太明显的场景。

实体和属性

当人们第一次开始用Neo4j建模的时候，他们用了个看上去如图2-7所示的图形。

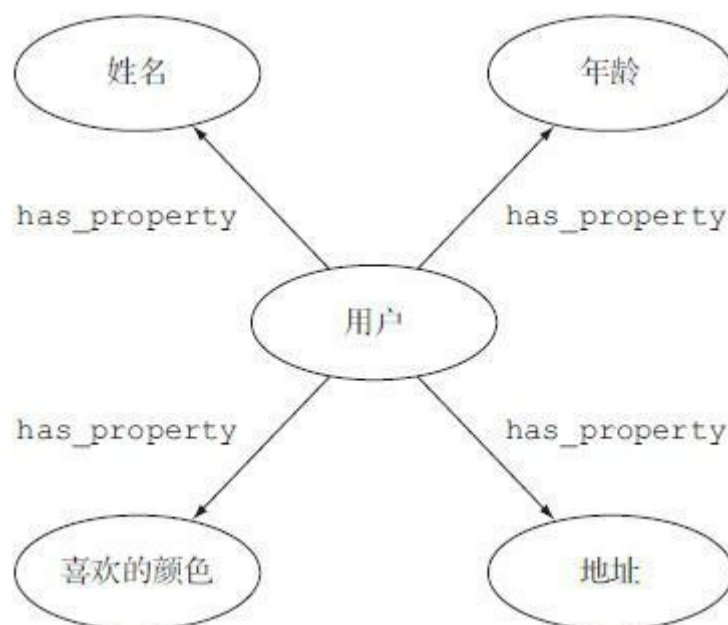


图2-7 一个具有某些属性的用户

可能有一些场景，这也是建立用户及其属性模型的完全正确的方式，但是这里有一些警示标志。如果看到一个标有具有属性（`has_property`）或类似标识的关系，应该考虑连接节点另一端的节点是否真的必须是一个节点。除非，出于某些原因，你想基于用户自己喜欢的颜色将他们连接起来，可能没有必要对喜欢的颜色作为模型的一个节点建模，而以用户节点上的属性为宜，对于姓名和年龄属性也同样是这样的。

如果你将这些节点转换成用户节点上的属性，然后重新命名剩下的具有属性的关系，使其能更确切地描述用户所具有的关系和其相关的地址，那么这时的示意图就如图2-8所示。

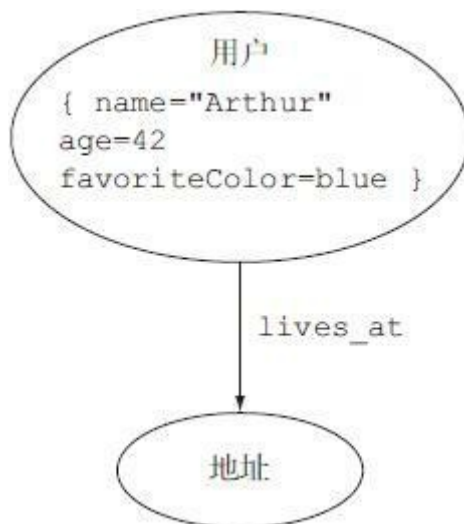


图2-8 一些节点转换为属性的用户

地址与姓名、年龄和喜欢的颜色不同，因为你可能会使用共享地址的方式将用户关联起来。一个用户搬家后可能会有一个当前的地址和原来的地址，或者一系列曾经住过的地址。因此，将地址作为实体建模，用它们自己所属的节点表示地址比较合理。

现在，假设你想要区分一个用户的当前地址和以前地址，一个方法是附加一个现在的（`is_current`）标志作为属性到地址节点上。但是，如果两个用户一直住在同一个地址，一个是当前的地址，而另一个是以前的地址，那么就无法确定这个标志值在什么时间为哪个用户取什么值。用户的当前地址不是地址本身或者用户的属性，但它们之间是属于关系，关系可以随着时间的变化而改变，就像用户从一个地址迁移到另一个地址。

至少有三种方式来表述一个地址是用户的当前地址。第一种方式是标注不同的关系如图2-9所示。

第二种方式是对关系本身设置属性如图2-10所示。如果不考虑用户是否住在现在的地址中而只是把用户和地址连接起来，你可以采用这种方式。具有不同的关系类型将使查询变得复杂。

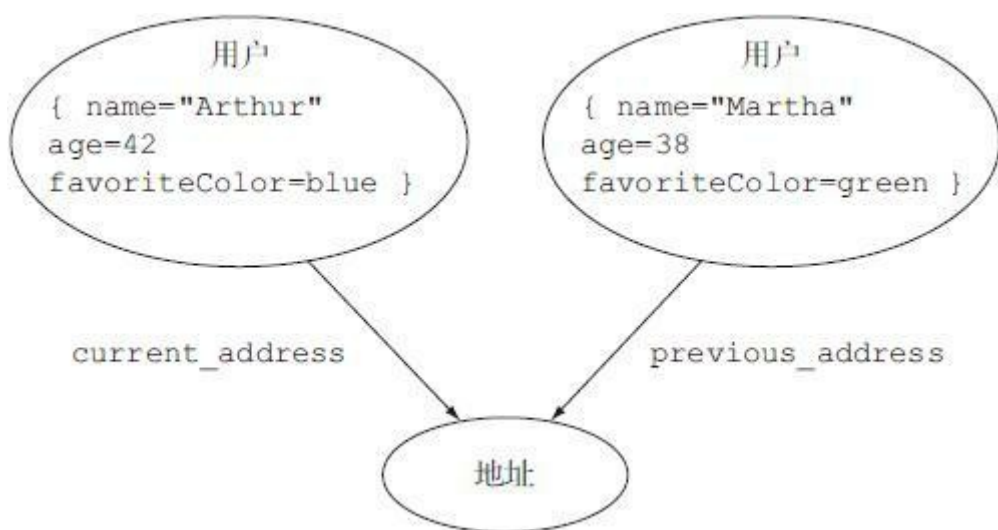


图2-9 两个用户对同一地址具有不同的关系

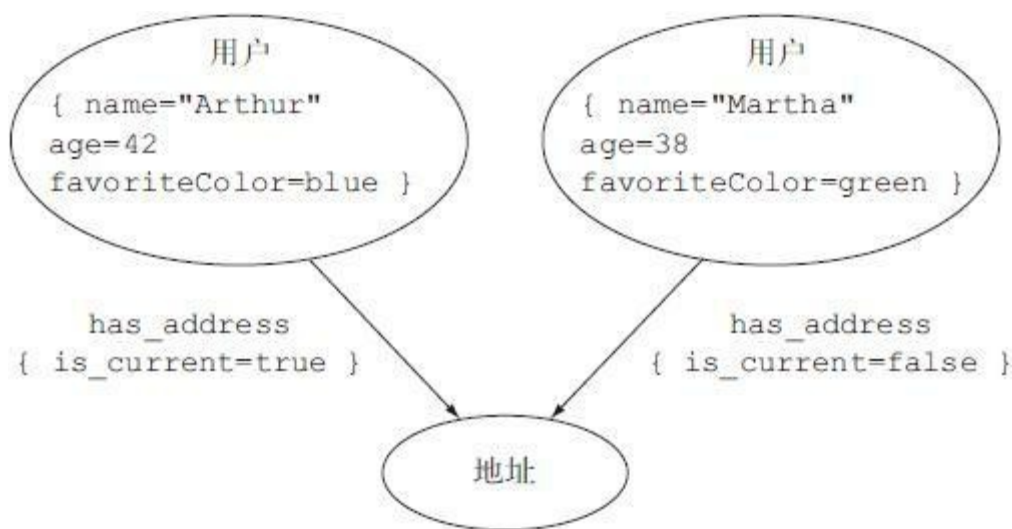


图2-10 两个用户对同一地址具有不同条件关系

第三种方式是将用户的一个占有地址作为其自身的一个节点。这是一种称作**物化**的技术（字面含义是使其成为一种东西），其中两个节点之间的关系被分成由一个中间节点连接的两种关系。当关系本身与其他实体有关系时，这种方法是很有用的。例如，如果一个用户的占有地址在中介机构管理的租约下，图2-11给出了这种情况是如何处理的。

在这种情况下，标签表示该占用是否是当前地址已经添加到占有节点本身，但也可以将其设置为具有（has_held）关系的属性，或者用两种不同标

签来表示现在的地址与以前的地址。

当决定是否使用节点建模实体的属性时，另一个需要考虑的因素是用查询引擎遍历图形数据库时的效率。例如，把一个节点的数据扩展为与另一个节点的关系上，目的仅仅是为了具有额外的属性，通常这并不是一个好的方法，如图2-12所示。

这种情况的一个更好方法是对用户节点使用一个**标签**（label）表明它具有扩展属性组，并将额外属性直接放在用户节点本身。标签是一个强大的机制，在Neo4j的2.0版本中引入，用于给集合分配节点。一个节点可以有多个标签，所以节点可以视为属于多个集合，也可能具有交叉重叠。通过编写针对特定标签的节点查询，可以做两件事情：

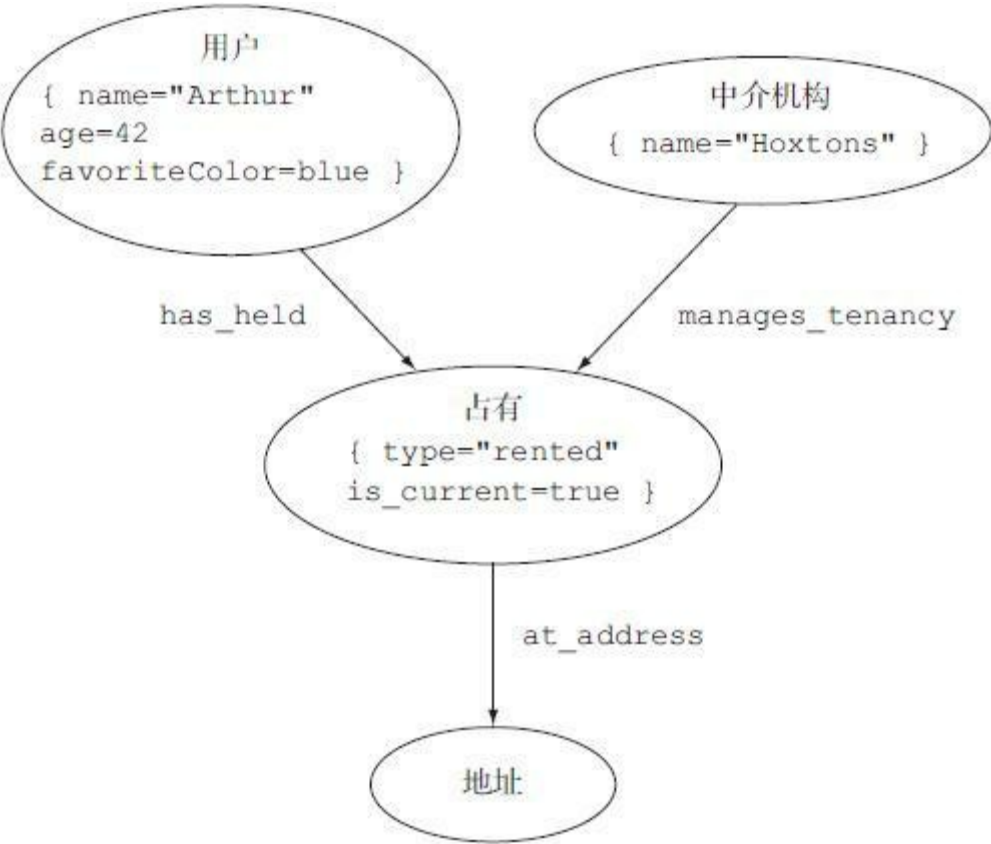


图2-11 为了引进其他参与者而具体化用户和地址之间的关系

·当在遍历图形数据库搜索匹配模式时必须考虑减少节点改进查询遍历的性能。

·使用标签说明节点具有的特定属性以简化查询，以便查询不必处理缺少属性值的可能性。

你已经在本章中见过Cypher查询实例中的标签使用方式：一个类似u:User的表达式匹配一个节点与用户标签，而忽略任何其他的节点。

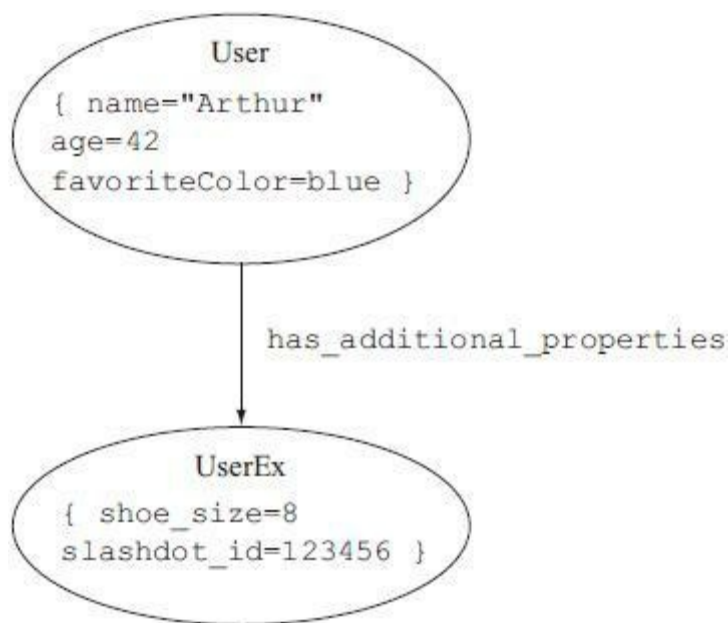


图2-12 一个在相关UserEx节点存有额外属性的用户

决定使用哪种描述方法的关键是能够容易查询和遍历数据中包含的数据模式。如果很难看出两条信息是如何相连接的，但实际上它们的关系是非常简单的，那么考虑更直接地将它们联系起来。如果有这样的节点，其唯一的目的是存放属于其他实体的一些信息，那么考虑将这些信息放入该实体的属性中。

在这个阶段，不要太担心能否找到合适的模型。使用Cypher很容易重构图形数据库，将现阶段模型中的仅仅为隐式关系明确为显式关系。对于图形

数据库没有标准模式——只有最适合具体应用的形式。

2.3 更多实例

我们将以几个来自于不同领域的例子结束本章，这些例子既说明了Neo4j可以建模的领域范围，也阐述了两对相互对立的概念，即在图形数据库中，模型中的元素是隐式描述还是显式描述，或者数据通常是不变的还是可变的。

让我们先回顾一下前面提到的地铁站的例子。

2.3.1 地铁车站实例

我们的第一个例子涉及在什么时候应该明确地使用节点与关系对数据直接建模和在什么时候应当隐式建模（但可通过查询获得数据）的问题。在这个例子中，我们对伦敦地铁系统的车站、线路和连接进行了建模。我们对两个方面感兴趣：哪些车站直接相连，哪些车站在哪条线路上。

这里最大的问题是要不要把地铁线路直接作为模型的节点，或是作为隐式的关系属性。如果将它们建模为节点，那么你可能需要连接各个车站及轨道线上的每个轨道段，如图2-13所示。

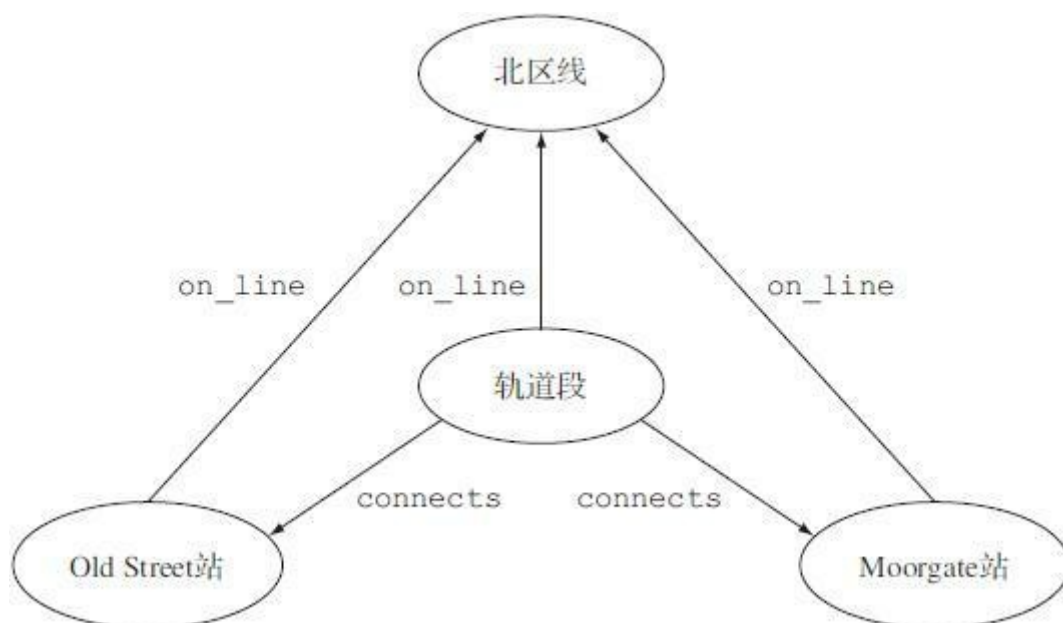


图2-13 两个具有连接的地铁站视为单一的一个轨道段

这是一个非常繁忙的模型，如果你试图对多条线路通过多个车站进行建模，那么你的模型很快就会变得相当复杂。相反，如果你将地铁线路作为关系的属性建模，那么你的模型就简单多了，图2-14显示了两个站由两个不同线路的连接。

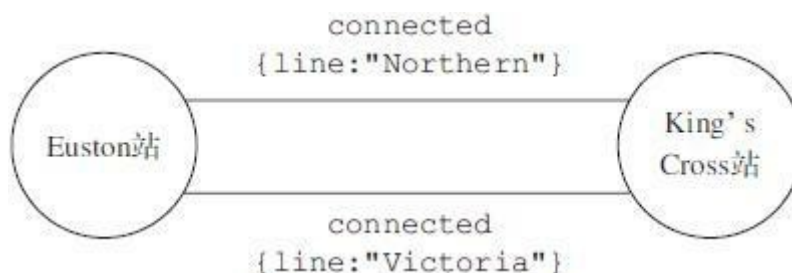


图2-14 连接于两条不同线路的两个地铁车站

然而，为了知道Euston站在哪条线路上，你不得不找出它与哪个车站相连。幸运的是，使用Cypher查询可以很容易地做到这一点：

```

MATCH (s1:Station)-[r:connected]-(s2:Station)
WHERE s1.name="Euston"
RETURN DISTINCT r.line
  
```

查找与 Euston 站相连的所有车站，返回不同线路的名称

这将会找到所有Euston站与其他车站的连接，并返回所有那些连接的不

同线路属性值。

这两种方式都可以对地铁系统进行建模并包含完全相同的信息。可以通过查询将第一种方式建模的图形数据转换为第二种方式建模的图形数据，反过来也可以，并不会丢失任何信息。那么，到底选择哪种方式好呢？答案主要取决于你如何去填充数据库和打算如何进行查询。

在第一种建模方式中，如果必须创建和管理显式节点和关系来表示地铁线路、哪些车站在哪些线路上，则数据库将比只用节点代表车站、关系代表车站之间的连接笨重很多，并且也更难理解。运行查询以查找某一特定线路上的所有车站将非常快，因为Neo4j数据库引擎将仅仅从一个节点沿着关系到所有与它直接相连的节点：

```
MATCH (l:Line)<-[:on_line]-(s:Station)
WHERE l.name="Central"
RETURN s.name
```

查找代表名字为“中心线路”的节点和直接与它相连的所有车站，返回车站的名字

如果对姓名（name）属性和线路（line）节点做过索引，这个查询是非常快的，因为Neo4j查询引擎会使用这个索引找出它的索引节点。

第二种建模方式的相应查询则需要对图形数据库更多的遍历才可以得到结果。

```
MATCH (s:Station)-[r:connected]-()
WHERE r.line="Central"
RETURN DISTINCT s.name
```

查找通过“中心线路”连接到另一个车站的所有车站并返回它们的名字

在这种情况下，数据库引擎必须考虑一对车站节点之间的每一个连接关系，统计具有指定的线路属性的所有节点，然后返回特定的车站名字。如果这是一个可能会经常使用的查询，那么就值得通过创建代表那些信息的显式节点和关系以便更容易获得这些信息。

本例子说明在不同的领域建模时，有时将一些实体在图形数据库中以隐式建模而不是用专用的节点以显式建模是合理的。相反，有时则将隐式信息表面化并以显式建模，以便Neo4j查询引擎能更快的查找到它们。

2.3.2 乐队成员实例

第二个例子涉及处理随时间不断变化的信息，以便分辨不变的问题和变化的问题。

假设你想追踪哪一个音乐家在不同的摇滚乐队演奏哪一种乐器，一个简单的模型可能会如图2-15所示。

用该模型的一个问题是乐队的成员会经常发生变化。为了处理这种变化，可能需要创建表示特定时间事物状态的中间节点。可以引入表示乐队阵容的节点，记录某一特定专辑录音时的阵容，如图2-16所示。

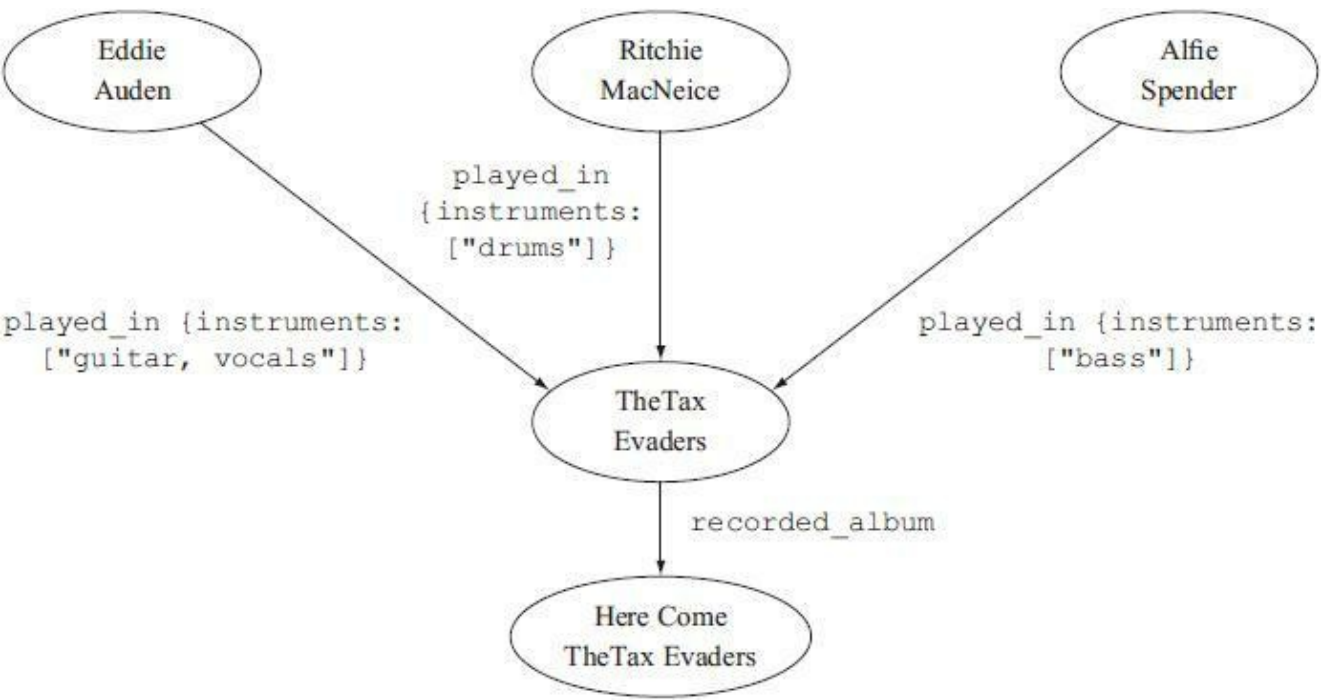


图2-15 一个表示乐队成员和录音专辑的简单例子

注：The Tax Evaders是乐队的名称，Here Come The Tax Evaders是专辑名称。

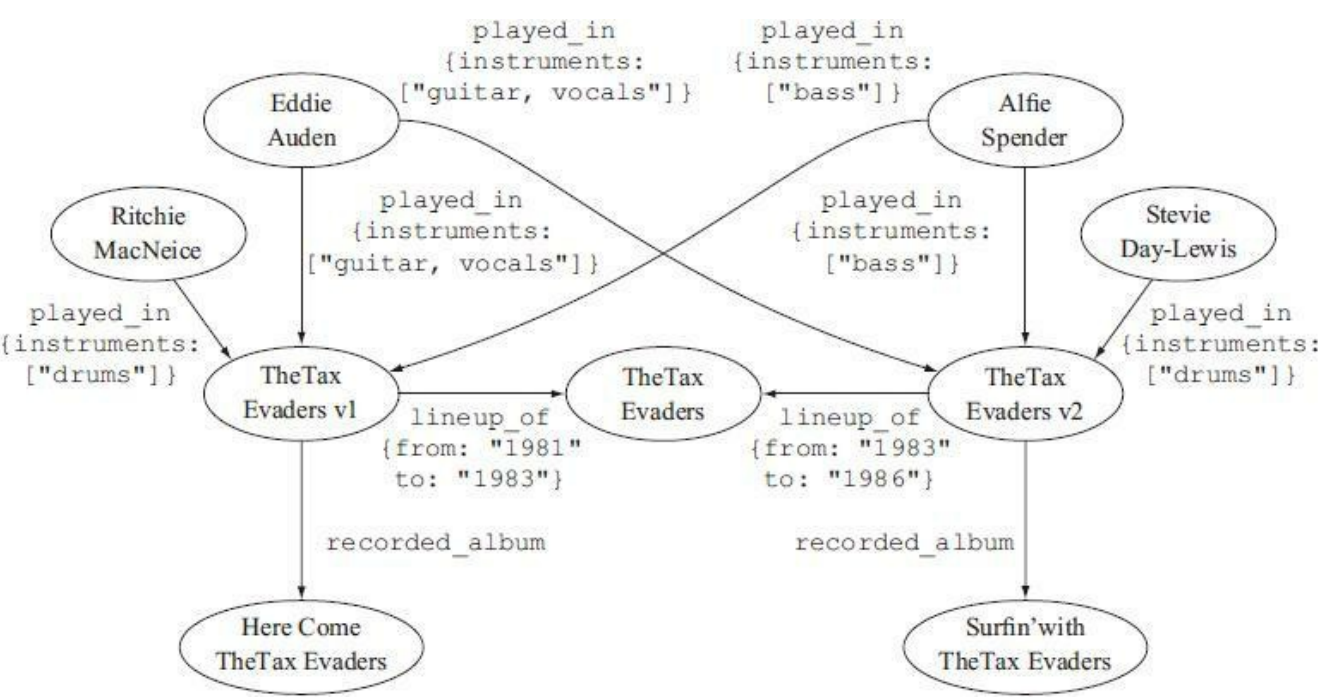


图2-16 表示同一个乐队不同阵容的一个较复杂的例子

用该模型时，你可以很容易地查询到Tax Evaders乐队（任何阵容）录制的所有录音专辑：

```
MATCH (a:Album)<-[:recorded_album]-(l:Lineup)-[:lineup_of]->(b:Band)
WHERE b.name="The Tax Evaders"
RETURN a.name
```

返回 “Here Come The Tax Evaders” 和 “Surfin’ With The Tax Evaders”

但是，你可能很容易地查询到Ritchie MacNeice演奏鼓的专辑：

```
MATCH (a:Album)<-[:recorded_album]-(l:Lineup)<-[:played_in]-(m:Musician)
WHERE m.name="Ritchie MacNeice"
RETURN a.name
```

仅返回 “Here Come The Tax Evaders”

在图2-11中有一条不变的信息，即有一个名为Tax Evaders的乐队，即使其成员变化，该乐队仍保持其完整性。可以将Tax Evaders乐队构成的变化用

节点描述其不同时间的即时状态，并用阵容（`lineup_of`）关系连接这些状态到中央节点，表示它们是同一事物的不同状态。同样，不同音乐家在乐队中演奏的身份保持不变，尽管他们与乐队（也可能是其他乐队）的关系将随时间发生改变。

这个例子展示了一个具体化的实例——描述实体之间的关系，例如音乐家和乐队之间的关系，乐队作为一个节点与其本身的关系。阵容是乐队在特定时刻的构成，在摇滚乐快速变化的世界里，这个概念必不可少。毫不夸张地说，具体化同样有用。例如，建模人与其居住的地址、软件许可与其所激活的电脑。

2.4 本章小结

在本章中介绍了Neo4j数据建模的灵活性，以及对同一领域可以有多种可能的描述方法。仅仅以铅笔和纸就可以很容易地开始数据建模，但是你应该准备好尝试把相同的数据放在不同的地方——属性可以作为节点、作为关系、或拉出来放到自身的一个节点上。

我们已经了解了一些数据建模的策略，以便适合于不同的查询模式，选择哪些数据用节点以显式描述，哪些节点作为隐式描述并可以在以后的数据查询模式中进行推断。我们也看到了随时间变化的数据，并看到了变化数据和不变数据的一个混合建模模式。

本章的例子显示了图形数据建模适合于多种领域的应用。在下一章中，你将看到如何遍历数据模型，以高效和强有力地检索获取数据。

第3章 Neo4j开发入门

本章包括以下内容：

- 图形数据建模
- 使用Neo4j Java API建立连接的节点
- 使用属性为节点和关系添加额外信息
- 区分不同节点类型的策略

第1章演示了应用Neo4j作为图形数据库对改进性能和扩展性的可能性，也讨论了对图形建模的数据如何正好适应于Neo4j数据模型，现在到了该动手实践的时间了。

在本章中，通过建模和建立一个Neo4j图形数据库的实例，介绍Neo4j核心Java API（Neo4j Core Java API）。第1章中，我们建立了社交网络的例子，但现在我们会让它稍微复杂一些，允许用户对他们看过的电影进行评级。

社交网络中的用户可能是朋友。此外，用户可以标记他们已经看过的电影，根据他们对电影的喜欢程度，用1~5颗星给这些电影评级。

在本章的第一部分，我们将模仿软件的设计过程，使用一系列图表为本例的需求建模。在第二部分，我们将演示如何使用Neo4j核心Java API创建同一个社交网络。

3.1 图形数据结构建模

在深入探索Neo4j之前，我们将这个社交网络域模型描述为一个通用的图形。如果在办公室建模，我们可能在白板上画出模型。在本章中，我们将图示说明这种白板建模的方法。

首先，社交网络中需要有几个用户，用户用方框表示，如图3-1所示。



图3-1 方框表示社交网络中的用户

我们的目标是建立电影爱好者的一个社交网络，因此，下一步很自然就是为模型添加社交元素。首先用箭头连接是朋友关系的用户。比如说用户1有两个朋友，分别是用户2和用户3。图3-2给出了朋友之间相互连接的示意图。

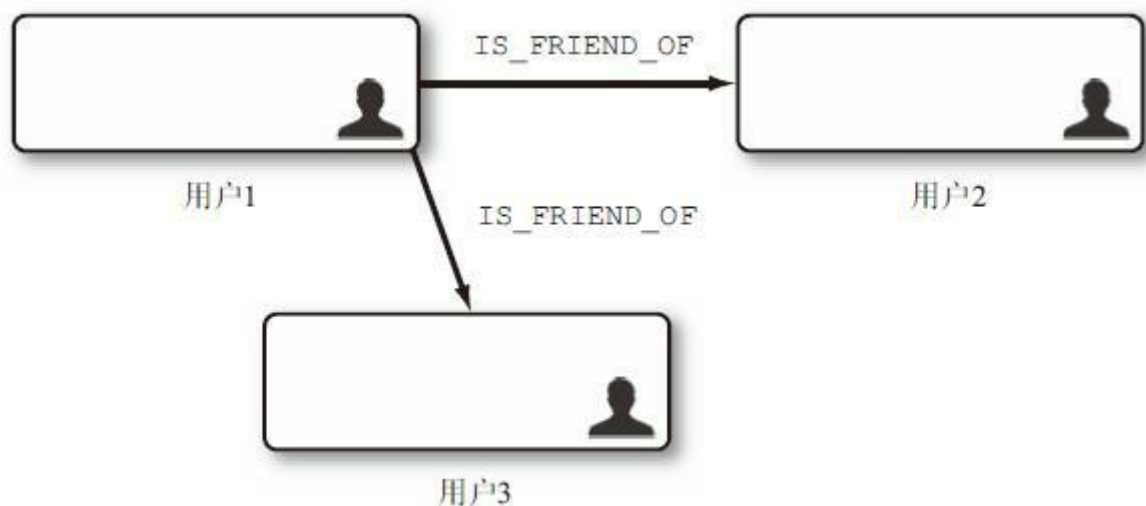


图3-2 朋友之间相互连接的一个简单社交网络示意图

注意

为了在句法上正确，朋友关系应该是双向的，但是，Neo4j图形数据库是单向的，这意味着每一个关系已经定义好了起始和终止节点。在Neo4j中，双向关系可以用两个独立的关系建模，每一个关系代表一个方向。为了简单起见，我们将关系定义为单向关系。正像在后面的章节中你会看到的那样，这并不会影响Neo4j的查询能力，因为关系可以在两个方向上跟踪，这将会大大简化模型。

现在，有了几个用户并用方框表示，他们在社交网络中相互是朋友。为了区分不同的用户，可以为每个用户设置一个name属性，如图3-3所示。

图3-3最终看上去像一个合适的社交网络。但是我们的目标是建立电影爱好者的一个社交网络，因此，下一步是添加一个新的电影节点。

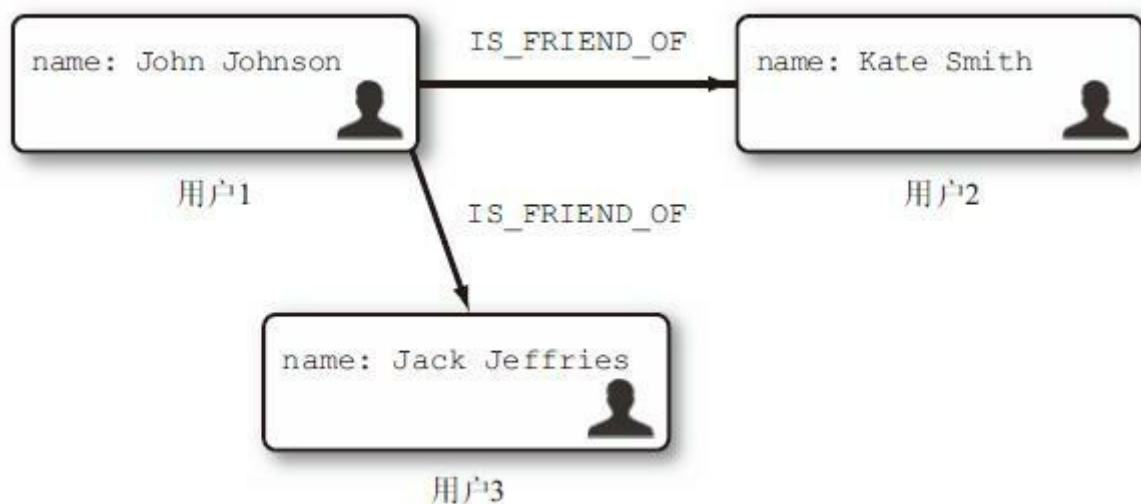


图3-3 给每个用户添加name属性的复杂社交网络示意图

可以用一个具有name属性的方框代表电影，如图3-4所示。

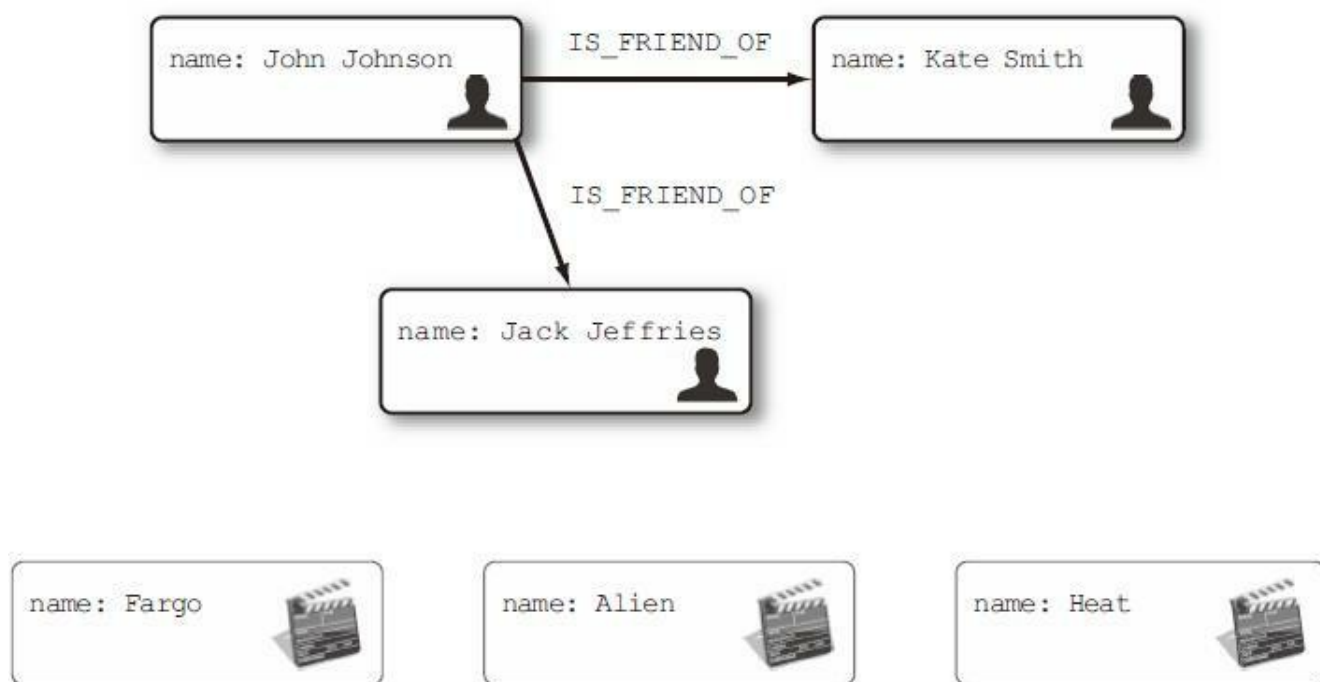


图3-4 模型中引进电影节点

这里有一个问题：因为用户和电影都用具有name属性的方框表示，所以没有任何办法区分它们。最容易区分user方框和movie方框的方法是引进一个type属性，用不同的值代表不同的对象。具有相关类型的该模型如图3-5所示。

现在已接近例子的尾声——最后一步是将电影爱好者元素添加到社交网络中。譬如说John喜欢《Fargo》并给它打5颗星，Kate看了《Heat》并给它打了3颗星，Jack喜欢《Fargo》和《Alien》，并给了《Fargo》4颗星，《Alien》5颗星。加上电影评级的模型如图3-6所示。

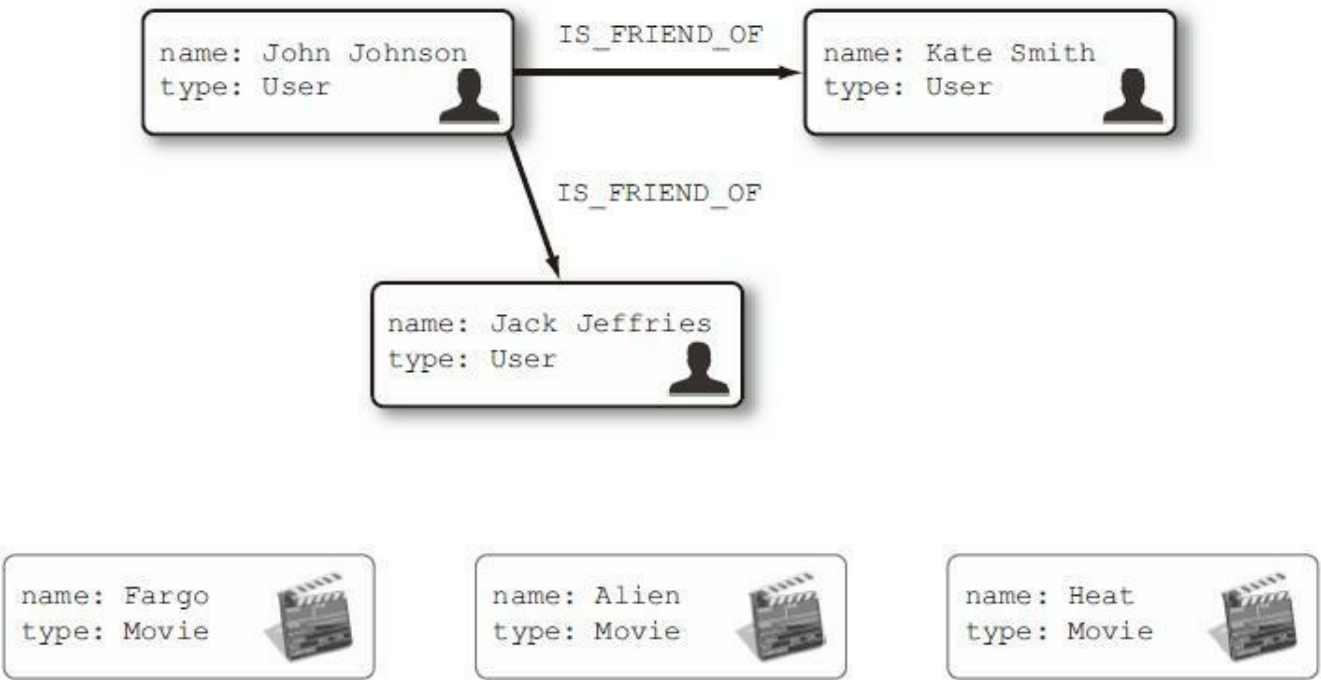


图3-5 引入type属性区分用户与电影

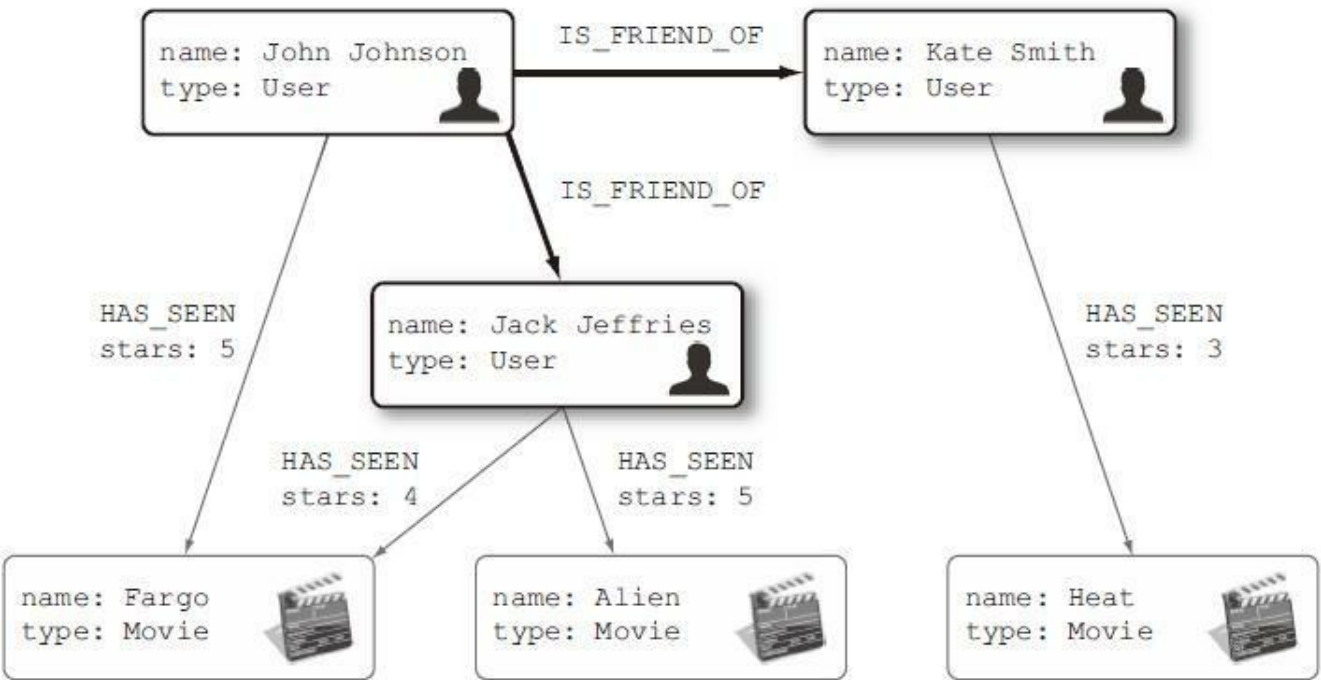


图3-6 电影爱好者社交网络的完整模型

综上，我们有了一个代表社交网络的模型，它用图表表示。

这个建模过程是在每一个软件项目开始阶段的典型做法。这些图形帮助我们理解项目的需求并使得决策模式容易和透明。建模练习的结果经常以关于模型、需求、模式决策、建议以及图表的文件形式出现。

模型准备好后，开发团队就会接过项目，基于模型和架构开始编写代码并形成软件产品。作为流程的一部分，模型应该是标准化的，并映射到数据的持久层，这通常是指关系数据库。

有趣的是，图3-6所示的社交网络表示为一个图，用户和电影作为节点，关系连接着朋友（IS_FRIEND_OF）和电影的评级（HAS_SEEN）。使用面向对象编程（ODP）技术，同样的图形结构一般会用到代码中。数据也会存到几个表格中，并使用关系代数访问。这种从图形对象模型到关系数据库或从关系数据库到图形对象的转换给项目添加了很多复杂性并增加了编写代码的工作量，影响了项目的效率并延长了项目的交付时间。这仅仅是因为不得不考虑、实施和配置一个转换机制，例如，使用对象关系映射（ORM）工具。

这里正是Neo4j的用武之地。不像在关系数据库中以额外的精力创建映射机制储存类似于图形的结构，而是用Neo4j本身具有的节点和关系存储图形对象。这可以让我们对软件应用程序建模和编写代码同时进行，并或多或少使用来自于白板的结构和术语直接编写代码。

3.2 使用Neo4j API

当使用Neo4j作为图形数据结构的底层存储时，建模和编写代码可以同时进行，这将在下面的章节中介绍。

3.2.1 创建节点

在社交网络中，我们可以从代表用户的节点实例化图形数据库开始，图3-1（在这里重新绘制为图3-7）是这一阶段的模型图，包含三个用户。

我们可以编写Java代码创建图3-7所示的图形数据。最开始的两行（程序3-1和程序3-2）建立了单个user节点，使用了Java 6（Neo4j 1.9.X）和Java 7（Neo4j 2.0.X）的习惯风格。

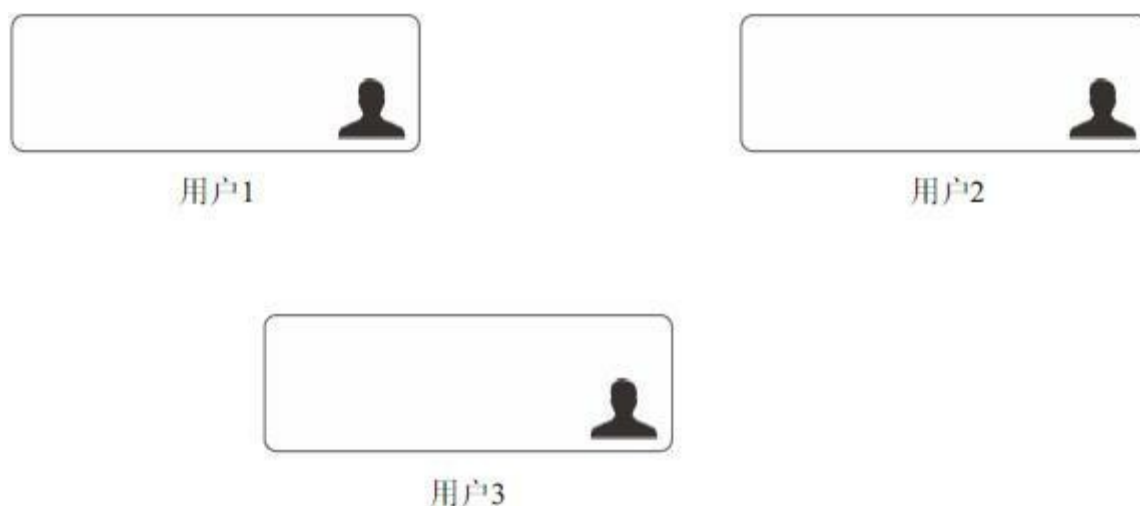
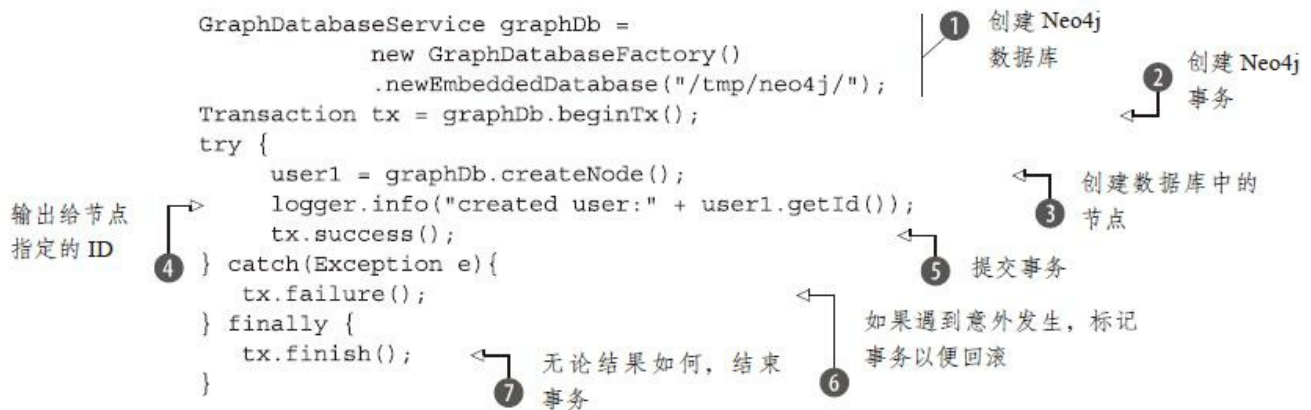


图3-7 只有一个节点实体的图形

这两种版本的最大区别是对事务的处理。创建节点的API本身是相同的，但是，由于这是我们给出的第一段程序代码，因此我们展示了两个版本的示例代码。对于以后，除非另有说明，否则本章和本书的后面章节将会使

用Java 7事务管理模式。

【程序3-1】 在Neo4j（Java 6/Neo4j 1.9.X风格）创建单个用户节点



【程序3-2】 在Neo4j（Java 7/Neo4j 2.0.X风格）创建单个用户节点



在这两种风格中，第一步是实例化Neo4j数据库（**1**），需要提供文件系统存储数据的位置作为构造函数参数。如果这是一个已经存在的Neo4j数据库，就会使用这个数据库。如果指定的位置没有数据库，Neo4j就会在那个位置创建一个空的数据库。数据库将只能建立一次并在整个应用程序周期中使用。在下面的程序代码示例中，我们假设使用的是一个已经存在的空的Neo4j数据库实例。

前面已经提到Neo4j是一个与ACID完全兼容的图形数据库，就像其他的数据库所期望的一样，支持所有的标准事务型属性。当我们使用Neo4j的内嵌式数据库时，事务的边界是由Neo4j核心Java API程序化管理的。在两个程序中的步骤**2**和步骤**5**（程序3-1中的步骤**6**和**7**）是Neo4j核心Java API

的部分事务管理（如果还不能全部理解，也不要着急，第7章将对Neo4j的事务进行深入的讨论）。由于所有的Neo4j操作将在一个事务中完成，因此你将会在代码实例中看到用Neo4j编写的几行相同的程序代码。没有这几行程序代码，程序就不能运行（在Neo4j 1.9及以下版本中，只有事务是强制性的操作，并用于更新数据库。但是从2.0及以后的版本中，读操作也是强制性的）。现在，需要注意的是每一个访问Neo4j数据库的代码段要么在try/catch/finally（Neo4j 1.9/Java 6）块中执行，要么在try（用资源尝试）块中实施（Neo4j 2.0/Java 7），在执行代码中就是以这样的模式管理事务的边界。

在这两种方法中，例子的核心部分是try代码块中的代码（❸、❹）。出于一致性的原因，我们将把事务管理代码放入本章的所有例子中，但是第7章将深入讨论事务处理和API。

GraphDatabaseService.createNode（）方法创建了Neo4j数据库中的节点（❸）。这个方法返回了创建的节点本身，并可以检查其属性或对其做进一步的处理。在前面的程序中，正在使用返回的节点实例输出其由Neo4j数据库产生的内部编号（❹）。

现在我们已经看到了创建单个user节点的过程，但是我们是在创建一个社交网络，所以需要一些用户。程序3-3创建了另外3个用户。

【程序3-3】 在一个事务中创建多个节点

```

try (Transaction tx = graphDb.beginTx()) {
    Node user1 = graphDb.createNode();
    logger.info("created user:" + user1.getId());
    Node user2 = graphDb.createNode();
    logger.info("created user:" + user2.getId());
    Node user3 = graphDb.createNode();
    logger.info("created user:" + user3.getId());
    tx.success();
}

```

经过程序3-3的操作后，使用Neo4j图形数据库存储的图形将在对象模型和硬盘中与图3-7完全一样。

下一步是用关系将创建的用户连接起来。

3.2.2 创建关系

在开始实施将用户作为朋友连接起来之前，提醒自己注意图3-2所示的模型图，在这里重新绘制为图3-8。

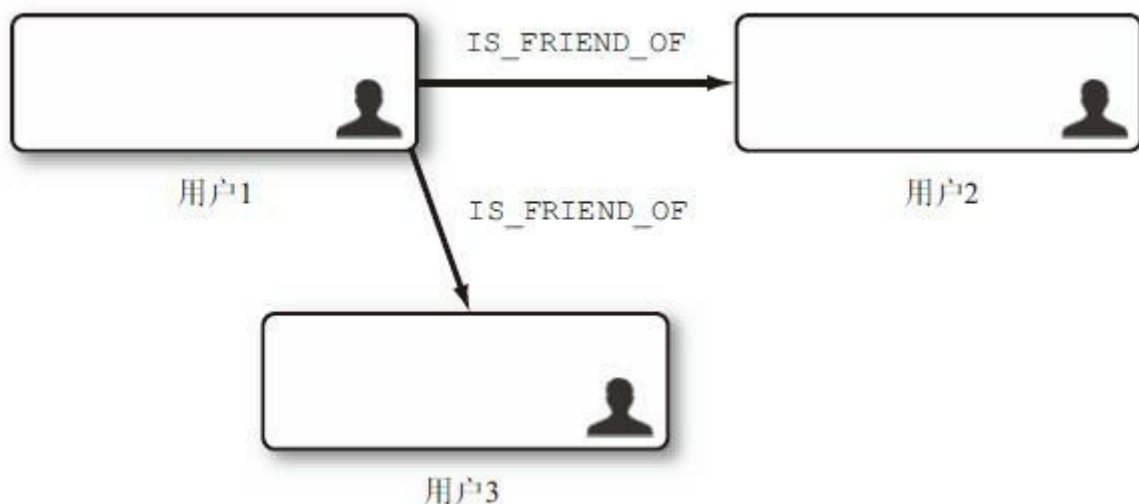


图3-8 用朋友关系连接用户的一个简单社交网络图

关系是图形数据库除节点以外的另外主要实体。每一个关系都有一个名字，名字代表标签或关系的类型。在Neo4j核心Java API中，关系是用

RelationshipType接口定义的，它定义了单个方法返回关系的名字，下面的程序给出了示例。

```
public interface RelationshipType{
    public String name();
}
```

在社交网络的例子中，我们将在用户之间建立朋友关系。要实现这一目标，需要使用RelationshipType接口：

```
public class IsFriendOf implements RelationshipType{
    public String name(){
        return "IS_FRIEND_OF";
    }
}
```



The diagram shows two annotations with arrows pointing to the code. One arrow points from the text "IsFriendOf 类实现 RelationshipType 接口" to the line "public class IsFriendOf implements RelationshipType{". Another arrow points from the text "name() 方法返回你将使用的关系标签" to the line "return \"IS_FRIEND_OF\";" inside the name() method.

RelationshipType接口有一个有趣的副作用，它定义的唯一方法（String name（））的签名看上去与Java的所有枚举方法的签名完全一样。这就给了我们一个选择，当使用Java枚举实现Neo4j的关系类型时，可以以一种更强类型化、更富有表现力和更简洁的方式。下面的程序示例用Java枚举实现同样的IS_FRIEND_OF关系类型。

```
public enum MyRelationshipTypes implements RelationshipType{
    IS_FRIEND_OF;
}
```

按照标准的枚举约定，枚举的名字代表关系的名字。

DynamicRelationshipType类

如果你知道在编译时需要什么样的关系类型，定义一个实际的类，或者按推荐的方法去做，最好使用枚举习语代表关系。然而，也可能有这样的情况，只有在运行时才能知道关系类型，并且仍然需要一种方法来表示它。在

这种情况下，org.neo4j.graphdb.DynamicRelationshipType类的使用将变得非常方便。

下面的程序展示了用DynamicRelationshipType类如何创建IS_FRIEND_OF关系。

```
String runtimeVal = "IS_FRIEND_OF";  
RelationshipType rel = DynamicRelationshipType.withName(runtimeVal);
```

现在已经定义了将使用的关系，就可以用这些关系建立user节点之间的联系。建立两个节点之间的关系，需要调用节点之一的Node.createRelationshipTo (...) 方法并需要把连接和连接类型传递给目标节点。



就像前面提到的，Neo4j图形数据库是直接关系定义起始节点和终止节点的。当调用createRelationshipTo方法时，调用方法所在的节点将成为起始节点，作为参数传递的节点将成为终止节点。这不会影响查询的灵活性和性能，这一点你将会在下一章中见到。

与在所有的图形数据库中写操作语句一样，关系的创建需要运行事务。程序3-4显示了怎样在以前已经创建的user节点之间创建IS_FRIEND_OF关系。

【程序3-4】 使用Neo4j核心Java API在节点之间建立关系


```
try (Transaction tx = graphDb.beginTx()) {
    user1.createRelationshipTo(user2,
    ➤ MyRelationshipTypes.IS_FRIEND_OF);
    user1.createRelationshipTo(user3,
    ➤ MyRelationshipTypes.IS_FRIEND_OF);
    tx.success();
}
```

为建立关系创建事务

从 user1 节点至 user2 节点
建立 IS_FRIEND_OF 关系

用同样的技术将 user1 与
user3 作为朋友连接起来

下一步，我们将利用name属性识别图形中的用户。

3.2.3 为节点添加属性

Neo4j是一个直接的属性图形。**属性图形**的意思是每一个图形实体能够具有多个属性值。属性以键值对存储并以属性名为键引用属性值。为了说明属性的概念，下面我们将演示如何给user节点添加name属性。

给节点添加属性，需要调用目标节点上的方法Node.setProperty（name，value）。下面的程序3-5显示了如何使用核心Java API给所有的用户添加name属性。

【程序3-5】 给用户节点添加name属性

```
try (Transaction tx = graphDb.beginTx()) {
    user1.setProperty("name", "John Johnson");
    user2.setProperty("name", "Kate Smith");
    user3.setProperty("name", "Jack Jeffries");
    tx.success();
}
```

给 user1 设置 name 属性，后面的
user2 和 user3 也要设置 name 属性

现在已经有有了一个更加丰富的模型，这里的用户具有了名字并且可以通过名字区分不同的用户。



注意

在Neo4j中，必须给添加的每一个属性设置属性值。换句话说，Neo4j中不允许null属性值。

前面例子中的属性值是字符串型的。但是Neo4j支持许多不同的数据类型而且均可以应用于节点的属性值。表3-1列出了可以使用的属性类型及其与Java相对应的数据类型。

表3-1 Neo4j中的属性类型

描 述	Java 类型
Unicode 值序列	java.lang.String
单一 Unicode 值	char
(续)	
描 述	Java 类型
真 / 假	boolean
8 位整型	byte
16 位整型	short
32 位整型	int
64 位整型	long
32 位浮点型	double
64 位浮点型	float
以上所有类型的数组	[]

具有属性的节点是无机制、半结构化的元素。每一个节点可以具有任意数量的属性。程序3-6演示了如何给不同的节点添加不同的属性。



注意

关系也可以添加属性，本章后面将涉及给关系添加属性。

【程序3-6】 给节点添加不同类型的属性

```

try (Transaction tx = graphDb.beginTx()) {
    user1.setProperty("year_of_birth", 1982);
    user2.setProperty("locked", true);
    user3.setProperty("cars_owned", new String[] {"BMW", "Audi"});
    tx.success();
}

```

设置节点的属性为（字符）数组

设置节点属性为整型数值

设置节点属性为布尔型数值

我们已经通过存储节点的额外信息使得图形更加丰富。图3-9演示了通过对程序3-5的功能做了增强的社交网络（但并不是程序3-6的例子）。

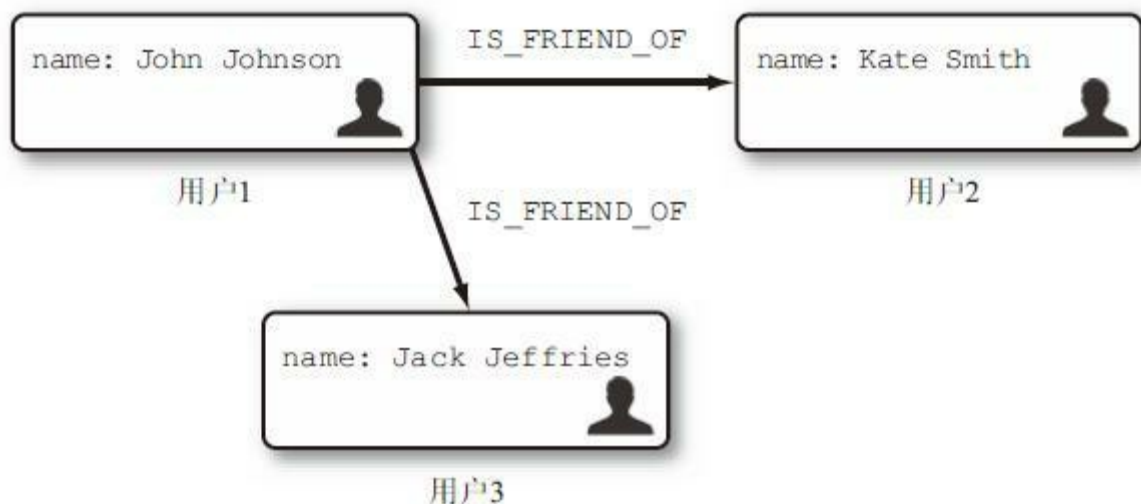


图3-9 一个具有多种属性的社交网络图形

注意

就像前面提到的，Neo4j不允许null属性。如果需要从节点上删除属性，必须明确地使用Neo4j核心Java API: `Node.removeProperty (String propertyName)`；删除。

下面需要添加几个用户看过和评级过的电影节点。

3.2.4 节点类型策略

本例的下一步是对图形数据库添加电影，用于用户进行评级。我们将按

照添加用户的同样方式添加3个movie节点。下面的程序3-7演示了如何使用Neo4j核心Java API添加代表电影的节点。

【程序3-7】 使用Neo4j核心Java API创建movie节点

```
try (Transaction tx = graphDb.beginTx()) {  
    Node movie1 = this.graphDb.createNode();  
    movie1.setProperty("name", "Fargo");  
    Node movie2 = this.graphDb.createNode();  
    movie2.setProperty("name", "Alien");  
    Node movie3 = this.graphDb.createNode();  
    movie3.setProperty("name", "Heat");  
    tx.success();  
}
```

创建 movie 节点

给新建立的 movie 节点
添加 name 属性

图3-10表示了在这一阶段的图形。

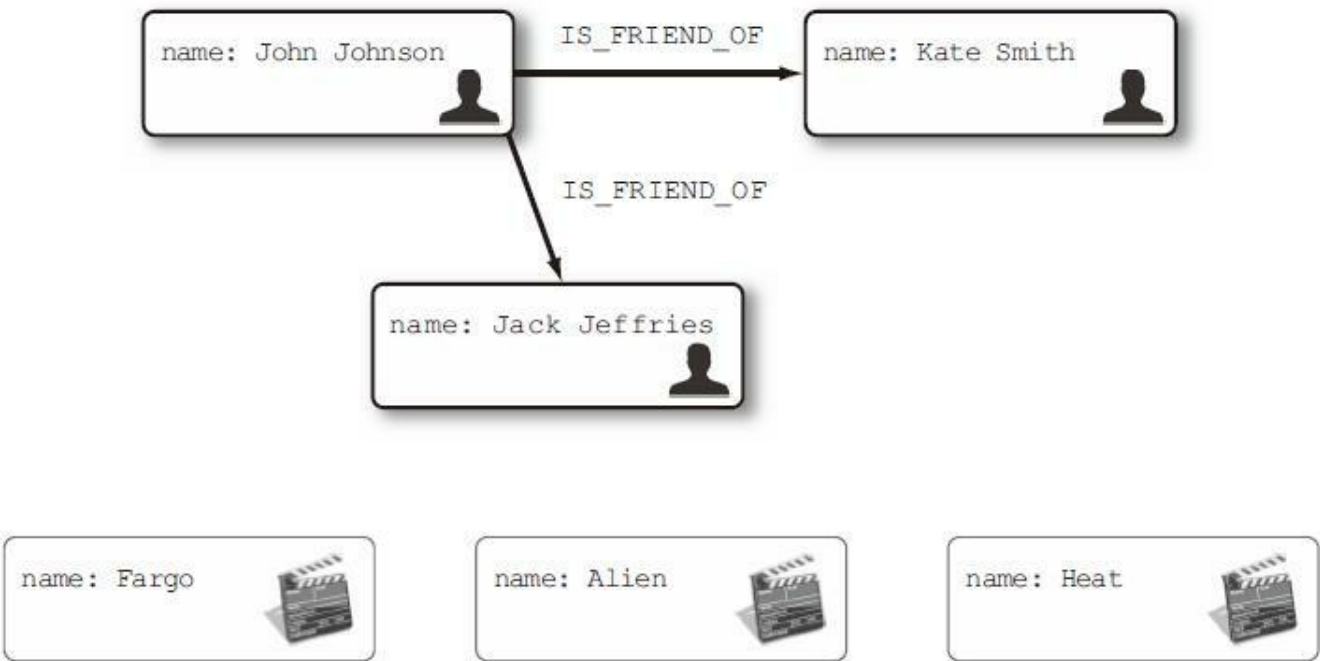


图3-10 具有代表用户和电影节点的图形数据库

正如在图3-10中能够看到的，user和movie节点都用方框表示。对于Neo4j数据库，这是非常重要的一点。从Neo4j数据库的角度来讲，user节点和movie节点之间没有什么不同。在Neo4j中，节点在默认情况下是没有类型的。确定领域模型中每一个节点应如何描述是开发者应考虑的问题。

在Neo4j 1.9及以下版本中，能够使用的最简单的策略是给每一个节点添加一个type属性，用属性值确定节点的类型。这是在其他NoSQL数据库技术（例如文件数据库）中被广泛使用的一个策略，因为这个策略很容易理解和实施。从Neo4j 2.0版本往后，建议的策略是利用称为“标签”的新特性，这将在3.3节中详细介绍。现在，我们将从兼容Neo4j 1.9版本的非标签策略开始。

使用type属性策略包括给每一个节点添加一个type属性，就像以前给所有的节点添加name属性一样。

【程序3-8】 添加type属性以确定节点类型

```
try (Transaction tx = graphDb.beginTx()) {  
    user1.setProperty("type", "User");  
    user2.setProperty("type", "User");  
    user3.setProperty("type", "User");  
  
    movie1.setProperty("type", "Movie");  
    movie2.setProperty("type", "Movie");  
    movie3.setProperty("type", "Movie");  
  
    tx.success();  
}
```

	为所有的用户节点设置 type 属性的值为 "User"
	为所有的电影节点设置 type 属性的值为 "Movie"

像以前所有的例子一样，所有图形数据库的操作必须包装在try事务处理代码块中。

有了节点的类型，现在可以简单地检查节点的属性值以确定节点的类型。

```
if ("Movie".equals(node.getProperty("type"))) {  
    //(这是一部电影)  
}  
if ("User".equals(node.getProperty("type"))) {  
    //(这是一个用户)  
}
```



注意

使用该方法，可以通过属性并使用索引找到给定类型的所有节点（例如所有的电影），这将在第5章中详细介绍。

图3-11展示了对每一个节点设置了type属性的图形数据库。

本例子到这里已经基本完成了，剩下的工作就是创建从用户到用户喜欢的电影之间的关系了。

3.2.5 为关系添加属性

要使得这个社交网络的图形数据库完整，只需要给用户和用户给电影的评级之间添加关系（称为HAS_SEEN）即可。HAS_SEEN关系将从user节点开始直接到movie节点，与以前用过的IS_FRIEND_OF关系是一样的。

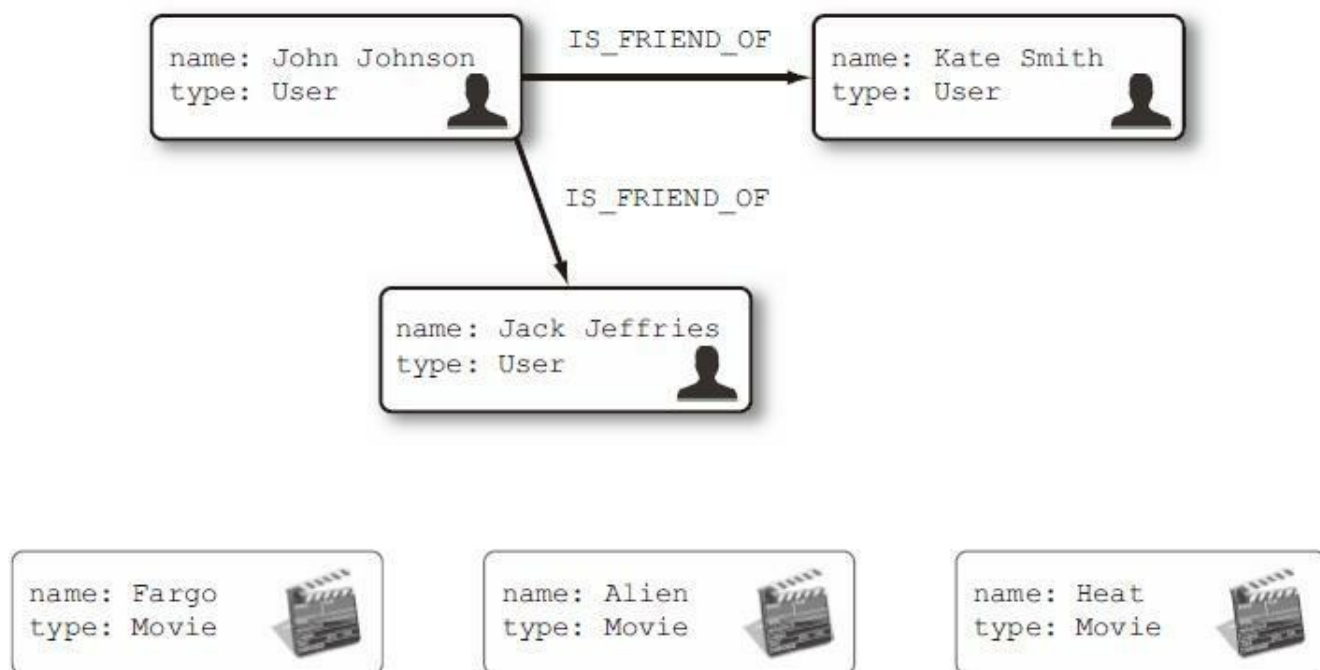


图3-11 使用类型属性策略的节点

在HAS_SEEN和IS_FRIEND_OF关系之间有一个重要的区别：

HAS_SEEN关系包含用户喜欢电影的程度的额外信息（以用户给电影评级的星的数量表示）。因此星的数量不是user节点或movie节点的属性（因为一个用户可以为多部电影评级）。我们已经知道如何给节点添加属性，但是给关系添加属性，到目前为止还是一个新的概念。

前面提到Neo4j是一个属性图形。在Neo4j中，属性图形适应于所有的图形实体——关系以及节点。在Neo4j中给关系添加属性就像给节点添加属性一样简单。我们将引导你通过使用Neo4j核心Java API为关系添加属性。

在继续往下进行之前，需要给这个枚举类关系添加一个关系类型。以下程序给出了具有新的HAS_SEEN关系的更新的MyRelationshipTypes枚举。

```
public enum MyRelationshipTypes implements RelationshipType {  
    IS_FRIEND_OF,  
    HAS_SEEN;  
}
```

下一步是创建节点之间的命名关系，这可以用本章前面在用户之间添加朋友关系的方法实现。创建关系的API调用将会返回Neo4j的Relationship对象，并且可以通过调用Relationship.set-Property（String name，Object value）方法使用它。

程序3-9给出了创建HAS_SEEN关系并添加stars属性的代码。

【程序3-9】 创建具有属性的关系

```
try (Transaction tx = graphDb.beginTx()) {  
    Relationship rel1 =  
        user1.createRelationshipTo(movie1,  
            MyRelationshipTypes.HAS_SEEN);  
}
```

创建 HAS_SEEN 关系；
调用 API 返回关系对象


```

    rel1.setProperty("stars", 5);
    Relationship rel2 =
        user2.createRelationshipTo(movie3,
    ➤ MyRelationshipTypes.HAS_SEEN);
    rel2.setProperty("stars", 3);
    Relationship rel3 =
        user3.createRelationshipTo(movie1,
    ➤ MyRelationshipTypes.HAS_SEEN);
    rel3.setProperty("stars", 4);
    Relationship rel4 =
        user3.createRelationshipTo(movie2,
    ➤ MyRelationshipTypes.HAS_SEEN);
    rel4.setProperty("stars", 5);
    tx.success();
}

```

← 给上一行创建的关系
添加 stars 属性

一旦为图形添加上了HAS_SEEN关系，最终就用Neo4j核心Java API完成了这个社交网络。图3-12显示了具有所有节点、关系和属性的最终图形结构。

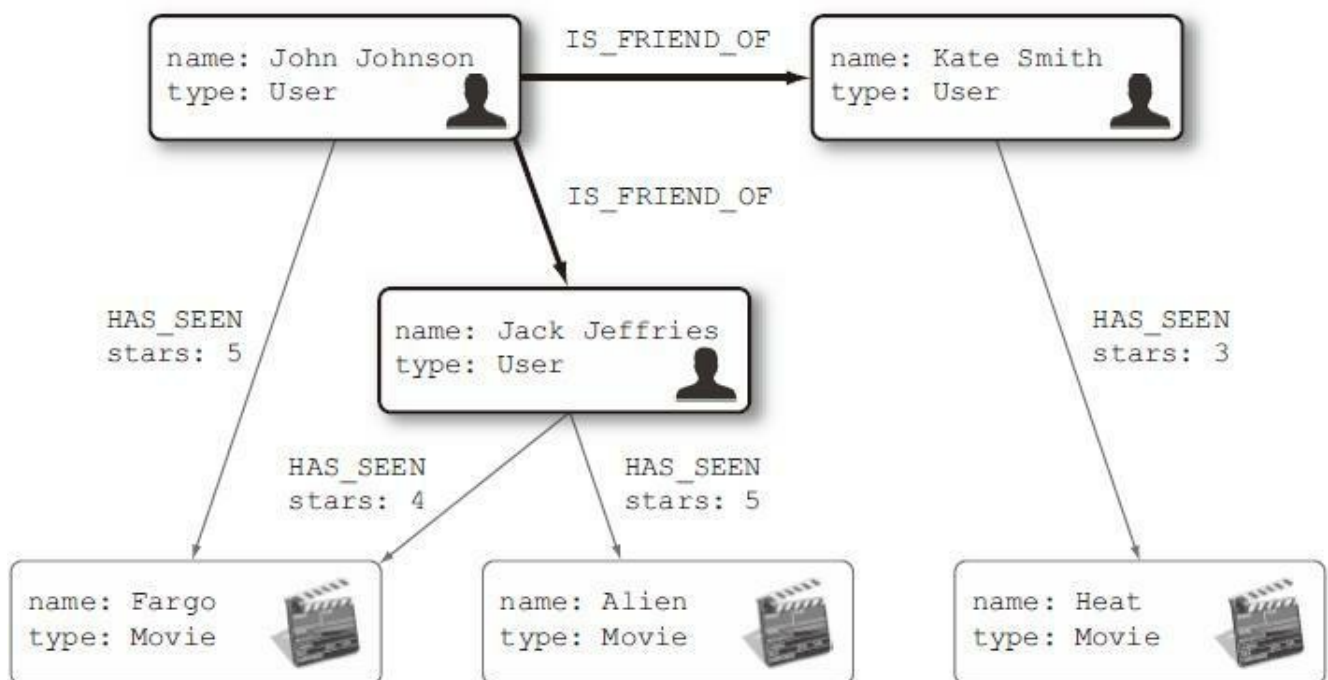


图3-12 电影爱好者社交网络的完整模型

图3-12与图3-6是一样的，我们已经设法在Neo4j图形数据库中用Neo4j核心Java API创建了同一个图形。

3.3 节点标签

在前面的几节中，我们提到了Neo4j数据库中的节点没有类型——它们仅仅是一些“方框”。为了区分movie节点和user节点，我们为每一个节点添加了属性类型及相应的数值。

在2.0版本中，Neo4j引进了（内置）节点标签的概念，帮助我们分组相似的节点。节点标签与先前讨论过的关系类型名字非常相似。

每一个节点可以有一个或多个文字描述，它称为节点标签。具有同样标签的节点用一种专用的方式存储，因此它们能分为一组并可以一起使用。Neo4j支持下列来自核心Java API且与标签相关的操作：通过标签加载所有的节点和通过标签及属性查找标签。模式索引也是通过标签定义的。



注意

标签也可以在Cypher查询中用于节点查找操作。标签的这种用法将在第6章中讨论。

正像在关系类型中，节点标签是用单个方法String name（）在一个简单的界面中定义的。

```
public interface Label {  
    java.lang.String name();  
}
```

要创建一个标签，可以简单地实现这种接口，或者像关系类型一样，使用Java枚举习语，满足Label接口的约束。

```
public enum MyLabels implements Label{
    MOVIES, USERS
}
```

程序3-10演示了如何使用Neo4j Java API创建标签。

【程序3-10】 给节点添加标签

```
try (Transaction tx = graphDb.beginTx()) {
    movie1.addLabel(MyLabels.MOVIE);
    movie2.addLabel(MyLabels.MOVIE);
    movie3.addLabel(MyLabels.MOVIE);
    tx.success();
}

ResourceIterable<Node> movies =
    GlobalGraphOperations.at(graphDb)
        .getAllNodesWithLabel(MyLabels.MOVIE);
```

① 给选定的节点添加 MOVIE 标签

② 查找具有 MOVIE 标签的所有节点

要添加标签，应该对选定的节点使用addLabel（）方法（①）。为了查找所有具有给定标签的节点，可以在GlobalGraphOperations类上使用静态的getAllNodesWithLabel（Label label）方法（②）。

图3-13是添加了标签的图形。

注意

在本例中，为了确定哪个节点代表电影，用节点标签取代了类型属性，因此，可以删除二者中的任何一个。只要可能，建议使用标签，因为这是Neo4j数据库的一个内置功能，这个功能给予了查询和基于模式索引的极大灵活性。

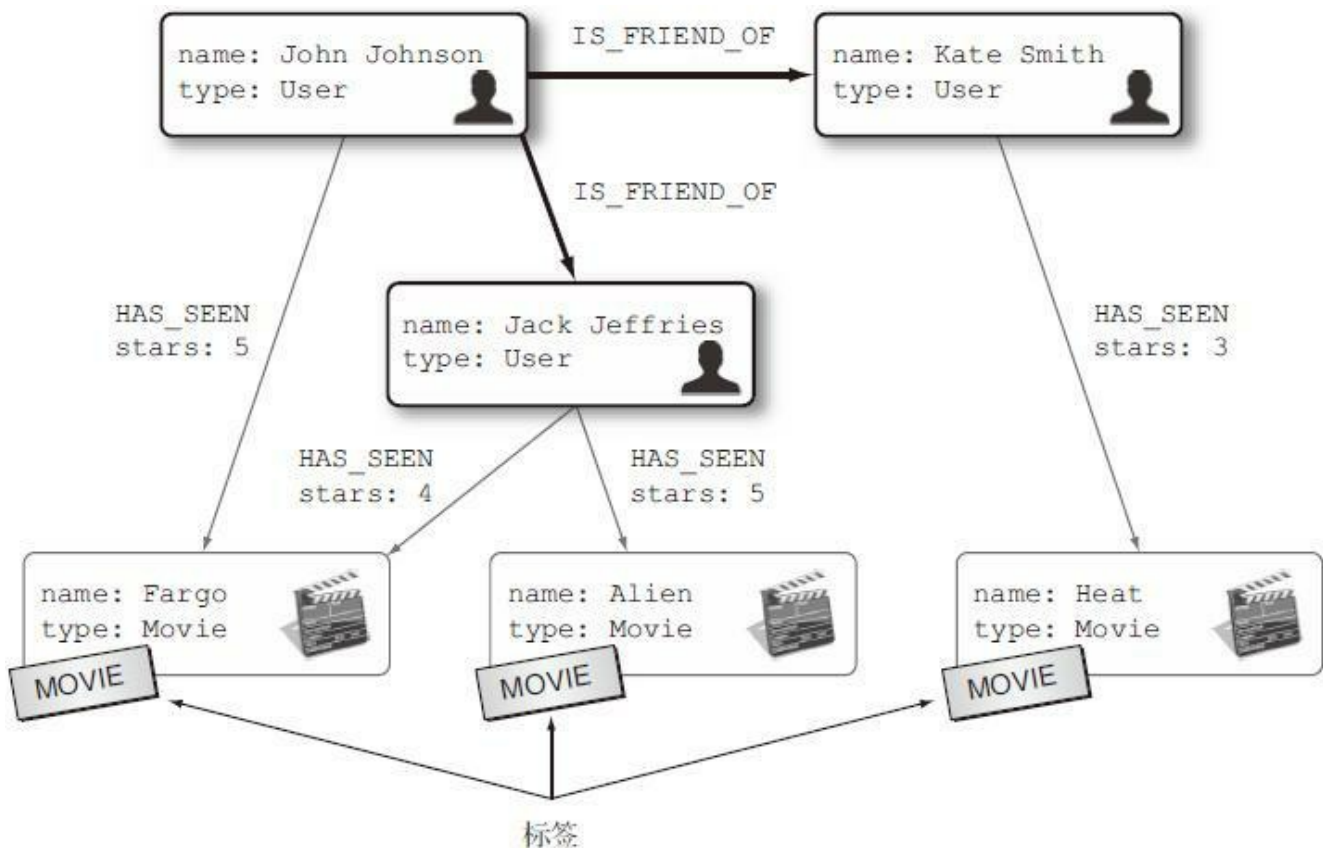


图3-13 用MOVIE标签分组所有电影节点

除查找具有给定标签的所有节点以外，Neo4j核心Java API具有查找具有给定标签和属性的所有节点的操作，如程序3-11所示。

【程序3-11】 查找给定标签和属性的节点

```
ResourceIterable<Node> movies =
    GlobalGraphOperations.at(graphDb)
    .findNodesByLabelAndProperty(MyLabels.MOVIE, "name", "Fargo");
```

查找所有具有 MOVIE 标签并具有名字“Fargo”的节点

在默认情况下，Neo4j引擎通过标签和属性查找节点是以蛮力执行的（通过循环查找所有具有给定标签的节点，并比较需要的属性名和值）。但是，如果对标签和属性定义了模式索引，Neo4j引擎将会使用更快的索引查找。使用模式索引将在第5章中讨论。



正如你在程序3-10中所看到的，寻找所有具有给定标签的节点是对图形的全局操作（正像设计的那样，可以获取图形的大部分），这个操作可以通过GlobalGraphOperations类实现。另一方面，通过标签和属性找到的所有节点（程序3-11），用GraphDatabaseService接口交互，因为只期望返回图形中的小部分节点。

标签是Neo4j的一个很好的补充，不仅仅是一个分类的策略。因为节点可以有多个标签，所以可以创建标签把经常一起使用的节点分组（即使不同的类型也可以），从而不必使用属性。例如，RED_THINGS（对所有红色的节点）、FLYING（对所有代表能飞的节点）等。

3.4 本章小结

在3.1节，我们演示了如何建模社交网络中电影爱好者需求的一个例子。可以用一系列图表作为需求的模型，或者说是一个虚拟的白板，就像在软件开发过程中使用的方法一样。

在3.2节，我们逐步演示了如何使用Neo4j核心Java API对同一个图形模型建立Neo4j图形数据库。

有趣的是两组步骤看上去几乎一样，就像是用自然的方式画图来描述图形，以便人类的理解，Neo4j核心Java API可以使我们遵循同样的模式用代码描述图形。使用图形的自然语言，使用Neo4j核心Java API，不需要其他的任何工具或框架，即不需要映射或翻译工具，就可以实现设计。

图形确实具有非常强大的结构，我们在日常生活中用它来可视化模型并解决与软件工程相关的问题。Neo4j也可以让我们使用清晰和简单的图形作为编程的模型，把软件设计和需求分析简单地转换成模型。

在本章中，我们揭开了Neo4j核心Java API的面纱，展示了如何在图形中建立节点、用关系连接节点和用属性描述节点和关系。下一章将讨论如何使用图形的强大功能进行数据查询。

第4章 强大的图形遍历功能

本章包括以下内容：

- 使用核心Java API查询Neo4j数据库
- 应用Neo4j的遍历API遍历节点和关系
- 扩展和定制Neo4j的内置遍历操作

第3章介绍了Neo4j核心Java API和创建了代表用户和他们喜欢的电影的社交网络。本章将讨论图形遍历——查询图形数据的一种强大功能。[图形遍历](#)是以一种特殊的方式在图形中按照节点之间的关系访问节点的过程。

我们将通过解决简单的遍历问题的示例来演示遍历选项，并接着介绍几个更为复杂的例子。在本章中，你将会更加熟悉Neo4j核心Java API以及具有更先进的Neo4j遍历框架的Java API（或简称为Neo4j遍历API）。

让我们从最简单的遍历开始，从起始节点开始按照关系使用Neo4j核心Java API查找直接的邻居。

4.1 使用Neo4j核心Java API进行遍历

我们将讨论的第一种遍历方法是使用Neo4j核心Java API提供的标准方法。这里将使用在第3章中创建的图形数据库做遍历。

第一次遍历可以描述为“从代表用户John Johnson的节点开始，查找这个用户看过的所有电影”。图4-1展示了我们感兴趣的这个图形的一部分。

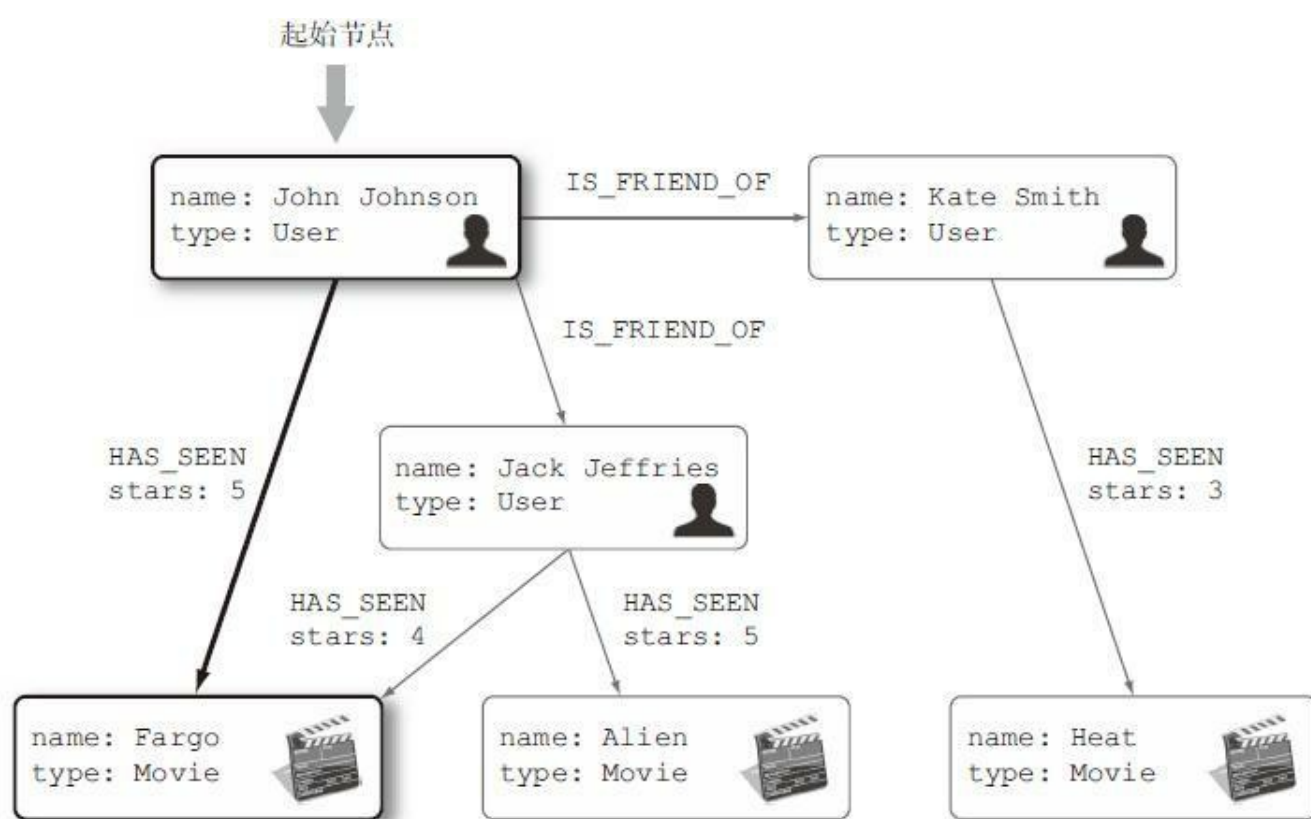


图4-1 选择的用户和他看过的电影（用加粗的边框标记）

要导航到代表John已经看过的电影的节点，我们将从代表John Johnson的节点开始，然后按照所有HAS_SEEN关系找到目标电影。我们要访问的节点和关系作为遍历的部分用加粗的边框做出了标记。

第一个任务是定位遍历的起始节点：用户John Johnson。

4.1.1 寻找起始节点

在处理遍历之前，我们需要寻找从哪个节点开始遍历，它通常称为**起始节点**。Neo4j核心Java API具有单个查找方法，可用于从图形数据库中加载节点——`GraphDatabaseService.getNodeById(Long id)`。这种方法通过内部标识符加载节点，在Neo4j中用`java.util.Long`数字表示。我们假设已知用户John Johnson节点的ID且ID以静态变量`JOHN_JOHNSON_NODE_ID`存储的。

```
public static Long JOHN_JOHNSON_NODE_ID = 3L;  
private GraphDatabaseService graphdb;  
public void getJohnNode() {  
    return graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);  
}
```

通过自己的标识符从 Neo4j
数据库中加载节点对象



注意

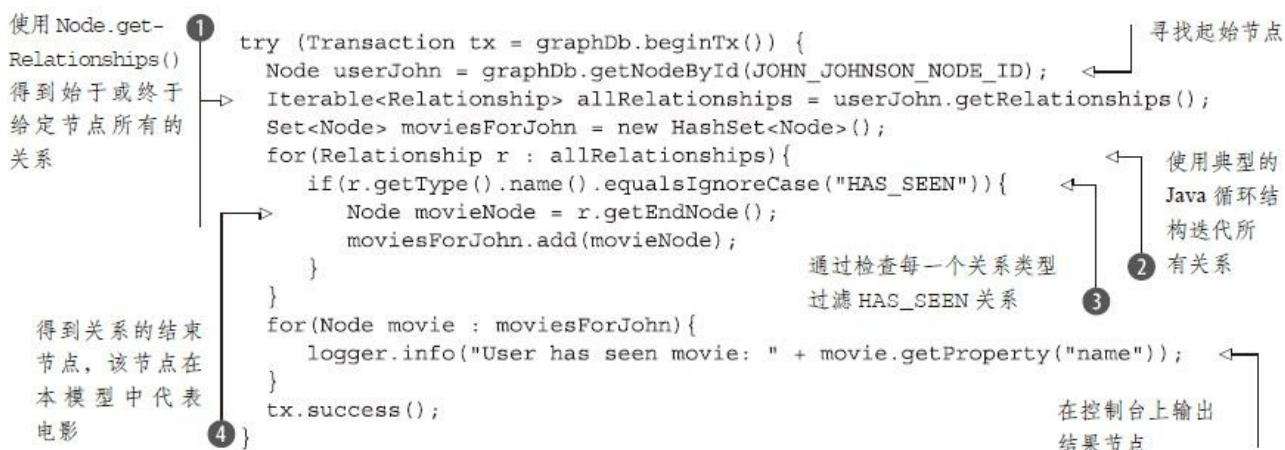
理想情况下，我们将基于一些属性值而不是节点ID寻找起始节点，例如，通过姓名或者电子邮件地址找到user节点。Neo4j的临时节点查找是使用索引执行的，这将在第5章中详细讨论。在本章中，节点的查找将以节点的ID操作。

既然已经有了起始节点，就可以通过遍历图形找到感兴趣的movie节点。

4.1.2 遍历直接关系

为了找到John已经看过的所有电影，首先要加载John节点，然后从这个节点开始循着所有的HAS_SEEN关系查找，如程序4-1所示。

【程序4-1】 从起始节点开始迭代所有的关系过滤电影



注意

回忆一下，第3章中讲的所有Neo4j操作（包括从Neo4j 2.0及以上版本的读操作）都需要包装到一个事务中。程序4-1包含了这个最初的样板包装代码，但是，我们将在以后的程序中将其删除，以减少混乱，并将重点放在讨论API核心问题。然而，不要忘记程序没有它是不能运行的。

在程序4-1中，使用Node.getRelationships（）方法以获得始于或终于给定节点的所有关系（**1**）。这个方法返回了一个java.lang.Iterable对象，该对象包含org.neo4j.graphdb.Relationship实例，它可以用典型的Java方法通过for循环迭代（**2**）。userJohn.getRelationships（）方法将返回起始和终止于起始节点的所有关系，包括并不感兴趣的IS_FRIEND_OF关系（参见图4-1）。要识别仅仅是感兴趣的HAS_SEEN类型的关系，需要在循环内检查每一个关系的名字（**3**）。

要得到与关系相关的节点，org.neo4j.graphdb.Relationship接口定义了两个方法：getStartNode（），返回关系的起始节点；getEndNode（），返回关系的终止节点。前面提到Neo4j融入了一个直接属性图形模型用以存储基本

图形。Neo4j中的每一个关系都是直接的关系（从起始节点到终止节点）。例如，HAS_SEEN关系总是从user节点到movie节点。因此，起始节点总是代表用户，而终止节点总是代表电影。

在程序4-1中，这正是如何找到John已经看过的电影的：而电影总是HAS_SEEN关系的终止节点，因此，通过调用Relationship.getEndNode（）方法得到一个对它的引用（4）。



注意

尽管所有的关系都是直接关系，但是实际的遍历可以任意选择两个方向之一——沿着关系往上或往下。

当程序4-1运行时，程序运行结果如下：

```
User has seen movie: Fargo
```

基于图4-1中的图形，程序运行的结果是正确的。用Neo4j核心Java API对图形数据库做了一个查询操作。



注意

Node.getRelationships（）方法不能保证遍历关系的顺序。在本例子中，电影的顺序不重要，但是如果需要排序的结果，就必须遍历完了再做一次排序。

在前面的例子中，从代表用户John Johnson的节点开始，通过逐一检查与起始节点相连的关系的名字过滤感兴趣的关系。用名字过滤关系是图形遍历的一个非常常用的方法，Neo4j提供了一个方便使用的专用API，因此，程

序代码中不必重复书写if语句检查关系的类型。程序4-2给出了使用Neo4j内置功能改进的程序代码。

【程序4-2】 使用Neo4j核心Java API的过滤功能过滤电影

使用 Node.getRelationships(RelationshipType) 方法获得具有给定类型（起始或终止于给定节点）的所有关系

```
Node userJohn = graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);
Iterable<Relationship> allHasSeenRelationships =
    userJohn.getRelationships(DynamicRelationshipType.withName("HAS_SEEN"));
Set<Node> moviesForJohn = new HashSet<Node>();
for(Relationship r : allHasSeenRelationships){
    Node movieNode = r.getEndNode();
    moviesForUser.add(movieNode);
}
for(Node movie : moviesForJohn){
    logger.info("User has seen movie: " + movie.getProperty("name"));
}
```

正像所能看到的一样，代码中并没有以前用过的通过需求类型过滤关系的if语句，这使代码更可读和简洁，并给出了与以前一样的结果。

```
User has seen movie: Fargo
```

如果你需要非常具体的结果，Neo4j核心Java API可以过滤除关系类型外的关系说明，加载从起始user节点向外的HAS_SEEN关系，可以使用下面的程序。

```
Iterable<Relationship> allOutgoingHasSeenRelationships =
    userJohn.getRelationships(Direction.OUTGOING,
        DynamicRelationshipType.withName("HAS_SEEN"));
```

如上所示，这段代码是用Neo4j核心Java API更高级的过滤功能改进的程序代码。但是这是一个非常简单的遍历，仅仅查看选定节点的直接相邻节点。在下一节中，你将遍历越过节点的直接邻居以确定John的朋友喜欢什么电影。

4.1.3 遍历深度为2的关系

让我们把这个例子推广到下一个水平并实现一个简单的推荐引擎，用以查找John的朋友喜欢而John还没有看过的电影。

图4-2中，John有两个朋友，Jack和Kate。Jack看过Fargo和Alien，Kate看过Heat。但是John早已经看过Fargo，因此我们希望从遍历获得两部电影：Alien和Heat。图4-2显示了遍历过程中必须访问的节点和关系。

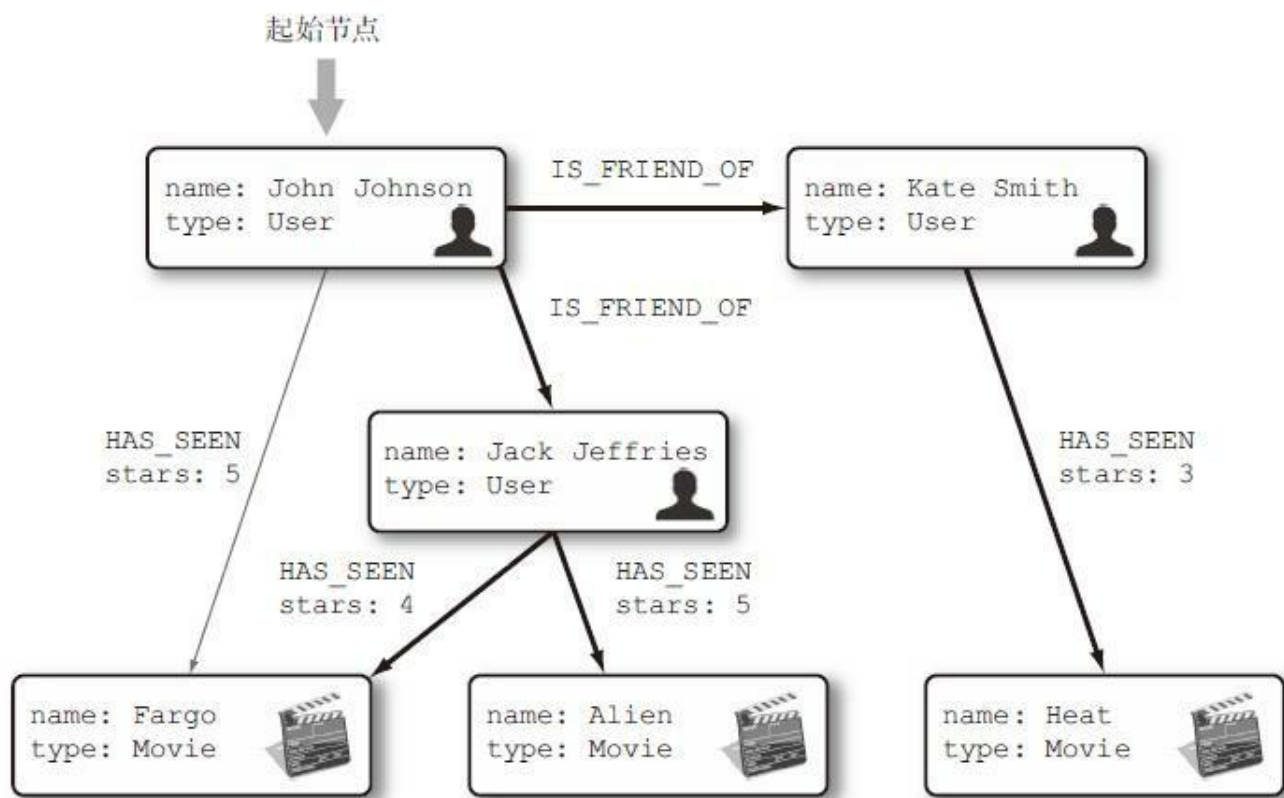
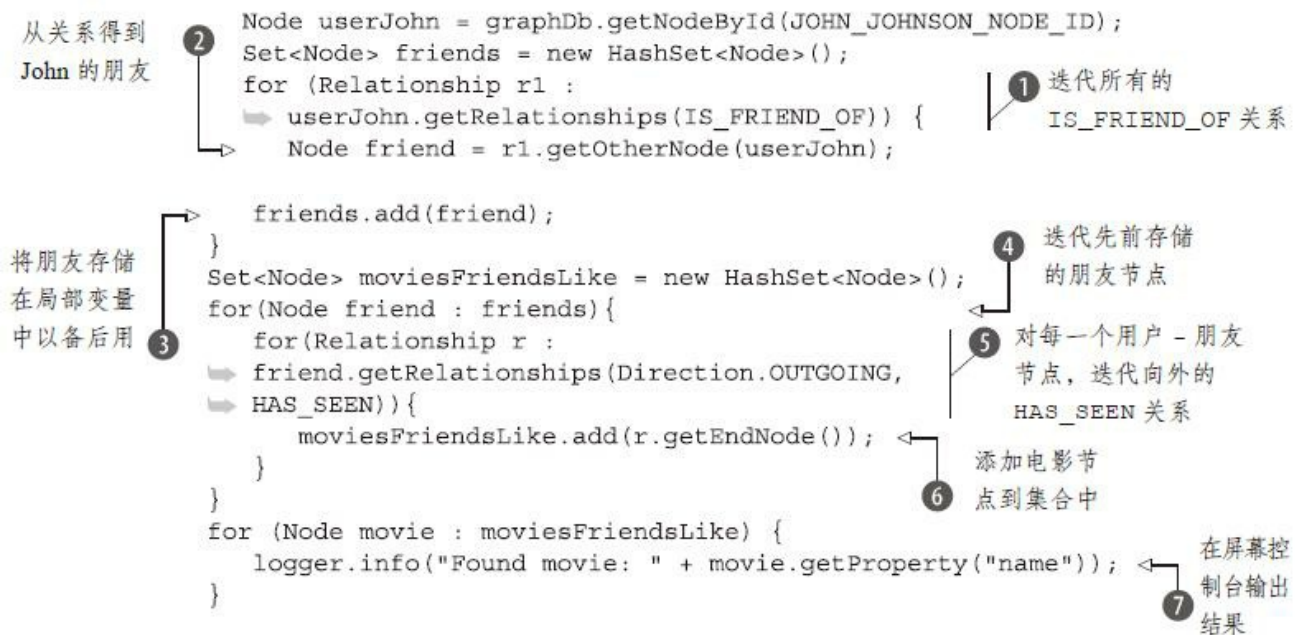


图4-2 为了查找John的朋友喜欢的电影遍历的节点和关系

为了解决这个问题，我们将把它分解为几步。首先，我们从起始节点开始按照IS_FRIEND_OF关系查找John的所有朋友。其次，从代表朋友的节点开始沿着HAS_SEEN关系查找他们已经看过的电影。程序4-3给出了这两步的具体过程，再一次简单地使用了Neo4j核心Java API。

【程序4-3】 查找John的朋友已经看过的电影



本程序的第一部分（①、②、③）用局部变量存储了John的朋友。首先，迭代从起点开始的所有IS_FRIEND_OF关系，而不考虑方向（①）。记住，尽管IS_FRIEND_OF关系具有一个起点和一个终点，但是既可以往上游查找也可以往下游查找朋友关系。由于并不知道John是否是关系的起点还是终点，因此不能使用Relationship.getStartNode（）或Relationship.getEndNode（）方法以确定关系的其他参与者。幸运的是，Neo4j核心Java API对这个问题有一个解决的办法：Relationship.getOtherNode（Node node）方法。给定关系的一端，该方法就给出关系的另一端。在这种情况下，因为知道关系的一端是John节点，所以可以利用该方法查找John的朋友（②）。最后，添加朋友节点到java.lang.Set局部变量friends中（③），以备以后使用。

在本程序的第二部分（④、⑤、⑥）找到了John的朋友看过的所有电影。对每一个存储在临时局部变量中的节点（④），检查所有向外的HAS_SEEN关系（⑤）。就像在前面的例子中一样，加载HAS_SEEN的终止节点（代表一部电影）并作为结果将其存到局部set变量moviesFriendsLike中

(6)。

最后，输出结果到控制台上(7)，输出的结果如下：

```
Found movie: Fargo  
Found movie: Alien  
Found movie: Heat
```

很明显，该程序有错的地方，尽管John的朋友已看过这三部电影（Jack看了Fargo和Alien，Kate看了Heat），因为John看过Fargo，所以不应该输出Fargo。

必须确保在输出中删除John看过的电影。要完成这项工作，需要找到John看过的所有电影（就像在4.1.2节中一样）并从结果中删除。程序4-4给出了完成这一工作的一种方法。

【程序4-4】 查找John的朋友已经看过但John没有看过的电影

```
Node userJohn = graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);  
  
Set<Node> friends = new HashSet<Node>();  
for (Relationship r1 :  
    ➡ userJohn.getRelationships(IS_FRIEND_OF)) {  
    Node friend = r1.getOtherNode(userJohn);  
    friends.add(friend);  
}
```



```

}
Set<Node> moviesFriendsLike = new HashSet<Node>();
for(Node friend : friends){
    for(Relationship r :
    ➡ friend.getRelationships(Direction.OUTGOING, HAS_SEEN)){
        moviesFriendsLike.add(r.getEndNode());
    }
}
Set<Node> moviesJohnLike = new HashSet<Node>();
for(Relationship r :
    ➡ userJohn.getRelationships(Direction.OUTGOING,
    ➡ HAS_SEEN)){
    moviesJohnLike.add(r.getEndNode());
}
moviesFriendsLike.removeAll(moviesJohnLike);
for (Node movie : moviesFriendsLike) {
    logger.info("Found movie: " + movie.getProperty("name"));
}

```

① 查找 John 看过的所有电影

② 从结果中删除 John 看过的所有电影

现在已经给遍历算法添加了另外一个步骤。首先查找John已经看过那些的电影（①），然后应用标准的Java Collections API从原来的结果集中删除John看过的电影（②）。运行程序4-4，可以得到预期的结果，即两部电影。

```

Found movie: Alien
Found movie: Heat

```

最终有了类似一个简单的推荐引擎的程序代码——由用户的朋友，而不是用户推荐已经看过的电影。

让我们回忆一下这个例子的实现过程。用一个简单的算法，在一系列的步骤中实现解决方案：找到所有朋友，然后找到朋友看过的电影，最后删除用户看过的电影。尽管程序可读性好并容易理解，但存在一个重大缺陷：用了大量内存。每一步都用Java的堆空间在局部变量中存储了中间结果。当处理少量用户和电影时，这不是个问题，但是数据集增长到成千上万个时，运行程序很可能出现可怕的OutOfMemoryError异常。在本例中这不是一个值得关注的问题，因为总共有三个用户和三部电影。现在到了该考虑Neo4j是如

何处理大的遍历结果的时间了。

4.1.4 内存使用注意事项

已经看到Node.getRelationships (...)方法返回了一个Iterable-<Relationship>对象。Iterable接口允许在Java的for循环中使用实现类，这在前面的例子中已经出现过几次。在Java代码中，有许多实现Iterable接口的例子——例如，所有的java.util.Collection类，像java.util.Set和java.util.List实现，以及java.util.Vector和java.util.Stack。但是Node.getRelationships (...)方法不返回任何这些著名的类。它们返回了Neo4j的Iterable实现，org.neo4j.kernel.impl.util.ArrayIntIterator既实现了Java Iterable接口，也实现了Iterator接口，并且是一个围绕java.util.Iterator的瘦包装类。在对结果迭代之前实际上还没有访问结果中包含的元素。这一实现是在第一次访问时延迟加载的，一旦使用，就不能再次使用——变为无效（这是Java迭代器期望的行为）。

当使用Neo4j核心Java API时，这是非常重要的。Iterable接口允许Neo4j返回非常大的数据集，开发者负责确保以正确的方式访问数据集中的元素。就像我们说过的，如果从Neo4j的Iterable接口加载大量的结果到你自己的Java list或set中，必须意识到你的程序代码需要大量的Java堆内存，这样在运行程序时就会有可能会出现OutOfMemoryError异常。这就是为什么尽可能在程序代码中使用Iterable接口被认为是一个好的做法，而不是将其转换为大量需要内存结构的代码。

程序4-5展示了一个解决与以前例子一样问题的方案——查找John的朋

友看过但是John还没有看的电影，但是这一次仅仅使用Java的for循环结构在Neo4j的Iterable接口实现，且不在局部变量中消耗不必要的内存。

【程序4-5】 使用迭代接口降低Java堆内存的消耗

```
Node userJohn = graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);
Set<Node> moviesFriendsLike = new HashSet<Node>();
for (Relationship r1 :
    ➤ userJohn.getRelationships(IS_FRIEND_OF)) {
    Node friend = r1.getOtherNode(userJohn);
    for (Relationship r2 :
        ➤ friend.getRelationships(Direction.OUTGOING, HAS_SEEN)) {
        Node movie = r2.getEndNode();
        boolean johnLikesIt = false;
        for (Relationship r3 :
            ➤ movie.getRelationships(Direction.INCOMING, HAS_SEEN)) {
            if (r3.getStartNode().equals(userJohn)) {
                johnLikesIt = true;
            }
        }
        if (!johnLikesIt) {
            moviesFriendsLike.add(movie);
        }
    }
}
for (Node movie : moviesFriendsLike) {
    logger.info("Found movie: " + movie.getProperty("name"));
}
```



注意

这个解决方案看上去比较好，但是，当面对有用户看过的几百万部电影时，存在一个潜在的瓶颈（因为在每一次循环中都必须检查John是否已经看过）。在有限的数据集中，这不会有什么影响，但是，必须指出的是，当面对大数据集时，编写简单的代码有时会更好，即使多写一点也没关系。在本例中，一旦往前查找，就会找到John看过的所有电影，然后用这个数据集在for循环中过滤电影。

我们已经设法重构了解决方案，以便节省内存并更好地适应潜在的大数据集。结果正如期待的一样，因此方案仍然是正确的。但是程序4-5仍然不是我们所见过程序中最好的。Neo4j核心Java API允许我们访问底层Neo4j数据结构以实现需要的遍历，但是存在付出多写代码的代价并担心其他的方面（如内存的消耗）。幸运的是Neo4j配备了一个高精炼的API，可以满足以简单、自然的方式描述遍历的需求。在下一节中，我们将用Neo4j遍历API进一步重构遍历解决方案。

4.2 使用Neo4j的遍历API进行遍历

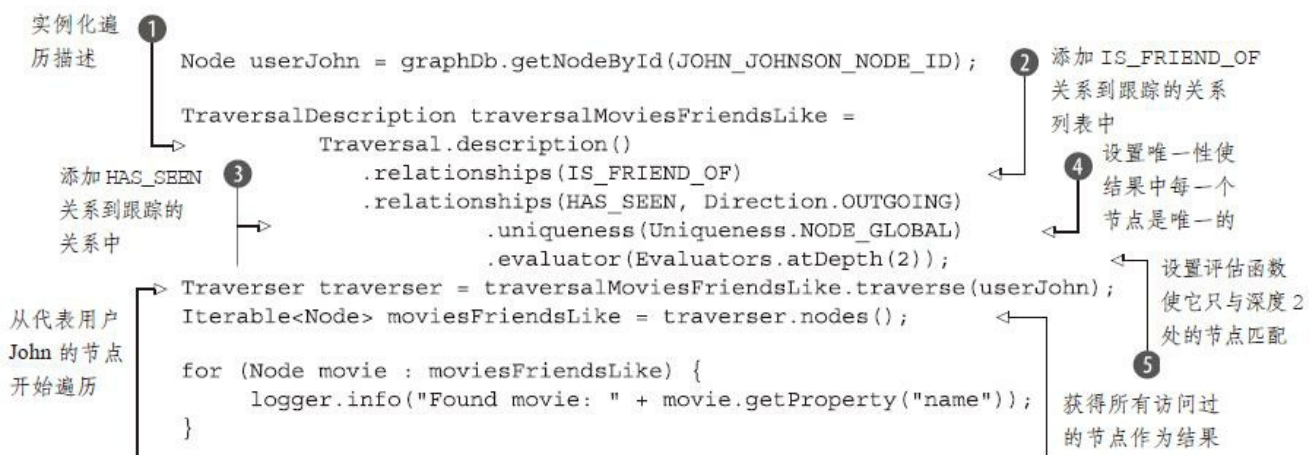
我们已经使用Neo4j核心Java API实现了一个推荐引擎，它允许我们使用底层的Neo4j数据结构获得我们的目标。在本节中，我们将使用更具有流畅性和表达性的Neo4j遍历API改进这个解决方案。

4.2.1 使用Neo4j的内置遍历结构

Neo4j遍历API是一个具有流畅创建器API且基于回调函数的框架，它允许你在一行代码中以可表达的方式创建遍历规则。遍历API的主要部分是`org.neo4j.graphdb.traversal.TraversalDescription`接口，它定义了创建器的方法用于描述遍历器行为。你可以将遍历比作一个机器人，按照一组定义好的遍历顺序规则和跟踪关系、在结果中需要包含的关系和节点等，通过关系从一个节点跳跃到另一个节点。**Traversal Description**（遍历描述）是一个不可变的对象，用于定义遍历规则。添加任何新的遍历规则（在`TraversalDescription`接口中调用创建器方法中的任何一个）总是返回新的`TraversalDescription`实例。

为了演示Neo4j遍历API是如何工作的，让我们使用Neo4j遍历API重构遍历。在程序4-6中，返回了John的朋友看过的电影的遍历已经实现，但没有删除John本人看过的电影。

【程序4-6】 使用Neo4j遍历API查找朋友看过的电影



Neo4j 提供了一个默认的 `TraversalDescription` 接口实现，它可以用静态方法 `Traversal.description()` 实例化 (1)。由于很少需要自己提供这个实现，因此，通常从此开始创建 `TraversalDescription`。下一步定义遍历中的要包含关系 (2、3)，`TraversalDescription` 维护使用 `TraversalDescription.relationships(...)` 方法添加的关系列表，并且只有在这个列表中包含的关系才能被遍历。可以添加关系类型而不必指定方向，在这种情况下，两个方向都可以查找，或者也可以定义一个相对于起始节点的查找方向 (3)。

在下一行指定遍历器如何处理在遍历期间遇到的节点和关系的唯一性。我们期望每一个节点恰好访问一次，因此，在 `Uniqueness.NODE_GLOBAL`（全局节点唯一性）中设置唯一性 (4)。例如，其他允许的值是 `Uniqueness.NODE_PATH`（节点路径唯一性），即当从起点到终点的路径是唯一的时候，它允许遍历同一个节点多次；而 `Uniqueness.RELATIONSHIP_GLOBAL`（全局关系唯一性）允许遍历每一个关系仅仅一次。我们将在第8章中详细介绍唯一性值之间的区别。

最后，给 `TraversalDescription` 添加一个评估函数（Evaluator） (5)。评估函数负责两个决策（作为 Neo4j 遍历 API 的一部分）：

- 决定正在遍历的节点是否添加到遍历的结果中。

- 决定遍历是否沿着当前图形的路径继续遍历下去，如果评估到现在的路径应该放弃，就移动到下一个路径（如有可能）。

Neo4j中的评估函数是由`org.neo4j.graphdb.traversal.Evaluator`接口定义的，Neo4j提供了大量可供使用的方便、现成的实现方法。这些方法可以通过`org.neo4j.graphdb.traversal.Evaluator`类中的静态工厂方法访问。在程序4-6中使用了`Evaluators.atDepth(int depth)`评估函数，该评估函数仅接受指定深度上的所有节点，并从起始节点计数。另外，该评估函数还阻止任何超出指定深度的遍历。

Neo4j提供的其他有用的评估函数将在第8章中讨论。评估函数是Neo4j遍历API的重要概念，很有可能会经常实现自定义评估函数。在下一节中，我们将实现一个自定义Evaluator接口。

4.2.2 实现一个自定义评估函数

现在需要改进前一节例子中的代码以从结果中排除John看过的电影。要实现这个目的，需要给遍历描述添加新的规则。前面提到Neo4j的评估函数实现定义了哪个节点保存在结果中和哪个节点丢弃。另外，还定义了什么时候遍历完全停止。基于这一点，我们可以实现另外一个自定义评估函数，用以排除用户已经看过的电影。

Evaluator接口定义了一个需要实现的方法`public Evaluation evaluate(Path path)`。这个方法接受`org.neo4j.graphdb.Path`类型的单一参

数，这个参数代表从起始节点到目前节点遍历的所有节点和关系。这个接口定义了许多便利的方法用以收集有关遍历当前状态的信息。例如，遍历的所有节点、遍历的所有关系、路径的起始节点和路径的终止节点等。表4-1列出了Path接口的所有可用方法。

表4-1 org.neo4j.graphdb.Path接口中定义的方法

方法签名	描 述
Node startNode();	路径的起始节点，不要与关系的起始节点混淆
Node endNode();	路径的终止节点，遍历当前节点，不要与关系的终止节点混淆
Relationship lastRelationship();	遍历的最后关系
Iterable<Relationship> relationships();	按遍历顺序，到当前节点遍历过的所有关系
Iterable<Node> nodes();	按遍历顺序，路径中的所有节点
int length();	返回路径的长度，实际上就是遍历过的节点数（或者是节点数减1）

 注意

不要混淆路径的起始节点终止节点与关系的起始节点终止节点是很重要的。关系的起始与终止节点依赖于关系的方向。路径的起始与终止节点依赖于遍历顺序。由于Neo4j允许按任意方向遍历关系，这对不太熟悉Neo4j的人来说，有时很容易引起混淆。

evaluate(..)方法能够返回org.neo4j.graphdb.traversal.Evaluation类型，这是一种Java枚举类型，具有四种可能的值。基于返回的Evaluator，遍历器决定是停止（Neo4j术语为修正）还是继续。另外，返回的Evaluator值用以确定是否将现在的节点保存在结果中（包括）还是丢弃（排除）。这两个变量的结合定义了Evaluator枚举的四个值。表4-2解释了不同的Evaluator值。

表4-2 Evaluator枚举类的可能取值

方法签名	描 述
INCLUDE_AND_CONTINUE	包括结果中的当前节点并继续遍历
INCLUDE_AND_PRUNE	包括结果中的当前节点但停止本路径的继续遍历
EXCLUDE_AND_CONTINUE	丢弃当前节点并继续遍历
EXCLUDE_AND_PRUNE	丢弃当前节点并停止遍历

现在已经理解了Path接口和Evaluator枚举，那就该实现一个自定义评估函数了。程序4-7给出了实现的代码。

【程序4-7】 自定义评估函数用于排除用户看过的电影

构造函数中
为评估函数
提供的起始
用户节点

```
public class CustomNodeFilteringEvaluator implements Evaluator {
    private final Node userNode;

    public CustomNodeFilteringEvaluator (Node userNode) {
        this.userNode = userNode;
    }

    public Evaluation evaluate(Path path) {
        Node currentNode = path.endNode();
        if (!currentNode.hasProperty("type") ||
            !currentNode.getProperty("type").equals("Movie")) {
            return Evaluation.EXCLUDE_AND_CONTINUE;
        }
        for (Relationship r :
            currentNode.getRelationships(Direction.INCOMING, HAS_SEEN)) {
            if (r.getStartNode().equals(userNode)) {
                return Evaluation.EXCLUDE_AND_CONTINUE;
            }
        }
        return Evaluation.INCLUDE_AND_CONTINUE;
    }
}
```

类实现 org.neo4j.
graphdb.
traversal.
Evaluator

得到遍历中当前节点的引用

如果当前节点不是一部
电影，丢弃
它并继续。

通过当前电影节点所
有向内的 HAS_SEEN
关系迭代

如果关系的起始节点与
用户节点相同，丢弃当
前节点（因为用户已经
看过）并继续

否则，包含结果中的
当前节点并继续

现在就可以把这个实现的自定义评估函数包含于一个遍历定义中。程序4-8给出了改进的遍历定义。

【程序4-8】 用自定义评估函数改进的遍历定义

```

Node userJohn = graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);

TraversalDescription traversalMoviesFriendsLike =
    Traversal.description()
        .relationships(IS_FRIEND_OF)
        .relationships(HAS_SEEN, Direction.OUTGOING)
        .uniqueness(Uniqueness.NODE_GLOBAL)
        .evaluator(Evaluators.atDepth(2));
        // ① 存在的评估函数仍然在用
        .evaluator(new CustomNodeFilteringEvaluator (userJohn));

Traverser traverser = traversalMoviesFriendsLike.traverse(userJohn);
Iterable<Node> moviesFriendsLike = traverser.nodes();

// 自定义函数加到遍历定义中
// ②
for (Node movie : moviesFriendsLike) {
    logger.info("Found movie: " + movie.getProperty("name"));
}

```

其实已经有了一个评估函数，这个评估函数仅包含深度2的节点（①）。最好是不要删除它，只把新的加进去即可（②）。在Neo4j遍历API中，可以同时建立多个评估函数，因此可以添加多个评估函数到单个遍历描述（Travelsal Description）中。如果在遍历的过程中包含多个评估函数，那么使用布尔代数：对包含在结果中的当前节点，所有的评估函数都应该返回带一个INCLUDE元素（INCLUDE_AND_CONTINUE或INCLUDE_AND_PRUNE）的评估值。类似地，要沿着同一路径继续往下遍历，所有的评估函数都要返回一个CONTINUE评估函数（INCLUDE_AND_CONTINUE或EXCLUDE_AND_CONTINUE）。

如果以同一组数据运行改进的应用，将会看到期待的输出。

```

Found movie: Alien
Found movie: Heat

```

然而，这一次我们使用了更流畅的Neo4j遍历API并设法实现了一个更具有表现力的解决方案。Neo4j遍历API具有一个声明特性：简单地声明你希望如何遍历，然后按照声明开始遍历。如果你使用的是Java API，我们建议你在可能的情况下使用Neo4j遍历器API框架遍历做图形遍历，因为Neo4j遍历器API框架在处理复杂图形遍历时产生可读的、可维护的和高性能的代码。

在第8章中我们将回过头来讨论更先进的遍历概念。

4.3 本章小结

本章介绍了遍历，遍历是一个查询相互联系的数据的强有力的方法，是专为图形数据库而设计的。

4.1节介绍了如何使用Neo4j核心Java API遍历例子中的图形数据，找到了用户的朋友看过的所有电影。遍历图形时，Neo4j核心Java API功能强大并且灵活性好，但是，对于复杂的遍历，Neo4j核心Java API的实现将很快变得复杂起来。

这就是Neo4j遍历API所解决的问题，是在4.2节中讨论的。使用流畅的Neo4j遍历API，可以以简单的声明方式描述遍历查询，而不会影响任何的图形遍历功能，并使性能的影响降到最小。

在本书的大部分例子中，因为Neo4j遍历API的流畅性和可读性，我们选择Neo4j遍历API。但是在一些复杂情况下，我们也会偶尔使用Neo4j核心Java API——两种方法都有它们各自的注重点，因此在图形遍历时应该选择合式的方法。

但是，在遍历一个图形数据库之前，需要选择一个或多个节点作为起始节点。本章使用了一个已知的节点ID寻找遍历的起始节点，但是在实际中不可能知道一个特定节点的ID——更多情况下是基于某一个属性选定一个特定的节点。使用这种“单调的”节点查找，需要使用索引，这正是我们将要在下章中讨论的。

第5章 数据索引

本章包括以下内容：

- 创建和维护手动索引
- 使用模式索引和自动索引
- 创建索引策略时的权衡

在前几章中我们已经看到了在Neo4j图形中通过遍历关系在节点之间移动是多么容易。在节点之间移动可以快速、容易地找到相连的节点，例如，一个人的朋友和他们喜欢的电影。Neo4j已经过优化以确保快速遍历图形，但是通过知道从哪里开始减少需要遍历的节点数是很重要的，随着数据集的增加，需要遍历的节点数也在增加。

Neo4j使用索引以确定在图形数据库中从哪里开始。在关系数据库中，一个索引提供了通过列的特定数值快速和容易地查找表中的行。同样，Neo4j的索引使得它通过特定的属性值容易地查找节点或关系。与关系数据库不同，Neo4j需要应用程序代码创建和维护索引项。

因为应用程序的代码负责索引，所以需要认真考虑索引策略。关于索引的不明智决策将导致性能低下或硬盘使用超量。在本章中，我们将介绍如何用Neo4j创建、维护和使用索引。然后我们将探索为什么要进行索引并讨论当创建一个索引策略时需要做的和不可避免的权衡。

在开始叙述有关权衡问题之前，让我们看一下如何创建索引项。

5.1 创建索引项

当做索引的时候，最常用的是显式创建索引，然后就像创建节点一样添加索引项。每一个索引项通常标识一个节点或关系属性值。索引项包含对正在索引的属性具有特定值的一个或多个节点的引用。

图5-1给出了像这样的节点索引，可以认为是一个或多个指向节点的指针相关的数值项。在这种情况下，我们期望电子邮件地址是唯一的，因此我们期望对每一个用户有一个主电子邮件地址。

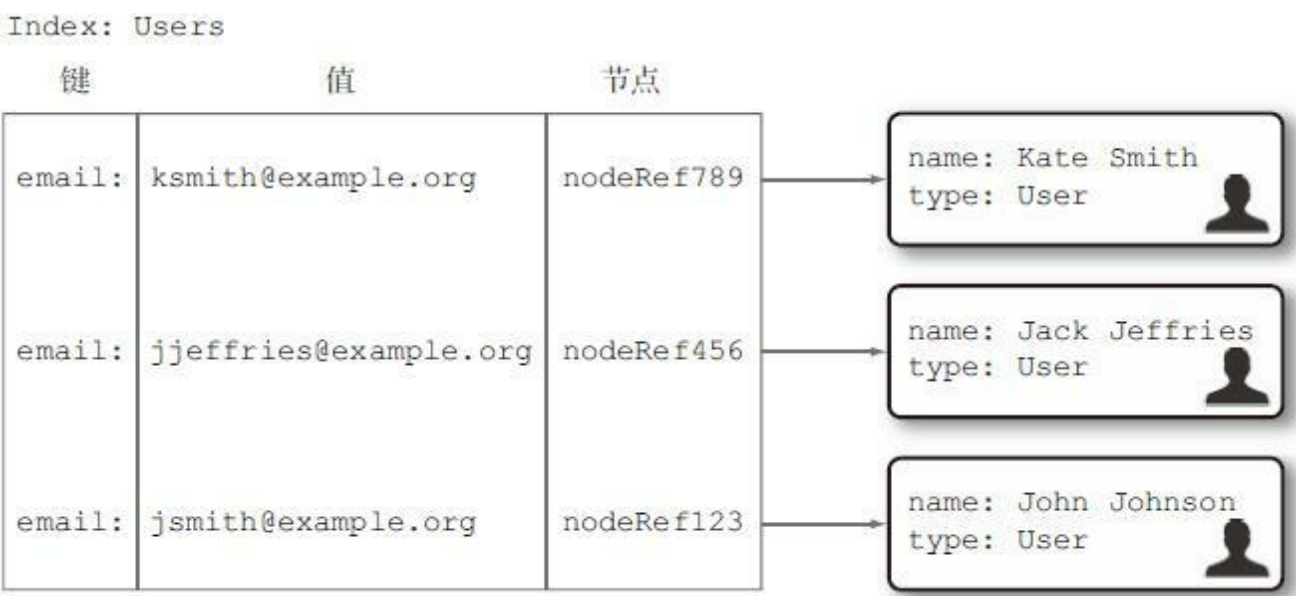


图5-1 使用电子邮件属性作为一个键值指向用户节点的索引

在Neo4j中，IndexManager（索引管理器）使用一个简单的字符串作为索引键提供访问索引。在社交网络的应用中，通用的起始节点是一个用户。要唯一标识一个用户，需要用户用自己的电子邮件地址登录。程序5-1中的代码显示了如何建立一个新用户并建立索引项，这意味着通过电子邮件地址可以快速找到代表用户的节点。在Index<Node>接口中定义所有可能的索

引操作，从Neo4j索引管理器部件访问索引的实例如下：

```
IndexManager indexManager = graphDB.index();  
Index<Node> userIndex = indexManager.forNodes("users");
```

注意只是简单地以需求的名字向Neo4j索引管理器请求索引。从代码的角度看，索引是否存在并没关系，如果索引不存在，当有请求时就会建立起来。参见程序5-1。

【程序5-1】 使用Neo4j API对一个节点创建索引项



首先，使用Neo4j核心API创建节点并添加属性（**1**、**2**）。其次，创建索引并将其命名为users（**3**、**4**）——这个名字将成为这个新建索引的唯一标识符。

最后，将节点添加到索引中（**5**）。要添加一个节点到索引中，需要提供如下三个参数：

- 需要索引的节点（`personOne`）。
- 索引键（`"email"`）。
- 索引的值（`jsmith@example.org`）。

然后，索引键和值与对这个用户节点的引用一起由Lucene（Neo4j默认

的索引实现）存储，如图5-1所示。

如果想要配置Lucene索引，当索引建立后，Neo4j允许传递不同的设置选项。这个可用以下的方法实现：

```
IndexManager.forNodes( String indexName, Map<String, String>  
    customConfiguration );
```

映射customConfiguration（自定义设置）可以包含任意的合法Lucene设置。对Lucene的设置一览表，可以参阅<http://docs.neo4j.org/chunked/stable/indexing-create-advanced.html>上的Neo4j/Lucene手册。

当然，将一个节点的真实值进行索引能够迅速和容易地找到要找的节点。这就是我们将在下一节中看到的，用用户的电子邮件地址索引查找用户节点。

5.2 通过邮箱地址查找用户

网络中有用户很棒，但是现在需要让用户有一个成熟的网页使他们能与朋友进行联系并存储电影信息。

在这个网页应用中，需要做的首要事情之一是显示用户的朋友清单。查找用户的朋友将涉及遍历朋友关系，但是首先需要查找代表已登录用户的节点。

由于用户使用他们的电子邮件地址作为登录ID，这里以电子邮件地址对用户进行索引，因此可以非常容易地查找一个已登录的用户。然后，可以往外遍历朋友关系以寻找他们的朋友，如图5-2所示。

要通过电子邮件查找一个用户，需要通过以索引的专用名字标识的节点索引获得一个引用，在本例中，使用users索引。然后可以得到一个针对特定的键和值的索引。程序5-2给出了用电子邮件查找用户节点的示例。

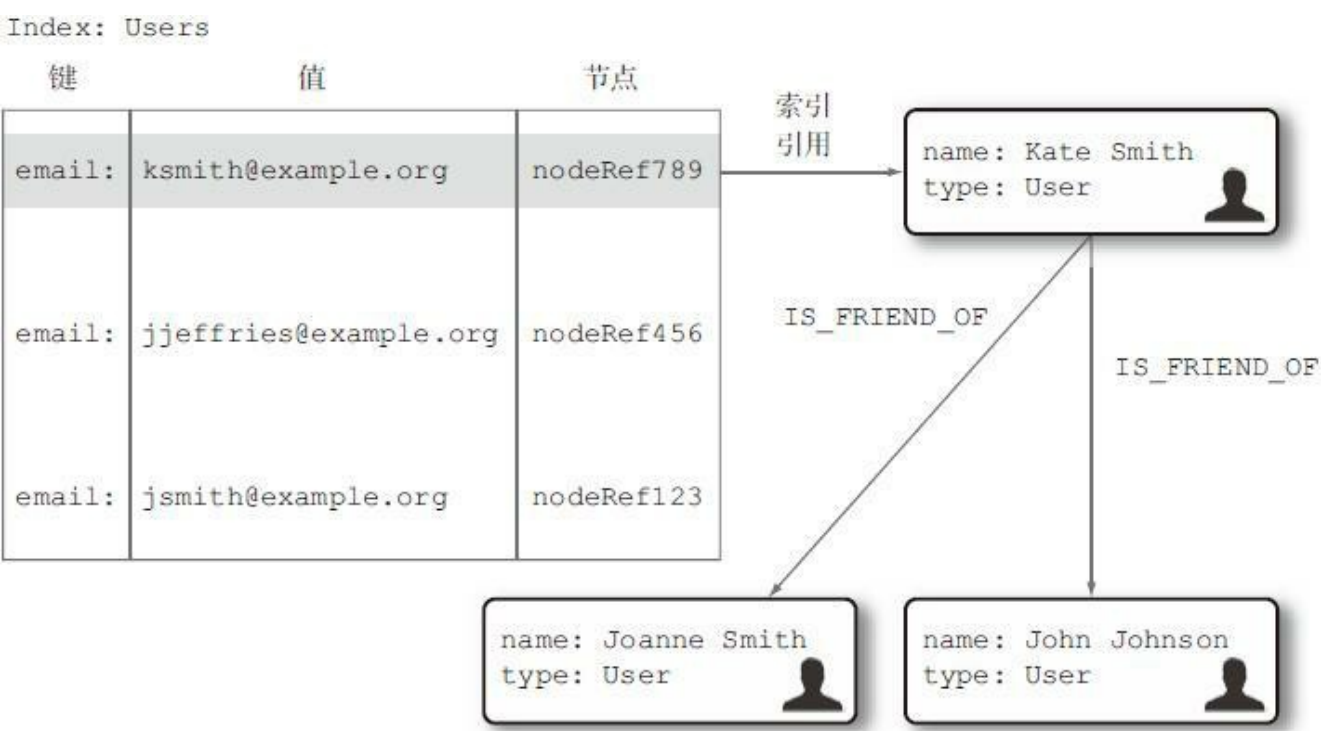


图5-2 通过电子邮件地址属性从索引中查找用户节点

【程序5-2】 使用电子邮件属性通过索引查找单个用户

```
String userEmail = "jsmith@example.org";

IndexManager indexManager = graphDB.index();
Index<Node> userIndex = indexManager.forNodes("users");
IndexHits<Node> indexHits = userIndex.get("email", userEmail);
Node loggedInUserNode = indexHits.getSingle();

if (loggedInUserNode == null) {
    throw new NoSuchUserException("No user with email "
        + userEmail + " found");
}
```

得到一个结果：如果多于一个，而不是没有，就提示没有如此唯一的元素

获得索引管理器

找到用户电子邮件索引

查找对应用户邮件地址的已知索引键（邮件）并获得位置

检查找到的匹配节点

一旦找到登录的用户节点，通过核心Java API使用一个快速简单的遍历就很容易地找到他们的朋友。

```
Iterable<Relationship> outboundFriendRelationships =
    loggedInUserNode.getRelationships(
        DynamicRelationshipType.withName("is_friend_of"),
        Direction.OUTGOING);

for(Relationship frRel: outboundFriendRelationships){
    friends.add(frRel.getOtherNode(loggedOnUserNode));
}
```

通过关系迭代

获得通过索引查找返回的登录用户即将停映的关系类型

在关系的另一端储存节点

在本节中，我们演示了Neo4j索引的典型工作模式——通过索引查找找到一个节点，然后按常规做遍历。程序5-2确保最多找到一个与用户邮箱地址相匹配的邮箱地址。现在让我们看看如何处理有多个匹配的情况。

5.3 对多个匹配结果的处理

在前面的例子中，对每一个用户的邮箱地址进行了索引，并确保邮箱地址的唯一性。因此，索引查找的结果应该是一个邮箱地址。

但是不一直都是这种情况。让我们给用户添加另一个索引属性——年龄。很显然，多个用户节点可能会有相同的age属性值。这种索引的键值能引用多个节点值，如图5-3所示。

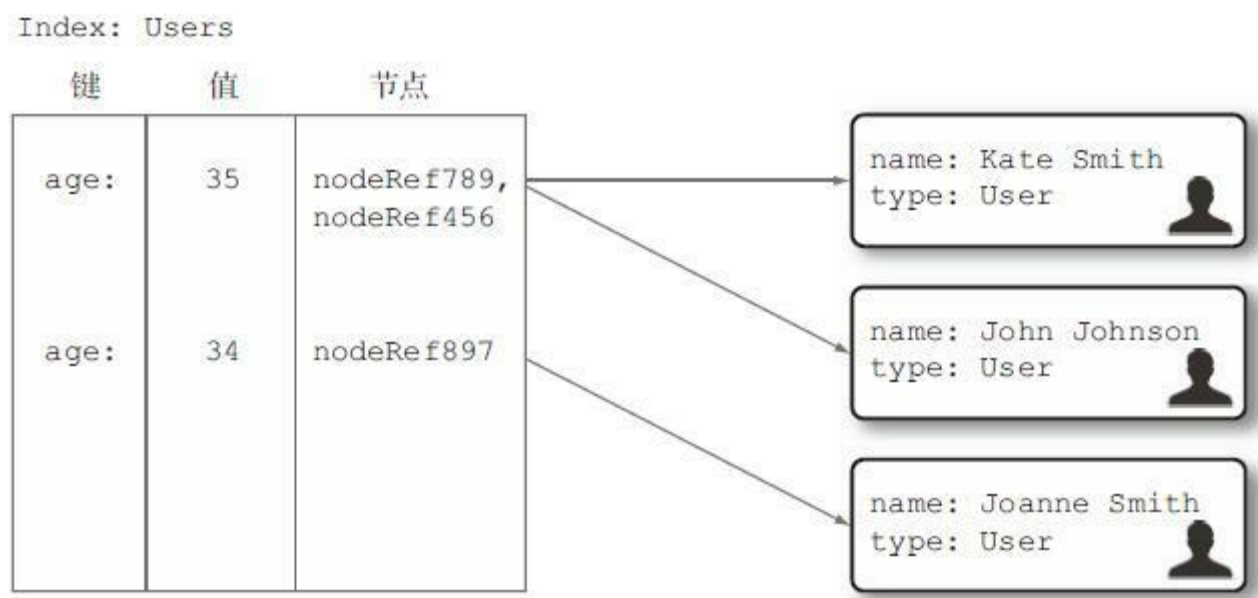


图5-3 对年龄属性索引的用户节点且每一个键值潜在引用多个节点

前面我们假设最多找到一个与给定用户相匹配的邮箱地址。如果情况不是这样，则需要对返回的索引结果进行迭代，如程序5-3所示，返回了所有年龄为34岁的用户。

【程序5-3】 对索引查找操作多个结果的迭代

```

IndexManager indexManager = graphDB.index();
Index<Node> userIndex = indexManager.forNodes("users");
IndexHits<Node> indexHits = userIndex.get("age", 34);

for (Node user : hits) {
    System.out.println(user.getProperty("name"));
}

```

像得到单一结果那样调用 get

在一个或多个结果中迭代，输出结果

注意

IndexHits（索引结果）是一次性迭代的，一旦用过，就不能再次使用。这是Lucene的规定。更多信息可以参考网站<http://lucene.apache.org>上的Lucene手册。

可能某些情况会需要找到所有34岁的用户，更有可能的是要找到一个范围内的用户。Lucene支持范围查询，还有许多其他类型的现成的查询方法。Lucene的详细介绍超出了本书的范围，但是可以在Lucene手册中找到详细的介绍（特别是在<http://mng.bz/Lq6r>）。

注意

使用完IndexHits后应该关闭。如果在所有的结果中迭代（如前面的例子），IndexHits将会自动关闭。如果没有迭代所有的索引结果，必须确保通过调用IndexHits.close（）手工关闭。这是Lucene的规定，更多信息可以参照<http://lucene.apache.org>上的Lucene手册。

已经看到如何创建和查询索引项，下一个需要关注的问题是如何处理图形数据库的变化导致索引的变化。

5.4 对索引过的数据进行修改的处理

索引项的创建并不仅仅是创建和删除。即使在本章的简单例子中，也需要处理用户改变他们的邮箱地址或者想要删除他们的账户的情况。

因为应用开发者对使用的索引策略非常谨慎，所以Neo4j并不自动更新由手动做的索引引起的图形数据库本身的数据改变。后面将会看到如何通过其他一些机制获得这类灵活性的例子，但是现在我们将重点关注如何处理涉及手动创建索引而引起的变化。

当一个用户想要更改他的邮箱地址时，将会发生什么情况？我们已经提到，所有可能的索引操作都在Index<Node>接口定义。

```
Index<Node> userIndex = indexManager.forNodes("users");
```

当看到Index<Node>接口时，也许需要注意的第一件事情是没有更新方法，一旦已经查找过一个索引，就没有方法对这个已经存在的索引进行修改。解决的方法非常简单：当处理Neo4j索引时，“先删除后添加”等于“更新”。程序5-4展示了如何使用Neo4j核心Java API完成这个操作。

【程序5-4】 使用顺序的删除和添加操作更新索引


```

String userEmail = "jsmith@example.org";
String updatedUserMail = "jsmith@shinynew.example.org"

IndexManager indexManager = graphDB.index();
Index<Node> userIndex = indexManager.forNodes("users");
IndexHits<Node> indexHits = userIndex.get("email", userEmail);
Node loggedInUserNode = indexHits.getSingle();
if (loggedInUserNode == null) {
    throw new NoSuchUserException("No user with email "
    + userEmail + " found");
}
userIndex.remove(loggedOnUserNode, "email", userEmail);
loggedInUserNode.setProperty("email", updatedUserMail);
userIndex.add(loggedOnUserNode, "email", updatedUserMail);

```

使用索引
获取已有的
用户节点

删除给定节点
邮箱地址项

更新具有新
邮箱属性值
的节点

给索引
添加新
的属
性值

索引关系

我们已经了解了如何创建索引项并在图形数据库中快速查找节点，但是也可以索引节点之间的关系。这非常像索引节点，但是使用关系属性作为主键。

按以往的经验，并不经常对关系索引，对关系索引也并不是一个好的解决方案。这并不是说关系索引是一个低劣的操作，但是如果添加了很多的关系索引，那就很值得问问为什么了？

通常一个图形数据库将会以节点代表一些形式的实体，例如，人、电影和城市，关系就代表了这些实体之间的关系。那么经常问的问题是这些实体与谁相连。例如，谁喜欢这部电影，谁是一个特定人的朋友，或者离开这个城市的交通路线。

然而，很可能想到回答这些问题需要用关系索引。如果选择在电影数据库中以不同的关系对应不同的评级来代表星的级别，索引这个关系将会快速找到所有五星级的评价。

了解更多有关关系索引可以参见Neo4j手

册：<http://docs.neo4j.org/chunked/stable/indexing-relationships.html>。

我们将在本章的后面详细讨论有关索引的权衡，但是，我们首先要通过一个自动索引来完成我们对索引选项选择的过程。

5.5 自动索引

本章前面讲的是人工创建和维护索引项。如果你是从关系数据库世界来的，你将会好奇为什么数据库不能为我们做这项工作。毕竟，在关系数据库中，你只需声明表示哪一列需要索引，然后让关系数据库在插入、更新和删除行时维护那个索引即可。

Neo4j有两种自动维护索引的方法——模式索引和自动索引，首先让我们看一看模式索引。

5.5.1 模式索引

在2.0版本中，Neo4j引进了模式索引的概念，从概念上讲，这与传统的关系数据库使用的索引处理方法很相似。

模式索引与节点标签的概念紧密相关，我们在第3章中已经做过介绍。每一个模式索引专门对应着一个标签和一组属性。例如，可以对一个用户的姓名属性name定义索引，或者对电影的名字和出品年份属性year定义索引。你要做的就是定义索引，因此，Neo4j会负责维护它们。这意味着当创建一个具有标签和属性的节点并匹配一个或多个索引时，所有的索引都会以那个值进行更新。当删除一个节点时，那个节点的所有与相关索引有关的索引项都会被自动删除，当更新节点关系时也是如此。

让我们看一个真实的例子，参见程序5-5。

【程序5-5】 与Java API一起使用模式索引

```

Label movieLabel = DynamicLabel.label("MOVIE");
Label userLabel = DynamicLabel.label("USER");

Node movie, user;

// Create schema
try (Transaction tx = graphDb.beginTx()) {
graphDb.schema().indexFor(movieLabel).on("name").create();
graphDb.schema().indexFor(userLabel).on("name").create();
tx.success();
}

// Add Labels and properties
try (Transaction tx = graphDb.beginTx()) {
movie = graphDb.createNode(movieLabel);
movie.setProperty("name", "Michael Collins");

user = graphDb.createNode(userLabel);
user.setProperty("name", "Michael Collins");
tx.success();
}

// Verify results
try (Transaction tx = graphDb.beginTx()) {
ResourceIterable<Node> result =
graphDb.findNodesByLabelAndProperty(
movieLabel, "name", "Michael Collins");
assertEquals(1, IteratorUtil.count(result));
assertEquals(movie.getId(), result.iterator().next().getId());
tx.success();
}

```

1 创建将要使用的标签

2 创建电影名字属性索引

3 定义用户名属性索引

4 定义电影“Michael Collins”节点

5 创建名字为“Michael Collins”的用户节点

6 通过名字索引查找电影

7 验证加载的是正确节点

这个例子使用了两个标签：一个是电影，称为MOVIE，另一个是用户，称为USER，这些是前面定义的（❶）。

现在要定义以后需要的模式索引。这与在关系数据库中定义索引非常相似。在图形数据库服务模式（`GraphDatabaseService.schema()`）方法中可以使用Java API创建索引。首先，对name属性创建MOVIE标签索引（❷），然后对USER标签进行同样的操作（❸）。注意这段代码包含在它本身的事务中，与本例子的其他部分独立存在，这是因为这部分代码仅仅在应用的最前面编写一次。更为重要的是，如果试图在一个事务中做这一切，则将得到出错的信息——“`org.neo4j.graphdb.ConstraintViolationException: Cannot perform data updates in a transaction that has performed schema updates.`”。

定义索引后就可以继续创建节点。首先为电影Michael Collins创建具有MOVIE标签的节点（④），因为电影的标签和name属性与早已定义的索引相匹配，这个节点将自动加到索引中并可以以名字搜索。下一步，是为了有趣，创建一个用户名字为Michael Collins的节点，与电影的名字相同（⑤），然后提交事务。

假设所有的事情都按照计划做完，那么现在真实期望所创建的节点能被索引。可以通过查找电影Michael Collins来证实。要搜索模式索引，使用GraphDatabaseService.findNodesByLabelAndProperty(...)方法（⑥）。正像名字中所包含的，这个方法将按指定的标签和属性值搜索所有的节点。在这种情况下，将搜索与Michael Collins值相匹配的name属性的所有MOVIE标签。返回的结果是一个包含匹配节点的Java可迭代集。这个搜索结果正是所期望得到的那一个。有两个节点具有相同的名字Michael Collins，但只有一个具有MOVIE标签，并且这也就是所期待得到的那个。这个过程证明了结果的正确（⑦）。

每一个节点可以附加一个或多个标签，每一个标签可以有一个索引。如果节点具有多个标签，Neo4j将会确保相关的索引按要求更新。程序5-6是一个演示例子。

【程序5-6】 更新多个模式索引

```

Label userLabel = DynamicLabel.label("USER");
Label adminLabel = DynamicLabel.label("ADMIN");

try (Transaction tx = graphDb.beginTx()) {
    graphDb.schema().indexFor(userLabel).on("name").create();
    graphDb.schema().indexFor(adminLabel).on("name").create();
    tx.success();
}

try (Transaction tx = graphDb.beginTx()) {
    Node user = graphDb.createNode(userLabel, adminLabel);
    user.setProperty("name", "Peter Smith");
    tx.success();
}

try (Transaction tx = graphDb.beginTx()) {
    ResourceIterable<Node> adminSearch =
        graphDb.findNodesByLabelAndProperty(
            adminLabel, "name", "Peter Smith");
    assertEquals(1, IteratorUtil.count(adminSearch));
    ResourceIterable<Node> userSearch =
        graphDb.findNodesByLabelAndProperty(
            userLabel, "name", "Peter Smith");
    assertEquals(1, IteratorUtil.count(userSearch));
    tx.success();
}

```

1 创建用户和管理员标签索引

2 创建具有两个标签（管理员和用户）的节点

3 验证管理员索引的更新是正确的

4 验证用户索引的更新是正确的

在本例中，对两个标签做了索引：一个是常规的用户标签USER，另一个是管理员标签ADMIN（❶）。然后创建了单个节点，使用两个标签代表常规用户和管理员（❷）。我们希望通过搜索常规用户或管理员用户找到这个用户，下面两个步骤正是确认这种情况（❸、❹）。

到目前为止，我们已经看到了如何使用索引创建节点。我们也提到当删除一个节点时模式索引会更新。下面的程序表示了这种操作。


```

try (Transaction tx = graphDb.beginTx()) {
    user.delete();
    tx.success();
}

try (Transaction tx = graphDb.beginTx()) {
    adminSearch = graphDb.findNodesByLabelAndProperty(
        adminLabel, "name", "Peter Smith");
    assertEquals(0, IteratorUtil.count(adminSearch));

    userSearch = graphDb.findNodesByLabelAndProperty(
        userLabel, "name", "Peter Smith");
    assertEquals(0, IteratorUtil.count(userSearch));
    tx.success();
}

```

删除节点（在事务中）

验证管理员索引现在是空的

验证用户索引现在是空的

模式索引方法通过节点标签分组索引数据。这意味着每一个索引均包含标签的节点。如果需要对给定属性的所有节点进行自动索引（不考虑节点的标签），可以使用自动索引特性。

5.5.2 自动索引

要使用自动索引，需要告诉Neo4j打开节点或关系，或两者同时自动索引。但是，仅仅是打开自动索引并不会引起任何变化。在大型数据集中，索引所有的东西不一定可行，如果每一个值都在Neo4j内并进行索引，则存储容量将增加两倍或更多倍。由于维护索引需要额外工作，每当做一个变更操作时，也将会使其在性能上降低。因此，Neo4j对索引采取了更多可选性的方法。即使在自动索引打开的情况下，Neo4j也将仅仅维护指定索引的节点和关系属性。

如何设置自动索引取决于Neo4j的运行模式，即以嵌入式模式还是以服务器模式运行Neo4j（第10章将详细讨论这两种模式）。

1.在单机模式下配置自动索引

在单机运行模式下打开自动索引，需要以额外的属性修改配置文件。配置文件在\$NEO4J_SERVER/conf/neo4j.properties文件夹中，需要添加以下两行：

```
node_auto_indexing=true
relationship_auto_indexing=true
```

指定要索引什么，需要另外两行：

```
node_keys_indexable=name, dateOfBirth
relationship_keys_indexable=type, name
```

2.在嵌入式模式下配置自动索引

在嵌入式模式下打开自动索引，当创建图形数据库实例时，需要传递额外的值。以下的值需要作为一部分包含在java.util.Map的含有配置的参数中：

```
Map<String, String> config = new HashMap<String, String>();
config.put( Config.NODE_AUTO_INDEXING, "true" );
config.put( Config.RELATIONSHIP_AUTO_INDEXING, "true" );
EmbeddedGraphDatabase graphDb =
    new EmbeddedGraphDatabase("/var/neo4j/data", config );
```

自动使用设置映射（config map）需要另外的两个属性，里面包含需要索引的一系列主键名字：

```
config.put( Config.NODE_KEYS_INDEXABLE, "name, dateOfBirth" );
config.put( Config.RELATIONSHIP_KEYS_INDEXABLE, "type, name" );
```

3.使用自动创建的索引

一旦添加了适当的属性配置，图形数据库将通过自动索引变为可搜索。例如，假如给定了前面的配置，图形数据库中的所有关系都具有了type和

name属性，并且图形数据库中所有的节点都有了name和dateOfBirth属性，这些都可以通过自动索引进行搜索。

你可能已经注意到，在配置中没有一种指定索引名的方法，因此不能在一个索引中指定索引的节点名而在另一个索引中指定索引的出生日期。这一点与手动索引不同，在手动索引中总是指定索引的名字。这是因为Neo4j有一个索引用于关系自动索引而另一个索引用于节点自动索引。

下面的程序代码给出打开了具有name主键节点属性的自动索引，可以以特定的name属性值访问这个索引并使用它查找节点。

```
AutoIndexer<Node> nodeAutoIndexer =
    graphDb.index().getNodeAutoIndexer();
IndexHits<Node> nodesWithMatchingName =
    nodeAutoIndexer.getAutoIndex().get("name", "John");
Node userNode = nodesWithMatchingName.getSingle();
```

返回了一个可迭代的索引连接

给节点索引器一个引用

查找名字是“John”的索引项

使用Neo4j提供的系统和自动索引设施可以使开发的工作量大为减少，但是也会减少对索引结果的控制。索引所有属性的想法是不现实的，因为图形变化操作对性能的影响和对磁盘空间的需求是很大的。下节将会详细讨论对这一问题的权衡处理。

5.6 索引的成本/效益权衡

看到了Neo4j提供的不同索引方法，现在我们将把注意力放在如何权衡哪一种索引策略适合于你的应用。可能你已经开始发现，当使用Neo4j的索引和建模数据库时，Neo4j具有极大的灵活性。

例如，你可能以一种完全不同的方式建模社交网络的数据，这会对索引策略有影响。图5-4给出了另一种描述数据的方法，用一个表示Kate看过的电影和朋友节点作为中间节点建模。

这是一个描述社交网络数据更好的方法吗？很不幸，答案取决于你的应用。判断最好的数据描述方法和索引策略主要取决于想存储的数据和以什么的方式与这些数据进行交互。在本书的后面我们将会讨论在不同的数据描述方法中进行权衡，但是现在我们将重点放在索引上。

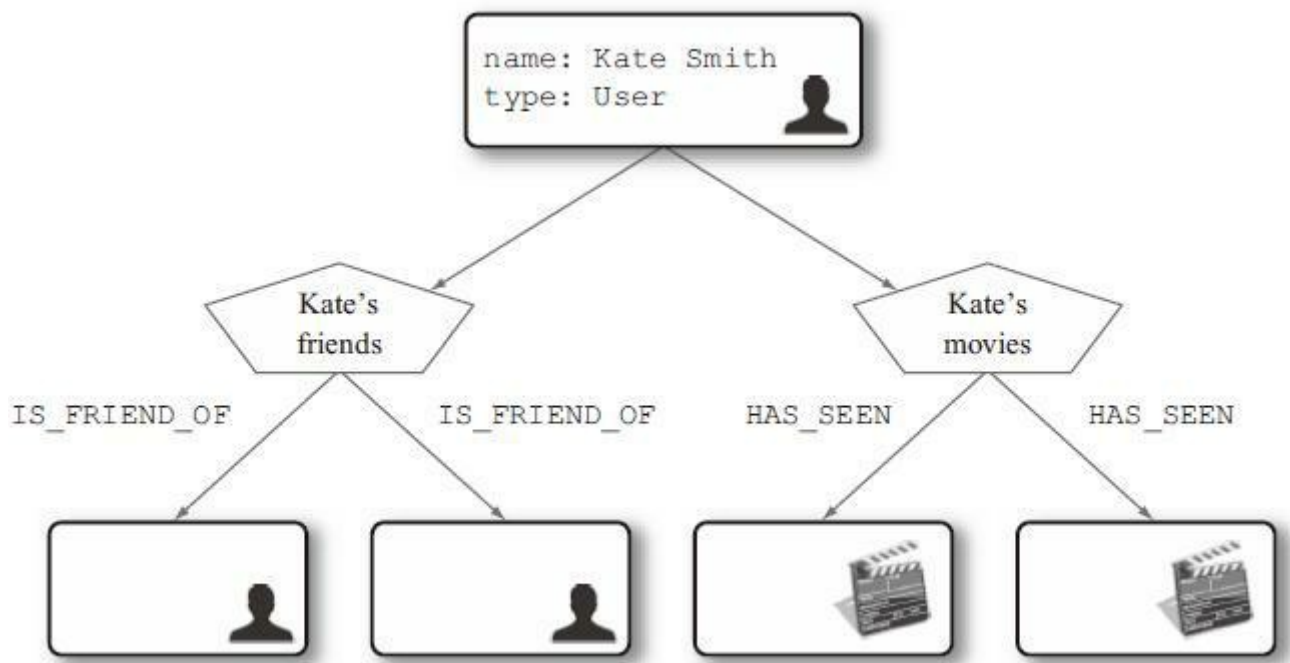


图5-4 用中间节点区分用户和电影节点的社交网络图

当涉及索引时，主要的权衡点是索引越多，所需的磁盘空间越大，当插入和更新图形数据库时对性能的影响也越大，需要写的管理创建和更新索引的代码也就越多。当涉及查询时，如果索引策略能够减少大量的图形，则能增强性能。

5.6.1 索引查询的性能优势

图5-5显示了简单搜索图形中的所有用户节点与使用索引的明显不同。图中给出的是随机查找一个用户节点的平均时间。

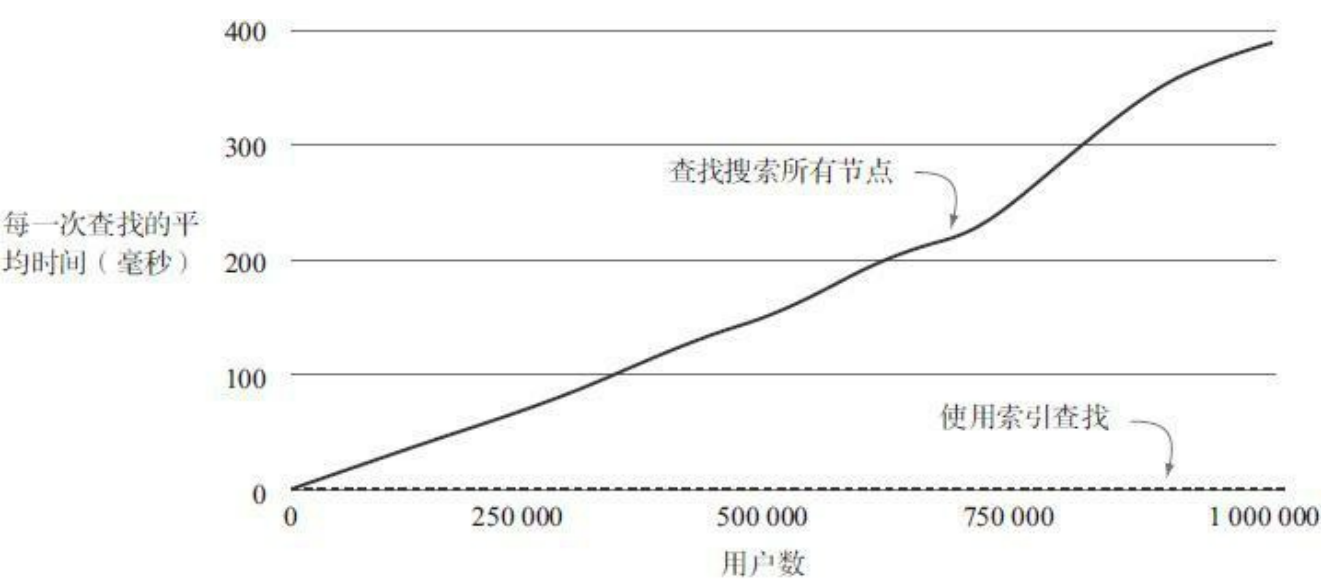


图5-5 使用索引与对所有节点迭代查找节点的性能比较

实线表示的是找一个节点消耗的平均时间，如果简单搜索所有节点，消耗的时间与图形中的节点数量成正比。这并不令人吃惊，节点越多，在图形中碰上要找的用户节点的搜索就越多。

在明显的对比中，虚线表示的是使用对邮箱地址索引查找用户节点消耗的时间。当这个数据稍微变化时，消耗的时间变化很小，每次搜索的平均变化是0.1~0.3毫秒。当测试达到100万用户时，搜索所有节点平均比使用索引

查找用户慢上千倍。

查找标准程序的注意点

注意，将Neo4j编号存到外部用作查找是一个不好的操作，因为在有些情况下这些编号很可能发生变化。应该把节点编号看作为Neo4j内部实现的细节，并不应该为了快速访问而存储。

另一种得到图形数据库中节点的方法是通过`getAllNodes`方法，可以返回一个可迭代量（`Iterable`），用这个量迭代图形数据库中的所有节点。对小数据集，这是一种切实可行的方法，但是每次加载图形数据库的大量数据查找需要的一个特殊节点会导致性能的降低。

测试使用了`getAllNodes`访问所有用户节点。在实际的应用程序中，这种方法将会导致性能比测试时还低，因为实际应用中会遇到大量的不是USER类型的节点。

就像我们一直说的，围绕索引的决策都是一种权衡。下一节中将会看到，查找提高的性能很容易被索引的性能影响而抵消。

5.6.2 当更新和插入数据时索引对性能的影响

通过索引查找一个用户几乎需要一个固定的时间，可能你会感到不解，面对额外的正在做的索引维护，到底哪方面原因造成了性能的降低。

图5-6显示了随着数据集中用户数量的增加创建用户节点所用的平均时间。所用平均时间并不随着用户数量的增加而增加很多，然而，创建一个节

点并索引邮件地址几乎用了仅仅单独创建这个节点的两倍时间。

在这种情况下，即使对上百万的用户，额外的百分之一毫秒可能没有什么区别，因为当这个用户注册时就会创建这个用户并经常进行更新，但对所有的应用程序，并不一定总是这种情况。当在创建和更新数据比读的频率高时，应该考虑快速更新和快速查找性能的权衡。这也可能以不同的方式访问图形数据库，因此可能有许多索引，例如，邮政编码、年龄和性别，这当中每一个的插入和更新都会增加性能的开销。

除索引对性能的影响外，每一个索引都需要额外的存储空间，下面我们来简单地看一下。

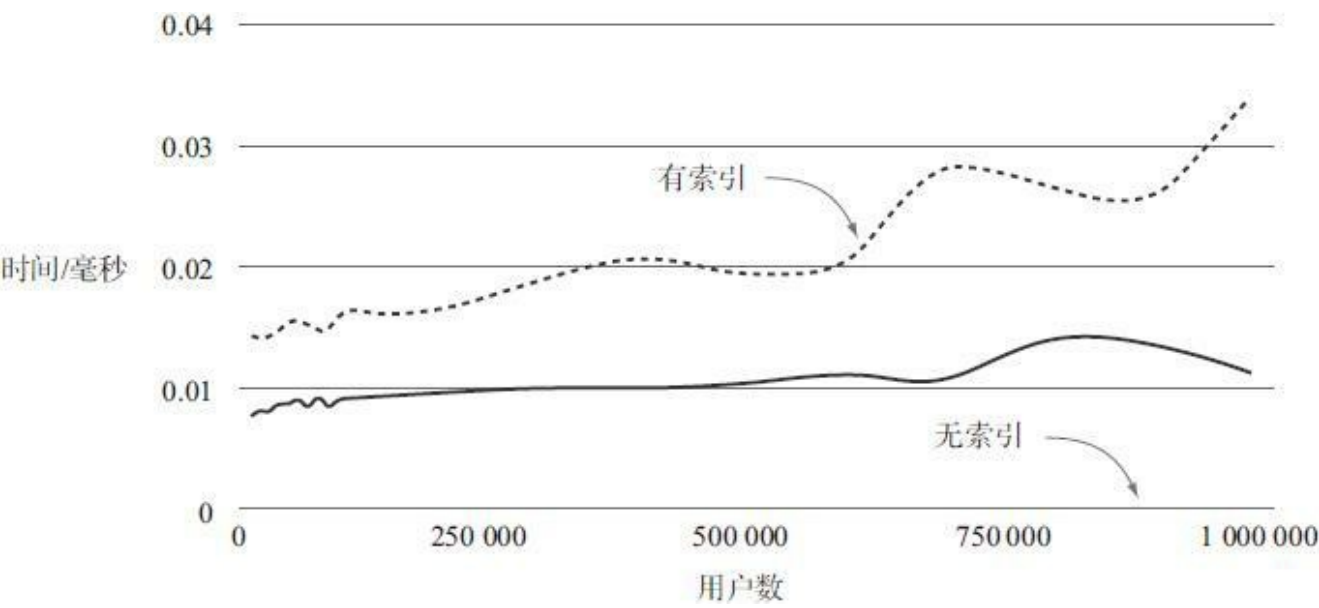


图5-6 有索引和没有索引存储用户节点的平均时间

5.6.3 索引的存储

索引的过程实质上是涉及创建了较小查找表（相对于数据集来讲是比较小的），这可以对图形数据库中的某个位置进行快速访问。这意味着除了写

索引数据而增加的性能开销外，还需要更多的磁盘空间用于存储所有的这些数据。

5.7 本章小结

在本章中，我们看到Neo4j提供了灵活的机制可以使Neo4j应用程序快速找到对图形数据库感兴趣的部分。但是，随着灵活性的增加也带来了权衡问题，要考虑对什么应该索引和对什么不应该索引。

到这里结束了本书的第一部分，在这一部分中，我们介绍了处理Neo4j图形数据库的所有基本概念和技术。在第二部分，我们将深入探讨Neo4j的一些关键概念，首先我们将学习Neo4j的人类图形查询语言——Cypher！

第二部分 Neo4j应用开发

·第6章 Neo4j的查询语言Cypher

·第7章 事务

·第8章 深度遍历

·第9章 Spring Data Neo4j

在第二部分，我们将讨论使用Neo4j建立应用程序的相关问题。

第6章介绍查询语言Cypher——人类易懂的图形查询语言。Cypher提供了不依赖于语言的查询和处理图形的方法，这章讨论如何使用查询语言Cypher与图形领域模型进行交互。由于Neo4j是一个完全ACID兼容的数据库，第7章重点是事务以及如何控制和利用这一特点确保你的应用查询和存储数据的一致性。第8章返回到遍历API，更深入地探讨一些高级选项，通过这种专用的API，在图形数据库中导航。

第9章介绍Spring Data Neo4j（SDN），即一个图形对象映射库，对有些应用情况，如处理一个非常丰富领域模型（rich domain model），可加快速度和简化开发。

第6章 Neo4j的查询语言Cypher

本章包括以下内容：

- 介绍查询语言Cypher
- 执行Cypher查询
- 写只读Cypher查询
- 使用Cypher处理图形数据库

在前几章中，我们使用Neo4j核心Java API处理和查询图形数据库。这明显需要一些Java技术来写和理解程序代码。另外，随着查询复杂程度的增加，程序代码的长度和复杂程度都会增加，在有些情况下变得很难理解，这时候查询语言就派上了用场。查询语言的语言表达结构很强大，并且简单易读。

Neo4j的查询语言称作Cypher。在本章中，我们将解释Cypher的性质，演示其图形操作的基本语法并包括一些日常开发中有用的高级特性及Neo4j数据库的维护。

6.1 Cypher简介

Cypher是对图形的声明查询语言，使用图形模式匹配作为主要的机制作图形数据选择（包括只读和变更操作）。Cypher的声明模式匹配性质意味着可以通过描述想从它那里得到什么查询图形数据。

为了解释Cypher的工作原理，让我们看看实际程序中的Cypher是如何应用的。

6.1.1 Cypher入门

图6-1是一个社交网络的简单图形表示。

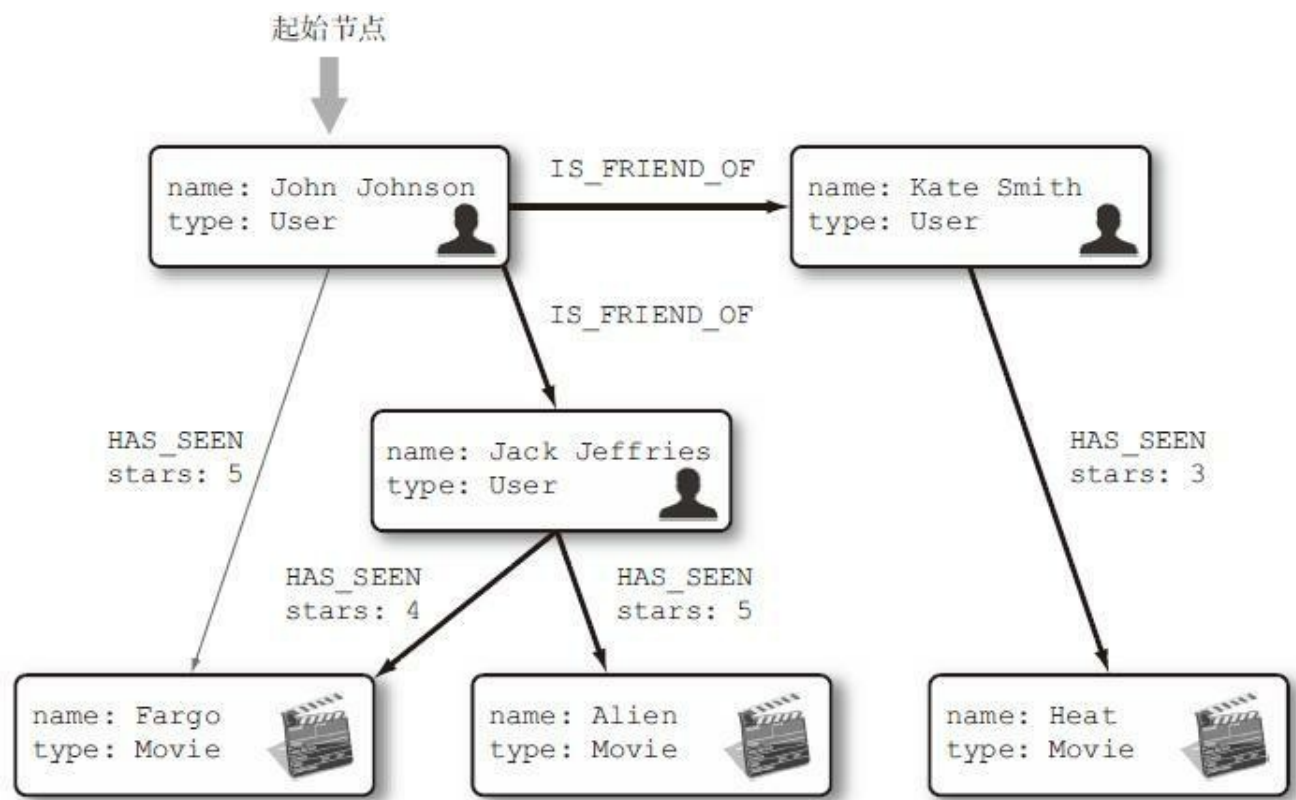


图6-1 要查询的一个社交网络图形数据库

在第4章中，我们讨论了Neo4j遍历API，演示了怎样使用Java代码查找一个用户看过的所有电影。作为提醒，程序6-1给出了用过的Java代码。

【程序6-1】 使用Java API遍历图形查找用户看过的所有电影

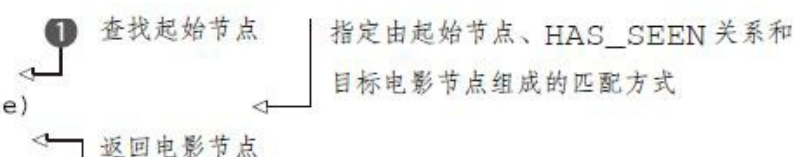
```
try (Transaction tx = graphDb.beginTx()) {
    Node userJohn = graphDb.getNodeById(JOHN_JOHNSON_NODE_ID);
    Iterable<Relationship> allRelationships = userJohn.getRelationships();
    Set<Node> moviesForJohn = new HashSet<Node>();
    for(Relationship r : allRelationships){
        if(r.getType().name().equalsIgnoreCase("HAS_SEEN")){
            Node movieNode = r.getEndNode();
            moviesForJohn.add(movieNode);
        }
    }
    for(Node movie : moviesForJohn){
        logger.info("User has seen movie: " + movie.getProperty("name"));
    }
    tx.success();
}
```

遍历有三个阶段：

- 1) 通过节点编号查找一个起始节点。
- 2) 从起始节点跟随HAS_SEEN关系遍历图形。
- 3) 返回HAS_SEEN关系最终的节点。

现在让我们看看如何使用Cypher编写同样的查询代码，同样遵循遍历的三个阶段。

```
start user=node(1)
match (user)-[:HAS_SEEN]->(movie)
return movie;
```



① 查找起始节点

指定由起始节点、HAS_SEEN关系和目标电影节点组成的匹配方式

返回电影节点

与Java代码一样，首先需要使用节点编号查找起始节点，这个可以使用Cypher的start和node关键词获得（①）。注意起始节点的名字（user），在

以后同样的Cypher查询中可以引用。

下一步，设置需要输出结果的匹配模式。模式就是对需要检查数据库中的子图形的描述，它是由关系相互连接的一些节点构成。一个关系连接的两个节点是典型的图形模式，它是用 `()-[]-()` 描述的。节点用圆括号 `()` 指定，关系使用方括号 `[]` 指定。节点和关系使用连字符连接。在 `start` 句子中的节点（前面例子中的 `user`）用作确定模式的左边界。整个模式有两个已知元素（左边界节点和关系），将用作查找所有与右边界（在我们的例子中是 `movie` 节点）匹配的所有节点。

最后，要查找所有匹配的电影，因此，使用 `return` 关键字返回匹配的电影（再一次使用前面介绍的用名字引用节点）。使用冒号标志Cypher查询的结束。

当运行这个Cypher查询时，将会看到与使用相应的Java API方法完全一样的输出结果。Cypher非常简单易读，开发者、业务人员和任何具有足够域知识的人都很容易理解。



注意

从Neo4j 2.0版本起，`start`语句是可选的。如果省略`start`语句，Neo4j将会试图在`match`查询语句中从节点标签和属性推断`start`语句。为了清晰和与早期版本的Neo4j兼容，本章的所有查询都有显式定义的`start`语句。关于更多的推断`start`语句，请访问<http://docs.neo4j.org/chunked/stable/query-start.html>。

那么，究竟应该如何执行Cypher查询呢？别着急，我们将在下一节作出解释。

6.1.2 执行Cypher查询

有很多种方式可以执行Cypher查询，与Neo4j一起发布的几个工具都支持Cypher执行，并且Cypher也可以从Java代码执行，与SQL非常相似。表6-1给出了Cypher执行选项。

表6-1 执行Cypher查询的工具和技术

工 具	描 述
Neo4j Shell 命令	命令行工具
Neo4j 网页管理控制台	基于网页的接口
Java	程序执行
REST	在 HTTP 上使用 REST 接口

下面详细介绍前三个选项，第四个选项使用REST执行Cypher查询将在第10章中讨论。

1.使用Neo4j Shell命令执行Cypher查询

Neo4j Shell命令是一个命令行工具，它是随Neo4j一起发布的，可以使用它与以下任何一个连接。

- 本地Neo4j数据库——通过指定shell到Neo4j数据库存储的目录连接。
- 通过RMI连接远程Neo4j服务器——通过给shell提供主机名字和连接端口连接。

Neo4j Shell命令开始脚本语言在\$NEO4j_HOME/bin/目录，\$NEO4J_HOME是Neo4j二进制代码安装的目录。

表6-2给出了基于Linux和Windows系统如何连接本地和远程Neo4j的实

例。

表6-2 Linux和Windows环境下的Neo4j Shell命令开始脚本语言句法

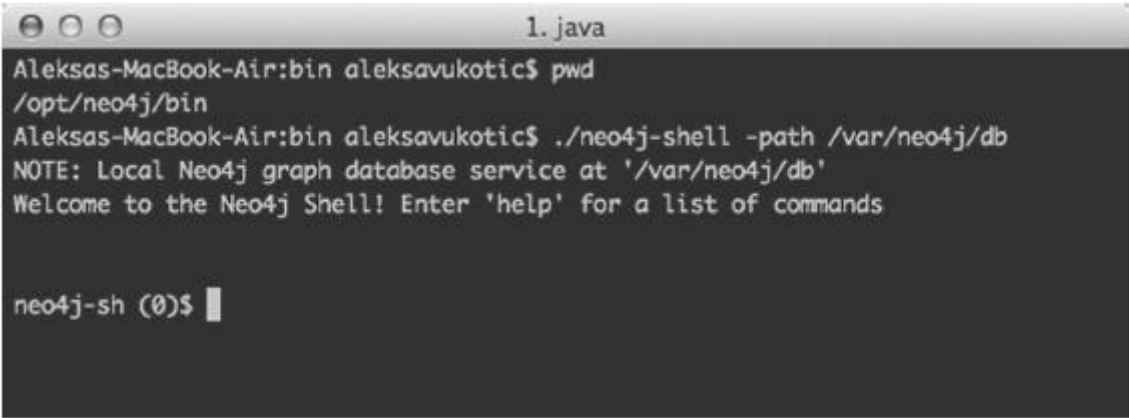
数据库类型	Linux 命令	Windows 命令
本地 Neo4j 数据库	<code>\$NEO4J_HOME/bin/neo4j-shell -path=/var/neo4j/db</code>	<code>\$NEO4J_HOME/bin/neo4j-shell.bat -path=/var/neo4j/db</code>
远程 Neo4j 数据库	<code>\$NEO4J_HOME/bin/neo4j-shell -host=localhost -port=1337</code>	<code>\$NEO4J_HOME/bin/neo4j-shell.bat -host=localhost -port=1337</code>

在开始运行查询前，需要填充图形数据库。对于这个例子，我们将使用第3章与第4章例子（电影爱好者社交网络）中的数据集。使用的数据库在硬盘上的位置是/var/neo4j/db，因为我们使用的是Linux操作系统，但是可以基于所使用的操作系统更换目录（例如，Windows操作系统c: \neo4j\db）

使用Neo4j Shell在\$NEO4J_HOME/bin目录中运行以下的程序连接Neo4j数据库。

- neo4j-shell-path=/var/neo4j/db（Linux）。
- neo4j-shell.bat-path=/var/neo4j/db（Windows）。

按回车键后，将进入Neo4j Shell命令状态，如图6-2所示，准备接受命令。



```
1. java
Aleksas-MacBook-Air:bin aleksavukotic$ pwd
/opt/neo4j/bin
Aleksas-MacBook-Air:bin aleksavukotic$ ./neo4j-shell -path /var/neo4j/db
NOTE: Local Neo4j graph database service at '/var/neo4j/db'
Welcome to the Neo4j Shell! Enter 'help' for a list of commands

neo4j-sh (0)$
```

图6-2 Neo4j Shell准备好接受命令



当开始shell时，要确保对Neo4j数据目录有读的权限。

Neo4j Shell本身支持Cypher，因此不需要任何特殊的命令，仅仅在控制台上输入Cypher查询并按回车键即可。结果以表格形式输出，非常类似于在关系数据库中运行SQL查询的输出。图6-3给出了Cypher查询一个指定用户看过的所有电影的输出结果。

```
neo4j-sh (0)$ start user=node(1)
> match (user)-[:HAS_SEEN]->(movie)
> return movie;
+-----+
| movie |
+-----+
| Node[4]{name:"Fargo",type:"Movie"} |
+-----+
1 row
51 ms

neo4j-sh (0)$
```

图6-3 在Neo4j Shell下执行Cypher查询返回了表格输出

正如所看到的，结果正是基于图6-1的图形数据库所期望的结果。用户1（John Johnson）确实只看过一部电影：Fargo。在查询中返回的是movie节点，因此结果就是这样。Fargo是数据库中的一个节点，内部编号是4，两个属性：name和type，这些值也正是所期望的值，这也证实了查询结果的正确性。

这就是我们期待的第一个Cypher查询，通过简单描述感兴趣的子图形得到了期望的结果，而没有写任何的Java代码，也不用担心事务的操作。

2.使用网页管理控制台执行Cypher查询

Neo4j服务器自带一个操作工具：网页管理控制台，即一个基于浏览器的对Neo4j实例多样的网页接口。它有许多特性，可以查询、处理、可视化Neo4j图形数据库和管理Lucene索引、维护和监控Neo4j设置。更详细的网页管理控制台将在第11章介绍。

要访问网页管理控制台，需要运行Neo4j服务器设置指定数据库的位置，例如，前一个例子中是/var/neo4j/db。按照以下的步骤：

1) 下载正确版本的Neo4j，按照本书附录A中的详细说明安装Neo4j服务器。

2) 编辑位于\$NEO4J_HOME/conf/neo4j-server.properties的Neo4j主服务器配置文件，定位下列的行：

```
org.neo4j.server.database.location=data/graph.db
```

替换数据目录并使其指向上一节中创建的图形数据库：

```
org.neo4j.server.database.location=/var/neo4j/db
```

3) 保存配置文件。

4) 使用以下命令打开Neo4j服务器：

```
$NEO4J_HOME/bin/neo4j start
```

随着服务器的加载和运行（可能需要几秒钟的时间），通过浏览器打开[http://localhost: 7474](http://localhost:7474)，就可以访问网页管理控制台。网页管理控制台的主页

如图6-4所示。

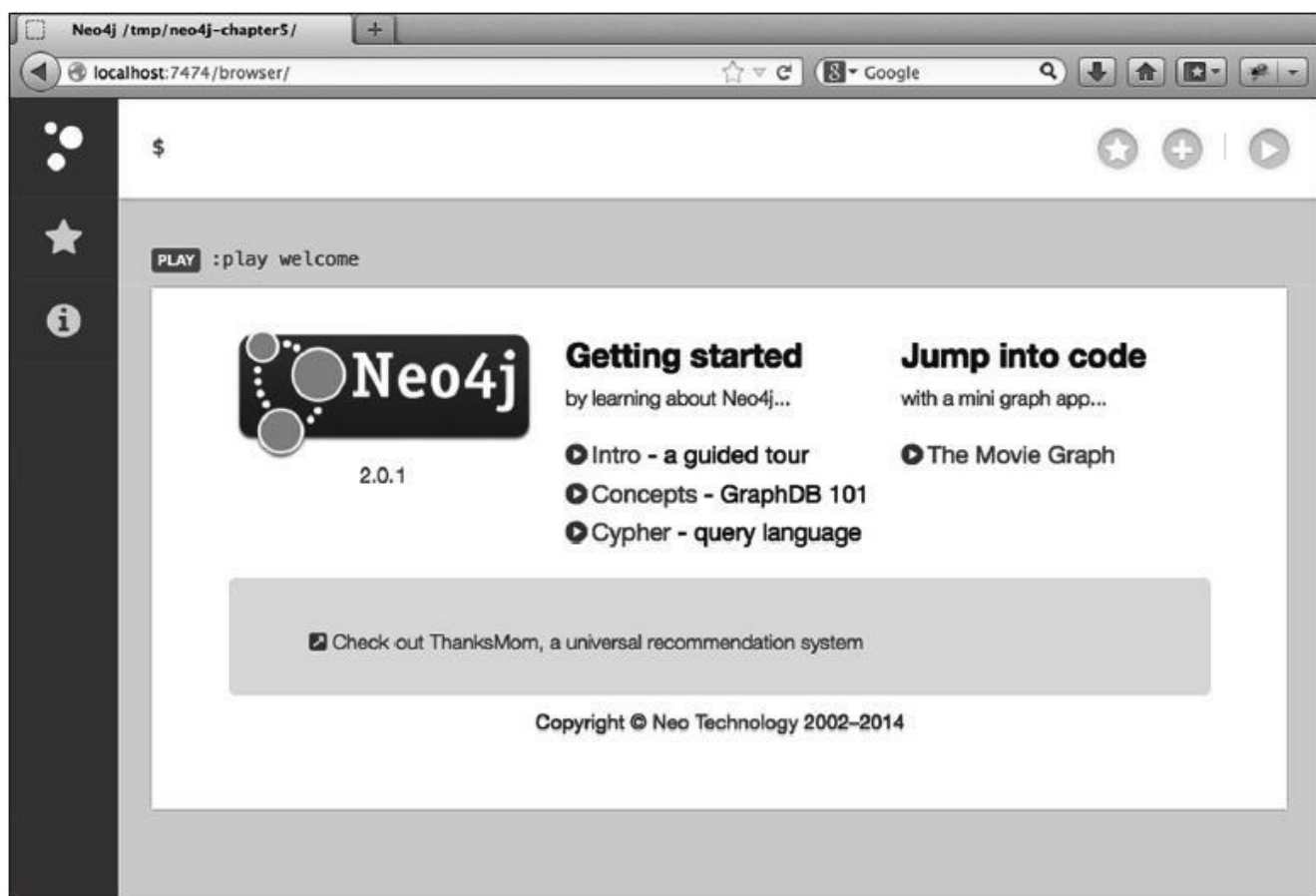


图6-4 浏览器中的网页管理控制台主页

网页管理控制台在浏览器（使用一些杰出的JavaScript脚本语言）内置了一个完全功能的Cypher查询执行引擎。可以在页面顶部的文本字段里输入Cypher查询，然后按回车键，将会在浏览器窗口中看到查询结果。图6-5给出了一个在网页管理控制台执行Cypher查询的例子。

不用命令行访问运行中的Neo4j服务器实例，在网页管理控制台内运行Cypher查询是非常方便的。



注意

Neo4j网页管理控制台还有另外一个非常优秀的特点——可以在浏览器

中可视化Cypher查询的结果。要切换表格和可视化结果，可以使用浏览器右下角的两个按钮。

到目前为止，这里描述的是手工运行Cypher查询，使用的工具是随Neo4j一起发布的。我们说过不写Java代码也可以非常容易地查询图形数据库。但是若把Cypher查询集成到Java的应用程序中将会怎样？Neo4j核心Java API包含一个Cypher API，使得从Java代码中运行Cypher查询非常简单。

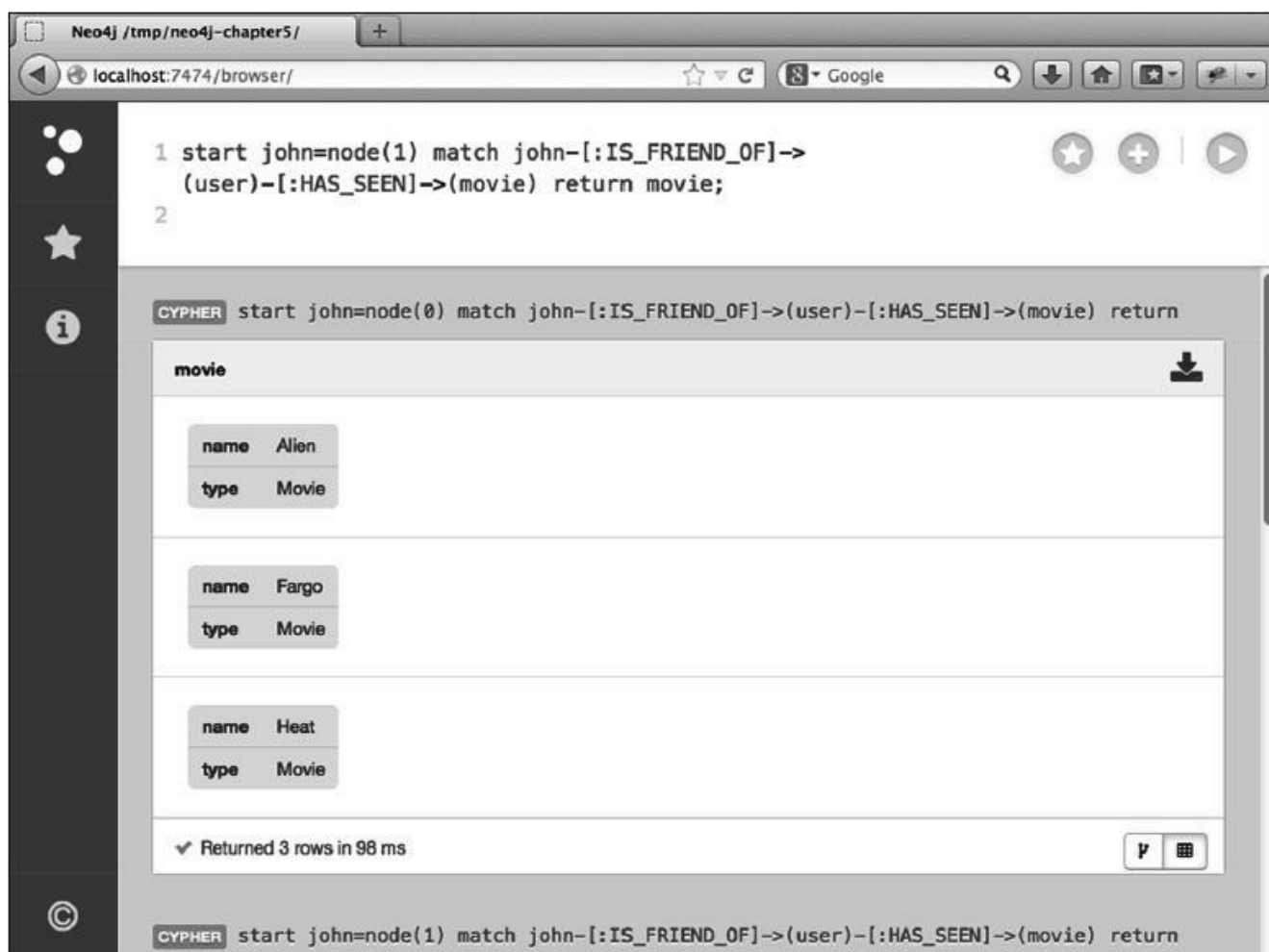


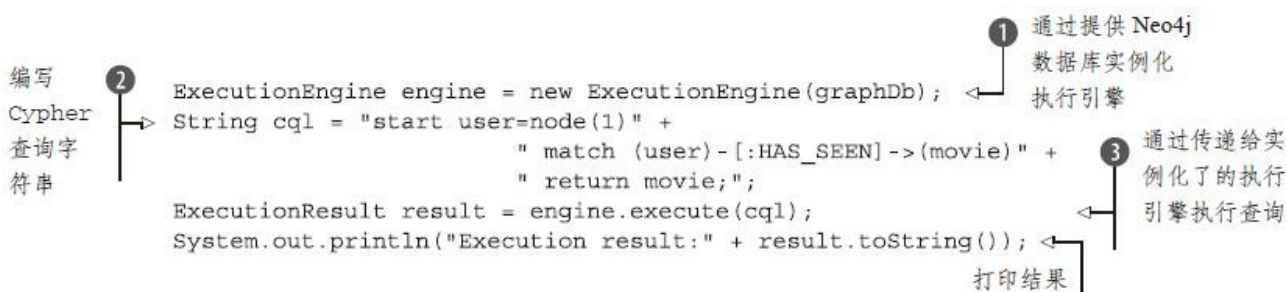
图6-5 在网页管理控制台中执行一个Cypher查询

3.用Java代码执行Cypher查询

在Java中使用JDBC驱动程序和JDBC API很容易查询关系数据库。获得连接，创建和执行声明，通过结果集迭代，立刻就会得到JDBC数据库查

询，在Java中执行Cypher查询显然更为简单。

以下代码演示了在如图6-5的网页管理控制台中执行的同一个样本Cypher查询，用以查找一个特定用户看过的所有电影。



可以看到，对Cypher查询，需要做的是实例化Neo4j的执行引擎（ExecutionEngine）（❶）并且给它传递有效的Cypher查询（❷、❸）。ExecutionResult.toString（）方法与Neo4j Shell的输出格式相同，这对调试程序和排除问题非常有用。

在Java中通过Cypher查询结果迭代，所需要做的就是对execute（...）方法返回的ExecutionResult对象进行迭代：

```
ExecutionResult result = engine.execute(cql);
for(Map<String,Object> row : result){
    System.out.println("Row:" + row);
}
```

每一个迭代元素都包含着一个结果行，代表一个Java映射，这里的主键是列的名字，值就是实际的结果值。

前面的小段程序逐行迭代了Cypher查询结果。另一个选项是通过单个的列迭代结果，当不是对返回的所有列都感兴趣时非常有用，就像下列的程序演示的那样：


```

List<String> columns = result.columns();
for(String column : columns){
    Iterator<Object> columnValues = result.columnAs(column);
    while(columnValues.hasNext()){
        System.out.println("Value:" + columnValues.next());
    }
}

```

从结果表中得到所有列的名字

对指定的列定位

通过结果值迭代

Cypher和Scala 尽管核心Neo4j图形引擎是用Java实现的，但是Neo4j Cypher引擎是用Scala实现的。Scala是一种流行的、混合的功能编程语言，运行于Java虚拟机上。关于更多的Scala知识，请参阅www.scala-lang.org/。

Cypher的替代者

Cypher并不是唯一的图形查询语言，另一个常用的图形查询语言是Gremlin，Neo4j也支持Gremlin。Gremlin实际上是一组定义如何遍历和处理图形的接口，这些接口是通过每一个与之兼容的图形引擎实现的。这意味着理论上与Gremlin相同的查询可以在任何与之兼容的图形数据库上使用，使得代码可移植。Gremlin是一个开源库，并且是流行的图形库Tinkerpop栈的一部分（<https://github.com/tinkerpop/gremlin>）。

Cypher和Gremlin的主要区别是它们的性质——Gremlin是命令式语言，用户需要描述如何遍历图形，Cypher是声明性语言，它使用模式匹配描述需要哪一部分图形数据。这就使得Cypher易读且易于理解。Gremlin可以在不同的图形数据库中移植，而Cypher是由Neo4j团队开发，并仅应用于Neo4j图形数据库。

Gremlin的句法与使用超出了本书的讨论范围，但是可以在以下网站查阅更多的资料：<https://github.com/tinkerpop/gremlin>。

现在知道了使用基于网页的、基于命令行应用的和基于使用Java API的

执行Cypher查询，下面开始使用Cypher查询。

6.2 Cypher的基本句法

Cypher句法由四个不同的部分组成，每一部分都有一个特殊的规则：

- start**——查找图形中的起始节点。
- match**——匹配图形模式，可以定位感兴趣数据的子图形。
- where**——基于某些标准过滤数据。
- return**——返回感兴趣的结果。

Cypher的模式匹配性质使得图形模式成为任何查询的重点问题。这就是为什么我们要首先讨论Cypher的图形模式匹配技术。

6.2.1 模式匹配

模式匹配是Cypher的重要方面，它提供了强大的功能以人可读的句法描述子图形结构。以下的程序演示了以单个命名的关系连接的两个节点的模式匹配：

```
start user=node(1)
match (user)-[:HAS_SEEN]->(movie)
return movie;
```

当描述关系时，在方括号里面的冒号（:）后面指定关系的类型。当建立关系（句法上大小写敏感）时，类型必须与它定义的类型严格一致。这个简单的查询使用[:HAS_SEEN]句法描述了单个HAS_SEEN关系。

关系方向的描述是在Cypher中用ASCII实现的。关系的连接是用ASCII箭头（单个连字符接着一个大于号或前面有一个小于号[]->或<-[]）连接着它的端点。关系的起点使用单个连字符连接（[]-）。在前面的例子中，匹配用户看过的电影（`match (user) -[: HAS_SEEN]-> (movie)`）模式指定了从user节点指向movie节点的HAS_SEEN关系。

如果不在意关系的方向，或者不清楚哪一个是关系的起始节点或终止节点，则可以在关系的两端都使用单个连字符连接，在这种情况下解读为“任意”。如果想知道两个用户是否是朋友，忽略掉方向，可以使用下面的模式。

```
start user1=node(1)
match (user1) -[:IS_FRIEND_OF] - (user2)
return user2;
```

现在让我们看看在Cypher中如何使用节点和关系标识。

1.使用节点和关系标识

在Cypher查询中，节点和关系都可以与标识关联，这种关联使得以后可以在同样的查询中引用同一个图形实体。下面的例子在返回语句中引用了movie节点。

关系也可以命名，使用稍微不同的语法：

```
start user=node(1)
match (user) -[r:HAS_SEEN]->(movie)
return movie;
```



要命名一个关系，在关系类型之前，简单地要为要命名的关系在方括号里面指定名字即可。

当创建复杂的模式时，命名节点和关系是非常有用的，因为在以后的Cypher查询的其他地方可以引用它们。但是，如果以后不需要引用图形实体，避免使用标识将会使查询可读和易于理解。

在前面的例子中，节点是被命名的（`user`是起始节点而`movie`是查询需要返回的节点），但是关系没被命名，因为没有任何地方引用它。要使关系匿名，只需在方括号内保留冒号后面的关系类型，就像在原始的查询中做得一样：

```
start user=node(1)
match (user)-[:HAS_SEEN]->(movie)
return movie;
```



类似地，节点也可以匿名。要使节点匿名，使用空的圆括号（`()`）指定节点。为了演示这种方法，我们编写另一个Cypher查询，这里想要从`user`节点返回所有的HAS_SEEN关系，而不必担心`movie`节点：

```
start user=node(1)
match (user)-[r:HAS_SEEN]->()
return r;
```



使用空的圆括号描述一个期望的节点，但是不在意其任何属性，以后也不想引用它，因此不给它命名。



注意

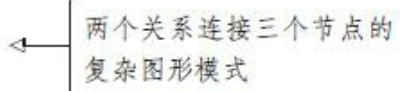
在Cypher查询的`match`语句中可以不使用圆括号指定命名节点，例如`start user=node(1) match user-[:HAS_SEEN]->movie return movie`。在复杂的查询中，如果可能，也可以去掉圆括号，以简化查询且使之可读。匿名节点必须有圆括号。

到目前看到的所有模式都非常简单，即由单一的关系连接的两个节点。下面让我们看看如何使用Cypher匹配更复杂的模式。

2. 复杂模式匹配

Cypher查询支持相当复杂的模式匹配。让我们看看另一个例子：查找用户1的朋友看过的所有电影。在第4章中已经使用Neo4j遍历API解决了这个问题，因此，现在看看如何使用Cypher查询解决这个问题：

```
start john=node(1)
match john-[:IS_FRIEND_OF]->()-[:HAS_SEEN]->(movie)
return movie;
```



从已知的名字为john（节点编号是1）开始，匹配IS_FRIEND_OF关系指向的另一个节点（匿名），依次具有HAS_SEEN关系指向另一个movie节点。

没有必要命名所有的关系以及紧接着的节点。所有需要返回的是代表John的朋友已经看过的电影节点。

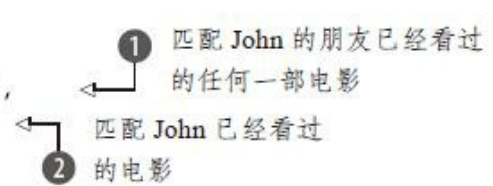
运行这个查询将会返回John的朋友已经看过的所有电影节点，不管John是否看过：

```
| Node[5] {name->"Alien",type->"Movie"} |
| Node[4] {name->"Fargo",type->"Movie"} |
| Node[6] {name->"Heat",type->"Movie"} |
```

如果想要这个社交网络基于John的朋友的喜好为John推荐电影，那么应该跳过他自己已经看过的电影。要得到想要的结果，在前一个例子的结果中使用匹配John已经看过的电影模式进行过滤。使用Cypher查询，在同一个查询中，如果需要，可以在两个模式中引用相同的节点，匹配多个模式。

看看以下程序的查询：

```
start john=node(1)
match
john-[:IS_FRIEND_OF]->()-[:HAS_SEEN]->(movie),
john-[:HAS_SEEN]->(movie)
return movie;
```



Cypher查询中的多个模式使用逗号（，）作为分隔符分开。这里的第一个模式是前一个例子中使用的模式——匹配John的朋友已经看过的电影（❶）。第二个模式匹配John已经看过的所有电影（❷）。



注意

作为结果的节点必须匹配所有逗号隔开的模式，相当于一个“并”（AND）语句。

这里感兴趣的位是使用同一个节点作为两个模式中的锚点（两个模式都从同一个john节点开始，都匹配同一个movie节点）。

如果运行这个查询，将会得到单一的结果：Fargo。

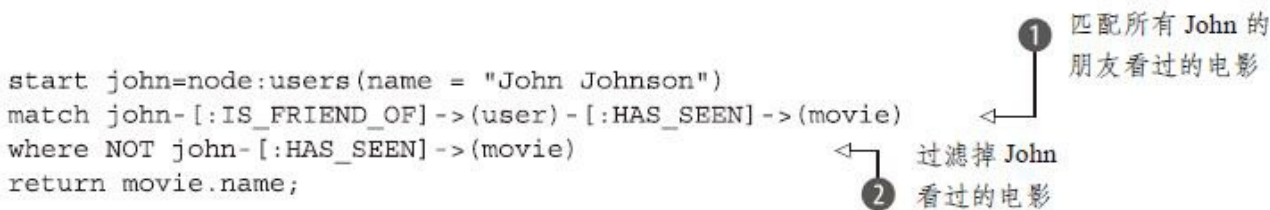
```
| Node[4] {name->"Fargo",type->"Movie"} |
```

很明显这并不是所要的结果。匹配了所有的John的朋友已经看过的电影（结果是三部电影），然后匹配那些John已经看过的，结果是John和John的一些朋友看过的。

尽管这个可以成为一个有用的查询（如果要回答John和John的朋友共同看过的电影这个问题），但在这里不是我们想要的答案。

需要用匹配不存在的关系：John没有看过的电影，替换第二个模式。下

面的程序演示了如何使用NOT语句句法查找John的朋友看过，但是John没有看过的电影。



第一个模式与以前的完全一样——匹配John的朋友看过的电影（**1**）。第二个模式，where语句的一部分，过滤第一个模式返回的电影，通过HAS_SEEN关系有选择的连接（**2**）。

注意

在本例的where语句（**2**）中，在结果中过滤掉不感兴趣的内容，与关系数据库中的SQL语句非常相似。本章后面将详细讨论where语句。

最后的结果包括两部电影（Heat和Alien），这也是期望的结果，图6-1给出了数据集。图6-6显示了在Neo4j Shell中运行查询的结果。

```
neo4j-sh (?)$ start john=node:users(name = "John Johnson")
> match john-[:IS_FRIEND_OF]->(user)-[:HAS_SEEN]->(movie)
> where NOT john-[:HAS_SEEN]->(movie)
> return movie.name;
+-----+
| movie.name |
+-----+
| "Alien"    |
| "Heat"     |
+-----+
2 rows
293 ms
neo4j-sh (?)$
```

图6-6 查找基于他们的朋友看过的电影为用户推荐的电影在Neo4j Shell执行查询的结果

现在已经使用Cypher通过基于用户对电影的评级找到了推荐的电影，但是不包括用户已经看过的电影。在第4章中使用Java API对同一个问题的解决方案需要编写40多行程序。这次用5行的Cypher查询就解决了问题。Cypher查询也更有可读性，并且对任何了解该领域应用的人来说更易于理解，即使他们没有任何编程技术。



在本章中，我们讨论了Cypher的基本概念和句法，还有一些Cypher查询的高级应用。关于Cypher查询的全部特性可以参阅Neo4j手册中扩展在线文件<http://docs.neo4j.org/chunked/stable/cypher-query-lang.html>。

现在让我们对Cypher查询做一个更深入的剖析。

6.2.2 查找起始节点

Neo4j图形数据库的查询遵从标准的模式：查找起始节点并且运行遍历。到目前为止，在所有的遍历中，我们一直使用Neo4j核心API、遍历API和Cypher遵从这个模式查询。可以通过特殊的节点编号直接查询以定位起始节点，或者使用Lucene索引查找，像在第5章中学习的那样。首先让我们回顾一下通过编号直接查找。

1.通过编号查找节点

就Cypher查询而言，在Cypher查询的开始，在关键字start之后就指定了起始节点。在前面的例子中，到目前为止，我们一直通过直接编号查找加载起始节点，如下面的程序所示。

```
start john=node(1)
return john;
```

在 Cypher 查询中通过编号使用
① 直接节点查找得到起始节点

通过在node关键字后面的圆括号中简单指定编号加载节点（①）。

如果有一个起始节点，这就是可行的。但是，有时需要对多个起始节点使用同样的模式并得到所有的匹配结果——例如，如果想要查找John或者Jack看过的电影。

2.通过多个节点编号加载多个节点

在start语句中使用多个编号，需要列出以逗号隔开的编号参数值，如下面的程序所示。

```
start user=node(1, 3)
match user-[:HAS_SEEN]->movie
return distinct movie
```

① 使用节点编号 1 和 3 开始
匹配
返回找到的电影，不包括重复的
匹配①中指定用户看过的所有电影

start语句中使用单个标识符指定了两个节点编号：user（①）。下一步是模式匹配查找用户看过的所有电影。user标识符引用了节点1或节点3，因此该模式匹配了用户1（这里是John）或者用户3（Jack）看过的电影。输出的结果如下：

```
|Node[5]{name->"Alien",type->"Movie"}|
|Node[4]{name->"Fargo",type->"Movie"}|
```



注意

最后一个查询的return语句中distinct关键字用以删除重复的结果。distinct的作用与SQL查询相同——在结果集中排除重复的结果。Jack和John

都看了Fargo，因此，如果不使用关键词distinct，则Fargo将在结果中出现两次。

但是，大部分情况下，我们并不知道感兴趣节点的节点编号——可能知道一些其他的属性，如用户的名字、电影的名字或其他一些相似的东西。在下一节，我们将学习如何使用这些信息加载起始节点。

3.使用索引查找起始节点

我们在第5章介绍了索引在遍历中是作为推荐的方法查找起始节点。Cypher也支持直接访问Lucene索引，因此，Neo4j中常用的遍历术语便成为：

- 1) 使用Lucene索引查找一个或多个节点。
- 2) 从步骤1中找到的节点开始做遍历。

如何在Cypher中使用这种模式？Cypher查询通过使用模式匹配工作，因此，常用的用法如下：

- 1) 使用Lucene索引查找一个或多个节点。
- 2) 通过将从步骤1中找到的节点附加到模式中做模式匹配。
- 3) 返回感兴趣的实体。

现在唯一的问题是如何在Cypher查询的start语句中做Lucene索引查找，而不是通过编号加载节点。这非常简单，下面的程序做出了演示。

```
start john=node:users(name = "John Johnson")
return john
```

← 在用户索引中查找具有名字属性“John Johnson”的节点

现在的start语句句法稍微有一点不同。`node: users`部分指定了一个感兴趣的来自users索引的节点。当给索引添加节点时，索引的名字必须匹配使用的索引名字。圆括号中的参数指定了在索引中查找的键——值对；在这种情况下，匹配的是name属性与值“John Johnson”。如果这样的索引入口在users索引中找到，那个节点将从Neo4j数据库中加载并在Cypher查询中使用。

这种索引查找句法需要正好键-值对的匹配，并且是大小写敏感的，因此具有name属性“john johnson”或者“John Johnso”将不会返回。同时，如果多个节点以同一键-值对（如果在数据库中有多于一个的John Johnson）索引，所有匹配的节点都要加载；这与前面具有多个节点编号的例子非常相似。

如果使用Lucene查询的完全功能查找起始节点，可以使用与本地Lucene原始查询稍微不同的句法。

```
start john=node:users("name:John Johnson")
return john
```

← 使用本地 Lucene 查询索引查找节点

这个例子与前面介绍的例子完全一样，唯一的区别是索引查找的参数。这一次所有的索引参数都传递到双引号中，并且包含一个本地Lucene查询。这允许利用强大的Lucene查询结构，例如，通配符或多属性匹配。



注意

关于本地Lucene查询功能的文档可以在Lucene网站上找到：http://lucene.apache.org/core/3_6_0/queryparsersyntax.html。

这里是在使用Cypher查询中的本地Lucene查询。

```
start john=node:users("name:John* AND yearOfBirth<1980") return john
```

从 John 开始以名字属性且出生日期小于 1980 年进行索引查找所有节点（匹配 John Johnson、John Edwards 等）



注意

第5章介绍的自动索引也能够以同样的方法用作Cypher查询的一部分。需要做的是对索引名字使用节点自动索引（node_auto_index），而不是前面查询中的对users使用节点自动索引。

下一步，让我们看一看如何对标签使用基于模式的索引在Cypher查询中查找起始节点。

4.使用基于模式的索引查找起始节点

在前面的几章中，我们讨论了如何通过给节点指定标签确定Neo4j节点类型。我们也提到如何使用标签作为基于模式的、内置索引查找图形。也可以使用同一个索引在Cypher中查找节点。

让我们看一看如何使用标签通过名字查找用户节点。

```
match (john:USER)
where john.name='John Johnson'
return john
```

- 1 使用基于模式的索引结合用户标签查找 John 节点
- 2 在 where 语句中指定索引查找查询——通过名字查找用户

指定使用模式索引的标签被定义为节点标识符的一个部分，用冒号分隔（**1**）。必须在圆括号内放一个与标识符相关的标签。索引查询被指定为where语句的一个部分，与在SQL所做的非常相似（**2**）。



注意

基于标签的索引只能用于完全属性值的查找。对全文的搜索，必须使用手动方法创建索引，就像前一节讲的那样。

现在，我们知道了如何在Cypher查询中的start语句使用索引查找节点。但是，在执行Cypher查询时，如果要在图形模式中确定多个节点，并要求对它们分别匹配时应如何做？如果要查找John和Jack都看过的所有电影应该怎么做？使用Cypher通过指定多个不同的起始节点作为图形数据库入口点也是很容易实现的。

5.Cypher查询中的多个起始节点

要指定图形数据库的多个入口点，可以在Cypher查询的start语句中使用逗号分隔列表，程序如下所示。



起始节点以逗号分隔列表出现，每一个节点有自己的标识符，这个标识符可以在整个查询中使用（❶）。进行匹配时，所有的起始节点可以用作单个模式的一部分，或者可以像在本例中用作独立的模式（❷）。本例中的两个模式都是movie节点（在两个模式中具有同样的名字，因此代表同一个节点），这也是Cypher查询的结果（❸）。

6.2.3 过滤数据

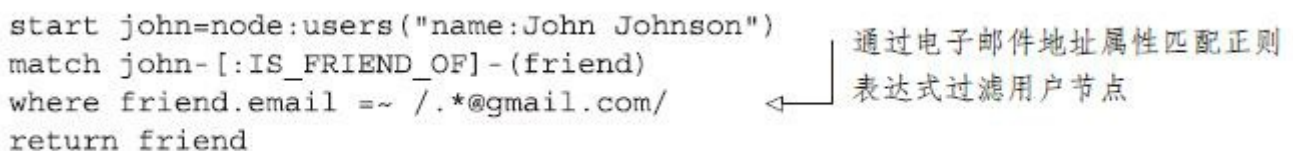
使用Cypher查询只靠起始节点和模式有时并不足以获得需要的结果。在一些情况下，需要对节点和关系再做一些过滤以决定返回哪些结果。与在关系数据库中的SQL查询相同，Cypher查询过滤使用where语句实现。

一般情况下，where语句基于节点或关系的一些属性过滤结果。让我们看一个例子，查找用户John出生于1980年后的所有朋友。我们假设每一个用户节点都有一个出生年（yearOfBirth）的整型属性存储用户的出生年份。具体程序如下所示。



通过索引查找寻找起始节点（①），并用IS_FRIEND_OF关系模式匹配朋友节点，而不考虑方向（②）。下一步，过滤匹配了的朋友节点，找出出生年份大于1980的朋友节点（③）。因为出生年份具有整型属性，因此可以在where语句中做标准的数字比较。

对于字符串属性，除标准的等于号（=）比较外，还可以使用正则表达式过滤掉指定的值。例如，要查找所有邮箱地址具有“gmail.com”的朋友，可以运行下面的查询。



如上所示，Cypher中的正则表达式比较符是等于波浪号（=~）。正则表达式放在两个正斜杠之间（/）。表达式本身遵从标准的Java正则表达式句法（参看<http://docs.oracle.com/javase/tutorial/essential/regex/for more information>）。

但是，如果所有的用户或节点没有存储邮箱地址，则将发生什么情况？就像我们以前讨论的那样，Neo4j允许半结构化、无机制的数据结构，因此没有强制两个节点必须有相同的属性。本书例子的数据集有些user节点具有email属性，但并不是都有。让我们看看如何使用Cypher查询并不是所有节点都具有的属性。

要过滤具有给定属性的节点而不管属性的值（例如，具有Twitter账号的所有朋友），则需要使用Cypher的has函数。

```
start john=node:users("name:John Johnson")
match john-[IS_FRIEND_OF]-friend
where has(friend.twitter)
return friend
```

使用 has 函数过滤具有 twitter 属性的节点

正如所看到的，在Cypher中过滤结果并不困难，看上去与SQL的where语句非常相似。

在下节中，我们将看看Cypher查询中能返回的不同类型的结果。

6.2.4 获得结果

Cypher查询的结果用return语句返回。到目前为止，我们已经使用它返回了感兴趣的节点，但是return语句不仅仅能返回节点——也可以返回关系、节点和关系的属性，甚至子图形的整个路径。



注意

Neo4j的路径代表通过关系连接的节点集，例如，`node1-[: relationship1]-node2-[: relationship2]-node3`。

1.返回属性

在前面的例子中，使用Cypher查找John的朋友已经看过但John没有看过的所有电影，并且返回了匹配的movie节点。如果使用这些数据在网页上显示推荐的电影名字，则应该返回具有所有属性的整个movie节点——比需要的数据还多。更好的方法是仅仅从查询中返回感兴趣的node属性。

下面的程序做出了演示：

```
start john= node:users(name = "John Johnson")
match
john-[:IS_FRIEND_OF]->(user)-[:HAS_SEEN]->(movie)
where not john-[:HAS_SEEN]->(movie)
return movie.name;
```

从查询中仅返回名字属性

例如，如果从Neo4j Shell运行该程序，程序的输出结果将比以前看到的简单得多。

```
| "Alien" |
| "Heat"  |
```



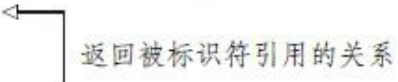
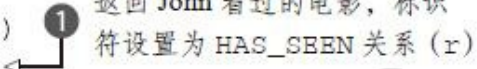
注意

如果引用的属性并不是结果集中的所有实体都具有，那对不具备引用属性的实体将返回null。

2.返回关系

有时查询图形数据库中的某些关系是合理的。假设有显示一个用户对看过电影的评级，评级是HAS_SEEN关系的一个属性（实际属性名是stars，代表给予电影星的数量），因此需要返回这些关系。下面的程序做出了演示。

```
start john=node:users("name:John Johnson")
match john-[r:HAS_SEEN]-(movie)
return r
```



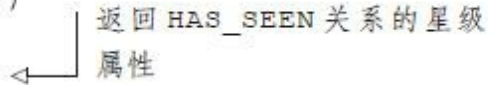
查询通过从索引查找用户John Johnson开始并匹配所有他看过的movie节点（❶）。然后返回例子中的HAS_SEEN关系。注意要返回一个关系，需要有一个标识符（这里是r）。

这个查询的输出结果如下所示：

```
| :HAS_SEEN[2] {stars->5} |
```

如果只对关系属性的星级感兴趣，可以仅仅从查询中返回那个属性。就像节点属性一样，关系属性可以在return语句中引用。下列的程序演示了这一用法。

```
start john=node:users("name:John Johnson")
match john-[r:HAS_SEEN]-(movie)
return r.stars
```



这一次不是整个关系实体，而是仅仅返回了感兴趣的stars属性。

3.返回路径

除节点和属性外，还可以从Cypher查询返回整个路径。例如，当要基于John的朋友看过的电影推荐电影时，可能对每一部电影是如何被推荐的感兴趣，这样可以从开始的用户节点，通过所有的朋友到推荐的电影，返回整个

路径。下面的程序对这一方法做了演示：

```
start john=node:users(name = "John Johnson")
match
recPath = john-[:IS_FRIEND_OF]->(user)-[:HAS_SEEN]->(movie)
where not john-[r:HAS_SEEN]->(movie)
return movie.name, recPath;
```

① 给路径以标识符
recPath, 在查询
的任何地方都可以
使用

← 返回引用的路径及其
相关的电影名字

要返回路径作为查询的结果，将需要给它一个标识符并能对它做出引用（①）。那么，在return语句中通过指定标识符就可以返回引用的路径。

返回的路径输出将包含属于它的所有节点和关系：

```
| "Alien"
| [
|   Node[1]{
|     name->"John Johnson",
|     year_of_birth->1982,
|     type->"User"
|   },
```

```

:IS_FRIEND_OF[1] {},
Node[3]{
  name->"Jack Jeffries",
  cars_owned->[Ljava.lang.String;@3927ff0d,type->"User"
},
:HAS_SEEN[5] {stars->5},
Node[5]{
  name->"Alien",type->"Movie"
}
] |
| "Heat"
| [
  Node[1]{
    name->"John Johnson",
    year_of_birth->1982,
    type->"User"
  },
  :IS_FRIEND_OF[0] {},
  Node[2]{
    name->"Kate Smith",
    locked->true,type->"User"
  },
  :HAS_SEEN[3] {stars->3},
  Node[6]{name->"Heat",type->"Movie"}
] |

```



注意

在前面的例子中，从Cypher查询返回了一个node属性和一个路径。这里讨论的所有可返回的图形实体可以被任意组合指定。

当在屏幕上显示或在纸上打印数据时，通常并不想一次显示所有的结果，而是把它们分为数页以便更为可读或可用。下面将要介绍如何对Cypher查询的结果分页。

4. 分页结果

结果中包含很多实体，可能需要将结果分成几页以显示在网页上。要对Cypher查询结果进行分页，Neo4j有三个简单明了的语句。

·**order**——在分页之前，对结果进行排序，因此分页返回的结果是一致的，无论是往前还是往后分页。

·**skip**——划分结果集以便跳到指定的页。

·**limit**——以页面尺寸限制返回结果的数量。

假设数据库中的一个用户是一个电影迷，他看过并评级了上百部电影，如果想在一页上显示这个用户评价的所有电影（给定的星级），则显示的字型将小的无法阅读（或者根本容纳不下）。

可以确定以电影名字排序，每一网页只显示10部电影。要查询图形数据库得到这样分页的结果，需要使用**order**、**limit**和**skip**语句。下面的查询返回了第三页（21~30项）。

```
start john=node:users("name:John Johnson")
match john-[:HAS_SEEN]->(movie)
return movie
order by movie.name
skip 20
limit 10
```

① 通过电影名字对结果排序

② 跳过最前面的两页 (2 × 10 = 20 项)

返回 10 个结果 (一页的大小)

就像我们讨论过的，匹配了这个用户看过的电影后，通过**name**属性对结果进行排序（①）。因为对结果的第三页感兴趣，所以跳过前两页共20项（②）。最后，限制返回的结果集为页面的大小（本例子中是10）。



注意

Neo4j在线手册包括了一份方便打印的Cypher参考卡，卡中有常用句法和详细的查询：<http://neo4j.org/resources/cypher>。

在简单的查询中，可以看到Cypher查询的性能与Neo4j遍历API或Java API大体相同，但是当运行更复杂的查询时（具有复杂的模式匹配或密集的过滤），Cypher查询的性能不如Neo4j遍历API或Java API的性能。在有些情况下，差距会有一个数量级或更大。

由于Cypher非常年轻，它的开发者首先着重于添加需求的功能，性能并没有优先考虑。然而，到了Neo4j 2.0，Cypher绑定到了Neo4j引擎比遍历API低的栈中。这将使得细微调整Cypher性能以好于遍历API成为可能。一旦这个特性组成为稳定的，那么Cypher解析器的性能和执行引擎的调节可能将会产生更好的结果。

鉴于其简单性和描述性，对于大部分的图形查询，Cypher是最好的选项。但是对非常复杂的遍历或插入一个大的数据集，可能要使用核心Java API和遍历API了。我们在第3、第4和第5章讨论了核心Java API的基础知识。对更深入的遍历API的讨论，参见第8章。在未来版本的Neo4j中，可以期待Cypher将成为最有效和最快的与Neo4j交互的方法。

现在已经介绍完了Cypher句法，我们学会了如何在Cypher中进行模式匹配，如何过滤结果，如何不仅返回节点还直接返回图形实体，如何排序和限制返回的结果集等。

到目前为止我们的讨论一直与从Neo4j读数据相关。在下一节中，我们将讲解如何使用Cypher创建、更新和删除图形数据。

6.3 用Cypher更新图形数据

从Neo4j 1.7开始，就可以使用Cypher对图形数据库运行变更操作如创建、更新、删除节点与关系以及它们的属性。

变更操作的句法是非常直观的，因此让我们看几个例子。



注意

Cypher图形变更操作是一个Neo4j数据库相对新的特性，因此，变更的句法在未来发布的版本中可能会有较大的变化。本书是基于Neo4j 2.0版的Neo4j句法。建议读者参考最新版本的句法手册

(<http://docs.neo4j.org/chunked/stable/query-write.html>)。

让我们从创建节点和关系开始。

6.3.1 创建新图形实体

要创建一个具有属性的节点，可以使用以下的句法：

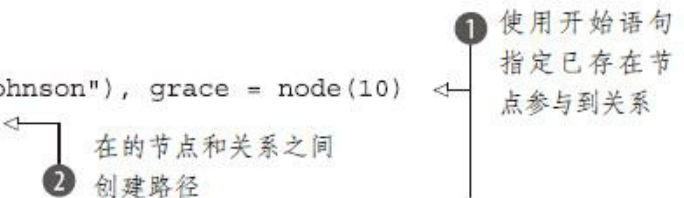


创建（create）图形实体命令，并不令人吃惊，就是create跟着节点标识

符 (❶)。紧接着, 指定一个用冒号分隔的属性列表加到节点上, 用花括号括着 (❷)。每一个属性用它的名字指定, 跟着一个冒号和一个属性值 (❸)。字符串属性值以单引号括着, 但是数值和布尔属性不需要任何引号 (❹)。最后, 使用它的标识符返回新创建的节点, 就像在前面的Cypher例子一样。

现在已经创建了一个节点, 那么如何创建一个关系呢? 让我们使新创建的用户Grace成为John的朋友。

```
start john = node:users(name = "John Johnson"), grace = node(10)
create john-[r:IS_FRIEND_OF]->grace
return r;
```



要想在已存在的两个节点之间创建一个关系, 需要首先引用它们。这个操作与在只读查询中指定起始节点完全一样, 使用start语句和节点查询 (❶)。下一步, 在create语句中, 简单地指定图形路径作为模式通过它们的标识符引用起始节点并且设置在它们之间想要创建的关系 (❷)。

Cypher和事务

当从命令行或网页界面运行Cypher时, 每一个查询执行都包含于一个事务中。不必担心事务的开始和完成或处理它们的失败——所有的事务都由Neo4j引擎负责。如果查询成功, 则事务就会自动提交, 如果查询失败, 事务就会回滚。

就应用程序来说, Cypher查询将会与已经存在的事务一起执行。如果由程序创建事务, 则可以在同一个事务中使用多个查询, 并在事务的最后提交

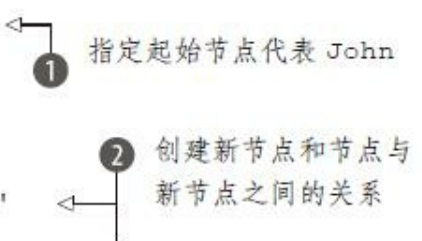
或回滚每一个事项。

因为在本章中我们正在使用Neo4j Shell或网页管理控制台运行Cypher查询的例子，每一个查询自动包裹到了一个单一的事物中，因此不需要额外的设置。

关于Neo4j事物的更深层次信息，请阅读第7章。

到目前为止，已经创建了一个具有一些属性的节点和一个关系。但是可以用一个命令完成所有的这一切，就像下面程序演示的一样：

```
start john = node:users(name = "John Johnson")
create john
  -[r:IS_FRIEND_OF]->
    (grace {
      name: 'Grace Spencer',
      yearOfBirth: 1982, email: 'grace@mycompany.com'
    })
return r, grace;
```



在本例中，固定一个已经存在的节点john（❶）。在create语句中，指定了一个通过不存在的IS_FRIEND_OF关系从john到一个不存在的节点grace的模式，同以前一样，具有相同的属性（❷）。Cypher聪明地知道哪个节点和关系是固定的（start语句中的john），哪个是新的并且应该在数据库中创建（IS_FRIEND_OF关系和grace节点）。这是一个具有强大功能的，可以创建具有属性的新节点和关系的整个路径的方法。

Cypher create语句将创建匹配模式中的所有元素，而不管它们是否存在。如果运行前面的节点程序，创建了John的朋友Grace Spencer两次，将得到两个Grace Spencer节点和两个从John节点的IS_FRIEND_OF关系。这可能不是想要的结果。要只创建不存在的图形实体，应该使用创建唯一的（create unique）Cypher命令，下面的程序作了演示。

```

start john = node:users(name = "John Johnson")
create unique john
    -[r:IS_FRIEND_OF]->
        (grace {
            name: 'Grace Spencer',
            yearOfBirth: 1982, email: 'grace@mycompany.com'
        })
return r, grace;

```

对前面例子的唯一改变是使用create unique命令。多次运行这段程序将不会产生实体数的乘积——将总是有一个Grace Spencer节点和一个在John和Grace节点之间的IS_FRIEND_OF关系。

现在知道了如何创建新节点、新关系和新属性，但是，怎样删除它们呢？下面将会讨论从Neo4j使用Cypher删除数据。

6.3.2 删除数据

要删除节点，Cypher提供了一个delete命令。让我们删除刚刚创建的用户节点Grace。

```

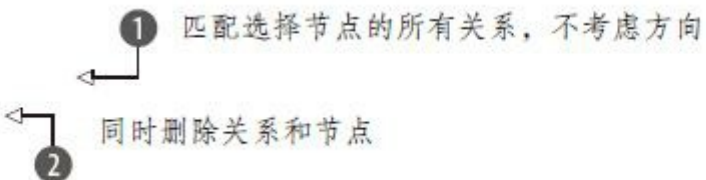
start grace = node(10)
delete grace

```



但是，运行这个命令将会出错，警告节点具有关系。在Neo4j中，只能删除没有任何关系的节点（无论是进还是出的关系）。要明确的是删除一个节点，同时也会删除它的关系，如下面的程序所示：

```
start grace = node(10)
match grace-[r]-()
delete grace, r
```



这里使用了标准模式匹配`grace`节点的进或出的所有关系（❶），并且删除了指定节点的开始或结束的所有关系（❷）。

`delete`查询的输出不包含任何数据，但是将指出删除的节点和关系数：

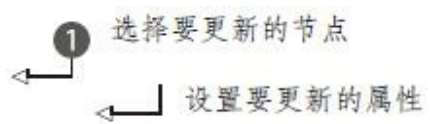
```
Nodes deleted: 1
Relationships deleted: 1
```

最后，让我们看看如何使用Cypher更新已存在节点的属性。

6.3.3 更新节点和关系属性

假设John的出生日期有错误，需要对其进行更新。完成这一工作的命令如下：

```
start john=node:users(name = "John Johnson")
set john.yearOfBirth = 1973
```



可以看到句法非常简单：首先选择要更新的节点（❶），然后使用`set`命令设置选择节点的属性并给予新值。如果要设置的属性并不存在，将会创建这一属性。

可以选择多个节点添加同一属性：

```
start user=node(1,2)
set user.group = 'ADMINISTRATOR'
```

Neo4j不允许图形实体具有空（Null）属性值。事实上，空属性值认为是不存在的属性。因此，如果想要删除一个属性，将使用delete命令。下面的程序做出了演示：

```
start n=node(1)
delete n.group;
```

Cypher中的图形变更操作是在本书的写作时Neo4j的一个相对较新的特性。我们已经演示了一些有用的例子帮助理解。关于更多和最新涉及Cypher句法的信息，可以在下面网页查阅Neo4j手册：<http://docs.neo4j.org/chunked/stable/cypher-query-lang.html>。

Cypher中的模式索引

在第5章中，我们介绍了模式索引的概念，并且改进了Neo4j自动索引的特性，那里有Neo4j引擎完全维护索引作为节点生命周期的一个部分，也学会了如何从Java代码使用模式索引，此外，也可以从Cypher做模式索引操作。

模式索引所需的Cypher句法的详细描述超出了本书的范围，但是可以参考Neo4j手册获得详细的信息（<http://docs.neo4j.org/chunked/stable/query-schema-index.html>）。

注意Neo4j的变更操作不支持手动创建索引的更新。如果需要对新创建的节点使用自定义（不是模式索引）索引，则必须使用Neo4j核心Java API、语言绑定或者REST API。

到目前为止，有关Cypher我们已经讨论了很多，但是，还有更多。下一节，我们将探讨一些有关Cypher更好的东西。

6.4 高级Cypher

目前我们已经学会编写相当复杂的Cypher查询，但是Cypher功能远远不止这些。在本节中，我们将讨论一些Cypher中的高级功能，例如数据聚合、Cypher函数和链式多个查询等。让我们从数据聚合开始吧。

6.4.1 聚合

就像在SQL中的GROUP BY一样，Cypher支持查询中的聚合函数。不是使用一个专用GROUP BY语句，Cypher中的分组关键词被定义为所有的查询非聚合结果。下面的程序展示如何计算图形数据库中每一个用户的朋友数：

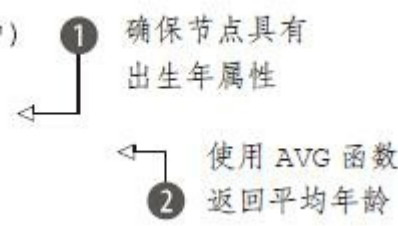


在本例中查找了整个图形数据库，因此，所有节点都作为起始节点（①）。对每一个user节点，通过跟随一级IS_FRIEND_OF关系匹配他们的直接朋友（②）。return语句包含一个非聚合条目（user节点）和一个聚合函数（count）（③）。这意味着在使用user节点分组结果，使每一个用户的结果为一行。也可以在order by语句中使用聚合值排序整个结果集（④）。

除count函数外，Cypher支持SQL的所有常用的聚合函数：SUM的数值求和、AVG计算平均值到MAX和MIN寻找数值属性中的最大值和最小值。

要求John所有朋友的平均年龄，可以使用以下查询：

```
start john=node:users(name = "John Johnson")
match john-[:IS_FRIEND_OF]-(friend)
where HAS(friend.yearOfBirth)
return avg(2014-friend.yearOfBirth);
```



1 确保节点具有出生年属性

2 使用 AVG 函数返回平均年龄

在用于数学计算之前，这里检查了这个节点具有一个出生年的属性设置（1）。然后返回计算的平均值（2）。



注意

要得到用户的平均年龄，用现在的年份减去出生的年份。在本例中，我们使用了本书的出版年。

现在让我们看一下Cypher的内置函数。

6.4.2 函数

Cypher支持多项函数可用于在查询中计算表达式，让我们看看其中的一些。

Cypher函数用于访问图形实体的内部属性，例如节点和关系的编号和节点标签（类型）。在计数的例子中，已经看到了用ID（node）函数获取指定节点的内部编号。另外，也可以使用TYPE（relationship）函数查找一个关系的类型。

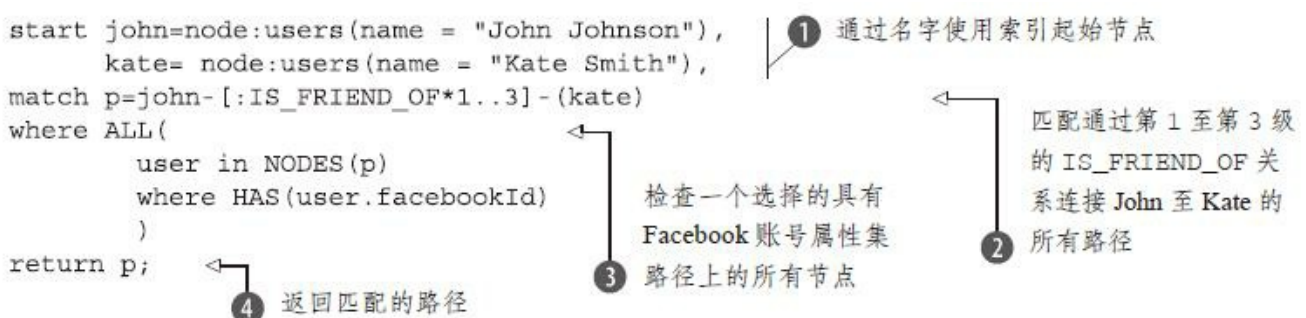
为了演示TYPE函数的使用，让我们对下面的问题寻找答案：“每一类型有多少个关系开始于或结束于用户John？”，下面的程序给出了实现这一功

能的Cypher查询:



本例中的前两步非常熟悉, 使用索引查找表查找起始节点 (❶), 并且使用match语句匹配查找所有的邻居节点 (❷)。在以后的查询中使用标识符rel引用每一个关系。最后, 使用TYPE函数返回每一个关系的类型与其数量 (❸)。由于TYPE是一个非聚合函数, 结果将会以关系类型分组。

Cypher也支持一些这样的函数, 可以在查询内以可迭代集的方式方便地查询。作为一个例子, 让我们解决下面的问题: John希望在Facebook上介绍给Kate。他不认识Kate, 但是想找出谁是Kate的朋友, 朋友的朋友认识她, 直至第三层的朋友。因为他需要使用Facebook, 所以他仅仅对Facebook上的人 (即有Facebook Id属性设置的人) 感兴趣。下面的程序给出了需要的查询:



通过在Neo4j数据库中定位知道的代表John和Kate的起始节点开始查询 (❶)。在match语句中, 将会找到通过IS_FRIEND_OF关系连接John和Kate的所有路径, 直到关系的第3级 (❷)。在后面的查询中使用标识符p引用匹配了的路径。

智能位来自于where语句 (3)。使用NODES (p) Cypher函数提取给定路径上的所有节点集。使用HAS函数检查每一个节点是否有facebookId属性。然后使用ALL函数对节点集中的每一个元素使用HAS函数。如果在给定的可迭代集中每一个元素匹配了HAS函数的定义，ALL函数将会返回true，因此，如果在路径p上的一个节点没有facebookId属性，这条路径将被放弃。

最终，返回了符合所有标准的路径 (4)。这些路径包含所有John将要联系的人，因此他可以在Facebook上被推荐给Kate。

在本例中，函数的使用如下，这只是很少的几个函数：

- HAS (graphEntity.propertyName) ——如果一个节点或关系具有给定名字的属性存在，则返回true。

- NODES (path) ——把一个路径转换成一个可迭代的节点集。

- ALL (x in collection where predicate (x)) ——如果collection中的每一个单个元素匹配了给定的predicate，则返回true。

Neo4j支持许多具有相似目的的函数。例如，与NODES (path) 函数类似、RELATIONSHIPS (path) 函数返回在一个给定路径上的所有节点集。

除ALL函数外，Neo4j也支持其他一些基于谓词表述的布尔函数：

- NONE (x in collection where predicate (x)) ——如果提供的集合中没有元素匹配谓词表述，返回true；否则，返回false。

- ANY (x in collection where predicate (x)) ——如果至少有一个元素匹配谓词表述，返回true；如果没有匹配的，返回false。

·SINGLE (x in collection where predicate (x)) ——如果正好有一个元素匹配谓词表述，返回true；如果没有或多于一个匹配的，这个函数返回false。



本书没有足够的篇幅详细讨论所有的Neo4j函数。相关的更多信息，请参阅Neo4j手册 (<http://docs.neo4j.org/chunked/stable/query-function.html>)。

6.4.3 with语句的管道功能

在Cypher中，可以将一个查询的输出链接到另一个查询中，从而创建功能强大的图形结构。Cypher中的链（或管道）语句是with。

为了演示它的用法，让我们在前面例子的基础上来演示，在前面的例子中使用Cypher统计从john节点的每一类关系的数量。让我们添加一个需求，仅包含出现多于一次的关系：如果John已经看了两部电影，应该包含HAS_SEEN关系，但是如果他只看了一部电影（或者一部也没看），这些关系不应该包含在内。

问题涉及在聚合函数上的过滤。在SQL中，在聚合函数上过滤是由HAVING语句完成的，但是Cypher并不支持这个。而在Cypher中可以使用with，就像下面的程序演示的一样。

```

start n=node(1)
match n-[rel]-()
with TYPE(rel) as type, count(*) as count
where count > 1
return type, count;

```

① 不是返回聚合计数，而是使用 with 语句管道输送计数

② 链接另一个 where 语句管道输送结果

③ 返回匹配了的类型和数量

查询从起始节点以索引查找开始，然后匹配所有与之相连的关系。但是，不是像以前一样返回聚合的结果，而是用 **with** 语句链接它（①）。在 **with** 语句中，重新命名输出，这个输出将用于链接命令（**type** 和 **count**）的输入。链接以后，输出用 **with** 语句定义为新的 **where** 语句的输入（②）。最后，返回匹配的结果。



注意

在被用作链接查询之前命名 **with** 语句中的输出字段是强制性的。

如果 **Cypher** 的这个功能进化得非常快，你是否必须在每次升级到下一个 **Neo4j** 版本时重新编写 **Cypher** 查询代码？下一步我们将讨论如何处理不同版本的 **Cypher** 句法。

6.4.4 Cypher的兼容性

我们已经提到 **Cypher** 是一种发展非常快的语言，它的句法经常发生变化。为了给 **Cypher** 添加有价值的新功能，开发者有时需要引入重大的更改，这将使得在早期版本的 **Neo4j** 中写的查询在新版本的 **Neo4j** 数据库中运行失败。

幸运的是，有一个简单的设置可以让你在任意版本的 **Neo4j Cypher** 引擎

上运行查询。即可以在运行查询之前指定查询句法符合的Neo4j版本。

```
CYPHER 1.8 start n=node(1)
match n-[rel]-()
with TYPE(rel) as type, count(*) as count
where count > 1
return type, count;
```

在查询开始之前指定确切的
① Cypher 分析器版本

可以在查询的开始，在start语句之前（①）使用CYPHER关键词跟着Neo4j版本指定需要的Neo4j版本。

注意

本章的这个Cypher例子已经在Neo4j 2.0版本通过测试。

除了在所有的查询中使用Cypher分析器版本外，还可以设置全局配置选项cypher_parser_version。这个选项在neo4j的属性文件里设置。关于这个文件在服务器模式和嵌入式模式下的位置，请参阅第10章，我们将在第10章讨论不同的配置选项。

6.5 本章小结

我们现在已经了解了Neo4j的查询语言Cypher，知道如何在图形数据库中执行Cypher查询，如何编写高效的Cypher查询从图形数据库中提取数据，如何使编写出的Cypher简单可读且没有损失太多的性能。图形数据库的处理特性也使得Cypher更加有助于所有的维护Neo4j的工作。

在下一章中，我们将深入研究Neo4j的事务。

第7章 事务

本章包括以下内容：

- 为什么事务是重要的
- Neo4j如何处理事务
- Neo4j如何与其他事务管理系统集成
- 如何利用事务事件的优点

Neo4j与其他一些没有结构化查询（NoSQL）技术的不同点在于，Neo4j是ACID完全兼容的（ACID是Atomic、Consistent、Isolated、Durable的首字母缩写——原子的、一致的、隔离的、持续的）。ACID完全兼容意味着Neo4j与使用传统的关系数据库一样有保证。

当使用Neo4j时，具有ACID保证是很重要的，因为对图形数据库在同一个事务中进行大量的实体（节点、关系和索引实体）进行变更操作是很普遍的。也很容易对系统完全兼容ACID的系统行为做出推理，完全兼容ACID是相对于最终一致系统而言，在最终一致系统里并没有实际担保在什么时间数据的改变能在一连串的操作中可见，相对技术来说，那里没有事务，因此，一个大的商务操作的部分结果可能会暂时可见以与事务竞争。ACID事务在可持续性方面也提供了高自信度，一旦事务提交，数据就不会丢失。

本书的其他章节中也提及事务，本章将对事务做比较详细的探讨。

7.1 事务的基础知识

首先，让我们考察一下在Neo4j中创建事务的方法，如下面的程序所示：

```
try (Transaction tx = graphDatabaseService.beginTx()) {  
    //用数据库做一些事情  
    tx.success();  
}
```

这段代码开头使用Java 7的try-with-resource语句，创建一个新的事务并定义一个本事务将会使用的代码块。任何与数据库交互和在块内执行的语句将会在同一个事务中运行。在try块的末尾对success方法的调用表示当一个事务结束时（或在try-with-resource术语中资源关闭时）应该有一个提交。当代码块结束（或关闭）时事务完成。然后，Neo4j会确保这个事务的提交（已经探测到前面调用了success方法）。如果在对数据库做操作时出现意外的情况，将不调用success方法，当事务结束时将回滚到原来的状态。Neo4j涉及是否提交或回滚的决策逻辑基于前面是否调用了success或failure方法。如果想显式地回滚一个事务，例如从一个条件代码块，可以调用failure方法，则事务将在程序块的结束做无条件的回滚。既不调用success方法也不调用failure方法也将导致事务回滚（默认）。

Neo4j 2.0以前的事务

在老版本的Neo4j（1.9和以前）中，管理事务有一点复杂，主要是因为Java 7以前没有自动关闭资源功能，Neo4j从2.0版本开始具有了该功能。

如果使用老版本的Neo4j，处理事务的方法如下：

```

Transaction tx = graphDatabaseService.beginTx();
try {
    //do something
    tx.success();
} finally {
    tx.finish();
}

```

这种风格是可行的，但是在Neo4j 2.0中不赞成这样使用。

在Neo4j 2.0中，包括节点、关系、模式和索引的图形对象，任何的读或写操作都需要激活一个事务。注意在先前的版本中，并没有对读操作提出这种要求。

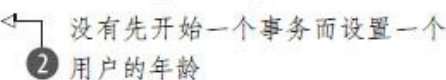
程序7-1给出了首先读一个代表人的节点，然后设置一个年龄属性。

【程序7-1】 没有事务的尝试更新

```

Node userNode = graphDatabaseService.index()
    .forNodes("byName").get("name", "John").getSingle();
userNode.setProperty("age", 34);

```


 ① 查找起始节点，在Neo4j 2.0中将会失败
 ② 没有先开始一个事务而设置一个用户的年龄

注意

本章的例子故意假设了一组非常简单的数据模型，仅仅由2个节点组成，John和Bob，代表设置了年龄属性的两个人。这个数据模型足以演示Neo4j的事务行为。

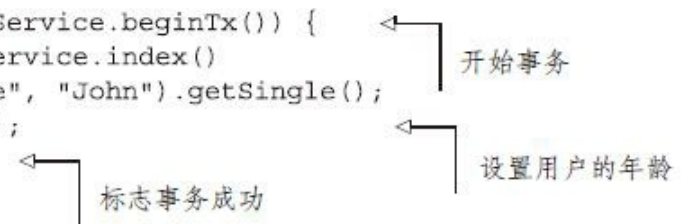
当以Neo4j 2.0以前的版本运行这段代码时，行①将通过，但是行②将会导致一个NotInTransactionException异常。这是因为在Neo4j 1.X版本中的变更操作需要一个事务，而只读操作没有事务。而在Neo4j 2.0及其以后的版本中，①和②都会失败，抛出NotInTransactionException异常。

7.1.1 添加事务

程序7-2来自程序7-1，但是加入了事务管理。本章的后面，我们将假设具有事务的风格以满足Neo4j 2.0的需求。

【程序7-2】 具有事务的更新

```
try(Transaction tx = graphDatabaseService.beginTx()) {  
    Node userNode = graphDatabaseService.index()  
        .forNodes("byName").get("name", "John").getSingle();  
    userNode.setProperty("age", 34);  
    tx.success();  
}
```



这段程序代表了开始一个事务的通常模式，执行一些操作，并假设没有异常地完成。在调用success方法之后代码块完成，数据的变化提交到数据库中。

基于模式的操作和事务

如果在图形数据库中做任意与模式相关的操作，这些代码需要在与所有其他Neo4j事务代码相隔离的一个事务中，否则，将抛出ConstraintViolationException异常并提出如下信息：“在做模式更新的事务中不能做数据更新”。

在第5章中讨论了模式索引，下面的代码是程序5-5的一部分用以演示现在的情况。扼要重述一下，在一个事务中定义USER标签的name作为模式可索引的属性，然后使用分离的事务实际设置一个真实用户的值。

```

Label userLabel = DynamicLabel.label("USER");

// 1. Do schema related work in one transaction.
try (Transaction tx = graphDb.beginTx()) {
    graphDb.schema().indexFor(userLabel).on("name").create();
    tx.success();
}

// 2. Do other work in a separate transaction.
Node user = null;
try (Transaction tx = graphDb.beginTx()) {
    user = graphDb.createNode(userLabel);
    user.setProperty("name", "Michael Collins");
    tx.success();
}
}

```

7.1.2 打好基础，循序渐进

在try块中创建一个事务，就像我们前面所说的一样，确保事务在适当时间完成。事务的状态特性需要对在一个事务中完成的工作量给予考虑。这仅仅针对大的事务，包括大量的插入和更新数据，如程序7-3所示。当执行这段代码时，很可能以出现内存不够的异常而终止，具体取决于Java虚拟机（JVM）的可用内存。

【程序7-3】 一个大事务可能出现内存不够用的情况

```

try(Transaction tx = graphDatabaseService.beginTx()) {
    for (int i = 0; i < 1000000000; i++) {
        Node n = this.graphDatabaseService.createNode();
        n.setProperty("number", i);
    }
    tx.success();
}

java.lang.OutOfMemoryError: Java heap space
    at java.util.HashMap.addEntry(HashMap.java:766)
    at java.util.HashMap.put(HashMap.java:402)
    at java.util.HashSet.add(HashSet.java:217)
    at org.neo4j.kernel.impl.api.DiffSets.add(DiffSets.java:98)
    at ....

```

为了避免在大的事务中出现内存不够的异常，应该将大的事务分成几个小的事务。然而，一般情况下不推荐对每一个新的节点使用分离的事务创建大量节点，这是因为事务的计算消耗将会对性能产生很大的影响。更好的策略是在一个事务中批处理成百也许上千个创建操作。确切的数量依赖于创建每一个节点所涉及的数据量。

前面的例子中混合了服务器端和客户端的行为，因为使用的是嵌入式模式。在一个更传统的客户端/服务器情形下，处理大事务的压力主要影响到服务器，因为服务器需要保留大量的潜在用户还没有提交的事务。

批处理插入和事务

对Neo4j提供批处理插入功能以快速批处理插入没有任何意义。然而，这个类是为了速度而优化，并不支持事务，而不是像它应该做的那样把数据的变化写到磁盘上。这样的功能在下面的情况下可能很有用，即有大数据集需要一次加载，但是需要充分注意使用该功能的影响。更多的信息请参阅Neo4j手册<http://docs.neo4j.org/chunked/stable/batchinsert.html>。

在本节中，我们已经看到了当程序需要与图形数据库交互时程序如何管理事务。这使我们有充分的信心来定义工作原子单位，成功地完成意味着数据变化的永久性和图形数据库始终保持一致性。在下一节中，我们将稍微深入地探讨一下事务，包括如何使用锁作为一种控制隔离级的读事务的方法。尽管这不是必须的，但在某些情况下有可能要做的。

嵌套的事务

Neo4j支持平面嵌套事务，这意味着嵌套事务是可能的，但是内外事务

最终将共享相同的环境。如果所有的事务都调用**success**方法，那么它们的所有修改都将提交给数据库。

如果要从内事务显式地提示操作的失败，可以调用**failure**方法，这将标志整个事务的失败。在这种情况下，事务将会回滚并且将会在最外层事务的结束时抛出**TransactionFailureException**异常。

7.2 事务的高级功能

我们已经讨论了Neo4j中事务的基本工作情况，这已经给予了足够的知识处理大多数情况下的事务需求。但是，如果需要获得比默认的隔离级更高的隔离保证，或者要编写健壮的程序代码而不出现死锁，那就需要理解Neo4j的事务管理模型。

我们将通过考察事务如何工作和Neo4j提供的默认隔离级开始。然后我们将看一看使用显式的锁获得更强的隔离保证。

7.2.1 事务的语义

让我们考察一些ACID事务提供的保证和它们在Neo4j中是如何管理的。

事务和线程的局域性

Neo4j中的每一个新事务都绑定到当前的线程上。因为这个原因，保证每一个事务都适当的结束和每一个事务的内容恰当清晰非常重要。否则，运行在同一线程的下一个事务很可能受到上一个事务的影响，导致错误的结果和不一致的输出结果。本章提供的代码将会帮助你避免这类问题。

永久性

永久性是指把数据永久写入到数据库中，以便在数据库重启时数据仍存在。如果一个事务成功，可以认为数据已经永久保存到了硬盘。

尽管Neo4j在内部保留了一个事务的日志以记录更新和保留每一个成功

的改变操作事务，当达到某一个程度时，或者当数据库关闭时，事务日志将变得繁忙起来并且硬盘上的适当储存文件将被更新。第11章将讨论事务日志的作用。

隔离级别与Neo4j锁

许多数据库管理系统使用锁机制来管理对同一个数据库的同时访问。Neo4j事务是由清晰的读锁和写锁来控制每一个图形数据库资源（节点和关系）。

读锁是很有用的，因为读锁可以保证相关资源一致性的锁定，当在读数据时不会发生意外的修改数据。更精确地说，如果没有写锁在同一个数据库中激活，则读锁可以在任何的图形资源上随意地获得。多个读锁之间并不相互排斥，在同一时间的多线程中，可以在同一数据资源中获得多个读锁。在给定的时刻，对同一数据资源只能获得一个写锁，还必须没有其他激活的锁，不管是读锁还是写锁。读锁和写锁之间的区别以及它们的交互方式使得开发者能灵活地权衡性能的一致性，具体取决于对正在处理的情况什么是可以接受的。

Neo4j事务的默认设置并不自动获取读锁。因此，读的数据是节点、关系、索引实体最新提交的状态，除非在它自己的事务中，局部的修改（即使没提交）是可见的。图7-1给出了一个演示这种情况的例子。

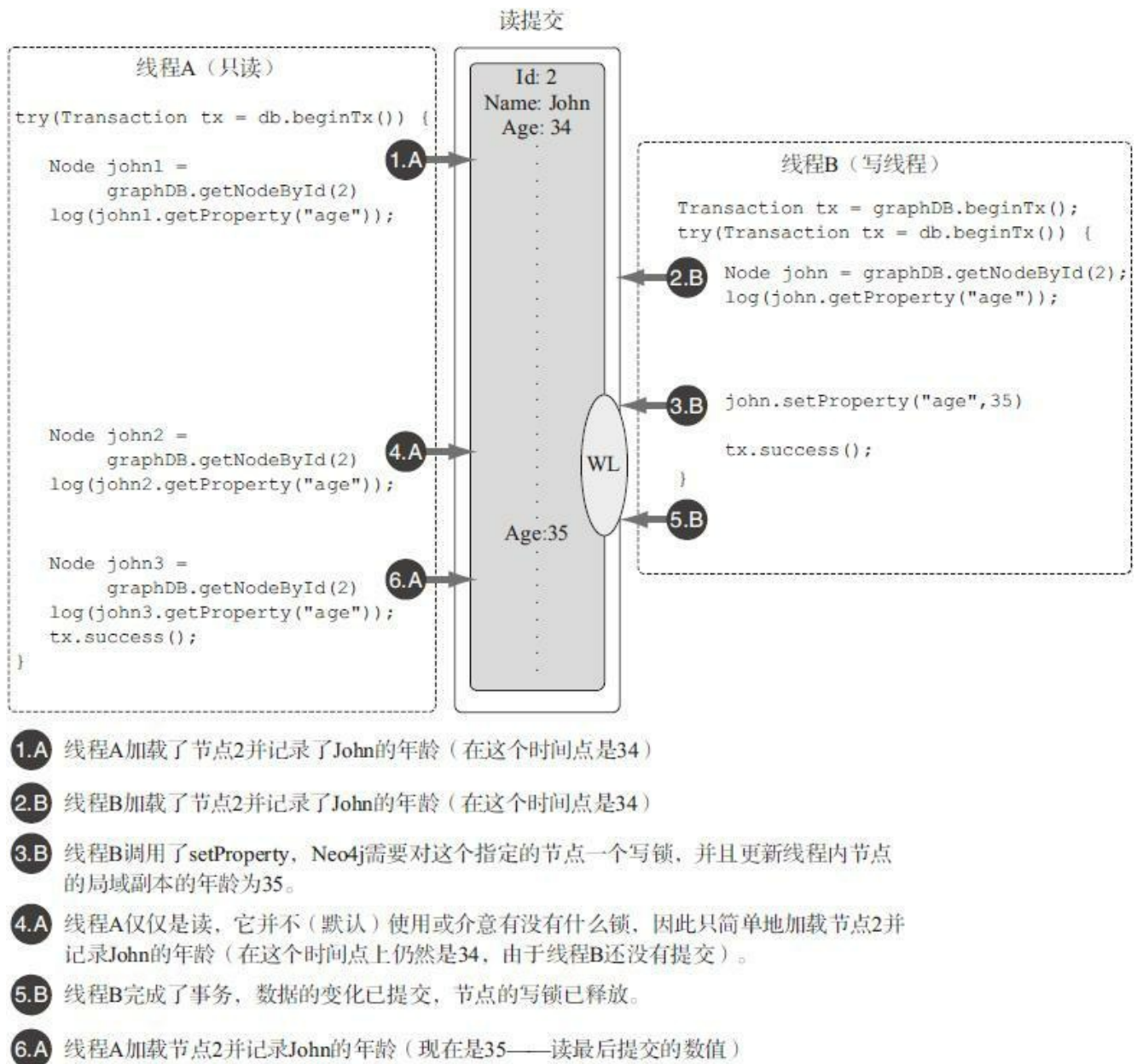


图7-1 默认隔离级

写锁则是在对任意图形资源试图做变更操作时自动获得，在事务的持续时间内都有效。每一个事务确保在完成时自动释放所有的锁。

这个默认行为与关系数据库中的读提交隔离级别非常相似，隔离级别除了潜在的脏读，但是仍然允许幻像读和不可重复读的发生。

锁的到底是什么？

哪个图形资源被锁定和什么时候锁？当试图修改一个图形对象时（即当创建、删除或者更新一个节点或关系的属性时），Neo4j必须获取一个在问题中对象的写锁以保护同时发生修改。同样，当创建两个已存在的节点之间的一个新关系时，两个节点由事务锁定。

幻像读是指一个事务有可能在不同的时间读到不同的节点和关系的情况，原因是另一个事务在相同的时间添加或删除了一些实体。

不可重复读是指在数据库访问中，一个事务范围内两个相同的查询却返回了不同数据，即原始读取不可重复。

图7-1是Neo4j的默认隔离级。在图中，两个线程竞争访问同一个图形资源，导致了一个读提交隔离级。

通过显式管理锁，有可能获得更高的事务保证，正像下一节将会看到的那样。

高可用模式下的事务

本章讲述的事务行为的重点主要放在单一的Neo4j实例内将发生什么。在高可用（HA）模式中，有两类Neo4j实例，主的和副的，并且事务确保写操作总是一致和永久的。尽管技术上能够做到对任意的Neo4j实例做写操作，写操作首先推给主机并且最终传递给副机。这个行为是自动的并且可以进行设置以获得持续性和性能之间的一些权衡。关于更多的Neo4j高可用模式和在实际中的应用可以阅读第11章。

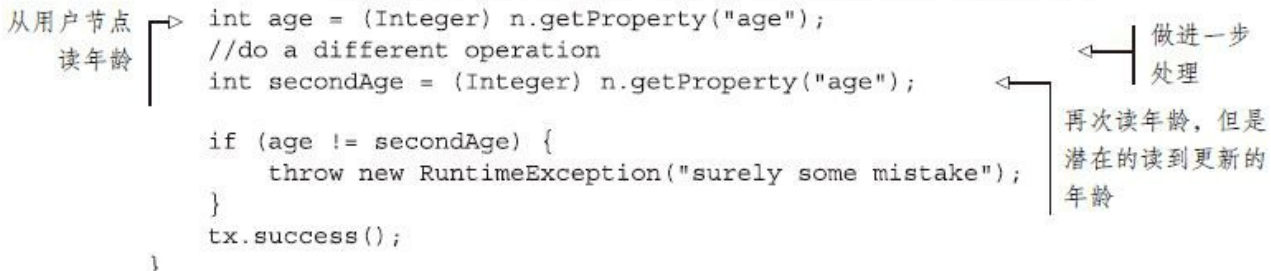
7.2.2 事务中的读取与显式读锁

正如前面所看到的，从图形数据库中读值返回的是最新提交的值（或当前修改的状态），这与读提交隔离级（Read Committed isolation level）是等效的。程序7-4给出了读一个节点属性两次的一些代码（在这种情况下是用户的年龄）作为商业操作的部分，在读之间做一些额外的读。

【程序7-4】 在不用事务的情况下读同样的属性两次

```
try(Transaction tx = graphDatabaseService.beginTx()) {
    Node n = graphDatabaseService.index().
        forNodes("byName").get("name", "John").getSingle();
    int age = (Integer) n.getProperty("age");
    //do a different operation
    int secondAge = (Integer) n.getProperty("age");

    if (age != secondAge) {
        throw new RuntimeException("surely some mistake");
    }
    tx.success();
}
```



由于其他处理的时间不确定，这里将会有有一个做外部修改的重要的窗口出现。如果程序混合着许多读和写，这可能成为一个问题。

如果需要一个更高级别的隔离级以确保其他人不能修改正在读的图形资源，Neo4j提供了一个潜在的显式获取对图形资源的读锁。程序7-5给出了相应代码，但是这一次在读取任何的属性之前有一个额外的显式请求获取对节点的读锁。

【程序7-5】 在增高隔离级的情况下读同样的东西两次


```

try(Transaction tx = graphDatabaseService.beginTx()) {
    Node n = this.graphDatabaseService.index()
        .forNodes("byName").get("name", "John").getSingle();
    // acquire a read lock on n. The lock will be released automatically
    // when the transaction completes
    tx.acquireReadLock(n);
    int age = (Integer) n.getProperty("age");
    //do something else
    int secondAge = (Integer) n.getProperty("age");

    if (age != secondAge) {
        throw new RuntimeException("should now never happen");
    }
    tx.success();
}

```

开始一个事务

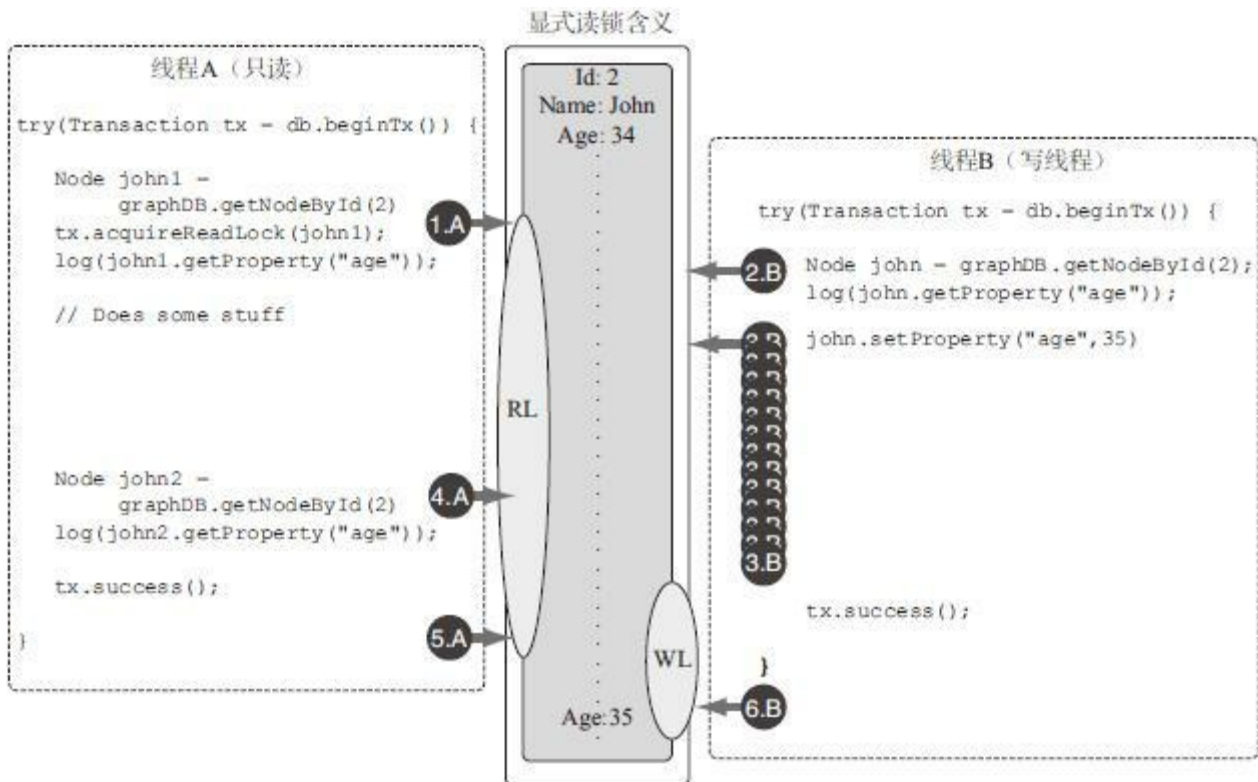
对节点请求一个读锁

图7-2展示了如何使用读锁获得更高的隔离级。

当读数据时，有两个很好的理由避免使用系统的锁。就像以前提及的，首先这将产生事务的状态，并且很占内存。第二，这里主要的权衡是同时写数据，因为线程希望更新任何有显式读锁的节点或关系，在锁被释放或事务完成前是锁定的。

7.2.3 事务中的写入与显式写锁

正如以前讨论的，当要修改每一个资源时，当前的事务会自动为你获取一个不同于其他的写锁。然而，这些锁并不是必须同时有的。可能会有你的业务逻辑需要以一致的方式修改一个节点或关系集的情况，并且要确保没有其他事务能够同时修改它们。如果你的应用程序需要这样高的事务隔离级（这很像关系数据库中的可串行化的（Serializable）隔离级），可以在修改数据之前，在事务的开始对一个图形资源集获取显式的写锁。通常，当事务完成后这些锁将会自动释放。



- 1.A 线程A加载了节点2并显式请求一个读锁，记录John的年龄（在这个时间点是34）
- 2.B 线程B仅仅是读，不管有什么锁（默认情况下）并简单地加载节点2记录John的年龄（在这个时间点仍然是34）
- 3.B 线程B调用了setProperty，Neo4j探测到这个节点已经存在一个读锁，并且不允许读数据，直到能够获取一个写锁时才能读数据。
- 4.A 线程A获得一个节点信息的局域副本并记录John的年龄（在这个时间点上仍然是34）
- 5.A 线程A完成了事务，默认会释放所有的锁，包括显式请求的读锁
- 3.B 线程B解除了阻止并可以获取自己的关于节点的写锁，并且更新它的John年龄局域副本到35。
- 6.B 线程B提交修改并释放写锁完成事务。

图7-2 显式读锁

在程序7-6中，两个节点显式地获取了写锁以确保它们以可串行化的方式共同更新。

【程序7-6】 显式获取写锁

```
try (Transaction tx = graphDatabaseService.beginTx()) {
    Index<Node> nodeIndex = graphDatabaseService.index().forNodes("byName");
    Node n1 = nodeIndex.get("name", "John").getSingle();
    Node n2 = nodeIndex.get("name", "Bob").getSingle();
    tx.acquireWriteLock(n1);
    tx.acquireWriteLock(n2);
```



```
n1.setProperty("age", 35);  
n2.setProperty("age", 37);  
tx.success();  
}
```

再一次提醒，在运行程序时必须非常注意权衡一致性与性能和资源的耗费。

7.2.4 无效锁的危害

尽管前面讨论的锁定技术允许灵活地管理隔离级，但是不可避免地引进了产生无效锁的可能性。无效锁可能在两个或多个事务对同一个图形资源竞争获取锁的时候出现。如果事务A试图按节点1和节点2的顺序锁定它们，而事务B试图对相同的节点按相反的顺序获取锁，那么每一个事务可能会锁定一个节点并且无限期地等待另一个节点释放，从而产生一个无效锁。

当无效锁发生的时候，Neo4j具有一个内置的机制对其进行探测，若有，则抛出一个DeadlockDetectedException异常并且事务回滚。当锁定是由Neo4j专门管理时，才能进行无效锁的探测，这意味着通过依赖于Neo4j的自动默认锁定机制，或者仅仅通过使用我们前面讨论过的Neo4j API的相关锁定方法（acquireReadLock和acquireWriteLock）。如果使用任何其他的锁定机制，例如使用Java的同步（synchronized）关键词，即使发生无效锁也不会被探测出，应用程序很有可能出现终止。

要有效处理无效锁，有几件简单的事情可以做。第一个是坚持使用Neo4j提供的核心API而不使用任何外部的锁定机制。这将确保Neo4j的无效锁探测机制是有效的，确保你的应用程序不会完全锁死，并且数据会保持一致性。在Neo4j核心API中的所有操作（除非另有指定）都是线程安全的，因

此，一般不需要外部的同步。另一个是通过确保图形资源能够以同样的顺序进行访问来避免无效锁。

我们现在已经详细地讨论了Neo4j的事务。在下一节，我们将把重点放在与其他事务管理系统集成Neo4j事务的相关问题上。

7.3 与其他事务管理系统的集成

到目前为止，我们已经看过的事务处理代码是专门用于Neo4j的核心API来开始和完成事务的，并没有对程序运行的环境的性质做任何的假设。

如果你是在建立一个典型的企业应用程序，你很可能会与一个已经存在的事务管理器集成，例如一个应用程序服务器提供的JTA事务管理器。`org.neo4j.kernel.impl.transaction`程序包包含着许多类支持这种情况。

同样的，如果应用程序是建立在流行的Spring框架上的，你将从Spring的声明事务支持受益并确保代码更健壮。程序7-7建立了一个Spring JTA事务管理器，与Neo4j的TransactionManagerService和UserTransactionService相关联。

【程序7-7】 设置Spring事务管理器



这种设置让APretendService组件能够以Spring的@Transactional注释法注释其公共方法，以避免人工管理事务。

程序7-8给出了APretendService的模样，包括使用注释声明事务管理。

【程序7-8】 声明事务管理

```

public class APretendService {
    private GraphDatabaseService gds;

    @Transactional
    public Node createNodeViaAnnotatedMethod(String name, int age) {
        Node node = gds.createNode();
        node.setProperty("name", name);
        node.setProperty("age", age);
        return node;
    }

    public void setGraphDatabaseService(GraphDatabaseService gds) {
        this.gds = gds;
    }
}

```

你已经看到了如何将Neo4j集成到一个典型的Spring应用中。对一个更面向对象的编程模型，第9章将讨论Spring Data Neo4j，它提供了一个在域类和Neo4j之间的映射解决方案。

下一步，让我们看看如何使用事务事件在Neo4j事务周围添加自定义逻辑。

7.4 事务事件

一些数据库实施引发自定义代码数据库事务的执行。Neo4j具有事务事件管理器来实现这个功能。

TransactionEventHandler接口包含三个方法：beforeCommit、afterCommit和afterRollback。当对数据库进行实施和注册时，它们将会在每一个事件的这些关键时间点被调用。beforeCommit方法可以有选择性地返回一个对象，如果任何的上下连接或状态需要进行交流，这个对象能被传递给其他两个方法。

下面的程序给出了GraphDatabaseService对象创建的一个事务事件管理器的骨架。

【程序7-9】 事务事件管理器

```
graphDatabaseService.registerTransactionEventHandler(new
    TransactionEventHandler<Object>() {
        @Override
        public Object beforeCommit(TransactionData data) throws Exception {
            // do some work

            // no need to return state
            return null;
        }

        @Override
        public void afterCommit(TransactionData data, Object state) {
            // do some work
        }

        @Override
        public void afterRollback(TransactionData data, Object state) {
            // do some work
        }
    });
```

TransactionData对象将会通知你事务内图形资源发生的变化。事件管理

器的典型使用情况包括对外部系统的审查和集成。例如，你可以设置一个事件在每一个新用户添加到系统时更新数据板。

要额外留意从事务事件管理器运行的程序代码，尽管它会与你的事务在同一个线程运行，不及时运行代码将会对系统的性能产生影响。从事件管理器出现的异常也会导致事务的失败。

事务和Neo4j服务器

本章的重点是在演示嵌入式Neo4j设置事务管理的使用。Neo4j也能够以服务器模式运行，这种情况下客户端用户是通过周密建立的基于HTTP REST的API访问Neo4j服务器的，第10章给出了详细的有关这种模式需要做的事情。当以服务器模式运行时，为了能够建立健壮的、性能优良的应用程序，理解事务的管理方式和对其考虑是很重要的。这段内容是对一些以服务器模式运行时应当注意的与事务相关的一些关键点，而没给出详细解释。请阅读10.4.3节获取更多的信息。

默认情况下，每一个对Neo4j服务器的请求都是通过调用REST API实现的，并且每一次调用都是在其自己的事务内实现的。但是做许多细致的REST调用很有可能对你的应用程序的性能产生负面的影响。除了通过每次请求创建的新事务这些前面的内容外，多重网络调用本身就是一个对性能影响的灾难性事情。为了帮助解决这些问题中的某些问题，在服务器模式下处理事务场景的以下选项应给予考虑。

- 使用REST Cypher的结束点——提供执行单个Cypher语句的能力，这可能导致多节点或关系需要更新。所有影响到的节点或关系将会作为单个HTTP请求的一部分进行更新。

- 使用REST事务结束点——提供在一个事务范围内执行一系列Cypher语句的能力，但是不像Cypher的结束点，而是通过多个HTTP请求。

- 使用REST批处理结束点——提供定义一项工作的能力，当发送至批处理REST结束点时作为单一的事务执行一组低层的REST API调用。

- 使用服务器插件或非托管的扩展——提供定义Neo4j服务器运行的Java编码能力，作为一个REST调用的结果，提供完全的程序控制（包括事务管理）。

7.5 本章小结

事务是一个非常复杂的主题，期望你到目前为止已经理解了Neo4j如何充分利用基于ACID的事务管理当前对图形资源的访问。

我们已经详细讨论了事务是如何工作的：默认的事务行为能满足大部分情况，但是我们考察了在同一时间对同一图形资源访问的事务含义，并且通过使用显式的读和写锁能够获得高得多的隔离级。

企业应用程序经常建立在像Spring这样的框架之上，我们讨论了如何利用Spring的声明方法将Neo4j事务集成到Spring的事务管理系统。

最后，我们讨论了事务事件管理器，事务事件管理器对添加自定义行为到事务中是非常有用的，如传统关系数据库的激发器（trigger）所起的作用。

现在你已经知道了如何使用事务，在下一章，我们将教你更多的关于遍历的操作，以便使用更复杂的方式查询数据。

第8章 深度遍历

本章包括以下内容：

- 遍历的顺序如何影响性能和内存的占用
- 遍历期间使用扩展器（`expander`）
- 使用`uniqueness`（唯一性）属性控制每一个节点的访问次数
- 使用双向遍历改进图形查询的性能

编写有效的遍历是成功查询图形数据的关键。在第4章中我们学习了Neo4j遍历API的基本概念。本章将深入探讨遍历API的内部工作原理，因此，你可以学习怎样以最有效的方法解决最复杂的图形问题。

8.1 遍历的顺序

每一次遍历器访问一个节点，它都需要决定下一步沿着哪个关系访问哪个节点，重复这个过程直至遍历结束。通过在图形中选择一个合适的路径，遍历可以很快结束并且使用很少内存。这是图论中的两个主要顺序算法，并且这两个算法用于大多数的Neo4j遍历中：[深度优先](#)（depth-first）算法和[广度优先](#)（breadth-first）算法。

要了解遍历的效率和性能如何依赖于选定的顺序，请看图8-1的图形。

图8-1中的图形有9个节点。它代表一个树状图，这是一个由两个节点通过单个路径相连的图形类型。换句话说，树状图是一个没有任何环的图形。为了使我们演示遍历的顺序更简单，我们将使用树状图作为一个例子（对一个具有环的例子，因此不是树状图，参见图8-4）。

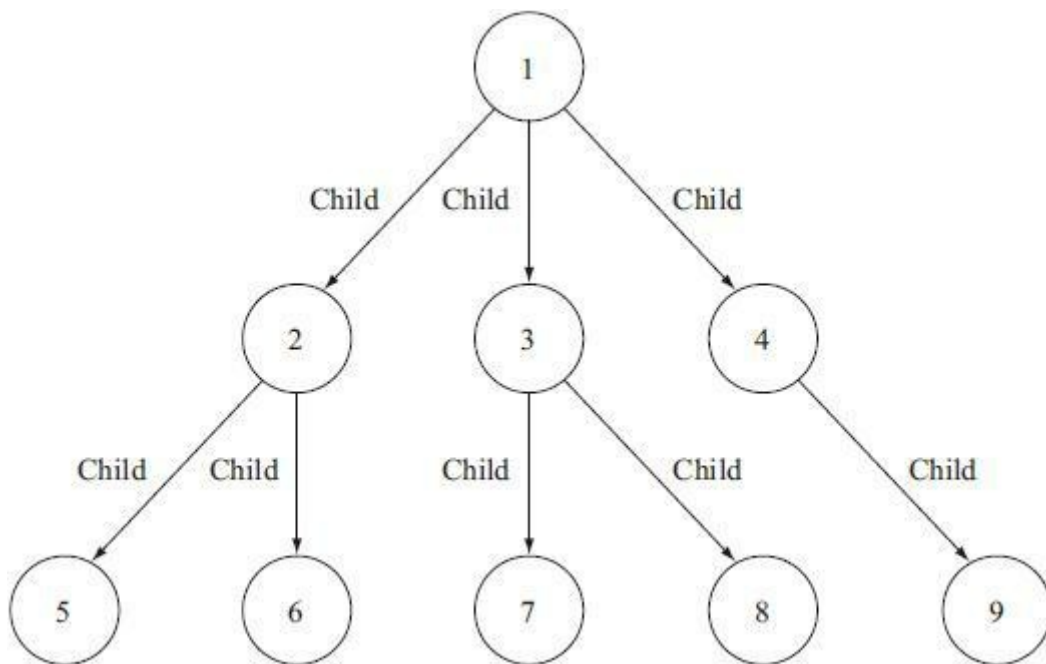


图8-1 具有9个节点和8个关系的简单图形

图形的根节点标记为1。直接与根节点相连的（分支）标记为2、3和4。其他的节点（叶）是两个离开根节点的关系（节点5~9）。

让我们看看按深度优先和广度优先的顺序遍历这个图形的区别。我们的目的是遍历这个图形，因此能够访问图形中的每一个节点正好一次。

8.1.1 深度优先

每一次访问一个节点，需要决定下一步沿着哪个关系，或者下一步到哪个节点。深度优先顺序表明你应该首先跳到现在所在且以前没有访问过的节点的第一级子节点。如果所有的下一代子节点都已经访问过，就应该返回具有没有访问过的子节点的第一个节点上。图8-2展示了使用深度优先遍历访问每一个节点的顺序。

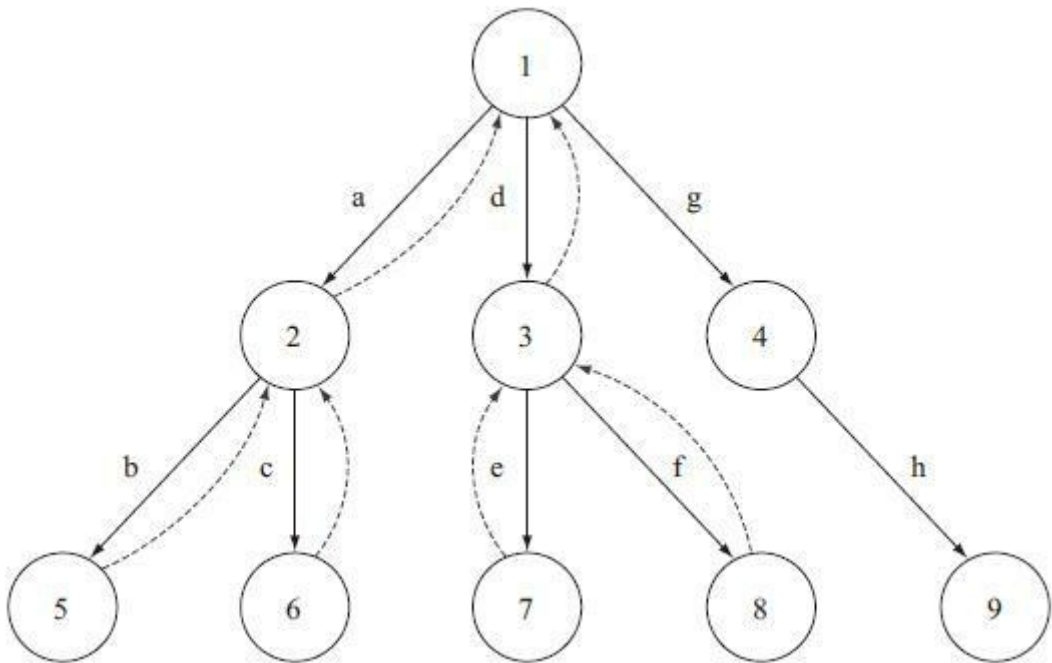


图8-2 使用深度优先遍历图形

从节点1开始遍历，节点1与其他三个节点（2、3和4）具有关系并且你

将访问前面没有访问过的第一个子节点——在这种情况下是节点2。你可以设想遍历器就像一个小的机器人从节点1通过关系a跳到节点2。

一旦你检查节点2，你不得不决定下一步往哪里走。记住深度优先的原则，如果有可能的话，访问前面没有访问过的子节点。因此你的下一步将从节点2到节点5（通过关系b）。仅仅两步跳跃，你就到了该图形的最深节点——从根节点（节点1）到节点5。这就是深度优先算法命名的原因：以最快的速度到达图形的最深处。

现在你已经在节点5上了，你应该到它的子节点上，但是节点5是该图形的叶，因此它没有任何子节点要访问。在这种情况下，深度优先算法有一个规则，回到前面没有访问过的具有子节点的第一个节点上。你返回节点2，然后使用原来的规则。节点2具有两个子节点（节点5和6），你已经访问过节点5，因此，下一站是节点6（通过关系c）。

使用相同的规则，你将会以下面的节点顺序遍历图形。

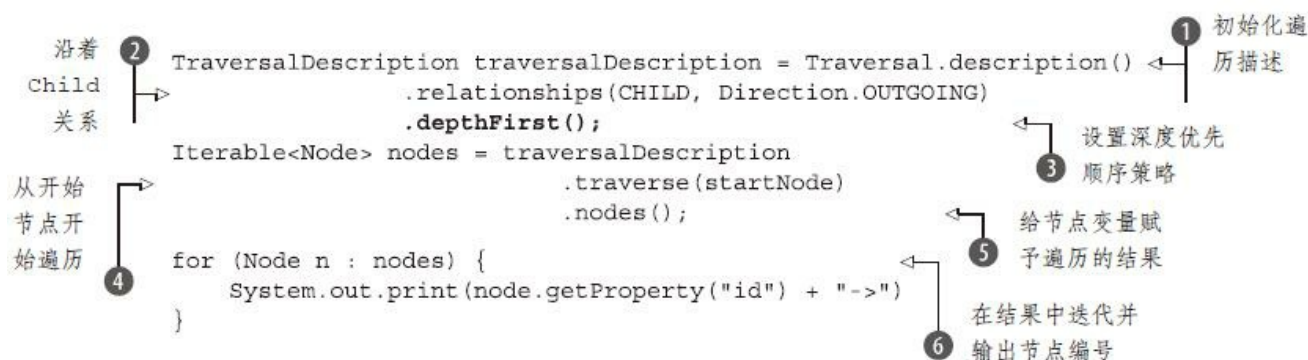
1→2→5→6→3→7→8→4→9

如果你看图8-2，在遍历期间你已经通过a、b、c、d、e、f、g和h的顺序进行了跳跃。

倒数第二个访问节点4是非常有趣的，尽管那个节点非常靠近根节点——开始点。你也会注意到图形左边部分的节点（节点1、2、5和6）比右边部分的节点（节点4、8和9）更早访问。这是深度优先遍历的结果，当在建模图形数据库和设计遍历时应充分利用这个顺序。当在比较深度优先和广度优先的顺序时我们将再一次进行讨论这一点。

现在的问题是如何在Neo4j遍历API中使用深度优先算法。使用Neo4j遍历API实施我们刚刚描述的遍历，你会在程序8-1中看到解决的方法。

【程序8-1】 使用Neo4j遍历API深度优先遍历整个图形



首先，需要初始化遍历描述，就像在第4章中当你使用Neo4j遍历API时一样（**1**）。然后，设置遍历只沿着往外的CHILD类型关系遍历（**2**）。下一步，设置遍历的顺序为深度优先。要实现这一做法，使用专用的方法depthFirst（）作为TraversalDescription创建器的一部分（**3**）。

一旦这个描述建立起来，通过指定起始节点开始遍历（**4**），并且返回遍历过程中访问的所有节点（**5**）。记住，遍历结果的返回类型是一个延迟Java可迭代的实例。仅仅指定起始节点并且在遍历中调用nodes（）并不能访问任何节点。需要在结果中迭代以便遍历器开始在图形中移动，这里你要做的正是这样（**6**）。在迭代期间，你可以以希望的方式处理访问的节点，这取决于你所用软件的需求。在本例中，只要求在控制台上输出节点属性的编号。

最终的输出将会证实遍历图形时确实用的是深度优先算法规则。

1 -> 2 -> 5 -> 6 -> 3 -> 7 -> 8 -> 4 -> 9 ->

深度优先是Neo4j默认分支顺序策略。当建立遍历描述时，如果不指定遍历的顺序，则默认为深度优先。

现在让我们看一看其他的内置分支选择策略：广度优先。

8.1.2 广度优先

顾名思义，广度优先算法试图在图形中从起始节点访问更远的节点之前走得尽可能广。作为广度优先算法的一部分，遍历在访问子节点之前将首先访问当前节点的所有同级节点。同级在这里是指从根节点开始与你正在访问的具有相同距离的节点。图8-3显示了广度优先算法对前面用过的同一图形的遍历顺序。

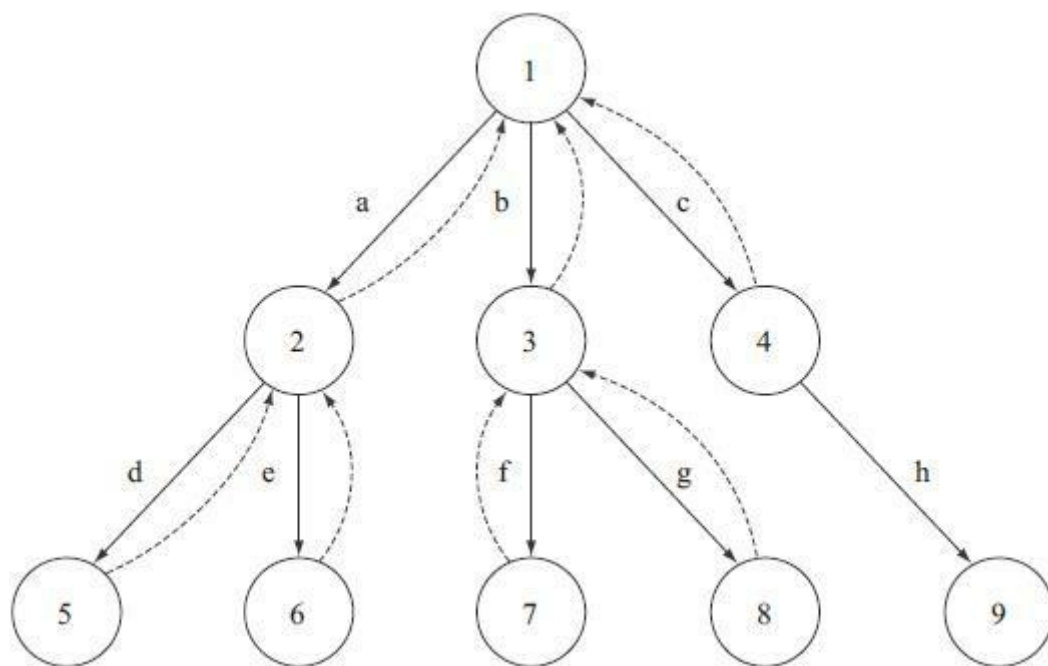


图8-3 样本图形的广度优先遍历

从根节点1开始，像以前一样，通过访问它的第一个后代节点2继续（通

过关系a)。下一步，访问节点2的同级节点，移动到节点3，然后节点4。一旦访问了节点4，没有与根节点同距离的节点需要访问了，因此移动到下一步深度级别上，从节点2的子节点开始，以这样的顺序访问节点5和节点6。然后继续到节点7和节点8，最后访问节点4的子节点（节点9）。使用广度优先算法访问的节点顺序如下所示：

1→2→3→4→5→6→7→8→9

在广度优先遍历中，离根节点越近的节点访问得越早，将远离根节点的节点放在后面访问。

让我们现在用广度优先策略更新遍历的实施。Neo4j遍历API的遍历描述创建器有一个专用的方法做这项工作，就像程序8-2给出的一样。

【程序8-2】 使用Neo4j遍历API的广度优先遍历

初始化
遍历
描述

为节点
变量指
定遍历
结果

```
TraversalDescription traversalDescription = Traversal.description()
    .relationships(CHILD, Direction.OUTGOING)
    .breadthFirst();
Iterable<Node> nodes = traversalDescription
    .traverse(startNode)
    .nodes();

for (Node n : nodes) {
    System.out.print(node.getProperty("id") + "->")
}
```

沿着 CHILD
关系

① 设置广度
优先策略

从起始节点
开始遍历

通过结果迭代并
对它们进行处理

就像你所看到的，代码非常相似。唯一的不同是建立遍历描述时使用了一个不同的方法来指定遍历的顺序（①）。控制台的输出将会与本节前面建立的顺序相匹配：

1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 ->

8.1.3 深度优先与广度优先顺序的比较

正像你在前面两节中看到的，深度优先遍历支持从图形的左边部分搜索解决问题，广度优先遍历在结果靠近起始节点时会比远离起始节点时更快。这两个特性在设计图形数据库和设计需要执行的遍历时是非常重要的。

取图8-1中的节点4作为一个例子。如果使用深度优先顺序从节点1开始遍历图形，在找到节点4之前，需要的遍历几乎走遍了整个图形，因为这将访问倒数第二的节点。如果用广度优先顺序做同样的工作，节点4将是起始节点算起的第四个节点，只访问了路径上的一半节点。由于遍历的速度与跳跃的次数（或路径上访问的节点数）相关，如果你使用广度优先而不是使用深度优先，你将用一半的时间找到结果。

如果你正在查找节点5，深度优先算法将会用三次跳跃找到结果，而广度优先则需要五次。

图形越大，遍历顺序对遍历性能的影响就越大。为了演示，创建一个每一个节点正好有三个子节点的图形数据库。创建一个起始节点并连接其三个子节点。然后，取三个子节点中的每一个节点，并添加三个子节点到每一个节点上。然后继续直至深度12级。结果图形将会有797161的节点和797160个关系。

现在运行一个遍历查找指定深度上的一个特定的节点。作为测量性能的一种方法，我们将使用访问过的节点数量作为测量依据。



移动的次数和访问的节点数直接与执行遍历所需要的时间相关。使用实际的时间测量性能将使得结果依赖于可用的硬件资源以及缓存情况，在这个

简单的实验中，我们将尝试避免硬件及缓存的影响。Neo4j每秒访问的节点数在几万至几百万之间，取决于硬件、缓存和遍历的复杂程度。

表8-1给出了本实验的结果。

表8-1 遍历的性能依赖于搜索的节点所在的位置和所用的遍历算法

深 度	节点所在位置	深度优先	广度优先
3	首先在左边	3	13
3	最后在右边	767 637	39
6	首先在左边	6	364
6	最后在右边	796 068	1092
9	首先在左边	9	9841
9	最后在右边	797 121	29 523
12	首先在左边	12	265 720
12	最后在右边	797 160	797 160

就像你在表中看到的，当结果节点靠近起始节点时，广度优先排序一般会给出较好的性能。但是远离起始节点时，广度优先排序总是比较慢，而深度优先排序可能非常高效，取决于结果节点是在图形的左边还是右边。

在最坏的情况下，是整个图形都需要遍历（当结果节点在图形的右下角时），深度优先和广度优先都需要访问所有的节点。由于广度优先遍历需要更大的内存记忆遍历的足迹，这种情况下最好使用深度优先。

除速度之外，当决定遍历的顺序时，另一个需要考虑的方面是遍历的内存消耗。再一次假设遍历器是一个小机器人通过关系从一个节点跳跃到另一个节点。这个小机器人将必须记住一些状态，从哪个节点来，哪个节点已经访问过。图形越大，存储这个状态需要的内存就越多。但是，选择的遍历顺序会影响需要的内存大小，下面让我们看看情况如何。

当使用深度优先时，你试图用最快的速度深入到图形。一旦你到了图形

的最底层（访问了一个以前没有访问过的且没有子节点的节点），你就可以完全忘掉图形的这个分支——从遍历的角度，可以认为已完成。在Neo4j Java世界，那意味着你可以从内存间接引用那条路径并且留作垃圾集的清
理。因为这些路径的一些将会在遍历的早期访问，你一旦开始遍历就可以开始解除内存的分配。

广度优先遍历在开始下一深度的节点之前，试图在图形中走得尽可能广。因此，在遍历期间，需要记住所有访问过的节点和哪一个子节点还没有访问。图形越大越复杂，需要记住的节点就越多，会导致一个巨大的内存占用。

一般来讲，每一个节点的关系越多，广度优先需要的内存就越多。

如何选择深度优先和广度优先？作为一个应用程序设计师和开发者，你必须认识到你有一个很大的优势：你知道和理解你所操作的域模型。基于你的域知识，你可以决定你所使用每一个遍历应该使用哪一个优先顺序（在同一个图形中，可能随着不同的情况而变化）。

如果结果靠近起始节点，广度优先顺序可能会更好。但是，如果图形非常密（即每一个节点有许多关系），则广度优先顺序在实际中可能会使用太多的内存。

对遍历顺序的理解和对问题领域的洞察将给你一个很好的开始，但是决定一个总体上最好的遍历，你需要以不同的选项做实验。如果Neo4j的内置顺序选项不匹配你的需求，你可以通过使用
`org.neo4j.graphdb.traversal.BranchSelector`接口实施你自己的顺序，这个接口是Neo4j遍历API的一部分。自定义遍历实施超出了本书讨论的范围，但是

可以通过查看Neo4j手册（<http://docs.neo4j.org/>）和访问Neo4j Google群（<https://groups.google.com/forum/#!forum/neo4j>）找到有用的信息。

让我们看看能帮助你决定如何在遍历期间从当前节点跟踪关系的Neo4j机制：路径扩展器。

8.2 扩展关系

扩展器是Neo4j遍历API的一个部件，负责决定遍历期间访问过的任意节点应该跟踪哪个关系。除选择关系类型和访向外，扩展器还负责跟踪关系的顺序。

Neo4j提供了许多现成的扩展器实施。默认的实施是一个最常用的StandardExpander。

8.2.1 标准扩展器

标准扩展器（StandardExpander）封装在StandardExpander中，它提供了一系列内置扩展器允许你指定想要跟踪的关系，包括关系的方向。关系的扩展是按照它们加入图形的顺序进行的。



注意

StandardExpander是Neo4j遍历API使用的默认实施。

为了演示StandardExpander的使用，考虑下面的例子：在一个社交网络中，一起工作的人们通过一个WORKS_WITH关系连接起来，并且是密切朋友的人们通过IS_FRIEND_OF关系连接起来。图8-4给出了我们将使用的样本图形。

现在我们想做的是查找John的直接关系（朋友和工作同事）喜欢的所有电影。看一下图8-4，你期待的结果将包含电影Fargo和Alien。Top Gun不应

该是结果的一部分，因为它只是John自己喜欢的。Godfather和Great Dictator不应该是结果的一部分，因为只是John朋友的朋友喜欢的（你只想要John的直接关系喜欢的电影）。

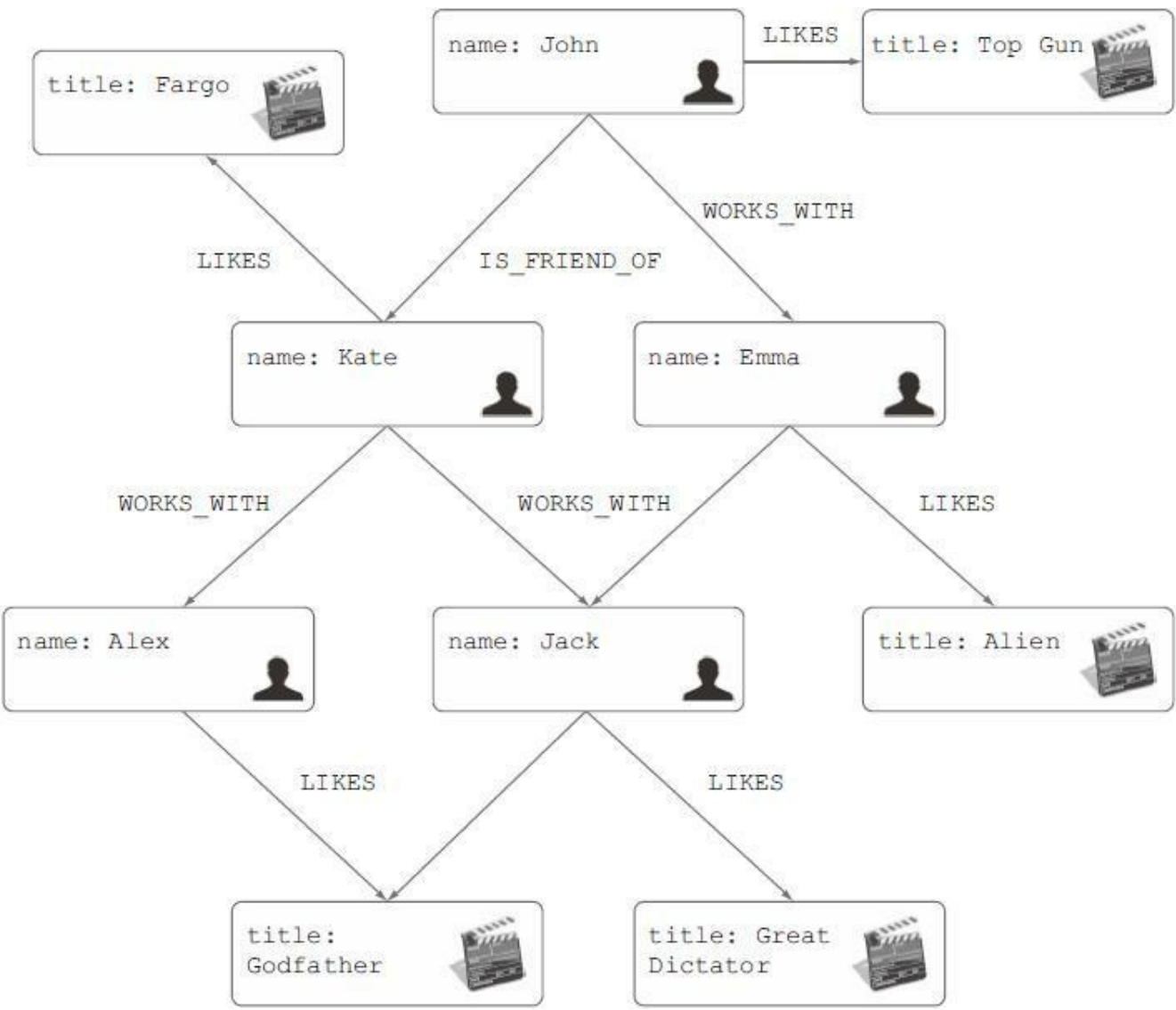


图8-4 一个用户和他们喜欢的电影的社交网络

程序8-3演示了这个遍历的实施。

【程序8-3】 查找John的朋友和同事喜欢的所有电影

```

TraversalDescription traversal = Traversal.description()
    .expand((PathExpander) StandardExpander.DEFAULT
        .add(WORKS_WITH)
        .add(IS_FRIEND_OF)
        .add(LIKES))
    .evaluator(Evaluators.atDepth(2))
    .evaluator(new Evaluator() {
        @Override
        public Evaluation evaluate(Path path) {
            if (path.endNode().hasProperty("title")) {
                return Evaluation.INCLUDE_AND_CONTINUE;
            }
            return Evaluation.EXCLUDE_AND_CONTINUE;
        }
    });

Iterable<Node> nodes = traversal.traverse(john).nodes();
for (Node n : nodes) {
    System.out.println(n.getProperty("title"));
}

```

使用出厂方法访问标准扩展器实例 ①

只包含结果中的电影 ④

添加要扩展的关系类型 ②

在最终结果中只考虑深度 2 上的节点；深度 2 后结束遍历 ③

运行遍历并输出电影名字到控制台 ⑤

使用 `TraversalDescription.expand(...)` 方法设置遍历要使用的扩展器（①）。使用静态出厂方法实例化 `StandardExpander`，然后添加要跟踪的关系（②）。因为你是在查看 John 的直接联系人看过的电影，你想要只包括深度 2 上的节点并且在这之后停止遍历（③）。确保在结果中仅仅返回电影节点（例如，没有用户节点），需要添加另一个评估器检查节点上的 `title` 属性，因为只有电影节点具有这个属性（④）。最后，从代表 John 的节点开始遍历，并将电影名字输出到控制台（⑤）。

一旦查询被执行，期待的结果输出到控制台上。

```

 Fargo
 Alien

```

在代码中，传递内置于 `StandardExpander` 中的扩展器到 `TraversalDescription.expand(...)` 方法中以选择每一个节点要跟踪的关系。但是因为 `Standard Expander` 是 `TraversalDescription` 创建器使用的默认扩展器，你可以方便地使用它的便捷方法选择要扩展哪一个关系，使代码更清晰、更简洁。下面的程序给出了如何从程序 8-3 使用这一句法更新遍历。为了清晰起见，程

序8-3中的部分已被省略。

```
TraversalDescription traversal = Traversal.description()
    .relationships(WORKS_WITH)
    .relationships(IS_FRIEND_OF)
    .relationships(LIKES)
    ...
```



使用 TraversalDescription.relationships(...) 方法添加需要的关系

①

在这段程序中使用了 `TraversalDescription.relationships(...)` 方法添加你想扩展的关系类型（①）。因为 `StandardExpander` 是默认的，这将导致在底层被执行的代码与程序8-3的代码完全一样，这里在句法上稍微紧凑一些。

程序8-3中的代码看上去更像本书中前面遍历用过的代码，但是你现在知道在它的背后是 `StandardExpander` 的行为。使用 `TraversalDescription.expand(...)` 方法，你可以控制任何遍历使用哪一个扩展器，下一节将介绍如何利用这种灵活性。

在下面的两节中，我们将学习如何使用非默认的扩展器在遍历中的实现。首先，让我们以顺序关系扩展器开始。

8.2.2 用于扩展的顺序关系

正如你所看到的，`StandardExpander` 具有一个限制——要扩展的关系顺序是固定的，它使用创建时的关系顺序。在大部分情况下，这是一个合理的限制，因为大部分遍历以同样的方式对待每一个关系类型。但是在一些情况下将会挑选一个特定的顺序扩展关系。让我们使用前面一节的同一个图形（图8-4）来演示使用顺序关系的情况。

就像在前一节中，我们将查找John的所有直接联系人喜欢的所有电影

（包括朋友和同事）。这一次，我们将挑选John的密友喜欢的电影，把它们放在结果清单的最前面。你将期待以指定的顺序（与以前不同）获得与以前同样的遍历结果。

1) Alien（由John的朋友Emma推荐）。

2) Fargo（由Kate推荐，Kate是John的同事）。

在扩展之前，你可以通过使用`OrderByTypeExpander`很容易地根据类型排序关系。而不是使用默认的`StandardExpander`，如程序8-4所示。

【程序8-4】 以关系的类型顺序扩展关系

```
PathExpander expander =
    new OrderByTypeExpander()
        .add(IS_FRIEND_OF)
        .add(WORKS_WITH)
        .add(LIKES);
TraversalDescription traversal =
    Traversal.description()
        .expand(expander)
        .evaluator(Evaluators.atDepth(2))
        .evaluator(new Evaluator() {
            @Override
            public Evaluation evaluate(Path path) {
                if(path.endNode().hasProperty("title")){
                    return Evaluation.INCLUDE_AND_CONTINUE;
                }
                return Evaluation.EXCLUDE_AND_CONTINUE;
            }
        });
Iterable nodes = traversal.traverse(john).nodes();
for(Node n: nodes){
    System.out.println(n.getProperty("title"));
}
```

1 实例化 `OrderByType Expander`

2 以需求的顺序添加要跟踪的关系

3 使用配置的 `OrderByType Expander` 遍历

使用默认的构造器实例化新的扩展器（❶）；然后添加要跟踪的关系类型（❷）。对每一个访问的节点，指定的关系类型确保以同样的顺序扩展，因为它们被添加到扩展器中。因为你想要`IS_FRIEND_OF`关系总是首先

被遍历，所以把它首先添加到代码中。

要使用新创建的扩展器，使用`TraversalDescription.expand(...)`方法指定它（❸）。程序的其余部分与以前的一样。

当你执行这个遍历时，结果将与期望的完全一样。

```
Alien  
Fargo
```

当你设计遍历时，`OrderByTypeExpander`可以提供更强大的功能。然而，在一些情况下，没有Neo4j内置的遍历提供需求的功能。在这种情况下，你可以使用Neo4j遍历API实现你自己的自定义扩展器；这就是我们将来在下一节中介绍的。

8.2.3 自定义扩展器

当你需要建立一个满足你的需求的自定义扩展器时，Neo4j有一个可用的遍历扩展点。这个扩展API是用`PathExpander`接口定义的，它是Neo4j核心库的一部分。

`PathExpander`接口定义了你需实施的两个方法。

`Iterableexpand<Relationship>expand (Path path, BranchState state)`；这个方法包含决定遍历从当前节点应该跟踪哪个关系的逻辑（并且以哪个顺序）。参数`path`包含在图形中遍历到当前节点的完整路径（包括节点和关系）。你可以像通常一样使用路径对象的所有可能方法。例如：

`-Path.endNode ()` 获得当前访问的节点。

-`Path.startNode ()` 获得遍历开始的节点。

-`Path.lastRelationship ()` 取回在图形中到达当前节点之前遍历的最后关系。

除遍历的路径之外，还访问了另一个方法的参数`BranchState`，它代表有关遍历的当前分支的状态。它是可选的，并且一般的扩展器并不使用它。

·`PathExpander<STATE>reverse ()`；这个方法以同样的扩展逻辑返回到扩展器，但是以相反的方向，并且一般用于双向遍历（在本章后面部分将进行讨论）。考虑`reverse`方法作为你需要使用的扩展器以获得从当前节点返回到遍历的起始节点。

现在让我们实现一个样本自定义扩展器。基于本节中前面使用的这个相同的社交网络图形（参见图8-4），我们将要设计一个遍历，将再次查找John的直接联系人喜欢的所有电影。但是，不是使用标准的Neo4j遍历API部件，包括自定义评估器，我们将使用自定义`PathExpander`实现。

这个查询的主要难点是避开所有John喜欢的电影，还有John的朋友的朋友喜欢的任何电影。为了实现这个目标，我们将自定义一个具有以下特性的扩展器：

·仅仅跟踪从起始节点开始的`IS_FRIEND_OF`和`WORKS_WITH`关系（代表John）。

·对所有在深度1访问过的节点（代表John的朋友和同事）仅仅跟踪`LIKES`关系。

使用内置的TraversalDescription方法和StandardExpander，不可能创建一个具有这些性质的遍历。但是你可以实现一个自定义PathExpander。程序8-5给出了一个这样的扩展器的实现。

【程序8-5】 基于从起始节点的距离扩展关系



DepthAwareExpander从Neo4j核心API实现PathExpander接口 (1)。要映射从起始节点 (或深度) 到关系类型的距离，可以使用一个简单的 java.util.Map (2)。

要实现的关键方法是expand (...)，这个方法扩展了从你当前访问的节点到你应该跟踪的关系 (3)。当前访问节点的深度可以用Path.length () 方法找到 (4)。下一步，在给定的深度从配置的映射查找关系 (5)。最后，你从当前的节点返回指定类型扩展的关系 (6)。使用 Path.endNode () 方法定位当前的节点。

下一步是在遍历中使用自定义扩展器。程序8-6给出了一个输出结果到控制台的遍历的例子（John的直接朋友喜欢的电影名字）。

【程序8-6】 使用一个自定义扩展器查找John的朋友喜欢的所有电影

```
HashMap<Integer, List<RelationshipType>> mappings =
    new HashMap<Integer, List<RelationshipType>>();
mappings.put(0, Arrays.asList(
    new RelationshipType[]{IS_FRIEND_OF, WORKS_WITH}
));
mappings.put(1, Arrays.asList(
    new RelationshipType[]{LIKES}));

TraversalDescription traversal =
    Traversal.description()
        .expand(new DepthAwareExpander(mappings))
        .evaluator(Evaluators.atDepth(2));
Iterable<Node> nodes = traversal.traverse(john).nodes();
for(Node n: nodes){
    System.out.println(n.getProperty("title"));
}
```



1 配置深度 / 关系映射

2 使用 DepthAwareExpander 作为一个遍历扩展器

3 仅返回深度 2 的节点

4 从 John 开始遍历，并且输出结果到控制台

第一步是配置在每一深度上要跟踪的关系（❶）。映射是跟踪直接从起始节点（深度0）的IS_FRIEND_OF和WORKS_WITH关系和从深度1节点的LIKES关系。遍历实施的关键点是使用自定义扩展器配置遍历描述（❷）。由于你感兴趣的节点仅仅到深度2，可以使用atDepth评估器在指定的深度停止遍历（❸）。最后，从代表John的节点开始遍历并以通常的方法在结果中迭代（❹）。

当执行这个遍历时，控制台的输出将会显示期望的电影《Alien》和《Fargo》，这是John的直接关系人喜欢的电影。

我们将讨论的下一个主题是高级Neo4j遍历API的唯一性，因此，让我们继续阅读本书来学习如何控制遍历访问节点和关系的次数。

8.3 管理唯一性

遍历的唯一性问题决定在遍历期间你能访问一个节点多少次。在Neo4j的遍历API中，有几个不同的唯一性设置，它们能用于你的遍历中。在本节中，我们将演示它们的使用和行为。所有可用的唯一性选项是在Neo4j Java库中的Uniqueness类中作为常数访问的。

8.3.1 NODE_GLOBAL唯一性

Neo4j遍历中最典型的唯一性设置是NODE_GLOBAL。它基本上意味着每一个节点只能访问一次并且在遍历期间只能访问一次。



注意

NODE_GLOBAL是默认的遍历设置，因此，如果你不指定唯一性，这样的设置会在你的遍历中使用。

NODE_GLOBAL是遍历的一般需求，因此让我们用一个例子中来演示。让我们看一个简单的社交网络图形，具有6个代表用户的节点和互相认识的用户之间的KNOWS（认识）关系。图8-5给出了这个简单社交网络图。

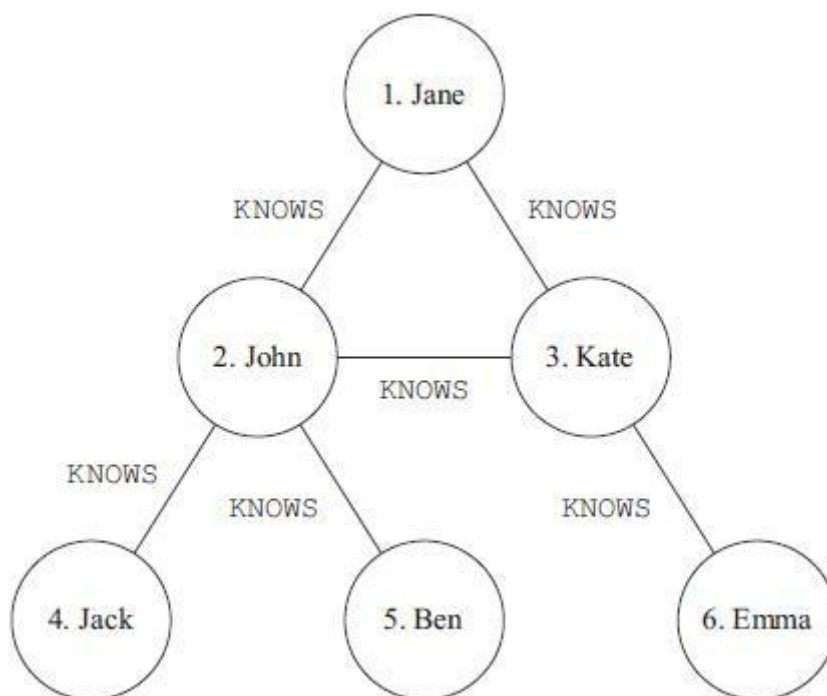
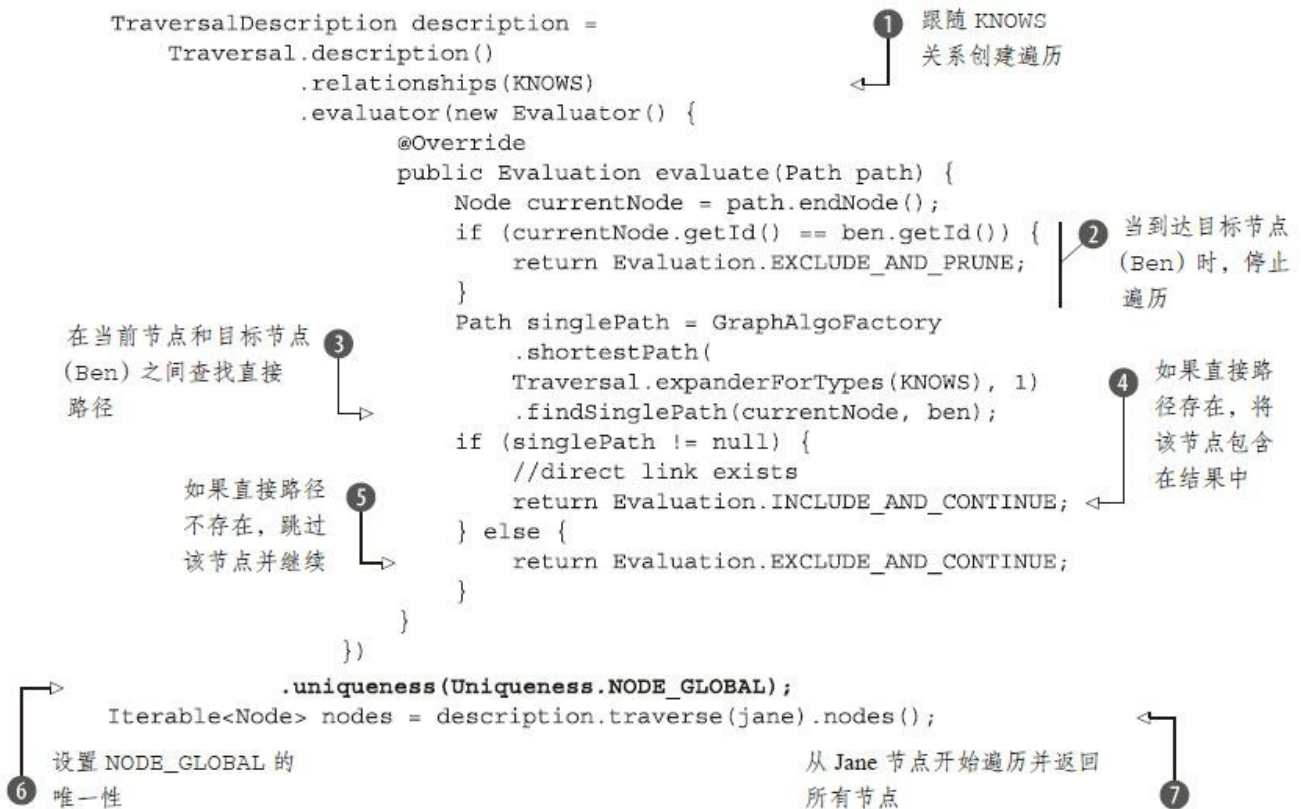


图8-5 一个简单的社交网络图

让我们实施一个遍历顺着问题寻找解决方案：谁是将用户1（Jane）介绍给用户5（Ben）的直接联系人？

程序8-7给出了使用NODE_GLOBAL唯一性实施的需求遍历。

【程序8-7】 查找将Jane介绍给Ben的直接联系人



用一般的方法创建遍历，并配置使其在任意方向跟随KNOWS关系（1）。然后利用一个自定义评估函数用以确定访问过的节点是否应该包含在结果中。评估函数的第一步是如果到达目标节点（在这种情况下是Ben），则停止遍历（2）——在她遇见他后，就没有地方检查谁将Jane介绍给Ben了。下一步，试图查找在当前节点与代表Ben的目标节点之间的一条直接的路径（3）。你正在使用Neo4j的内置图形算法计算直接路径（使用GraphAlgoFactory类）。如果一条直接路径存在，这意味着你找到了一个节点，他可以将Jane介绍给Ben，因此，应该将当前节点包含在结果中（4）。如果没有直接连接，则跳过该节点并继续遍历（5）。

做这个实验的主要原因是演示唯一性，因此使用遍历描述创建器的uniqueness（）方法设置NODE_GLOBAL到需要的设置（6）。最后，从代表Jane的节点开始遍历并返回所有的节点（7）。

如果运行这个遍历并将结果输出到控制台，你将会看到一个单一的结

果：

John

可以通过查看图8-5的图形确认John是Jane网络中唯一能将她介绍给Ben的人。

下面让我们扩展这个实验，了解NODE_PATH唯一性设置。

8.3.2 NODE_PATH唯一性

NODE_PATH唯一性设置指定了从起始节点到当前节点，没有路径可以被遍历两次。多次访问同一个节点是允许的，但只有它们属于不同的路径才可以。

让我们通过修改遍历NODE_PATH的唯一性更新程序8-7，并且保持其他遍历设置完全一样，再次运行这个查询。程序看上去将非常相似，如程序8-8所示。为了清晰起见，部分代码省略了。

【程序8-8】 使用NODE_PATH唯一性做连接

```
TraversalDescription description =
    Traversal
        .description()
        .relationships(KNOWS)
        .evaluator(new Evaluator() {
            @Override
            public Evaluation evaluate(Path path) {
                //... omitted for clarity
            }
        })
        .uniqueness(Uniqueness.NODE_PATH);
Iterable<Node> nodes = description.traverse(jane).nodes();
...
```

使用 NODE_PATH ① 唯一性

这段代码的唯一不同是用于设置唯一性的常数不同：NODE_PATH (①)

)。

如果你运行这个查询，输出将会如下所示：

```
John
John
```

输出John两次——在结果中有两个入口，尽管两个代表同一个节点。为什么会发生这样的事？仔细看一下图8-5的图形。Jane确实通过John连接到了Ben。但是还有一条路径连接着Jane和Ben，即通过Kate和John的路径。这是图形中的不同路径，因为你是在使用NODE_PATH唯一性，两者都代表了有效结果。

以前，你只得到了一个结果，因为两条路径（Jane—John—Ben和Jane—Kate—John—Ben）都经过John节点，并且由于你使用NODE_GLOBAL唯一性，第二条路径从来没有经过。

为了进一步演示这一差别，让我们稍微改变一下遍历查询，用以查找通过Jane自己的直接联系人到Ben都有什么路径。

为了回答这个问题，你必须明白你不仅仅只对认识Ben的用户感兴趣，而是在Jane和Ben之间路径上的所有节点。所有你不得不修改遍历的结果，如下面的程序所示：

```
TraversalDescription description = Traversal.description()
    .relationships(KNOWS)
    .evaluator(new Evaluator() {
        @Override
        public Evaluation evaluate(Path path) {
            ...
        }
    })
    .uniqueness(Uniqueness.NODE_PATH);
Iterator<Path> paths = description.traverse(jane).iterator();
```



遍历描述保持与以前完全一样，包括NODE_PATH唯一性（❶）。唯一的变化是从遍历返回路径而不是返回节点（❷）。

这段代码的控制台输出将会是这样：

```
(Jane) -- [KNOWS, 0] --> (John)
(Jane) -- [KNOWS, 1] --> (Kate) <-- [KNOWS, 4] -- (John)
```

这一次可以非常清楚地看到为什么有两个John节点——它们是不同的路径的部分。基于这些结果，Jane有两个选项。

- 她可以联系John，John直接认识Ben。

- 她可以联系Kate，Kate能够联系John。如果Jane仅仅认识John，而Kate是她的最好朋友并且与John一起工作！这可能就是一个有效的选择。

NODE_GLOBAL和NODE_PATH是使用的最典型的唯一性，但是Neo4j提供了其他一些唯一性类型，可能在某些情况下有用。在下一节中，我们将简要介绍一些其他唯一性类型。

8.3.3 其他唯一性类型

NODE_GLOBAL和NODE_PATH也许是最常用的两种遍历唯一性类型。但是为了完整性，Neo4j遍历API包括其他六种类型。

RELATIONSHIP_GLOBAL唯一性声明图形中的每一个关系只能被访问一次。如果图形中节点之间具有多个关系，每一个节点的访问次数是连接该节点的关系数。

RELATIONSHIP_PATH唯一性与NODE_PATH唯一性相似，其中的关系能被多次访问，只要从起始节点到当前节点之间的关系组合是唯一的。这种唯一性设置也可以使节点被多次访问。

当设置NODE_GLOBAL唯一性或者RELATIONSHIP_GLOBAL唯一性时，遍历可能耗尽内存，尤其是在有很多连接的大型图形数据库中。由于唯一性的限制原因，每一个节点（在NODE_GLOBAL唯一性的情况下）或关系（在RELATIONSHIP_GLOBAL唯一性的情况下）都必须记忆。这里NODE_RECENT和RELATIONSHIP_RECENT设置开始起作用。当使用这些设置时，唯一性的限制稍微可以放宽。

对于NODE_RECENT，记忆访问过的节点有一个上限数。这个规则对NODE_GLOBAL也是一样的，即一个节点只能访问一次，但是只是最近访问的节点集合才被记忆用于比较。最近访问需要记忆的节点能够以第二个参数传递给唯一性方法，如下面的程序所示：

```
TraversalDescription description = Traversal.description()  
    .uniqueness(Uniqueness.NODE_RECENT, 1000)
```

RELATIONSHIP_RECENT与RELATIONSHIP_GLOBAL具有同样的规则，因此关系只能被访问一次。但是在遍历期间，只有指定的一些关系被记忆用于比较。可以使用第二个参数对唯一性方法设置需要记忆的最近访问的关系，如下面的程序所示：

```
TraversalDescription description = Traversal.description()  
    .uniqueness(Uniqueness.RELATIONSHIP_RECENT, 1000)
```

从Neo4j 1.8版本开始，遍历API包括两个更多的设置：

·**NODE_LEVEL** 确保处于同一级的节点（从起始节点开始具有相同距离的节点）在遍历期间只被访问一次。

·**RELATIONSHIP_LEVEL** 确保处于同一级的关系（从起始节点开始具有相同距离的关系）在遍历期间只被访问一次。

在下一节中，我们将讨论双向遍历，这是可以大大增加遍历性能的一种类型。

8.4 双向遍历

在到目前为止你所尝试的所有遍历都是从选定的单一起始节点开始遍历直到找到答案，或者直到走遍整个图形。你还没有从多个起始节点开始遍历图形。这是直到Neo4j遍历API 1.8版本才实现的功能。下面我们介绍双向遍历的概念。

让我们考虑一个拥有一千个用户的典型社交网络图，与图8-5的图形相似。互相认识的用户通过KNOWS关系相互连接。在这样一个图形中的其中一个典型的遍历查询会是“被选定的两个用户（用户1和用户2）是在一个网络中吗”（换句话说，被选定的用户是否连着，如果连着，情况怎样）？

可以使用标准的遍历API解决这个问题：从用户1开始，跟着KNOWS关系；对每一个访问过的节点，检查这个节点是否是用户2。如果是用户2，你找到了这个连接；如果不是用户2，则继续查找，直到走遍全图形。这是一个完全有效的解决方案，但是如果这个图形非常大并且密密麻麻的连着，找到答案将相当费时间。

如果你从用户1沿着用户2的方向开始遍历，同时从用户2沿着相反的方向重新开始一个遍历将会怎样？当这两个遍历相遇时（称作碰撞点），你将已经找到了这个连接，并且可以确定用户之间的整个路径。这就是双向遍历允许你做的事情。图8-6演示了实践中的双向遍历。

双向遍历涉及额外的遍历API部件（我们以前还没有见到）。

· **Start-side traversal** 双向遍历开始侧（出侧或标准侧）的遍历定义。这

个遍历描述是使用本书通篇使用的标准一个方向的遍历API创建的。

- End-side traversal 双向遍历结束侧（进侧）的遍历定义。

- Collision evaluator 每一次起始侧与结束侧遍历碰撞，都有一个潜在的遍历结果。碰撞评估函数可以确定这个结果是否应该包含在遍历的结果中。

- Side selector 这个部件确定遍历在每一侧应该移动的多快。交替侧选择器是最常用的，它每次一步在移动起始侧和结束侧遍历之间做出侧的选择。

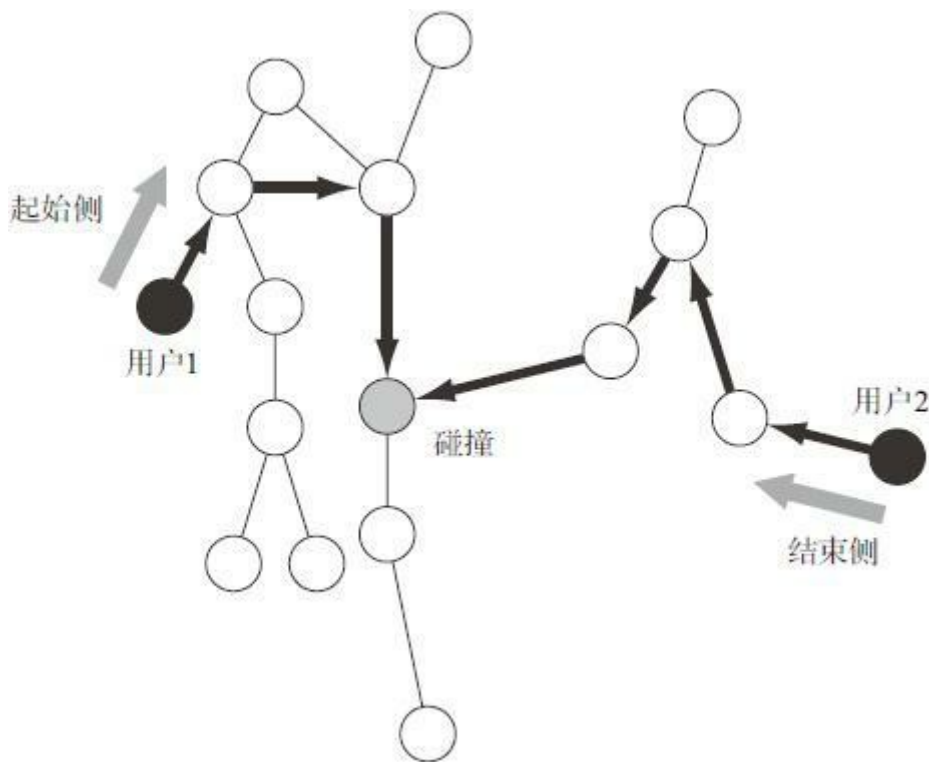
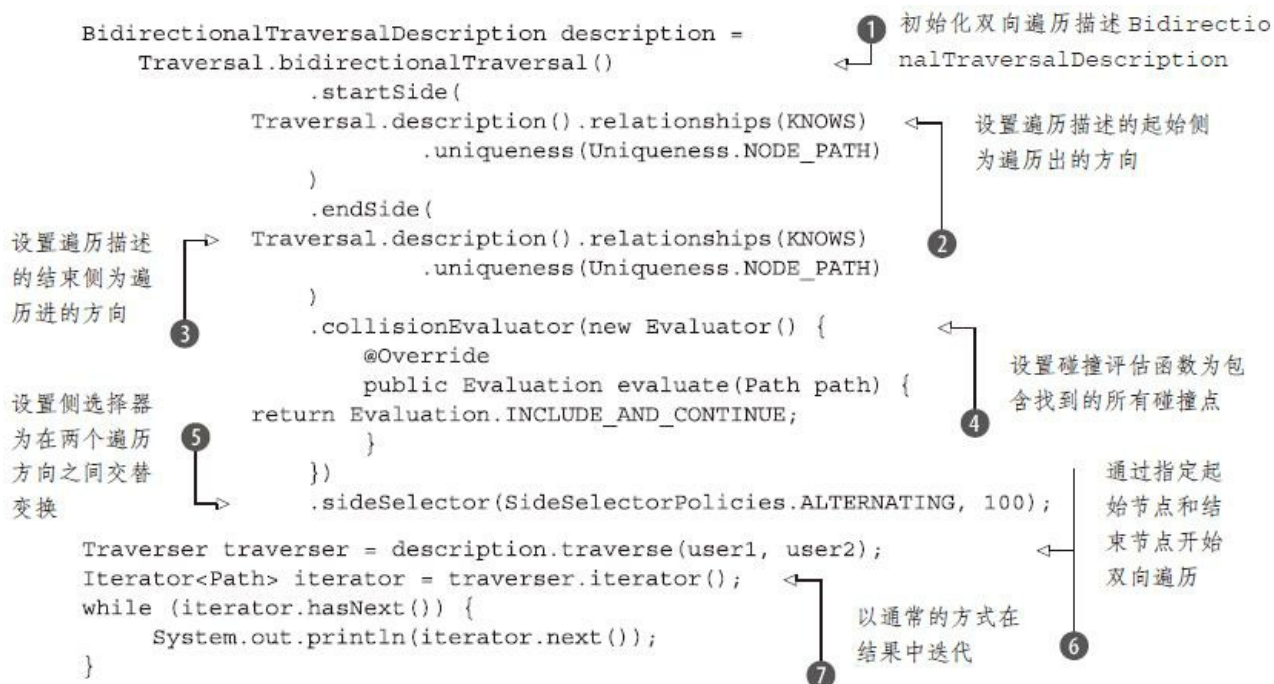


图8-6 用于查找两个节点之间路径的双向遍历

程序8-9给出了一个使用Neo4j遍历API实现的双向遍历的例子。

【程序8-9】 一个社交网络中查找两个用户之间路径的双向遍历



要初始化双向遍历描述，可以使用静态辅助方法 `Traversal.bidirectionalTraversal()`，这与正常遍历初始化相似（❶）。下一步，为遍历描述设置出的方向（❷）。用标准单向遍历API创建 `BidirectionalTraversalDescription` 并以 `NODE_PATH` 唯一性跟随 `KNOWS` 关系。

下一步对双向遍历的方向重复同样的过程，使用同样的遍历描述（❸），但是注意，没有任何东西阻止你从进入方向使用不同的遍历（例如，通过跟随不同的关系）。

再下一步是配置碰撞评估函数（❹）。碰撞评估函数是普通的 `Evaluator` 实现，用于确定当“出”遍历和“进”遍历碰撞时发生的情况。你想要的是返回所有这些碰撞，因此，评估函数简单地返回 `Evaluation.INCLUDE_AND_CONTINUE` 就可以了。

建立 `BidirectionalTraversalDescription` 的最后一步是设置侧选择器（❺），负责确定来自两侧的遍历如何进行。这里提供了内置的 `ALTERNATING` 侧选择器，这正像它的名字所述，在遍历方向之间交替（一步在一个方向，

而另一步则在相反的方向）。

有了这些，双向遍历就做好了准备，因此可以通过调用**traverse**方法并指定开始和结束节点开始遍历了（❷）。与往常一样，在结果中迭代直到结束遍历（❸）。

你从双向遍历中得到了什么好处？就像我们前面提到的，在一个大型图形上做一个标准单向遍历可能会有低于最佳的性能。在这种情况下通过使用双向遍历，有效地利用了两个遍历去遍历图形，把问题的尺度降低了一半。对于经典的单向遍历，遍历的关系数量随着遍历的深度成指数增长。当使用双向遍历时，每一侧的遍历将只需要访问一半的图形深度，做较少的节点—关系—节点跳跃，从而获得更好的性能。



注意

Neo4j中可用的内置图形算法，例如**shortestPath**或**a***图形搜索（来自于**GraphAlgoFactory**类），从Neo4j 1.8版本起可以使用双向遍历。

并不限定在一个方向上只用一个起始点。双向遍历两个方向上都可以有多个起始点。要使用这个特性，你必须使用超载的**BidirectionalTraversalDescription.traverse**方法。

```
BidirectionalTraversalDescription.traverse( Iterable<Node> start,  
                                             Iterable<Node> end )
```

所有这些都使得双向遍历成为一个非常强大的工具。用几行代码，可以写出快速的基于一些固定的节点进行查找的遍历，例如，查找认识某一给定用户组的所有用户。尽管进行双向遍历设置有一点困难，但它们具有更好的性能并且比标准遍历使用较少的内存，尤其是对长路径。

8.5 本章小结

遍历是一个查询图形数据非常强大的方式，在本章中你学习了一些高级概念和技术，能够帮助你写出快速有效的遍历。

使用正确的遍历顺序可以使得一个遍历在速度和内存占用上有极大的提高，尤其是对大型图形数据集。在8.1节你看到了常用的深度优先和广度优先顺序策略对遍历效率的影响。

你也学习了关于关系扩展的关键概念，了解了一个标准的实际扩展器适合于所有典型的遍历使用情况。但是对一些更特定的场景，我们演示了如何通过充分利用功能强大的Neo4j灵活的遍历API，用一个顺序扩展器排序关系和如何实施自定义关系扩展器。

访问过的节点和关系的唯一性是我们将讨论的下一个高级遍历课题。你学习了如何确保在遍历期间访问每一个节点或关系仅仅一次。我们也演示了如何使用不同的唯一性限制来解决一些查询。

最后，你看到了一个很少有人知道，但是Neo4j的一个强大功能：双向遍历。我们实施了一个从多个点开始并汇集到一点的遍历，大大缩短了求解的时间。

在本章中我们学习了许多Neo4j新的复杂概念和特点。但是Neo4j如何与现在流行的Java框架一起工作？是否有一种简单的方式可以用于将图形映射到面向对象的编程模型？这正是我们下一章将要学习的内容，在那里讲述了最流行通用的Java框架（Spring框架）和它们是如何与Neo4j集成的。

第9章 Spring Data Neo4j

本章包括以下内容：

- 使用Spring Data Neo4j（SDN）创建域模型
- 加载和保存SDN域实体
- 在SDN内以对象-图形映射模式工作
- 用SDN做查询

到目前为止，我们一直都在直接使用核心Neo4j图元（节点和关系），用不同的域模型概念进行表示和交互（即读和执行）。

尽管这种方法的功能强大且极其灵活，但低层Neo4j API的操作有时非常繁琐的，并导致许多样板化的代码，尤其是当遇到一些域模型实体时。在本章中，我们将介绍Spring Data Neo4j（SDN），这是一个以更简单、更熟悉为目标的基于Spring开发模型的Spring Data项目中的子项目，在这种情况下，尤其是指Neo4j。

为了演示SDN，我们将返回到第1章描述的并且在本书各章中都使用的第一个允许用户对电影进行评级的社交网络例子。我们将利用这个社交网络演示SDN如何允许使用老式的Java对象（POJO）对域进行建模，并且讲解映射到低层图形结构是如何产生的。我们也将演示如何读、执行和查询这些管理的实体。

9.1 SDN适合做什么

简而言之，SDN是一个对象-图形映射（OGM）框架，是为了简化开发者的工作而创建的（目前只有Java具有），或者说是满足那些需要或者想要用它与基于POJO域模型工作的开发者，在这种情况下，所有的数据都存储在Neo4j中。它的目的是通过处理所有低层工作和从Neo4j读域实体并写回去所必需的映射逻辑来提高效率。这将使你腾出时间把重点放在写代码上——即业务逻辑。图9-1给出了在更广泛的应用中适合SDN的场合。

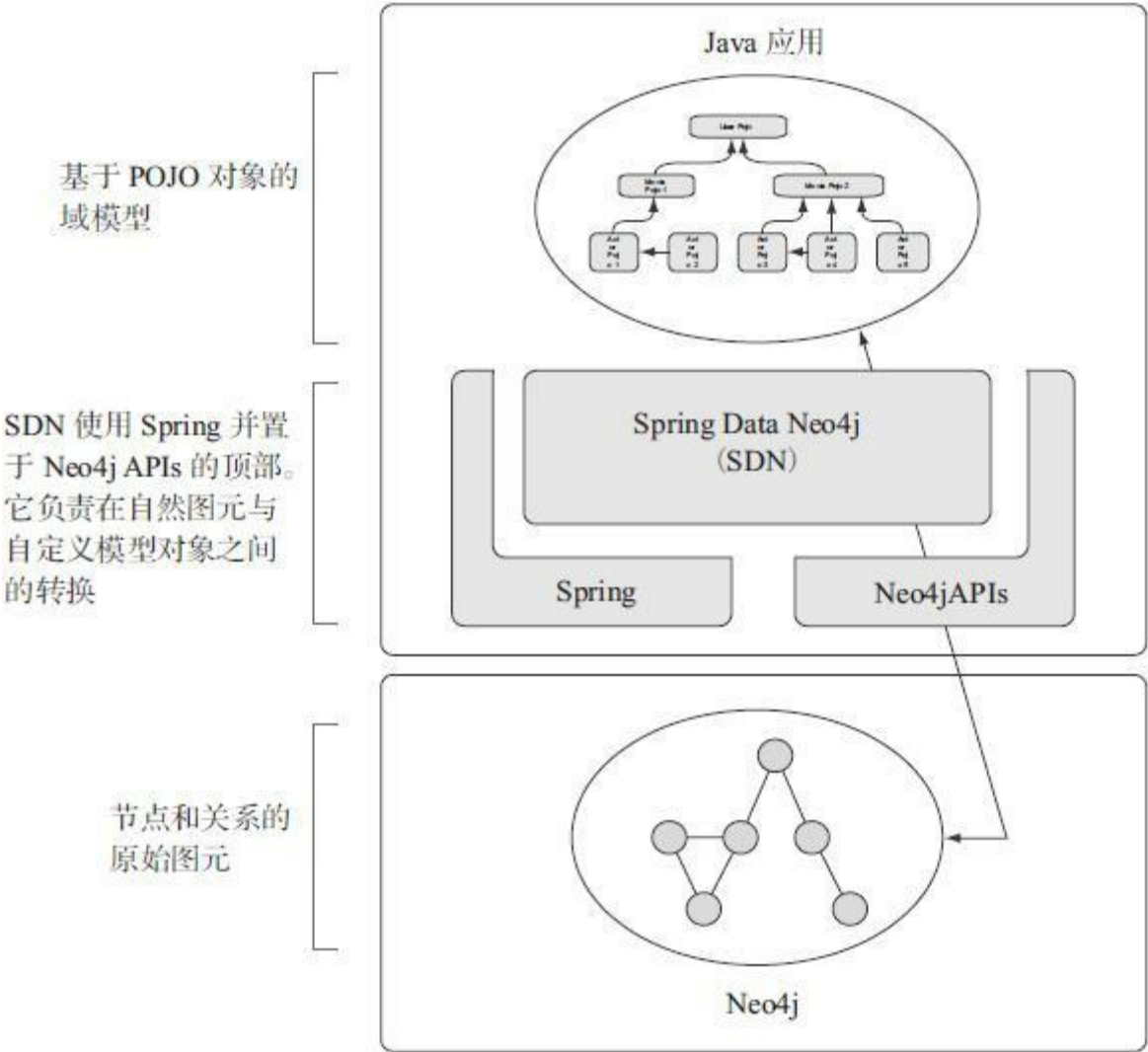


图9-1 在更广泛的应用中适合SDN的场合概况

在计算机软件中，从硬盘读数据和把它转换为应用程序专用的数据结构，然后再把它写回硬盘是非常常用的操作。在早期的Java关系数据库管理系统中（Java/RDBMS），许多开发者手工开发他们自己的持久性映射逻辑，例如，使用低层框架如使用Java数据库链接（JDBC）直接与数据库交互以创建和处理它们的域对象。这种方法被证明是相当容易出错而且在整个应用程序的开发过程中会花费大量的时间，需要开发者对每一个新的项目一遍又一遍地做同样的事情。其他很多对象-关系映射框架，例如Hibernate（<http://hibernate.org/orm>），将一些繁重的工作从开发过程中提取出来并弥补了在物理存储结构与内存中模型的转换之间的间隙。

Neo4j天生的基于图形的存储结构很自然地提供了一个比关系数据库更适宜的对复杂对象图形的存储和恢复。关系数据库常常需要引进额外的数据结构，例如联接表格处理基本的多对多关系；Neo4j很擅长处理这种情况。尽管也会有一些从永久性存储到域对象的数据转换，但如果低层持续性技术是Neo4j图形数据库，则会用SDN来处理。在SDN的情况下，这涉及映射节点和关系的自然图形概念到选定的POJO与模型类。

对非Java项目对象-图形映射将是什么情况？

对于其他的非Java对象-图形映射框架，其中的一些可以在<http://neo4j.org/drivers>这个网站上找到（在普通的Neo4j客户端之间穿插）。下面列举了一些框架，但是请参阅网站上的最新选项。

·neo4j.rb——一个包含Rails Active Model的对象-图形映射类型实现的Ruby绑定和一个Active Record API的子集，参见<https://github.com/andreasronge/neo4j>。

·Neo4j Grails插件——一个集成Neo4j图形数据库到Grails的插件，提供了一个GORM API，参阅<http://grails.org/plugin/neo4j>。

·neo4django——一个对Django的对象-图形映射，请参考<https://github.com/scholarly/neo4django>。

·neomodel——一个对Python的对象-图形映射，参阅<https://github.com/robinedwards/neomodel>。

9.1.1 什么是Spring以及Spring与SDN是怎样关联的

Spring (<http://projects.spring.io/spring-framework>) 是一个非常流行的基于Java的应用开发框架，它为Java的开发者提供生产力工具和实用程序。

Spring本身具有许多子项目处理不同的领域，其中一个Spring Data。在Spring Data中，其中的项目之一是SDN。Pivotal（前身是SpringSource）的工程师认识到有许多可行的数据访问框架可以用在传统的关系数据库中，但是，在NoSQL的全新世界中却并不多见。

开源Spring Data项目是作为一个大项目开发的，具有许多创建的子项目为指定的数据库提供逻辑（其中的第一个是SDN，由Emil Eifrem和Rod Johnson完成的初始版本）。在这一领域的其他NoSQL新人包括MongoDB和Redis。

在所有情况下，来自于Pivotal和低层数据库的开发者使这个框架发展到目前的情况。所有的Spring数据项目都充分利用核心Spring框架内可使用的现有好的做法和已得到证实的功能，用以提供它们的数据访问逻辑的特性。

9.1.2 SDN适合做什么（及不适合做什么）

SDN从本质上给需要或期待操作基于POJO的域实体的开发者提供了一种方便的使用代码或库函数的方法。如果是早已经使用Spring，或早已经在使用丰富领域模型并想映射到一个图形数据库，SDN正适合于做这样的工作。

除已经或想要使用其工作，对于丰富领域模型，另一个需要重点考虑SDN的因素是通常要处理的结果或实体的性质和数量。如果是在几百到几千的范围内，在这种情况下，它本身可以很好地进行设置。SDN不适合一次处理任意类型的大量数据的场景。要加载或存储的任何逻辑在一次操作中超过10000个单元对SDN来说不是一个好的选择。另外，通过提供了一个间接层，SDN会比仅仅使用核心API慢，因此，如果速度和性能是考虑的最重要因素的话，最好还是使用其本身的API。



注意

SDN框架提供了许多工具（提供访问低层GraphDatabaseService实例的代码），可以使用低层核心API获得最佳的性能和最大的灵活性。你仍然可以使用标准的Neo4j工具查看、查询和（在一定的限制内）更新SDN外的图形数据。本章的后面提供了一些有关该课题的更多资料。

9.1.3 从哪里获得SDN

SDN并不包含在主Neo4j中，但是可以以单独的库下载并与合适版本的

Neo4j配合使用。在程序的编写时间，最新的可用版本是3.2.1，与Neo4j 2.1.5版兼容，可以在<http://projects.spring.io/spring-data-neo4j>下载。本章（和随同的程序代码）将会使用这个版本。附录C提供了更详细的如何设置工程以便使用SDN的说明。更多有关介绍SDN 3总体重大变化的资料请参阅<http://blog.neo4j.org/2014/03/spring-data-neo4j-progress-update-sdn-3.html>上的Neo4j博客帖子“Spring Data Neo4j的进展与更新：SDN 3与Neo4j 2”。

9.1.4 从哪里获得更多的信息

SDN本身是一个巨大的项目，我们到目前也仅仅能够了解这些。在本章中，我们把目标放在提供框架中一些比较重要的方面进行介绍而不是一个全面的参考指南。如果你想查找更深入的在这里涉及以及没有涉及的某一方面的资料，有许多这方面的好书，例如Michael Hunger的《Good Relationships》，同时已经被编入官方的SDN参考文档，可以在这里找到：<http://docs.spring.io/spring-data/neo4j/docs/current/reference/html/>（在这里www.infoq.com/minibooks/good-relationships-spring-data免费下载）和Mark Pollak的《Spring Data》与其他的书籍。总体来说，SDN的文档（<http://projects.spring.io/spring-data-neo4j>）也是一个非常有用的参考资源，拥有有关核心Spring框架的更常用的信息，可以在<http://projects.spring.io/spring-framework>阅读。

9.2 用SDN建模

在本节中，我们将介绍如何使用SDN把POJO转换成在电影爱好者社交网络中代表的实体，这里的数据存储在Neo4j数据库中。我们假设都是从空白的画布开始并且将使用SDN作为主要的驱动、创建和设置图形数据库的机制。如果你早已经有了一个图形数据库并且想知道是否能用SDN，请参阅本章后面的内容。

在本节中，你将做以下事情：

- 1) 定义一个标准的POJO对象模型代表域。
- 2) 了解由SDN把这些POJO转换成Neo4j支持的实体的需求。
- 3) 稍微深入一些看看SDN建模的不同元素，包括：①建模节点实体。②建模关系实体。③建模节点之间的关系实体。

重新回顾一下，在这个社交网络中，用户之间可以互为朋友。用户也可以标识他们看过的电影并根据对电影的喜欢程度给它们评为1~5颗星。你将给用户添加一个userId属性，因此用户可以登录。这也会使你唯一标识和指定每一个用户。最后，你将为新用户添加权力以表示他们是否能在登录的时间被其他的用户引用。这可以用作推荐者给新成员在第一个月评价的每部电影的分配积分点，可能获得一张免费电影票或其他福利。图9-2展示了因为通过David的推荐，John最初加入了网络。

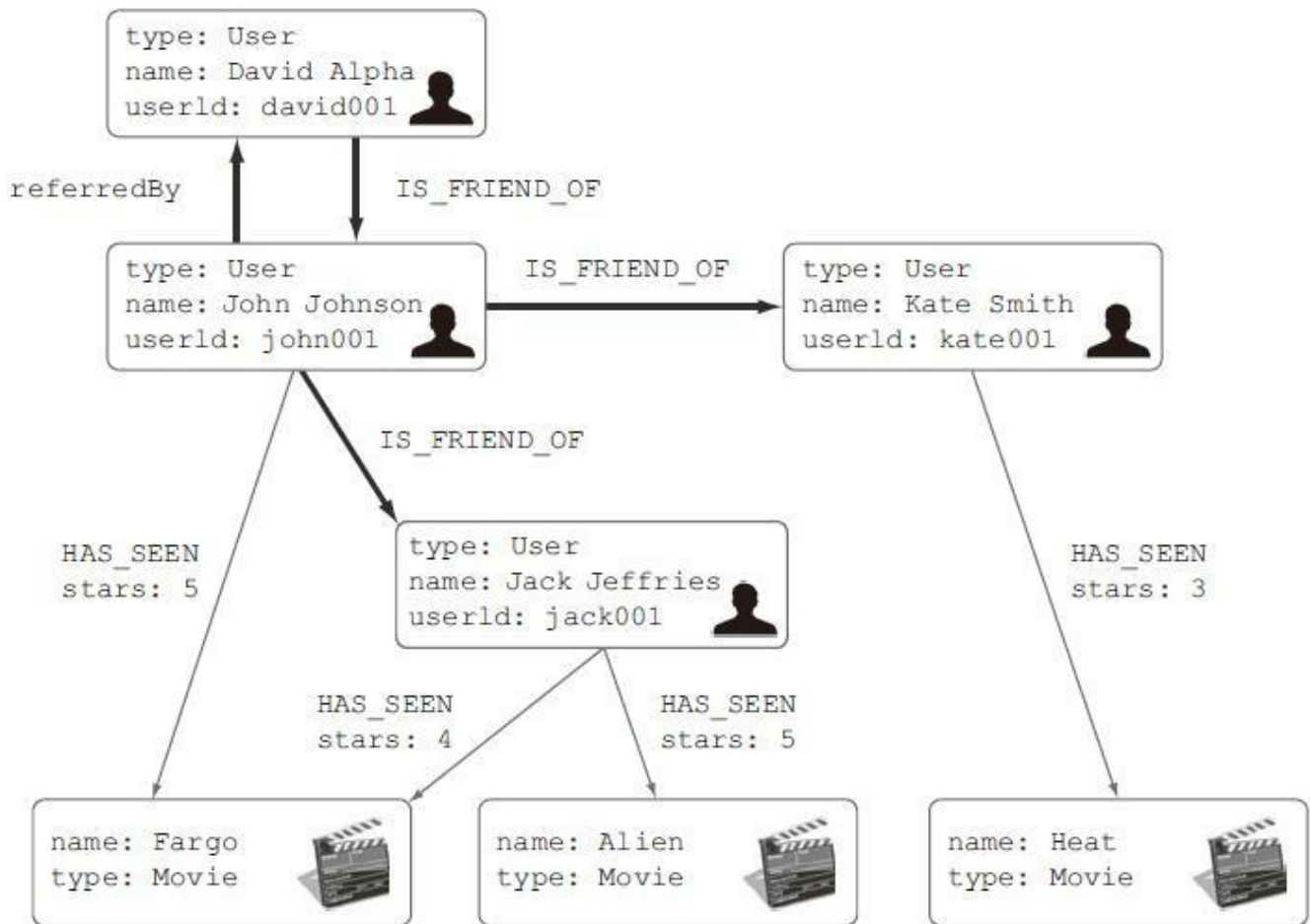
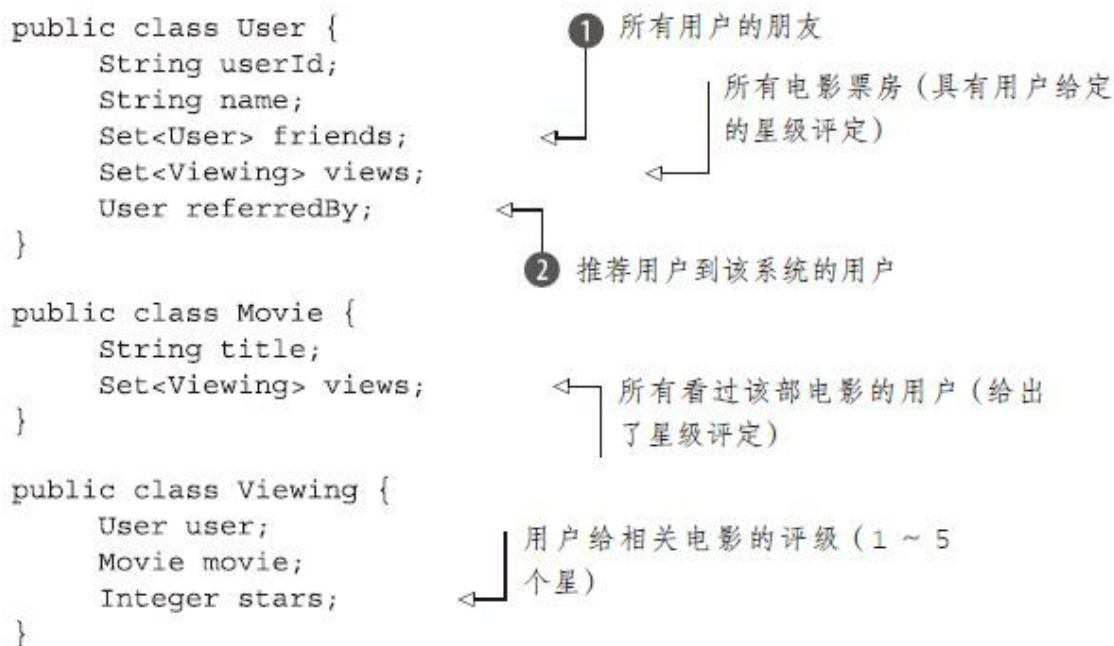


图9-2 具有推荐者的电影爱好者社交网络的概念性概述

9.2.1 原始POJO域建模

如果忽视数据实际上是储存Neo4j中的事实并且只想简单地尝试以POJO建模你的概念领域，则你的第一个尝试将会如程序9-1所示。注意，为了简单省去了getter和setter方法。

【程序9-1】 原始POJO建模尝试



这里并没有什么特别复杂的代码——这是基本的对象建模。User和Movie概念都被建模成第一类的实体，这看上去非常合理。你也将用户观看一部特定的电影建模成一个实体，即Viewing类。这主要是因为关系本身具有很重要的想要保留和使用的信息——即用户给定的评级stars。如果将这个建模成简单的User和Movie之间的连接类关系，则将丢失这些信息。

在这一点上，还没有引用任何的SDN或Neo4j特定的概念，只是POJO的内容。下一步，我们将需要把这些实体映射到低层Neo4j图形模型。

现在距离创建容易翻译成Neo4j领域的POJO模型有多接近了？可以不修改就使用它或者需要对它做修改吗？在这一特定的情况下，你似乎有一个不错的选择，将User和Movie类分别整洁地以相关的name和title属性映射到Neo4j节点图元概念。

新的referredBy关系（②）代表了推荐用户的一个推荐者，而IS_FRIEND_OF是用户与一组朋友之间恰当映射的关系（①）。唯一棘手的部分似乎是围绕着Viewing类的建模，它将试图代表一个用户看过一部电影

和可选的给予评价的情况。然而，进一步的检查显示这也非常适合Neo4j的关系概念。Viewing类代表了具有选项stars属性的HAS_SEEN关系，以及看过这部电影的User和Movie本身的引用。

到目前为止，一切都很好。现在该进行SDN映射了。



不可能总会找到一个逻辑POJO模型与Neo4j物理结构非常接近。如果有，那是非常幸运的，但是在其他情况下，有可能要修改模型以适应Neo4j的物理结构。在任何情况下，逻辑模型强调的重点是如何将通用的POJO建模概念转换成Neo4j结构。

SDN建模的挑战

在本章中我们使用的逻辑模型碰巧可以很容易地转换成Neo4j物理存储结构而不需要进行过多的调节。有必要强调有两种情况可能需要微量调整的对象模型来满足一些SDN映射的需求。

·使用基于非设置集合——当建模节点实体关系时（通过@RelatedTo，将在9.2.5节介绍），现在只能使用基于设置的集合。例如，不能使用以下的代码：

```
@RelatedTo
private Map<RelationshipType, Set<User>> users;
```

·实体平等性——SDN需要数据库编号（节点/关系编号）构成部分对象的标识。引自参考指南“实体平等是一个灰色地带，并且是否是自然键还是数据库编号最好的描述了平等性还是有争议的，还有随着时间的版本问题

等”。我们已经采用了Spring Data Neo4j数据库产生编号的传统方法是平等性的基础，并且这也会有一些后果。有一些技术以应对这种场景，但是本书中不涉及这些问题的内容。我们再一次向你推荐官方网站参阅更多的信息，地址是<http://docs.spring.io/spring-data/neo4j/docs/current/reference/html>。

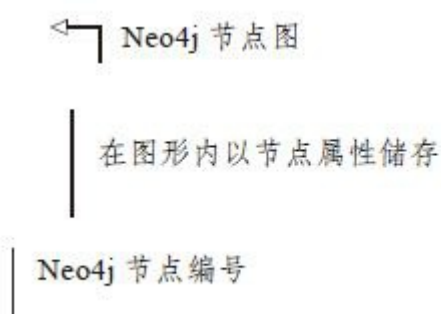
9.2.2 注释域模型

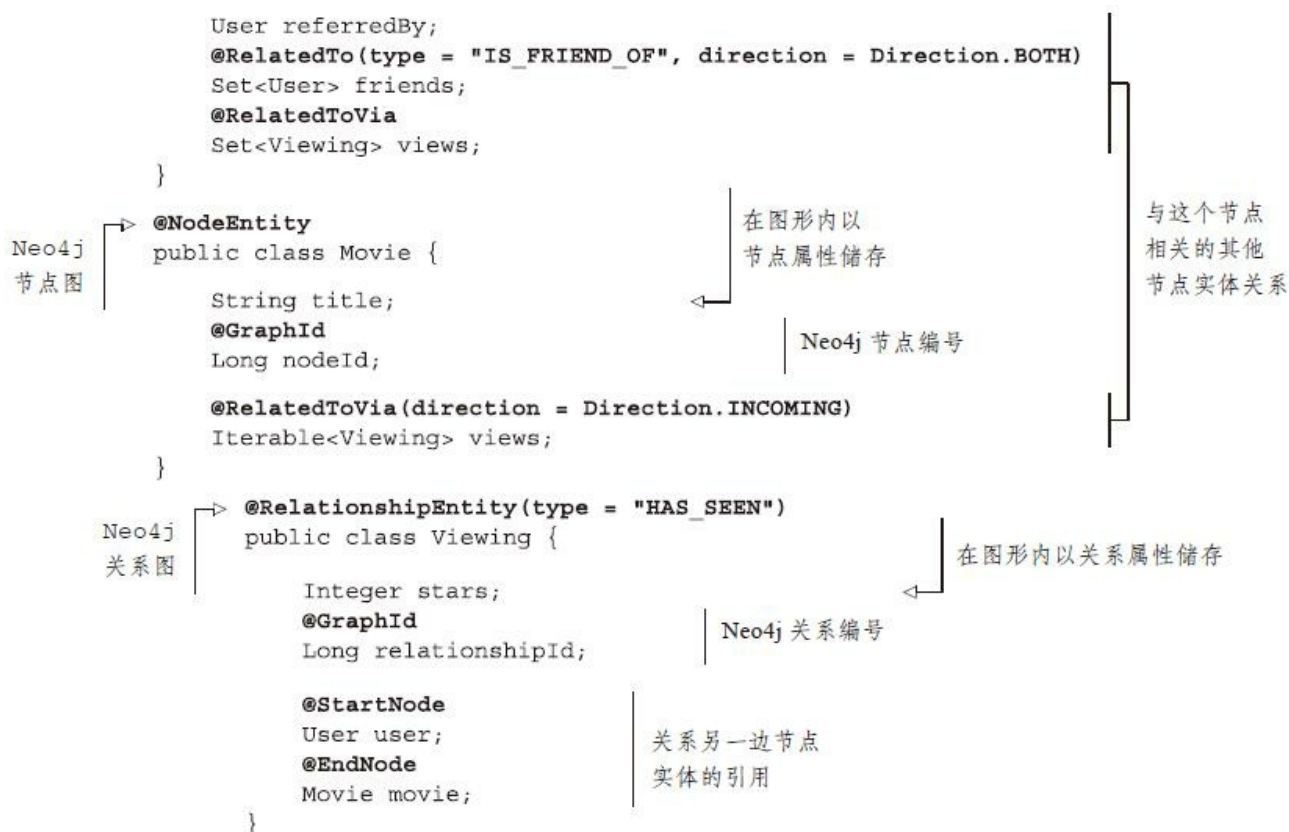
SDN是一个基于注释的对象-图形映射库。这意味着这是一个依赖于能识别一些附在你的程序中的SDN专用注释库。这些注释提供了如何转换相关的代码到图形数据库低层结构的指导。

有时你甚至会发现不需要注释程序的某个片段。这是因为SDN试图使用设置中的一些传统原理推理一些有意义的默认设置。OGM对图形数据库就像ORM对关系数据库一样。

程序9-2给出了Neo4j支持的添加到POJO中作为实体识别它们的SDN注释。

【程序9-2】 SDN注释的域模型

<pre>@NodeEntity public class User { String name; @Indexed(unique=true) String userId; @GraphId Long nodeId;</pre>	
--	--



这些注释，包括SDN合理的基于字段名的默认假设，直接连接着Java类元素和Neo4j中的物理实体。这意味着当用SDN建模时对数据模型有一个很好的理解是非常重要的。尽管SDN不需要你必须做低层的映射，但是期待你能够描述它是如何做的。

接下来一节将分解程序9-2并解释核心建模概念，更详细内容如下。

- 建模节点实体。
- 建模关系实体。
- 建模节点实体之间的关系。

我们从节点实体开始。

9.2.3 建模节点实体

在SDN内，一个节点实体是指一个Java类，用于代表一个特定的域实体，该实体是低层数据库中由一个Neo4j节点元支持和代表的。图9-3显示了社交网络域模型中的能够被建模成SDN节点实体的候选节点。

这里Movie和User类是非常完美的例子。使用@NodeEntity注释标记一个节点实体，一般放在Java类定义之前，如程序9-2和下面的程序所示。

1.属性

在用@NodeEntity注释的类中，在默认的情况下SDN将把所有的简单字段作为后备节点上的Neo4j属性的代表。在文字上，简单是指：

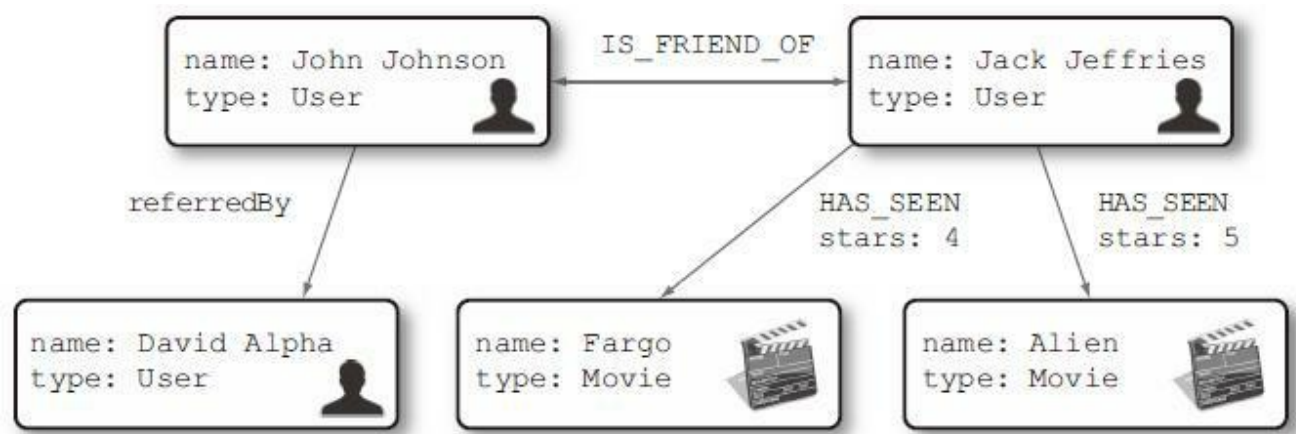


图9-3 具有加亮节点显示的社交网络模型

Neo4j 节点支持的类

```
@NodeEntity
public class User {
    String name;

    @Indexed(unique=true)
    String userId;
    @GraphId
    Long nodeId;
    ...
}
```

1 低层节点的名字属性图

2 确保用户编号节点被索引（使用基于模式的索引）并且是唯一的

3 包含 Neo4j 节点编号的属性

- 任何原始的或与它相关的封装类。
- 一个字符串。

·通过Spring传统服务能够被转换成字符串的一个对象。

这些类型的任何集合也包括在内，这些集合最终作为数组存到Neo4j节点上。在前面的例子中，这意味着分别定义在User和Movie类上的name（❶）和title字段将作为Neo4j属性获得与那些相同名字的映射，而不需要你做任何工作。

自定义属性类型是什么情况

核心Spring框架附带一个通用类型转换系统允许你编写和注册自定义类型转换逻辑，该逻辑可以转换指定的（非元）类到一个字符串描述，或者反过来也可以。SDN早已经注册了一些转换服务以处理枚举和日期类。例如，如果在一个节点实体上定义一个java.util.Date字段，当这个字段在存储的时候，Spring发现有一个自定义的日期转换器并使用它转换日期属性到一个字符串，然后存到图形数据库中；相反的情况发生在从图形数据库中读日期时，又转换成节点实体字段。

这意味着你可以利用同样的机制处理你想用的任何自定义类，例如电话号码对象。你将会需要写一个自定义转换器并在Spring注册。本书的样本代码提供了一个如何通过给User实体添加一个phoneNumber的例子。更多关于转换机制信息，请参阅《Spring Framework Reference》的第6

章，<http://static.springsource.org/spring/docs/current/spring-framework-reference/html/validation.html#core-convert>。

如果不注册一个转换器，SDN仍会在数据库中存储一个字符串版本的对象。然而，这将简单地依赖于从toString（）方法中返回的是什么内容。

你可能已经注意到一个额外的字段混进了User和Movie节点实体，也就是nodeId，由@GraphId注释的。当使用简单对象映射时，这是一个强制性的SDN需求。现在还不需要了解太深，简单对象映射是SDN使用的实施域实体与Neo4j支持的图形实体之间的物理映射的策略之一。使用这种策略，SDN需要一个可以用于存储低层支撑实体的节点编号的字段。@GraphId注释（）用于标记留出来给SDN用于存储这个值用的字段。在第9.4.1节可以读到更多有关简单对象映射的内容。

2.索引属性

由于一般认为对于一个应用程序依赖于Neo4j节点编号作为一个对象的外部唯一标识符不是一个好的习惯（参阅5.6.1节），有另外一些查找节点的方法是很重要的。前面程序中的userId字段添加@Indexed注释以确保查找和查询能够相对于这个用户并基于这个节点进行。默认的情况下，@Indexed使用依赖于Neo4j标签的基于模式索引。查找实体的代码将会在9.3节讲解，但是Neo4j内部使用与5.5.1节中完全相同的模式索引代码和逻辑来实现。



注意

如果你还是喜欢使用传统的索引，可以使用indexType属性指定，例如，@Indexed（indexType=IndexType.SIMPLE）。

对这些注释属性的哪些标签和名字索引

默认情况下，SDN使用一个基于标签类型的描述策略，这个策略关联一个实体层级结构中（这里是User）具有名字为简单版本名字类标签的节点实体的低层节点。然后，对这个标签定义的属性名字的模式索引就创建起来

了。

如果想要写一些Cypher查询直接在图形中的userId属性上进行查找，你需要以下的查询：

```
MATCH (n:User) WHERE n.userId = {userId} return n
```

注意如何命名和建立域实体结构。由于默认索引的性质，通常不建议两个实体具有相同的名字，即使它们俩在不同的包里。如果在core包里有一个User域实体，另一个在admin包里，他们最终会共享相同的标签，这通常不会是你想要的。通过对这个类设置一个@Alias注释，有可能覆盖这一默认行为。然而，对那些没有注意到这种行为的人可能是一种困惑，可能会导致意想不到的结果。

3.对其他节点实体的关系

节点实体之间也有关系。这些实体上作为参考字段的简单映射，将会在9.2.5节介绍。

在我们详细了解这能做什么之前，你需要学习关系实体。

9.2.4 建模关系实体

关系实体是一个节点实体的关系版本。它是指最终由一个低层数据库中的Neo4j关系代表和支持的Java类。Neo4j将关系作为第一类公民，就像节点一样，可以拥有它们自己的标识符，如果需要可以设置属性。因此，SDN也允许将它们表示为实体拥有它们自己的权利。图9-4显示了域模型内的这些关系。

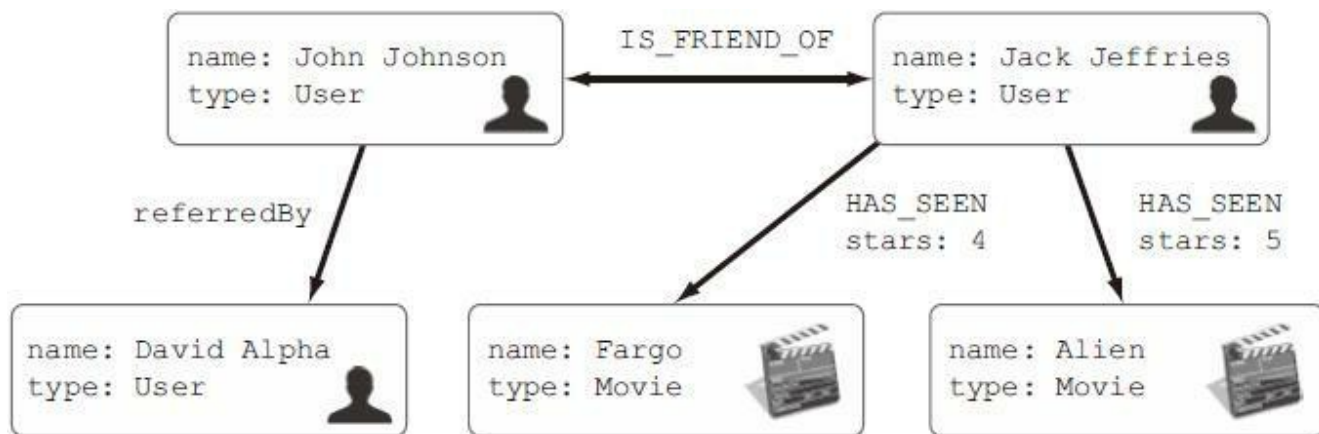


图9-4 具有潜在能够建模成关系实体的加粗关系显示的社交网络模型

在本节中，我们将讨论作为一个POJO本身建模的具体Neo4j关系实体的需求。在9.2.5节中，我们将给出从节点实体的角度通过这些模型关系以及通过更简单的机制引用其他节点实体的需求。

注意

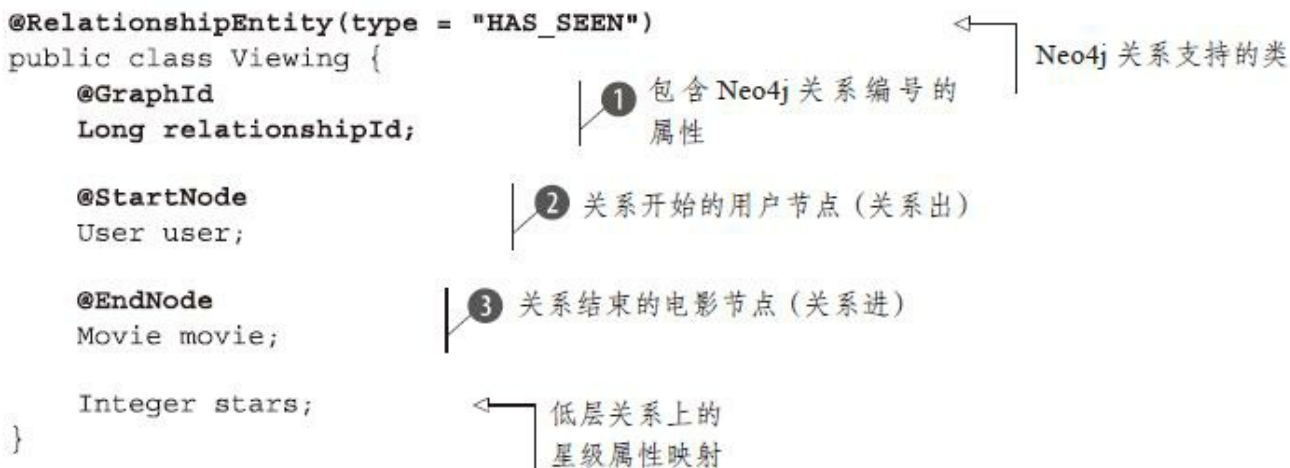
在Neo4j具体模型中，你或许定义了很多关系，但是这不会自动意味着它们当中的所有都需要在SDN中建模为关系实体。

SDN关系实体一般只在具有自己的属性设置关系中是需要的，并且与这些关系一起，提供了上下文联系的关系。我们将称这些关系为丰富关系（rich relationship），这是由于它们包含着超出了关系类型的额外数据。

HAS_SEEN关系是这种关系的一个很好的例子，具有额外的stars属性提供了这个关系的更多内涵。它表示了不仅仅是用户看过这部电影，而且还表示了用户对它的评级。在这个社交网络模型中，这个关系与它相关的信息被定义为Viewing类，如程序9-3所示。与之对比的是IS_FRIEND_OF关系，对于它所有需要理解的是两个用户之间的关系——即他们是朋友。这些简单的关系（这里的类TYPE单独就足以完全描述这个关系，例如IS_FRIEND_OF

关系)也能够被引用,你将在9.2.5节中看到这是如何实现的,但是定义一个全新的类来代表它们并没有额外的好处。

【程序9-3】 作为一个关系实体的Viewing类



`@RelationshipEntity` 注释应用于类表示它代表一个 Neo4j 关系。注释取一个 `type` 属性表示数据库内本身使用的 Neo4j 关系类的名字。如果这个 `type` 属性不是在注释内显式指定的, SDN 会默认将简单类的名字指定给它 (在本例子中是 `Viewing`)。对于节点实体, 一个 `@GraphId` 注释字段的关系实体具有相同的需求 (1), 这次存储的是低层关系的 ID。

如果你想访问这个关系实体上的任何一端的节点实体, 你将会需要提供这些节点中的每一个节点的一个字段并且用 `@StartNode` (2) 和 `@EndNode` (3) 注释它们。对于 `Viewing` 类的例子, 关系从 `User` 节点实体开始 (具有出的关系) 到 `Movie` 实体结束。

就一个 Neo4j 关系建模的需求而言, 这就是它的需求。然而, 一般来说在定义隔离中的关系实体类时没有多少需要注意的地方。它们几乎总是指相关节点实体上的一个或多个字段。在 9.2.5 节中, 我们将详细讨论节点实体如何通过简单引用来引用其他节点实体, 但是也可以通过 POJO 建模的关系引

用。为了提供这个例子的详细情况，程序9-4提供了一个先睹为快的User节点实体以及Movie节点实体如何通过它们的views字段引用Viewing关系实体的代码实例。

【程序9-4】 User和Movie节点实体程序节选

```
@NodeEntity
public class User {
    @RelatedToVia
    Set<Viewing> views;
    ...
}

@NodeEntity
public class Movie {
    @RelatedToVia(direction = Direction.INCOMING)
    Iterable<Viewing> views;
}
```

① 所有 HAS_SEEN 关系组
(从 User 的角度)

② 只读所有 HAS_SEEN 关系组
(从 Movie 的角度)

在User节点实体内，在views字段的RelatedToVia注释（①）实质上读到了作为“这个用户与这个用户看过的任意电影之间的所有HAS_SEEN关系（具有任意相关的属性）”（这个HAS_SEEN关系类型是推断的，因为Viewing类注释本身的属性类型定义所包含的内容）。Viewing类代表这两个包括评级字段的实体之间的关系的全部内容。注意在这种情况下（不像下一节详细讲述的Movie类），你不用在注释内显式指定direction=Direction.OUTGOING，因为这是默认的。

在Movie节点实体内，RelatedToVia注释（②）标志views字段作为“这部电影和任意看过这部电影的用户之间的所有HAS_SEEN关系的代表（具有任意相关的属性）”。

在两种情况下Viewing类都提供了一种对关系访问额外上下文信息的方式，在这种情况下，事实上这两类实体仅仅是相关，额外的上下文信息是每一次观看所指定的评级。在这一阶段你可能会惊奇为什么在一种情况下关系

会用一个Set（对User）建模而另一种情况（对Movie）是用一个Iterable建模。所有的这些都将在下一节中讲解。

在下一节中，我们将继续详细探讨定义节点实体之间的不同关系类型的细节，包括完全理解其内容所需要的丰富关系的详细解释。

9.2.5 建模节点实体之间的关系

到目前为止你仅仅能够建模具有隔离中的简单相关属性的节点和关系实体。当你能够实际上连接它们用以探索它们之间的关系时，模型就开始变得有趣了。在本节中，我们将讨论如何做这些。

上一节的结尾给出了一个如何通过Viewing关系实体建立一个User和Movie之间的这样一个连接的简单示例。看到User和Movie之间的HAS_SEEN关系是如何作为一个具体POJO（Viewing类）建模并在实体内被引用的。在这种特殊的情况下，使用一个完全分离的类（Viewing）来代表关系。但是其他的简单关系将会怎样？例如“John是Jack的朋友”。对这种情况是否也需要一个专用的关系实体类？答案是不需要——它们可以用一个更简单的方法来处理。

图9-5概括了你可能从其他节点实体感兴趣引用的节点实体之间的关系。

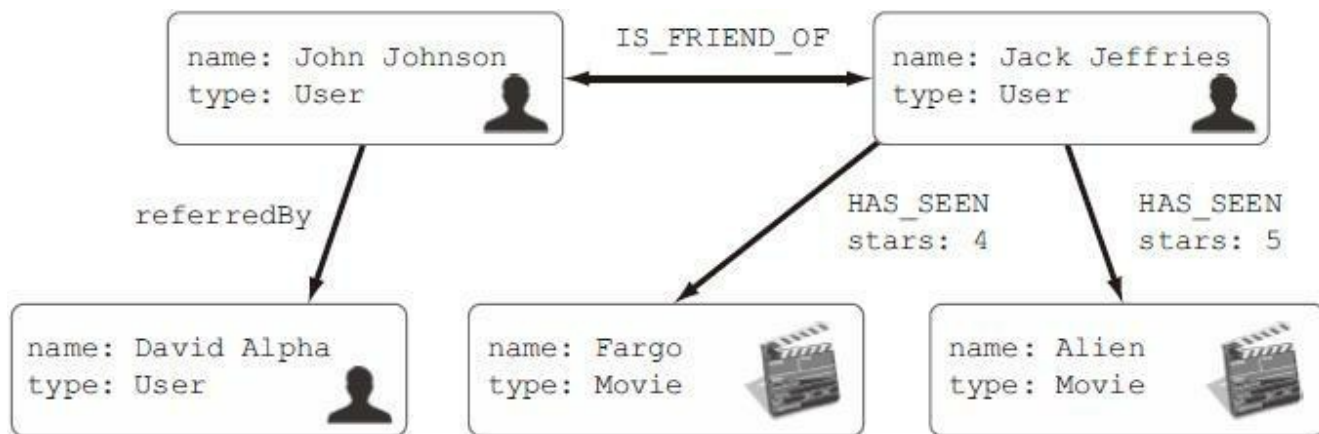


图9-5 具有加粗显示的节点之间引用关系的一个社交网络模型

从节点实体的角度看，关系也是被建模成普通Java对象引用，它们有很多的种类，取决于你想传达的是什么。我们已经预览了如何使用Viewing类引用HAS_SEEN关系，但是也让我们抓住其他更简单的关系，例如referredBy和IS_FRIEND_OF。

涉及User和Movie实体的关系显示在程序9-5中。

【程序9-5】 User和Movie节点实体程序节选

```

通过referredBy ① public class User {
关系与这个节点    User referredBy;
相关的用户(节    @RelatedTo(type = "IS_FRIEND_OF", direction = Direction.BOTH)
点实体)          Set<User> friends;
                  @RelatedToVia
                  Set<Viewing> views;
                  ...

public class Movie {
    @RelatedToVia(direction = Direction.INCOMING)
    Iterable<Viewing> views;
    ...
}

```

② 通过 IS_FRIEND_OF 关系与这个节点相关的用户 (节点实体)

③ 这个用户和电影之间的 HAS_SEEN 关系信息

④ 这部电影和给予评级的用户之间的 HAS_SEEN 关系信息的只读版本

由底层没有相关属性或不与任何其他节点实体相关的Neo4j关系代表的基本关系，能够在节点实体内被建模成标准的对象引用。在默认的情况下，将会使用属性的名字作为名字在Neo4j中映射到关系类，在没有任何元信息的情况下则会相反。新引进的一个用户引用另一个的概念到社交网络（以referredBy属性建模（①））是一个这方面的非常好的例子。注意这个属性

必须是另一个节点实体的引用。

对零个或对多个其他节点实体的基本关系可以用Set、List、Collection或Iterable类建模，把引用的节点实体作为集合类。Set、List或Collection类的使用表示从包含的节点的角度看字段被修改了；Iterable类表明这应该是只读。基于包含的节点实体的类型和它的注释，SDN能够知道这个字段代表一个基本关系。然而，如果你想覆盖任何的默认推断，你可以加一个@RelatedTo注释。用户之间的friends关系（❷）是一个例子。注意在这种情况下如何添加具有type属性的@RelatedTo注释来指定低层Neo4j关系为IS_FRIEND_OF，而不是在没有指定值的时候SDN推断的默认值。在没有注释的情况下，SDN将使用字段的名字friends作为低层关系类型属性的名字。

具有相关属性的Neo4j低层关系代表的丰富关系也是以同样的Collection类作为基本关系建模的。然而，在这种情况下，在集合中包含的实体的类型是一个关系实体而不是一个节点实体。作为概括，一个关系实体代表低层Neo4j关系与相关的属性，这些也是以实体建模的（参见9.2.4节）。这提供了一种巧妙的方法访问关系本身的丰富信息，而且仍然能够获得另一端的实体。由于只用基本关系，不用任何注释，SDN能够推测你正在创建这样的引用，这个引用仅仅基于在集合中包含的类类型已经被定义为一个关系实体。再一次说明，如果你想覆盖SDN假定的任何这些默认的关系，可以使用@RelatedToVia注释。正像你早已看到的，views字段引用（❸）所代表的一个User和一个Movie之间的关系是一个很好的例子；额外的stars评级增强了这两个实体之间关系的信息。注意在User和Movie节点情况下用在views属性中的不同Collection类，即Set<Viewing>和Iterable<Viewing>。这意味着从User节点的角度看views属性可以被修改，但是从Movie节点的角度则不可以

修改。从概念上讲，用户评价电影；电影不能评价用户。

`@RelatedTo` (❷)、`@RelatedToVia` (❸) 和注释 (❹) 都能够设置类型并且具有可选方向的元素，用以指定说明关系是否具有特定的方向（有效值是INCOMING、BOTH和默认值OUTGOING），并应该包含在返回的实体集中。

注意对friends属性你需要指定方向为BOTH（双向），如下面的程序所示：

```
@RelatedTo(type = "IS_FRIEND_OF", direction = Direction.BOTH)
Set<User> friends;
```

从逻辑上讲，一个friends关系是双向的；然而，在Neo4j内，关系实际上仅仅以一个方向存储。通过指定BOTH，你告诉Neo4j要考虑任意的与用户相关的IS_FRIENDS_OF关系，而不必考虑定义的实际关系的方向。

9.3 访问和持久化实体

你已经看到如何使用注释为SDN提供基本信息，用于映射POJO类到低层图形支持的数据库，而不需要很多的低层节点或关系类。这是一个在你的域模型代码中仅仅处理商业级概念的巨大进步。下一个逻辑问题是如何能与这些实体交互和怎样能加载并保存来自于或回到图形数据库中的POJO实体。

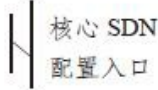
SDN在这一领域提供了几个选项，本节中我们将重点关注其中的两个，即Neo4jTemplate类和更通用的Spring数据库概念。在这之前，我们将看看支持Spring的配置，并对其进行配置以保证正常运行。

9.3.1 支持Spring的配置

程序9-6给出了激活SDN需要的最低XML Spring配制，SDN可以用以查找域实体并且生成许多不同种类的有用工具（例如Neo4jTemplate）。

【程序9-6】 基于XML的Spring配置

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:neo4j="http://www.springframework.org/schema/data/neo4j" ...>
  <neo4j:config storeDirectory="target/sample-db"
    base-package="com.manning.neo4jia.chapter09.simple.domain"/>
</beans>
```



核心 SDN
配置入口

SDN使用neo4j: config入口做许多初始化工作；例如，可以将storeDirectory属性作为一个方便的方式来引用图形数据库（如果数据库不存在就创建一个新数据库）。在SDN 3.0以前的版本中，使用base-package属性

指定域实体定义所在的目录或目录列表是强制性的。



如果你不喜欢XML，可以使用Spring Java设置获得同样的输出。

让我们继续看看如何使用Neo4jTemplate类与实体进行交互。

9.3.2 Neo4jTemplate类

Neo4jTemplate是一个SDN类，能用于实例化，可以直接使用或当Spring初始化时用于你的应用程序。相似于一些其他成功的Spring模板类的精髓（例如JdbcTemplate和JMSTemplate），Neo4jTemplate类的目标是提供一个方便、简单的API与低层类和低层Neo4j基于图形类的行为需求进行交互。

非常有趣的是，如果合适，SDN框架本身也会使用Neo4jTemplate做许多内部工作。但是模板本身并不限于仅仅做内部工作，也可以用作选项之一调用SDN周围实体的基本函数。

Neo4jTemplate：人们强有力的工具！

Neo4jTemplate类也提供许多其他低层应用方法用于操作、查询以及获得对节点和关系的访问。如果你真的需要“深入虎穴”并且想获得更加精细的控制，这是SDN提供的深入到核心低层API的方法之一。一定要在某种程度上探索所有可能的其他方法。

程序9-7是一个演示如何使用最基本形式的Neo4jTemplate类来保存和加载User实体的例子。

【程序9-7】 一个基本的Neo4jTemplate例子

```
GraphDatabaseService graphDB = getGraphDatabase();  
Neo4jTemplate template = new Neo4jTemplate(graphDB);  
  
try (Transaction tx = template.getGraphDatabaseService().beginTx()) {  
    User user = new User("john001", "John");  
    User savedUser = template.save(user);  
    User loadedUser = template.findOne(savedUser.getNodeId(), User.class);  
    tx.success();  
}
```

需要引用图形数据库

仅操作域对象
(模板本身除外)

首先，Neo4jTemplate类需要对它最终操作的低层图形数据库的一个引用来做它的工作。前几章用到的以本地Neo4j结构工作的GraphDatabaseService也是以同样的方式工作的。

在程序9-7中，尽管你只希望处理真实域对象，但是还会有许多“噪声”（那些样板事务代码）。通过充分利用具有依赖注入（Dependency Injection, DI）函数的Spring框架，可以很好地处理这种问题。按照这种方式，你的代码将转换成如程序9-8所示。

【程序9-8】 具有完全Spring集成的一个Neo4jTemplate例子

```
@Autowired Neo4jTemplate template;  
  
@Transactional  
public void saveAndLoad() {  
    User user = new User("john001", "John");  
    User savedUser = template.save(user);  
  
    User loadedUser = template.findOne(  
        savedUser.getNodeId(), User.class);  
  
    User loadedUserViaIndex = template  
        .findByIndexedValue(User.class,  
            "userId", "john001")  
        .singleOrNull();  
}
```

通过 Spring
编号使得可用

以前的样板代码被注释替换

1 保存节点实体
到图形数据库中

2 通过节点
编号查找

3 通过一个索引查找

这一段新代码非常接近Spring的完满功能，仅仅需要写核心业务逻辑，推迟所有的Spring低层工作。样板事务代码现在被@Transactional注释所代替，指导Spring以确保所有在方法中执行的代码包装在一个事务中。

聚焦到核心维护逻辑，你可以看到创建和保存一个新实体就像在Neo4jTemplate类上调用save方法（❶），作为参数传递新创建的基于POJO的实体一样简单。Neo4jTemplate在数据库内部为你创建了一个新的节点，赋予新的（或者重用的）图形节点编号，保存所有相关的属性（userId和name），并且索引userId属性。

程序9-7展示了两种方法加载一个实体。第一种方法（❷）需要相关的知识并且要访问图形节点编号，在这其中有作为前一个调用的结果保存这个实体（❶）中。回忆一下，在User实体上@GraphId注释nodeId属性是SDN存储低层节点编号所必需的。你现在只是为了你自己的查找目的简单利用了这一信息。

在节点编号不可用或不知道的情况下，第二种方法是通过索引userId属性查找实体（❸）。

尽管这段代码没有像它应该的那样清楚，但它确实能做这项工作。在下一节中，你将看到如何通过库查找实体，这将做得更好并隐藏需要做索引查找以及其他函数的内部低层工作。

9.3.3 资源库

用于加载和保存实体的第二个主要选项是通过一个资源库模式的实现来讨论的，在这里你可以发现一些在不同的书中讨论过的内容，这些书包括Eric Evans的《Domain Driven Design》和Martin Fowler的《Patterns of Enterprise Application Architecture》。

简单来说，一个知识库在你的业务代码和需求的所有逻辑之间提供了一个抽象层，用以转换这些实体到它们在任意低层中的格式或从它们在低层的格式转换到实体。你的商业代码应该以域语言去处理事情；这些域实体实际上如何加载和保存是一个特定知识库实现的工作。使用这种方法，如果你想交换出你的数据存取，你没有必要扔掉你所有的业务代码，而是用一个知道如何翻译域实体到或从数据仓库格式替换适当的资源库实现。

SDN资源库在某一段时间是专门设计用于一个域类，并且它们是针对那个域类提供整个范围的默认操作。SDN资源库真正的优异特色是它们不需要你写一行代码来实施这些默认操作。所有你需要做的是定义一个接口和指定哪一个域类你想要这些操作去创建。

使用User域类作为一个例子，让我们看看这项工作以及所涉及的东西。图9-6给出了我们开始的点。

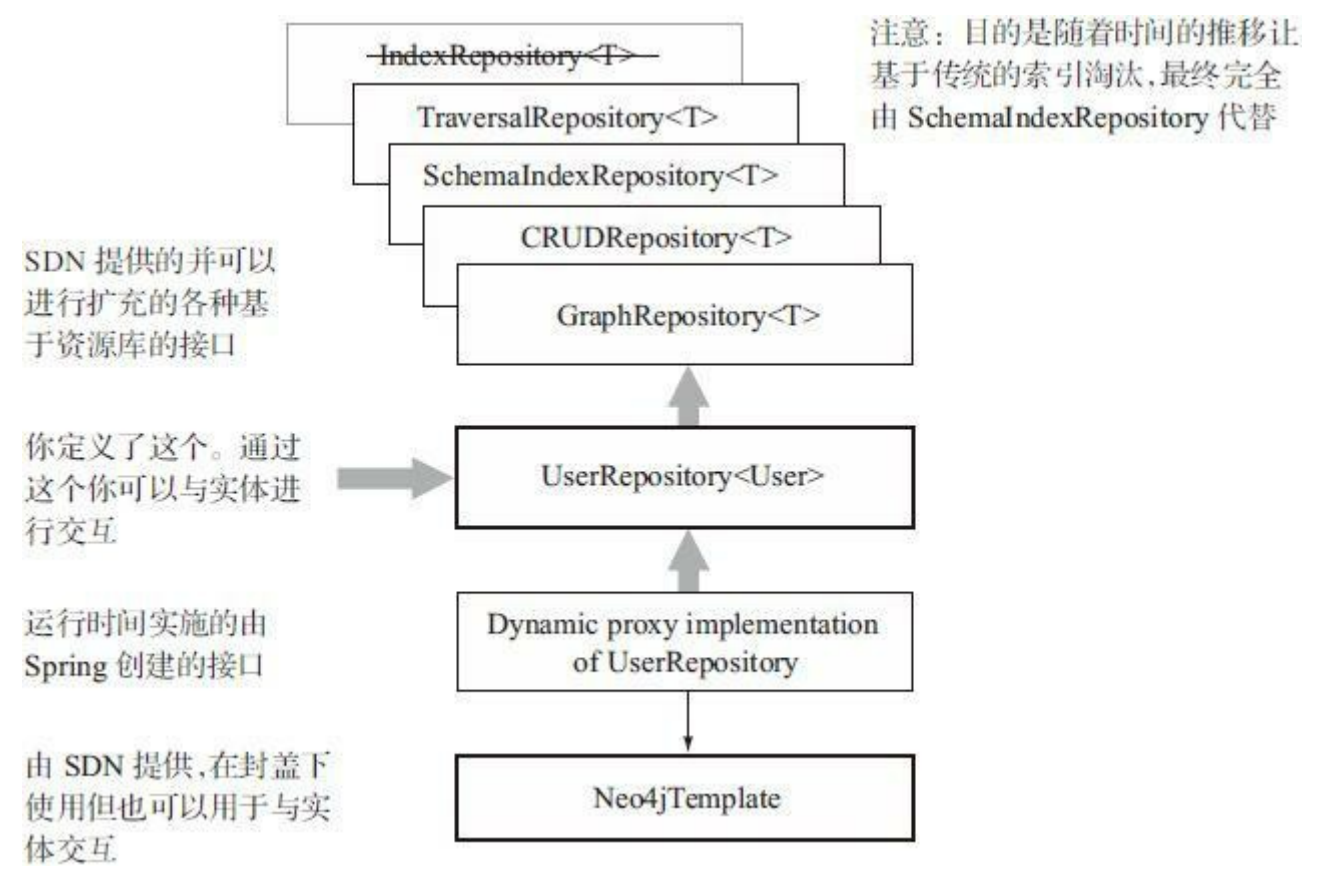


图9-6 涉及访问User节点实体的SDN资源库类概况

下面的一行程序是UserRepository接口的定义，尤其是对User域（不是节点域）类：

```
public interface UserRepository extends GraphRepository<User> { }
```

在这种情况下，UserRepository接口扩展了GraphRepository接口，它定义了对域类可用的默认操作的最广的集合。Java泛型的使用把这个知识库（在编译的时间）与User域实体联系了起来。

GraphRepository接口从实质上巩固了所有其他的接口，因此为用户定义了包括CRUD的操作、索引和遍历。下面的程序给出了GraphRepository接口包括的一些默认操作。

```
·public<U extends User>U save (U entity) ;  
  
·public<U extends T>Iterable<U>save (Iterable<U>entities) ;  
  
·public User findOne (Long id) ;  
  
·public boolean exists (Long id) ;  
  
·public Result<User>findAll ( ) ;  
  
·public void delete (User entity) ;
```

SDN建立在为Spring数据工程提供共享基础设施的Spring Data Commons模块定义的抽象概念基础上。SDN负责实施接口中知识库扩展的基本方法（例如，UserRepository情况下的GraphRepository接口），提供一套统一的函

数支持对一个实体的通用的加载、保存、查询、索引和遍历等操作。

程序9-1说明了如何使用UserRepository加载和保存用户。

【程序9-9】 通过UserRepository加载和保存数据

```
@Autowired
userRepository;

@Transactional
public void saveAndLoad() {
    User user = new User("john001", "John");
    User savedUser = userRepository.save(user);
    User loadedUser = userRepository.findOne(savedUser.getNodeId());
    User loadedUserViaIndex =
        userRepository.findBySchemaPropertyValue ("userId", "john001");
}
```

你会注意到这段代码与程序9-7使用的Neo4jTemplate方法（不需要提供一个目标类型除外）的代码非常相似。Repository.save方法保存实体到图形数据库中，是由findOne（基于节点编号）或findBySchemaPropertyValue（基于索引查找，在这种情况下是针对userId属性）加载完成的。为了让Spring知道你定义的任意知识库接口，它需要被告知。添加以下代码到你的Spring XML配置文件中就将实现这一目标。

```
<neo4j:repositories base-
    package="com.manning.neo4jia.chapter09simple.repository"/>
```

谁实现我的接口？

可能已经有人看出了我们从没有实际上对UserRepository接口定义任意具体的实现。我们只简单地通过Spring的@Autowired注释将其调入，然后使用它，设法读和保存User域实体。因此，如果我们从不提供任意的实施，是由谁完成，并且是怎样完成这项工作的呢？

这里并不涉及魔法。发生的这一切都是SDN使用在XML配制文件中提供的初始化软件包寻找你的知识库定义，然后动态地创建一个实施你的接口的一个代理，并且使其成为一个Spring组件。这个代理做实现在核心接口中定义的函数的所有艰苦工作。你仅简单地通过扩展一个接口就获得了一大堆函数。

知识库比Neo4jTemplate类高一级，它们也提供许多样板代码的实现，但以更针对域的方式。

9.3.4 其他选项

除使用Neo4jTemplate类和知识库以外，当保存实体时还可以用第三种方法。这涉及在实体本身上调用一个保存方法。由于它只在使用高级的映射模式时可用，我们将在9.4.2节中介绍，但是这里我们也简单地列了出来。

也会有一些这样的情况，你的域对象需要从多个数据源加载数据，这称作[交叉存储维护](#)（cross-store persistence）场景。也许你的一些数据存在于Neo4j中，而另一些存在于MySQL中。你将兴奋地知道SDN也适合于交叉存储维护，尽管我们在本书中不讨论这个问题。更多关于交叉存储维护的信息可以在核心Neo4j SDN文件中找到。

SDN与原始Neo4j能一起配合工作的很好吗？

从读数据、查询和遍历的角度看，任何从SDN写进低层数据库的数据，都可以使用到目前为止接触过的所有本地Neo4j工具，例如通过网页管理员控制台（Web Admin Console）的Cypher和Neo4j Shell等，以只读方式安全访

问。

SDN试图以最不引人注目的方式在图形数据库中存储数据，但是它需要存一些元信息以便做一些它需要做的事情。例如，有几种不同的方式可以在图形数据库中代表域模型的Java层级，并且这些选项可以用类型表示策略（type representation strategy）来管理。这些策略依赖于存储一些信息到图形数据库中。LabelBasedNodeTypeRepresentationStrategy（是SDN 3.0及以上版本的默认策略）对基于节点实体的简单类名的低层节点创建标签。IndexingNodeTypeRepresentationStrategy对每一个节点实体创建传统的索引以及添加一个_type_属性，而SubReferenceNodeTypeRepresentationStrategy可以在实体之间创建INSTANCE_OF和SUBCLASS_OF的关系。

如果你想要在SDN外更新低层实体，你将需要理解并考虑这一额外的元信息。例如，创建一个新的Person实体可能不只是涉及在图形数据库中添加一个具有基本属性的节点。

我们不可能涉及SDN存储和使用元信息的所有情景，但是我们可以在这里强调这一点以提醒你在使用SDN时，应注意到这一点和其可能对你的限制。

关于更多可用代表策略的不同类型信息，可以参见详细的“实体类型表示”<http://docs.spring.io/spring-data/neo4j/docs/current/reference/html>。

在最后介绍如何使用实体实现查询和遍历之前，让我们先讨论一下OGM。在本章中，我们偶尔会提到简单对象映射（simple object mapping），现在我们将说明这严格意义上指的是什么。

9.4 对象图形映射选项

SDN提供了两种执行域POJO和低层图形实体之间的映射模式：简单映射和高级映射。理解这两种映射操作方式的不同将帮助你评估权衡每一种方法，并且决定哪一种方法最适合你的项目（当你使用SDN时）。选择有一点像在DOM和SAX XML解析器之间做决定——两个都做同样的事情，但是解析器XML以非常不同的方式并以隐式方式做选择。

不管你选择的是简单映射还是高级映射模式，确定从映射得到了什么到两种模式的相同点的过程在9.2.2节中从概念上注释域模型做了描述。简单说，这种映射过程涉及SDN解析器和分析所有已知节点和关系实体。在Java反射API的一点帮助下，SDN建立了一个内存里的映射规则元模型。这些规则后面被用于做这些繁重工作，即在域模型和图形微元之间前后移动的工作，在每次合适的时候做转换。

补充信息

在Spring Data Neo4j 2.0.0版本之前，只支持高级方法（基于AspectJ的映射）。AspectJ是一个Java的面向方面编程（AOP）范例的实施——参见<http://eclipse.org/aspectj>。在SDN中，由于无缝集成的Neo4j数据库和Java域模型的机制，它看上去是动态拦截POJO上的方法调用。

对来自社区反馈的反应，添加这个简单映射模式用以解决许多与AspectJ工具有关的问题和关切，以及其他一些通过这个更复杂的设置而带来的更为一般的影响。应该注意到，现在更多的焦点应该集中到更新和使这个简单模型成为使用SDN的实际方式。

9.4.1 简单映射

简单映射模式是默认的模式，不需要特殊的设置，涉及在任何的加载操作中复制数据到一个节点实体中，只在调用显式保存操作时，才会出现写回到图形数据库中。图9-7给出了一个简单映射模式。

尽管这个方法简单，但应该注意这种简单映射对内存的固有影响。因为这种简单方法涉及在域实体和它们的低层代表之间的数据复制，你需要时刻注意你的内存占用，由于每一样东西实质上是加载了两次——一次通过节点，而后当所有的东西都复制时再一次通过域类。另外，这种方法也会带来性能上的损失，因为你不得不经常保持复制图形来做某些操作。

因此，让我们看看下面的程序并深入了解当加载和调用保存方法时将发生什么，以及两者之间将发生的一切。

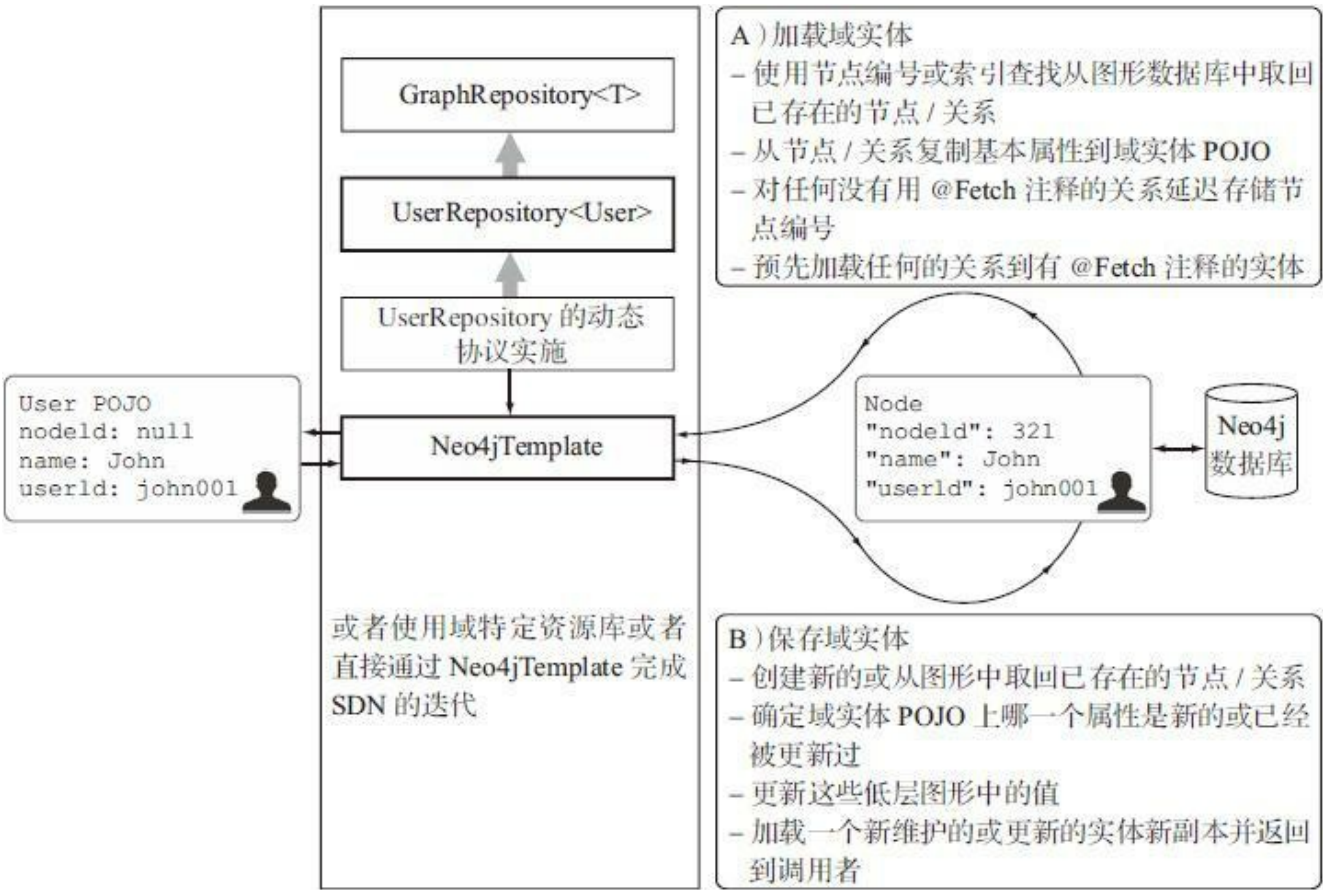


图9-7 简单映射逻辑概况

【程序9-10】 加载和保存

```
@Transactional
public void saveAndLoad() {
    Long johnNodeId = getJohnNodeId();
    User loadedUser = userRepository.findOne(johnNodeId);    ← ❶ 加载实体

    // Start: All changes between these Start/End
    //          markers are only performed in memory
    loadedUser.setName("New Name");

    User susan = new User("susan001", "Susan");
    loadedUser.addFriend(susan)
    // End

    User savedUser = userRepository.save(loadedUser);    ← ❷ 修改在这里保存到图形中
}
```

通过CRUDRepository接口可获得的findOne方法（❶）被分为加载方法之一。当这一行代码执行时，SDN将会做三件事情：

- 创建一个新的User对象。
- 加载低层节点类到内存。
- 对每一个属性和某些关系（那些没有@Fetch注释的，我们将很快接触到），从图形中复制出数据到POJO类属性。

下一次SDN与低层节点交互是在一个显式保存调用时（❷）。这些点之间从来没有导致一个对低层数据库的调用；所有的事情是在域对象上完成的。从技术上讲，刚刚创建或修改还没有保存的实体被称为处于分离状态。

1.传递的持久性

关于这些代码另一个值得一提的点是在John被保存时（❷），Susan也被保存。因此，John有什么特殊的？每当调用一个保存操作时，SDN将会看

看要保存的对象（在这种情况下是John）并且从被保存节点的角度检测是否有新的或变化的内容，包括关系。John具有了新年龄和名字并且有了一个新朋友。因此，SDN保存修改了数据的全部对象图形，Susan也会被保存。记住由只读句法定义的关系属性是（也就是说，那些Iterable集合类）不能被更新的。

2.预先加载VS延迟加载

当保存John时，默认会保存Susan，但是当John加载时Susan也加载了吗？官方的回答是“取决于不同的情况”。如果SDN是在默认的设置情况下，就像现在这种情况，答案是“不”。这是因为在默认情况下，当加载一个基于关系的实体属性时，SDN并不参与预先关系实例化。

预先加载将会涉及SDN主动跟踪一个实体可能存在的任何关系并且后面也会加载这些实体，包括它们所有的属性以及它们的关系。当不是显式的需要时，为了保留内存并且避免潜在的加载大量的图形到内存中，SDN默认使用延迟加载。

因此，如果你想了解延迟加载的实体将会发生什么情况？看一看下面的程序。

【程序9-11】 延迟加载的隐含含义

```

...
User john      = new User("john001" , "John");
User sally     = new User("sally001", "Sally");

john.addFriends(sally);
template.save(john);
User loadedSally = template.findOne(sally.getNodeId(), User.class);
assertEquals(1, loadedSally.getFriends().size());
User firstFriendOfSusan = loadedSally.getFriends().iterator().next();
assertEquals( john.getNodeId(), firstFriendOfSusan.getNodeId());
assertEquals( null, firstFriendOfSusan.getName());

```

用户集延迟加载，只加载了节点编号属性

对每一个延迟加载的节点实体，SDN仍然为低层实体实例化节点实体 POJO，但是那只会加载由@GraphId注释的属性。程序9-11的单元测试代码可以访问nodeId，但是，在此刻name仍然是null。

当想要加载整个实体时，你可以使用Neo4JTemplatefetch方法，如下面的扩展程序所示，这需要添加到程序9-11中：

```

template.fetch(loadedSally.getFriends());
assertEquals( "John", firstFriendOfSusan.getName());

```

另外，如果你想要SDN在加载时预先实例化实体，可以用@Fetch注释适当的属性，如下面的程序所示：

```

...
@Fetch
@RelatedTo(type = "IS_FRIEND_OF", direction = Direction.BOTH)
Set<User> friends;
...

```

现在你对简单映射模式有了一个简单的概况，可以概括为“简单且功能强大，但是存在潜在的内存危机”，接下来我们将介绍高级映射。

强制性的@GraphId属性应该被定义为一个长数据类型

简单映射要求域建模者提供一个用@GraphId注释的属性存储相关的节点或关系的ID，提供给库一个到低层节点或关系的连接以支持它。当实体还保存在Neo4j（通常是当你用关键字new创建实体时）时其值将是null，但是当它至少保存过一次时，它将会用于反映在图形中与其相关的低层节点或关系。

这里有几句警示语。确保你定义这个属性为长数据类型（Long data type）而不是长元型（long primitive）。包装类允许指定null，表示这个实体还没有保存到图形数据库中。如果你定义这个字段为长元型，这将意味着它的默认值总是0。然而，零可以代表一个真实的节点编号（在Neo4j 2.0之前，图形数据库中引用的节点总是节点零）。这将导致被存储和读取数据的所有类型的错误。如果一个实体还没有保存到图形中，你真正想要的是一个节点编号为null的节点。

9.4.2 基于AspectJ的高级映射

高级映射模式依赖于AspectJ库，这是由于它的机制是为了实现在注释的POJO域类和图形数据库中的低层实体之间的映射和转换逻辑（见图9-8）。

AspectJ（见<http://eclipse.org/aspectj>）是一个Java AOP范例的实现（http://en.wikipedia.org/wiki/Aspect-oriented_programming）。AOP，作为一个范例，目的是在代码库中增加模块化程度，尤其是为了允许常见跨领域关注的解决和从主代码中抽取最终分离存储的目的（作为某一方面）。跨领域关注是一些常见的逻辑，在代码库中的多个点需要，但是核心的本身并不需

要（引用的典型例子是登录和事务管理）。这个关切或问题在某种程度上能够被配置为在适当的时候应用于你的代码库（实现的方法不同）。

SDN是如何利用AOP（尤其是AspectJ）在POJO类和低层图形实体之间做它的常见转换和翻译逻辑的？

首先，不像简单映射，都是在运行时发生，高级映射取决于一些编译时的设置和配置。这涉及在你的项目中包含一个特定的感知AspectJ版本的SDN，以及使用AspectJ（ajc）编译器。详细的配置在附录C中讨论。

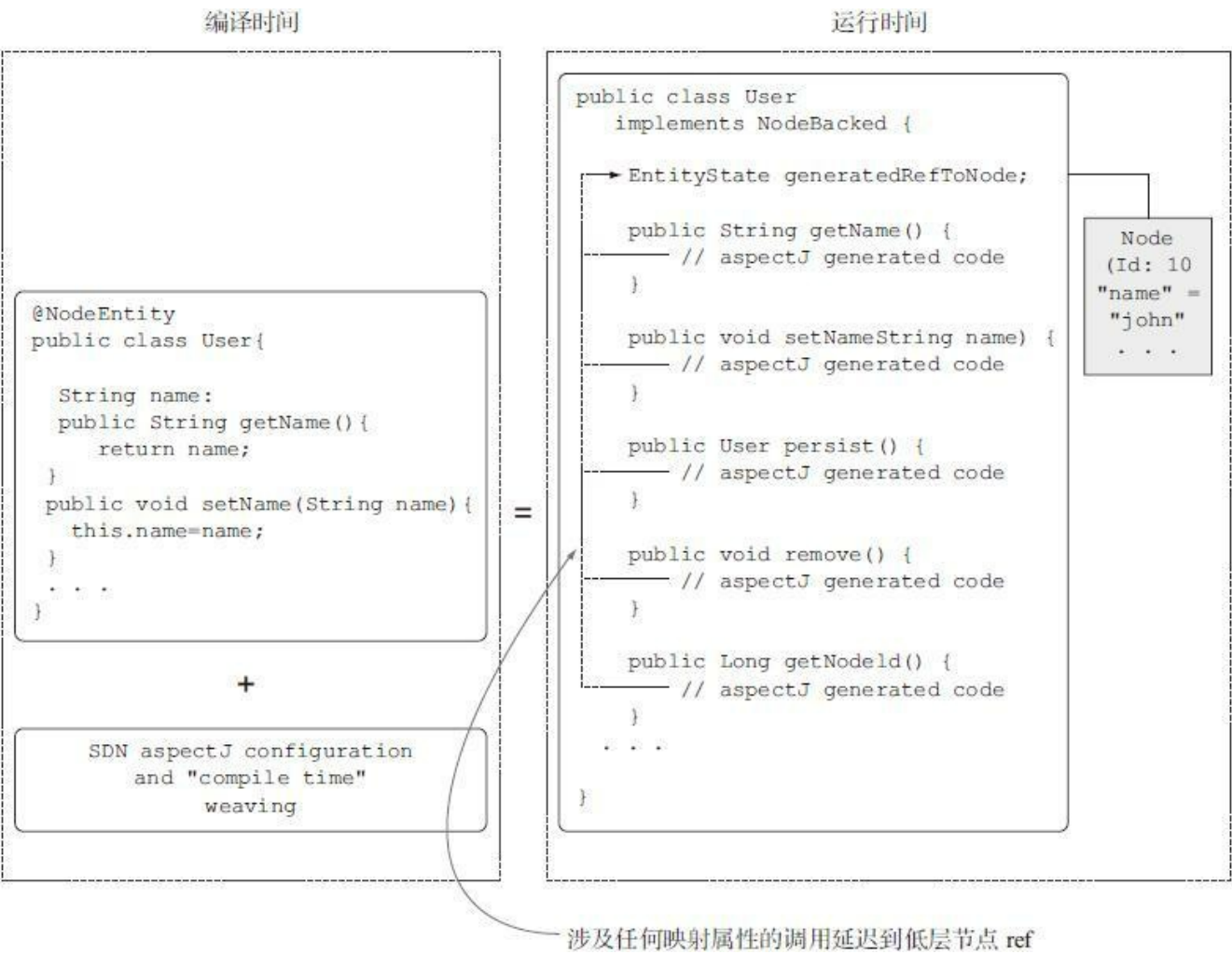


图9-8 高级映射概况

SDN将扫描你的代码查找所有的@NodeEntity和@RelationshipEntity类。

然后会使用特定的AspectJ编译器提取这些POJO域类，并用它们自己的自定义版本替换它们。

其次，这个自定义制作类具有两个值得注意的特点，第一个是对一个实际节点或关系的直接引用，作为一个内部隐藏的字段附带到类上。由AspectJ编译器产生的附加代码，与这个隐藏字段关联在一起，确保Neo4j支持的任何读或写到属性中的代码被修改，结束代表对低层实体的所有直接调用。

第二个值得一提的特点是额外的与维护相关的方法添加或混合进生成的类，与一个主动记录类型的方法相一致，也就是说，一项使维护代码与域代码一起存活的技术。这是由保证生成的类实现了一个NodeBacked或者RelationshipBacked接口来完成的，提供了适当的维护和查找功能的实施。下面的程序给出了如何使用这些与维护相关的方法直接操作实体。

【程序9-12】 隐式事物的主动记录维护

```
public void advancedUsageWithImplicitTransactions() {  
    User user = new User("john001", "John the 1st");  
    user.setName("John the 2nd");  
  
    ((NodeBacked)user).persist();  
    user.setName("John the 3rd");  
    ((NodeBacked)user).persist();  
}
```

没有事物注释或封装代码定义

1 隐式事物创建

如果你将眼光投向程序9-12的代码，可能会联想到几个问题，例如，“这段代码还能运行吗”？缺少事务的代码的确看上去有一点疑问，因为所有的操作都需要在事务内发生。

记住节点实体可以以两种状态存在：附加的或分离的。创建一个新节点

实体以分离的状态出现，这仅仅意味着这个实体还没有进入被写入低层图形数据库的状态。

在实体上（或者说Neo4jTemplate's save（）方法）调用persist（）（❶）附加实体到图形数据库中。附加确保实体的状态与图形数据库中的状态同步。对于创建的新实体、新节点和属性，还有那些前面某些时间点上附加的实体，它们需要重新附加，确保自从上一次附加后的任何变化写入到图形数据库中。

SDN可以检测你的代码是否在现有事务的范围内运行。如果不是（就像样本代码的情况），SDN将会确保与附加相关方法的调用（在这种情况下是NodeBacked's persist以及Neo4jTemplate的保存）将会为你创建一个隐式事务。隐式事务仅仅是为你在内部创建的一个新事务（❶），SDN会在后面使用它做变更调用并将状态保存到图形数据库中。

尽管在这里已经提供了详细的关于隐式事务是如何工作的，我们还是建议你提供显式事务。否则，库不得不一直在重新附加分离的实体。具有微小的隐式事务也会严重地降低应用程序的速度。

值得注意的是，每当访问域实体的一个属性或关系时，使用高级映射模式将会涉及通读数据库到底部。为避免多次读数据库，可能有时可以合理地存储数据或存储临时的结果，或者两者都存储，在局部变量中存储而不是通过域实体本身反复的查询数据。根据你如何与实体交互，这个局域变量可以局限于一个方法内部，实体类本身内部，甚至另外一个类内部。这将有利于提高性能，但是额外的变量也会加到你的整个内存占用中，因此，你应该时刻清楚这一点。

如果在一个已存在的事务中操作将会发什么？

在代码执行前，如果你在前面已经创建或开始一个显式事务（而不是依赖于隐式事务），SDN将不会试图创建自己的事务。SDN将会仅在已存在的事务已建立的边界范围内操作，提供给你使用标准事务的所有控制功能（例如，回滚），这些是你已经习惯了的功能。

到这里就结束了我们关于高级映射模式的简要概况。为了减少当你设定AspectJ开发环境会遇到有关配置的头痛问题（这也是主要抱怨之一并导致了简单映射模式的引进），相对于简单的映射模式，你可以较好地使用可用的内存。如果处理不当的话，也会有一些读取属性性能的损失。

9.4.3 对象映射总结

表9-1比较了简单和高级映射模式。

表9-1 简单和高级映射模式对比

	优 点	缺 点
简单映射模式	简单、直观、易于理解； 随手就用； 不需要特殊的开发环境	随着在实体和节点之间来回复制数据，可能耗尽内存； 慢
高级 AspectJ 映射模式	通读方式可能对内存使用更有效； 如果需要的话可以提供使用主动记录维护功能； 快	如果有太多的细小调用， 通读方式可能会对性能有影响； 涉及隐式事务程序的错误很难跟踪和解决； 工具，尤其是关于 AspectJ 的开发环境，例如，Eclipse 和 IntelliJ，被认为引起开发者许多头痛和无眠之夜，也是 Neo4j 社区的主要烦恼之一

9.5 执行查询和遍历

为了结束这一章，让我们讨论一下SDN如何延伸它的简易编程模型使之能够以非常简单的方式指定和使用查询与遍历。假设你想查找关于一个用户朋友的朋友的信息，我们可以使用一个主域的子模型来进一步探测这种情况，如图9-9所示。重点关注用户John，朋友的朋友最终标识为落在图形右边的最后部分的那些用户。

9.5.1 注释查询

这不应该令你感到惊讶，SDN提供了一个注释（在这种情况下是@Query）用于标识一个执行的查询。当你要返回一个过滤的实体列表或者一个运行时确定的信息子集时，这将非常有帮助。注释可以用于一个节点实体的一个字段上，或者知识库接口中的方法上。当一个节点实体上的字段被访问时，或在一个知识库接口上调用一个方法时，查询被执行并且返回结果。

1. 节点实体上的注释

让我们从如何使@Query注释使用于User节点实体开始定义一些涉及朋友的朋友的场景的查询：

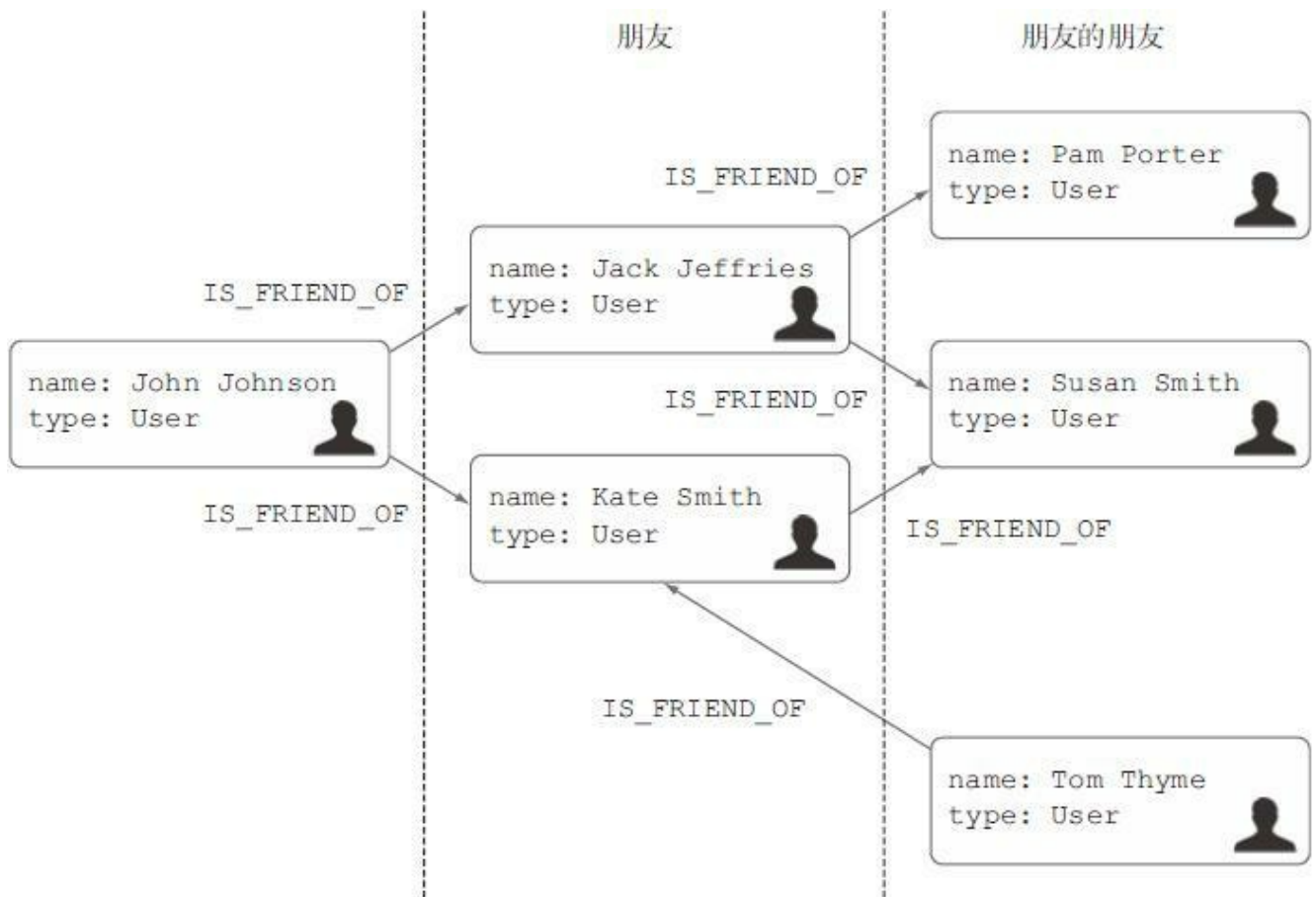


图9-9 朋友的朋友子模型

```

@Entity
public class User {
    ...
    @Query( value =
        "match (n)-[r:IS_FRIEND_OF]-(friend)-[r2:IS_FRIEND_OF]-(fof) " +
        "where id(n) = {self} " +
        "return distinct fof")
    Iterable<User> friendsOfFriends;
  }

```

每当加载User实体时（不论通过Neo4jTemplate还是UserRepository方法），注释中定义的Cypher查询将会被执行并且结果会存到变量中（在这种情况下，变量是friendsOfFriends）。

这是一个标准的Cypher查询，仅仅对{self}的引用是新句法，这个引用是指支持这个实体的当前编号。这是有道理的，作为定义在实体上的查询一般应该与那个实体相关。如果你的查询不是专门的与查询中的节点实体相关，这表示它可能需要作为一个更通用的方法在知识库中定义。

你可以通过注释的params属性提供一组关键值动态的指定查询中的参数。下面是另一段程序示例如何回应查询，“对一个指定的用户，对每一个他或她的直接朋友，统计朋友的数量并返回这些数量”。

```
@Query(value =
    "match (n)-[r]-(friend)-[r2:IS_FRIEND_OF]-(fof) " +
    "where id(n) = {self} " +

    "and type(r) = {friendRelName}" +
    "return friend.name as friendName , count(fof) as numFriends",
    params = {"friendRelName", "IS_FRIEND_OF"})
Iterable<Map<String,Object>> friendsOfFriendsCount;
```

由于你不是返回整个实体，但是也不是数据的一个子集，这个查询的结果将会是一个只读入口集合（Iterable），关键字是在查询的返回阶段定义参数名字，其值作为结果。使用图9-9中的图形作为例子，以John为例运行这个查询将得到如下的答案。

```
{friendName=Kate, numFriends=2}
{friendName=Jack, numFriends=2}
```

下一段程序给出了如何在Cypher中返回聚合类型的值，例如计数。在这种情况下，你是在查找朋友的朋友总数，这里根据图9-9所描述的子模型，结果应该是4。

```
@Query(value =
    "match (n)-[r:IS_FRIEND_OF]-(friend)-[r2:IS_FRIEND_OF]-(fof) " +
    "where id(n) = {self} " +
    "return count(distinct fof)")
Long totalNumFriendsOfFriends;
```

2.知识库接口注释

@Query注释也能用在你想定义在自己实体的特定知识库接口上的方法。回想一下9.3.3节的UserRepository。下面的程序给出了当对知识库定义

方法时查询朋友的朋友的程序。

【程序9-13】 知识库上的@Query注释

```
public interface UserRepository extends GraphRepository<User> {  
    @Query( value =  
        "match (n)-[r:IS_FRIEND_OF]-(friend)-[r2:IS_FRIEND_OF]-(fof) " +  
            "where id(n) = {0} " +  
            "return distinct fof")  
    Iterable<User> getFriendsOfFriends(Long nodeId);  
    ...  
}
```

{0} 映射到方法定义的第一个
参数 (nodeId)

注意你已不再使用{self}引用了。现在，你已经更有力地访问了传递到方法中的参数。这个参数可以作为从零{0}开始的增量值，{0}对应于这个方法中的第一个参数，{1}对应于第二个，以此类推。

你没有必要实现任何的逻辑去执行一个查询，或者做映射去转换结果到POJO和图元等。SDN为你做了这一切。

9.5.2 动态派生查询

另一个非常简洁的不需要写一行Cypher代码就能够用于定义查询的方法是利用动态命名查找器或get methods方法。每当SDN遇到一个没有显式@Query注释的在知识库接口定义的方法，它将试图基于你的方法的名字动态创建一个Cypher查询。SDN将把方法的名字分解为与元模型匹配的各个部分。所有的这些部分映射到定义在相关域实体类上的属性。

有相当多的可用选项和排列，包括指定顺序的分类、指定范围的查询、请求不同的结果等。我们不能讲述所有的内容，但是用于识别动态方法的核心规则可以总结如下：

- 任何以零开始的方法或以find、read、get之一开始的方法。

- 可选择地跟着任意的字母和数字组合的字符串。

- 后面由术语By跟着（大写B）。

- 后面跟着一个或多个相关域实体上的任意属性名，属性名的第一个字符是大写的，多个属性由And分开，或其他比较分隔符如GreaterThan或Like。

为了演示在实战中的这些规则，我们定义了一个新方法用以查找具有特定名字的任何用户（例如，查找所有的Susan）。看一看程序9-14并注意UserRepository接口上的所有方法的定义如何会最终导致完全一样的正在生成的低层Cypher查询。也要注意由于具有注释查询，与方法的某些方面相关的参数变量，在这种情况下，是代表属性名字的部分。

【程序9-14】 动态产生的查询方法

```
public interface UserRepository extends GraphRepository<User> {  
    Iterable<User> name(String name);  
    Iterable<User> getByName(String name);  
    Iterable<User> getUserByName(String name);  
    Iterable<User> readByName(String name);  
    Iterable<User> readUserByName(String name);  
    Iterable<User> findByName(String name);  
    Iterable<User> findUserByName(String name);  
}
```

所有这些方法解决同样产生的
Cypher 查询：匹配 n:User 与
n.name = {0} 返回 n



注意

如果你想看一下SDN为你的方法产生什么样的查询，使用SLF4J在应用程序中改变登录级别以进入到DEBUG.SDN，因此，任何支持SLF4J的框架都是可以的；样板代码中使用Log4j测试。

像以前一样，返回Iterable类型表示对SDN期待着多个可能的实体，而一个单一的域实体表示你只期待一个。如果你确实曾经指定只期待一个实体，而低层查询的结果多于一个，SDN将会跳出一个以下种类的异常。

```
NoSuchElementException: More than one element in IteratorWrapper(non-empty iterator)
```

1.多重和嵌入式属性

SDN能够理解相关的属性定义。如果你想查找被其他的某个用户（比如说Susan）推荐的所有用户，你可以如下定义一个方法。

```
Iterable<User> findByReferredByName(String name);
```

这里是引用用户的referredBy属性的第一部分。那么，由于这个属性本身就是用户类型，你可以基于那个用户的名字进行搜索。

你应该把所有的属性串到一起并且以And或Or把它们分开（现在And只在运行时支持）。假设你的用户也有一个年龄属性，在这种情况下，如果你想查找具有特定年龄的所有用户，你应该定义一个这样的方法：

```
Iterable<User> findByNameAndAge(String name, int age);
```

2.更多信息

这么简单的一节远不足以介绍所有可用的查询选项，包括排序、查询范围和通过Neo4jTemplate类获取的一般查询方法。我们希望我们已经给出了使用这类查询的基本模式，使你能够接近或已经给你指出了在这条路上应该选择继续的努力方向。

9.5.3 遍历

我们再一次发现我们不能涉及所有内容，不幸的是，SDN内部的遍历是我们不能深入讨论的一个领域。然而，为了不是一点也不涉及，我们将指出你在第4章中遇到的遍历框架也可以在SDN中访问。如果你以前没有猜到这一点，对它有一个注释，即@GraphTraversal，还有一个整体知识库接口（TraversalRepository）专用于帮助涉及域实体的遍历。

9.6 本章小结

在本章中，我们介绍了Spring Data Neo4j（SDN）框架，这个框架提供了允许你使用由强大的Neo4j数据库支持的标准的基于POJO的域模型的各种工具。你看到了以一种允许SDN无缝地照顾到域模型和低层图元之间的映射逻辑的方式注释一个POJO是多么容易。

现在你学会了如何使用Neo4j模板和知识库支持的实施对你的实体做CRUD^[1]和索引操作，以及如何定义和执行涉及节点实体的查询。

你也了解了两个主要对象-图形映射模式：简单映射模式和高级映射模式。你会发现尽管两者最终做的事情相同，但是它们却以相当不同的方式完成，这可能会对内存和性能产生影响，这需要仔细考虑。

就像我们在9.1.4节中所说的，我们仅仅能够对SDN的功能有一点皮毛的了解。我们希望本章能够起到激发你的探索欲望使你能够继续探讨SDN更多的功能。接下来，我们将深入理解使用Neo4j的嵌入式模式和服务器模式之间的不同。

[1] CRUD是指在做计算处理时的增加（Create）、读取（Retrieve）、更新（Update）和删除（Delete）几个单词的首字母简写。主要用于描述软件系统中数据库或者持久层的基本操作功能。——译者注

第三部分 Neo4j应用实例

·第10章 Neo4j的嵌入式模式与服务器模式

·第11章 Neo4j的架构与应用

在本书的前两部分，我们讲解了Neo4j的基础知识以及从应用开发的角度如何使用Neo4j。

最后的一部分将把重点转移到更多的操作型的领域和问题，这同样需要慎重的考虑和思考。本部分只有两章，但是涵盖了相当多的基础知识，因此，准备好开始上路吧！

第10章探索了Neo4j的两种主要使用模式，嵌入式模式和服务器模式，了解使用每一种模式的利与弊。本书的大部分把重点放在了嵌入式模式，本章提供了如何使用特定服务器功能，例如插件和扩展，当以这种模式运行时充分利用Neo4j的指导 and 例子。第11章通过一个Neo4j高层架构概览并演示如何扩展和配置Neo4j使之高度可用，以及如何备份和恢复Neo4j数据库来结束本书。

第10章 Neo4j的嵌入式模式与服务器模式

本章包括以下内容：

- 两种主要使用模式：嵌入式模式和服务器模式
- 如何权衡两种模式的利与弊
- 充分利用具有Cypher、插件、扩展和流的服务器

既然你已经有了对设计和建模Neo4j图形数据库所需的方法和实际技术的很好理解，现在就可以看看Neo4j的两种主要运行模式了，那就是嵌入式模式和服务器模式。在着手任何的重要Neo4j项目之前，第一件需要考虑的事情是决定最终要以什么模式运行Neo4j。这一选择将影响到许多因素，譬如你的应用程序将用什么语言和在什么环境下运行，因此这是一项非常重要的考虑因素。

你将会很高兴的知道，不管你选择的是什么模式，到目前为止你已经学到的知识仍然适用。使用每一种模式的句法非常不同，理解这些句法和如何适当的使用它们是很重要的，但是核心的原理还是一样。

在本章中，我们将讨论为什么会有两种模式，它们的主要区别是什么，如何权衡，优点缺点以及使用每一种模式的影响。现在让我们开始学习模式吧！

10.1 使用模式概述

当Neo4j第一次发布时，它的目标就是针对基于Java的领域，所以只是支持[嵌入式模式](#)。在嵌入式模式的设置中，你的Java应用程序和全新闪亮的Neo4j数据库很友好地绑在一起作为单一的应用实体，共同去征服基于图形问题的新领域。

然而，Neo4j的广泛能力和功能并没有被其他的语言所忽视，这些功能也感兴趣于能够起到杠杆作用和充分利用这一全新的图形数据库。尽管Neo4j是用Java语言写成的，但其本质上是一个基于JVM的产品。这意味着理论上任何基于JVM的语言（假设能找到或编写合适的库或绑定）也能够使用Neo4j数据库。因此，由于不同的库和绑定开始涉及并成为可用，Neo4j的应用范围自然地开始扩展到其他基于JVM的语言。但是，以网络更友好的结构运行是必需的，引进[服务器模式](#)背后的主要驱动因素是对支持其他非基于JVM的客户的需求。在服务器模式下，Neo4j数据库以自己的进程运行，客户端通过它的专用基于HTTP的REST API与其对话。

图10-1给出了这两种使用模式和不同客户通过这些模式使用Neo4j的主要方式的概况。

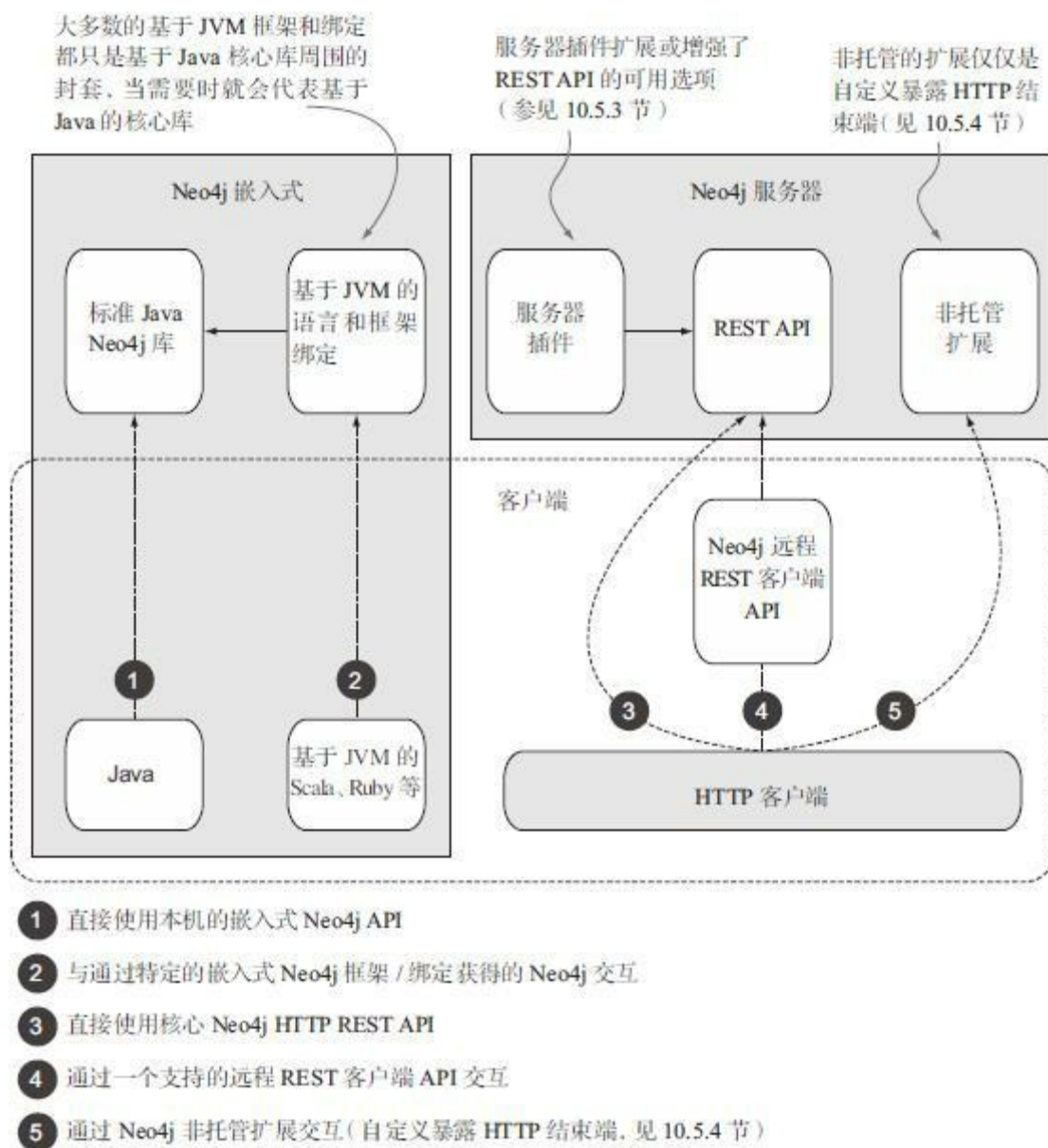


图10-1 Neo4j使用模式和客户端用户主要集成选项的概况

在嵌入式模式中, 任何能够在JVM中运行的客户端代码都能在Neo4j中使用。就像图10-1所演示的, 你可以以纯Java客户端直接使用嵌入式模式 (❶), 这里是直接使用核心Neo4j库, 或间接通过由不同社区提供的基于JVM语言的额外特定语言绑定和框架 (❷)。

在服务器模式中, 客户端代码通过HTTP协议, 尤其是通过明确定义 REST API与Neo4j服务器交互, 对于REST API, 还有额外的可用选项, 当需要时可以扩展REST功能。这个API可以被任何支持HTTP的客户端直接使用

(3)，或为了使得开发的工作变得更轻松一点，可以使用各种不同语言和框架可用的远程REST客户端API (4)。随着服务器模式引进的固有网络延迟，访问数据库的性能自然不如直接使用本机代码。为了给服务器模式的服务添加更大的灵活性，可以使用服务器插件和非托管扩展 (5) 加强其性能和功能。我们将在本章的后面讲解服务器插件和扩展。

注意

到目前为止本书中你看到的所有例子中的代码使用的都是嵌入式模式的设置。嵌入式模式是实验Neo4j的一种非常好的方法，它也经常用于探索Neo4j能够做什么。即使你决定使用服务器模式，仍然有机会在服务器本身使用本机的API，并且有时候可能是更必要的。另外，REST API经常仅仅为本机的嵌入式API提供一个门面，知道并理解嵌入式API的工作原理会是非常有用的。理解嵌入式的语义和选项将会给你很大的帮助以获得设置的最大益处，即使你选择的是基于服务器模式的设置。

10.2 嵌入式模式

到目前为止我们看过的所有例子都是使用嵌入式模式，因此，我们首先深入探讨这种模式也是很合理的。需要说清楚，嵌入式模式并不是指硬盘上的实际物理数据库嵌入到你的应用程序中，而是嵌入到Neo4j引擎（类和相关的过程）直接运行和管理Neo4j数据库。

10.2.1 核心Java集成

最通用的嵌入式情景涉及在一个基于Java的应用中直接嵌入Neo4j，如图10-2所示。

打包的和在你的应用程序内运行的核心Neo4j类比仅仅作为在物理数据存储和你的应用程序之间前前后后做数据传送的一种机制的作用大。类本身构成了集成整个数据库功能的一部分并提供管理，例如，所有的逻辑和做遍历、查询等的内存需求。从应用程序结构的角度看，这是非常有趣的，因为这意味着逻辑控制你的应用程序以及数据库需要能够在同一个JVM空间协调一致。这种在一起的影响将在10.4节讲解，但是现在知道嵌入式模式意味着你的应用程序和Neo4j代码将会驻留和运行于同一个JVM就足够了。

嵌入式模式需要适当的库（JAR文件）捆绑或当应用程序开始时你的应用程序可用这个库。然后，通过实例化GraphDatabaseService接口上的适当实例获得对库的访问后，就该由你的应用程序负责了。然后，你的应用程序就能够利用这个引用与Neo4j交互，通过到目前为止我们已经讨论过的所有API。通常对单机设置使用EmbeddedGraphDatabase类。高可用性（HA）将

在第11章介绍。

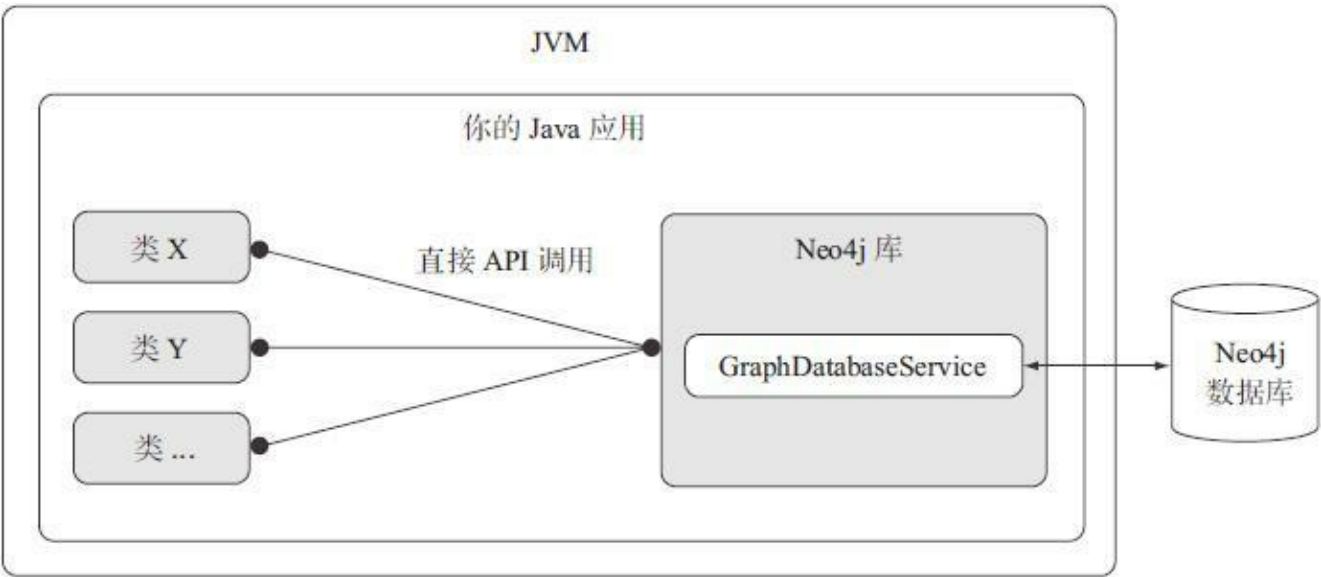


图10-2 典型的Java嵌入式部署方案，这里Neo4j库嵌入到Java应用程序中

1.需要的库

如果你是在项目中使用Maven（<http://maven.apache.org>），下面的程序给出了所有必须导入到你的应用程序中的适当的嵌入式Neo4j库。

【程序10-1】 嵌入式Neo4j依赖

```
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-community</artifactId>
  <version>2.0.1</version>
</dependency>
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-kernel</artifactId>
  <version>2.0.1</version>
  <type>test-jar</type>
  <scope>test</scope>
</dependency>
```

1 嵌入式开发所需的主要依赖

2 辅助单元测试有用的测试依赖

Maven将确保所有需要的依赖作为传递依赖管理系统的一部分下载下来（1）。换句话说，Maven将会知道除主Neo4j库外的那些需要下载的其他

支持库并确保这些是可获取的。

如果不使用Maven，你将会需要从Neo4j网站上下载合适的zip/压缩包（zip/tarball）并在lib目录下提取必要的库。下一段程序显示了Maven dependency: tree的执行结果（❷），显示了当你请求核心嵌入式库时，Maven会下载哪些额外的库。

【程序10-2】 核心Neo4j嵌入式库的依赖树

```
[INFO] --- maven-dependency-plugin:2.8:tree (default-cli) @ chapter10-embedded ---
[INFO] neo4j-in-action:chapter10-embedded:jar:0.1.1-SNAPSHOT
[INFO] +- org.neo4j:neo4j-community:jar:2.0.1:compile
[INFO] |   +- org.neo4j:neo4j-kernel:jar:2.0.1:compile
[INFO] |   +- org.neo4j:neo4j-lucene-index:jar:2.0.1:compile
[INFO] |   |   \- org.apache.lucene:lucene-core:jar:3.6.2:compile
[INFO] |   +- org.neo4j:neo4j-graph-algo:jar:2.0.1:compile
[INFO] |   +- org.neo4j:neo4j-udc:jar:2.0.1:compile
[INFO] |   +- org.neo4j:neo4j-graph-matching:jar:2.0.1:compile
[INFO] |   +- org.neo4j:neo4j-cypher:jar:2.0.1:compile
[INFO] |   |   +- org.neo4j:neo4j-cypher-commons:jar:2.0.1:compile
[INFO] |   |   +- org.neo4j:neo4j-cypher-compiler-1.9:jar:2.0.1:compile
[INFO] |   |   +- org.neo4j:neo4j-cypher-compiler-2.0:jar:2.0.1:compile
[INFO] |   |   |   \- org.parboiled:parboiled-scala_2.10:jar:1.1.6:compile
[INFO] |   |   |       \- org.parboiled:parboiled-core:jar:1.1.6:compile
[INFO] |   |   +- com.googlecode.concurrentlinkedhashmap:
[INFO] |   |       concurrentlinkedhashmap-lru:jar:1.3.1:compile
[INFO] |   |   \- org.scala-lang:scala-library:jar:2.10.3:compile
[INFO] |   \- org.neo4j:neo4j-jmx:jar:2.0.1:compile
[INFO] +- org.neo4j:neo4j-kernel:test-jar:tests:2.0.1:test
[INFO] |   \- org.apache.geronimo.specs:geronimo-jta_1.1_spec:jar:1.1.1:compile
[INFO] +- ...
```

对应程序 10-1 中❶的依赖树

对应程序 10-1 中❷的依赖树

2. 获得对嵌入式Neo4j数据库的访问

假设你已经有了代码所有需要的依赖，下一段程序详细地说明了如何获得嵌入式数据库的引用，并且该段程序也负责提供一个确保当JVM退出时数据库恰当关闭的一个机制。

【程序10-3】 开始和关闭一个嵌入式图形数据库

```

private String DB_PATH = '/var/data/neo4jdb-private';
private GraphDatabaseService graphdb =
new GraphDatabaseFactory().newEmbeddedDatabase( DB_PATH );
registerShutdownHook( graphDb );

...

private static void registerShutdownHook(
final GraphDatabaseService graphDb )
{
Runtime.getRuntime().addShutdownHook( new Thread()
{
    @Override
    public void run()
    {
        graphDb.shutdown();
    }
} );
}

```

← 创建一个嵌入式数据库，物理 DB 文件位于 DB_PATH。

← 注册一个关闭点确保当 JVM 退出时，执行一个彻底的关闭。

①

当你的应用程序退出时，确保Neo4j彻底关闭是非常重要的。在完成前即使你发送一个键盘中断信号（Ctrl-C）试图终止进程，程序10-3中的关闭数据库代码（①）也会被触发。

未能彻底关闭数据库将导致下次启动时出现问题。Neo4j能够探测出没有彻底关闭并试图恢复，这将会导致恢复进程中非常慢的初始化启动，有时也会导致出现其他的问题。当一个不彻底的关闭发生时，下面的警告信息将会在数据库下一次的启动日志中看到：探测到未彻底的关闭（non clean shutdown detected）。

3.嵌入式模式测试

测试是任何软件开发的重要组成部分。本书和本书中详细介绍的大部分代码都已经过单元测试，并在不同章节的情景中得到证实和演示。在许多情况下，这些测试使用一个非常方便的内存数据库实施

（org.neo4j.test.ImpermanentGraphDatabase），这是专门为单元测试设置的。

在测试Neo4j核心库中可以找到ImpermanentGraphDatabase类（程序10-1

中的❷）。以嵌入式数据库（参见程序10-3的❶）使用Neo4j会使物理节点和关系存到了硬盘。然而，使用非永久备份数据存储的Neo4j将导致只在内存中存储数据而不保存到文件系统中。强烈鼓励进行Neo4j代码单元测试，而且这个ImpermanentGraphDatabase实施提供了一个奇妙的方法做测试或证实你的问题领域里的一些图形场景。

10.2.2 其他基于JVM的集成

Neo4j社区拥有一大群各式各样并且积极的成员，并且他们早已经看到了很多语言和框架绑定以嵌入式模式用其他语言使用Neo4j，例如Scala、JRuby等。这通常是通过语言指定封装完成的，这个语言指定能够通过问题中的JVM语言和框架将Neo4j核心API改编为能用于本机的API（在某些情况下是惯用的）。图10-3给出了这些应用场景通常涉及的情况。

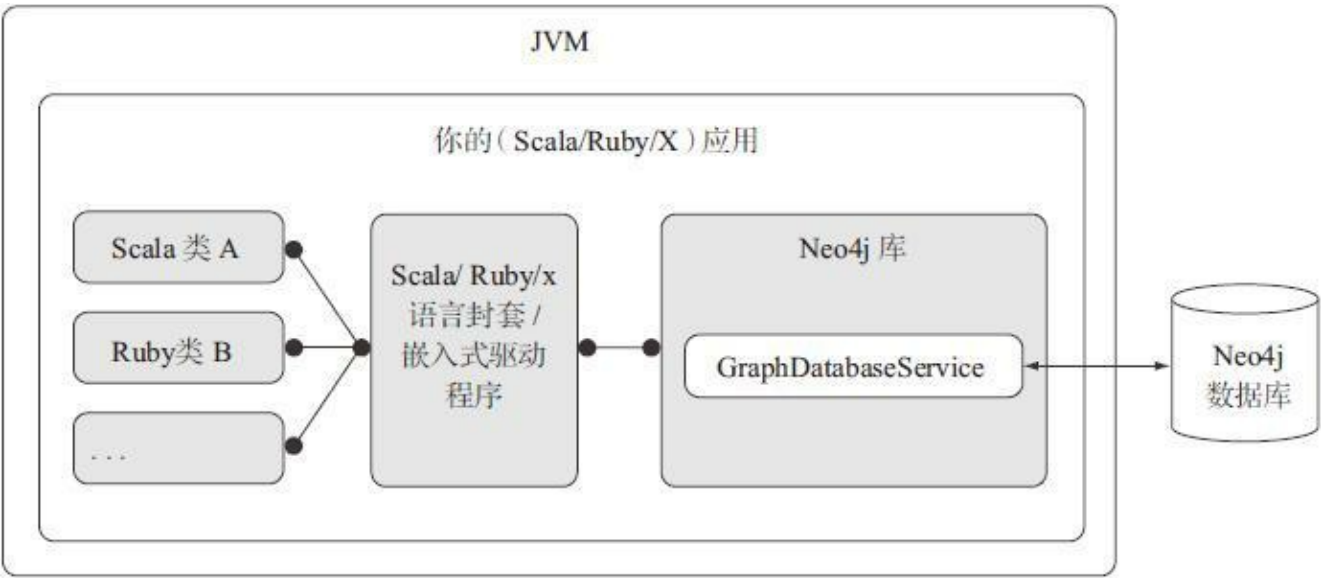


图10-3 其他包括语言指定封套和Neo4j驱动程序的基于JVM的嵌入式部署方法

重要的是这些封装的大多数仅仅是在基于Java的Neo4j类和API的前面有

效。这意味着所有的属性和与基于Java的嵌入式版本也会通常适用基于封装的实施，对于封装库本身引进的额外特点也需要进行考虑。

一个可用的最新语言封装列表可以参阅<http://neo4j.org/drivers/>。在本书写作时，这个列表包括JRuby、Django、JavaScript、Scala和Clojure。

这就是我们对嵌入式模式如何工作和使用嵌入式模式的需求的初步探讨。在我们转移到查看服务器模式的权衡、优点和缺点之前，接下来我们将以相似方法探讨服务器模式的性能和工作情况。

10.3 服务器模式

与嵌入式模式不同，以服务器模式运行Neo4j涉及要访问和处理的所有类和逻辑都与包含自身专用流程且完全与使用它的客户端分离的Neo4j数据库交互。如与许多其他的基于服务器的设置，客户端需要某个机制与服务器流程进行交互，在Neo4j的情况下，这是通过使用明确定义的，然而可扩展的基于HTTP的REST API获得的。

REST是什么

REST有时可看作有一点超负荷的术语，但它正式代表的是表述性状态传递。简而言之，它可以被认为是一种软件架构风格，拥有和利用了网络操作方法和网络结构，在大多数情况下使用HTTP作为选择的运输工具来帮助实现这个。对于REST的完全理论定义和解释，参阅Roy Fielding的博士论文，“结构风格和基于网络的软件结构设计”（2000年），这里是这个概念的原始提出和首先出版的地方

（www.ics.uci.edu/~fielding/pubs/dissertation/top.htm）。

总地来说，无论是从用户的角度导航他们还是从服务器和它们背面的支持结构来看，网页和人们进行交互的方式遵循一个一致的模式。作为一个用户，你通过提供URL请求网页（资源）。这些网页后来随着你请求的数据返回，还有另外的指向其他数据或相关资源的链接。服务器理解HTTP获取数据的GET请求以及修改和创建新资源的请求（POST和PUT）。简单地说，Fielding提出这些相同的原理可以用于同更一般的资源的交互，例如，那些由应用程序提供的服务（在这里是指读网页服务）。因此，如果它们是为利

用这些原理帮助网页发展成今天的这个样子而设计的，这些网页资源也可能是人或系统通过一致的和可预测的方式提供、导航和交互的。

类似RESTful的网页服务（例如，相对于SOAP）或许是事实上的当今网页服务实施，Neo4j也可以算作它们其中的一个。Neo4j的REST实施使用了JSON作为默认的数据格式并提供了一个面向服务的方式与Neo4j资源（节点与关系）交互及操作。

关于这一课题的更多信息和务实处理方式，参阅Jim Webber、Savas Parastatidis和Ian Robinson的《REST in Practice》。Jim Webber在Neo4j REST API的设计与开发中做出了非常大的贡献。

10.3.1 Neo4j服务器概述

图10-4描述了涉及具有一个以标准的、随手就用的REST API访问的，并以服务器模式运行的Neo4j核心部件。

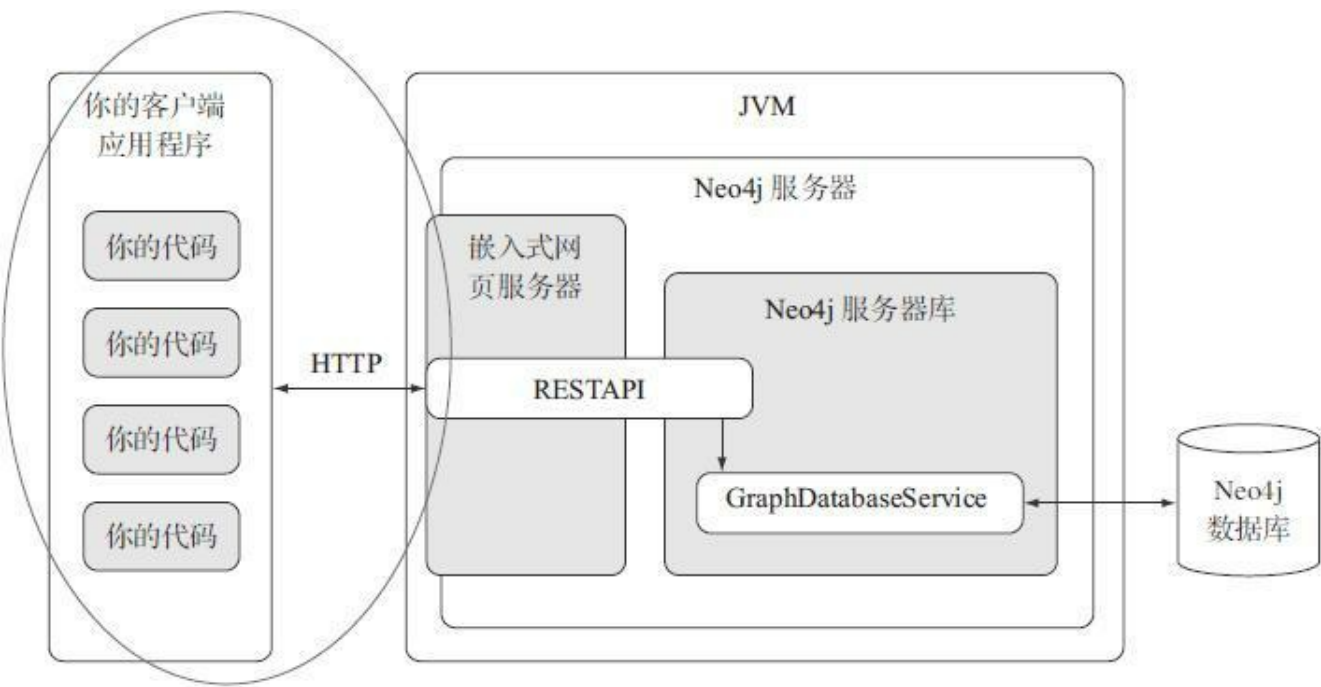


图10-4 一个典型的具有通过标准的REST API访问客户端的Neo4j服务器设置

Neo4j服务器本身只是一个基于JVM的应用。在后面它通过封装适当的Graph-DatabaseService接口（EmbeddedGraphDatabase或HighlyAvailable-GraphDatabase）实例提供它的功能，这些功能通过一个良好定义的接口显示给外部的用户。

为了能够侦听和与对服务器请求的REST交互，所有的Neo4j服务器实例将会启动一个嵌入式网页服务器（当前默认的是一个Jetty服务器侦听，在端口7474）。

1. 安装和使用Neo4j服务器

要安装Neo4j服务器，你需要下载适合你的操作系统的tar或zip文件，先解压，然后利用操作系统指定的脚本语言开始和结束服务器。附录A详细介绍了安装的各个步骤。

从客户端的角度看，你不需要任何指定的Neo4j库。只要你能发送HTTP请求，就足够了。如果你是在UNIX环境，curl客户端将会为你做这些。然而，本机的REST API是非常详细的，往往不是你想要的一些客户端库并通过注意一些更普通的低层REST映射使得你的工作更容易一些。使用合适的远程API库（10.3.4节的列表）能够在这方面给你很大的帮助。

2. curl和Java客户端例子

本章提供了使用两种不同客户端的例子：

·在大部分UNIX分布操作系统中，可以使用curl命令。这种客户端将会演示如何直接调用低层次的REST API。

·Java客户端使用官方的绑定到服务器的REST API上的Neo4j Java REST（neo4j-rest-graphdb）。这一选项显示了如何使用一个远程客户端库简化用于Neo4j REST API的代码。

再一次的，如果你使用Maven，下面的程序可以加到pom.xml文件中以调入适当的库。

```
<dependency>
  <groupId>org.neo4j</groupId>
  <artifactId>neo4j-rest-graphdb</artifactId>
  <version>2.0.1</version>
</dependency>
```

10.3.2 使用细粒度Neo4j服务器模式的REST API

原始细粒度Neo4j REST API的设计是从低端到超媒体驱动以及到你使用它的不同方面就能够“发现”它。每一个仅以很少信息量请求结果的返回，都以嵌入式的连接提供获得更多信息的机制。所有REST API的部件都能探测或获取的开始URL是<http://<domain-name>:<port>/db/data>。有时称它为服务的根目录（service root）。因此，Neo4j服务器运行于局域主机上，在默认的端口7474，将会有有一个服务根目录进入点[http://localhost: 7474/db/data](http://localhost:7474/db/data)。

下面的程序给出了一个请求所使用的数据和服务根目录的相关响应。

【程序10-4】 HTTP服务根目录的请求与响应



因为通过标准的HTTP协议能够访问REST，现在所有类型的客户端都可以照顾到，而不是那些仅仅能够使用JVM的客户端。下面的程序演示了标准的UNIX curl客户端产生的HTTP请求，请求的响应在程序10-4中一一详细列出。这段代码包括设置所有需要的头函数。

```
curl -X GET -H "Accept: application/json" -H "Content-Type: application/json"
http://localhost:7474/db/data/
```

假如你想从一个特定的起始点探索第9章中的社交网络图形数据库，例如，获得更多的关于Adam（系统中具有用户编号为adam001的示例用户）的信息。你首先需要识别你的起始REST URL，知道系统中的用户是用他们的用户编号索引的。使用程序10-4作为我们的起始点，如果你是使用传统的索引，你可以使用对node_index（❶）键详尽的基础URL作为起始点构建需要查找Adam的最终URL。使用基于模式索引，你将需要使用node_labels（❷）键构建一个URL。每一步的请求与相关的响应显示在下面的程序中。

【程序10-5】 通过他的userId获取Adam信息的HTTP请求与响应

```

Legacy Index HTTP request
GET http://localhost:7474/db/data/index/node/userids/userId/adam001
Accept: application/json
Label based index HTTP Request
GET http://localhost:7474/db/data/label/Person/nodes?userId=%22adam001%22
Accept: application/json

HTTP Response (partial response shown)
200: OK
Content-Type: application/json  ① 代表和能够用于获取这个节点（节点编号为0）
                                基础水平信息的 URL
{
  ...
  "extensions" : { . . . },
  "traverse" :
  "property" : "http://localhost:7474/db/data/node/0/properties/{key}",
  "self" :
    "http://localhost:7474/db/data/node/0",
  "all_relationships" :
    "http://localhost:7474/db/data/node/0/relationships/all",
  "data" : {
    "userId" : "adam001",
    "name" : "Adam"
  }
  ...
}

```

② 获取与这个节点相连的所有关系的更多信息的URL

③ 节点的数据，详细描述了关于这个节点的所有属性（例如键值对）

这一响应表示了返回的节点Adam碰巧与编号为0（①）的节点相关。它也包括与这个节点相关的所有属性（③）。如果你想探索这个节点的出和进的关系，将需要另一个调用。可以使用与all_relationships（②）相关的URL取回这组信息，其响应如下面的程序所示。

【程序10-6】 相对于Adam的所有关系的HTTP请求与响应

Logical HTTP Request

```
GET http://localhost:7474/db/data/node/0/relationships/all
Accept: application/json
```

HTTP Response

```
200: OK
Content-Type: application/json
```

```
[ { ... } ,
  { ... } ,
  {
    "start" : "http://localhost:7474/db/data/node/0",
    "data" : { },
    "property" : "http://localhost:7474/db/data/relationship/599/properties/
      {key}",
    "self" : "http://localhost:7474/db/data/relationship/599",
    "properties" : "http://localhost:7474/db/data/relationship/599/properties",
    "type" : "IS_FRIEND_OF",
    "extensions" : { },
    "end" : http://localhost:7474/db/data/node/600
  },
  { ... }
  { ... }
]
```

对每一个返回的关系，为客户端用户提供了数据和连接的混合信息，因此，用户可以发现关于有疑问特定关系的更多信息

利用这些响应，你可以按需求用另外的调用继续探索或发现这个图形数据库。尽管这种探索方式在REST背后的核心原理保持快速，但是它也使基本细粒度REST API成为一个相当流畅的协议，一般需要多个通过网络的请求来满足基本的图形查询，这并不预示着性能会好。在下面的几节中，你将看到什么样的方法能用于减少对话的次数，但是你现在应该对核心REST API是如何设计的和功能有一个基本的理解。

10.3.3 使用Cypher Neo4j服务器模式REST API端点

你能够在太多的情况下使用Cypher REST端点获得像客户端用户从各种不同细粒度REST API调用获得同样的结果，且具有更少的网络跳跃和更多的控制。正像你继续往前将会看到的，非常鼓励使用Cypher作为一种减少调用对话次数的方法。

下面的程序显示了如何能够通过REST API执行一个Cypher查询，并在

一个调用中获得Adam的节点基本信息及Adam所有关系的列表——相对于前一节演示的原始细粒度API的两个独立的调用。

【程序10-7】 通过REST API使用Cypher获得Adam信息，包括所有的关系

```
Logical HTTP Request
  POST http://localhost:7474/db/data/cypher
  Accept: application/json; charset=UTF-8
  Content-Type: application/json
  ← 执行 Cypher 查询的 REST API

{ "query" :
  "MATCH
    ( x:Person { userId: { uId } } )-[r]-()

    RETURN
      x as adam_node ,
      collect(r) as adam_rels",
  "params" :
    { "uId" : "adam001" }
  }
  ← 执行 Cypher 查询
  ← 任何参数中的值

HTTP Partial Response
  200: OK
  Content-Type: application/json

{
  "columns" : [ "adam_node", "adam_rels" ],
  "data" : [
    [ { // SAME AS CONTENT FROM LISTING 10.5 } ,
      [ // SAME AS CONTENT FROM LISTING 10.6 ]
    ]
  ]
}
```

使用Cypher端点的好处是能够在单一的执行中聚集和整合需求的数据，而不需要通过多个独立的网络调用。尽管程序10-7返回了原始节点和关系的数据，但是你可以很容易地修改它从而仅返回那些数据的一个子集，减少从网络来的有效载荷容量的大小。你可以用以下的查询选择仅仅返回Adam的名字和在IS_FRIEND_OF关系两端朋友的名字。

```
MATCH ( x:Person { userId: { uId } } )-[r:IS_FRIEND_OF]-(y)
RETURN
    x.name as name ,
    collect(y.name) as friend_names
```

10.3.4 使用远程客户端库帮助访问Neo4j服务器

10.3.2节介绍如何使用curl客户端用细粒度低层REST API进行操作，你还可能意识到，如果你不得不进行解析JSON构建适当的URL并在后来跟随它们时，这可能被证明是相当繁琐的。你还额外学到了如何使用Cypher端点作为访问Cypher服务器的另一种方法。然而，由于具有细粒度API，所以你需要完全理解执行的低层查询，包括解析和构建JSON结果。

你可以选择使用社区贡献的Neo4j远程客户端库之一来提供一个开发者更友好的接口，这可以为你做大量的低层工作。图10-5展示了这样的设置。许多语言和框架的结合也是有的——一个远程REST封装的列表（以及一般用途的封装）可以在<http://www.neo4j.org/develop/drivers>找到。在本书写作的时候，这个列表包括Java、.NET、Python、Django、PHP、JRuby、JavaScript和Clojure。

这些库和框架使得其与许多方面的REST API的集成成为一个更令人喜悦的经历而不是只尝试直接使用本机的API。程序10-8给出了一个Java客户端如何能够使用Java REST绑定库（java-rest-binding library）

（<https://github.com/neo4j/java-rest-binding>）通过Adam的所有关系开始导航。这一特定的绑定便在著名的GraphDatabaseService API后面绑定REST调用非常方便，这你已经在Neo4j的嵌入式模式中遇见过。

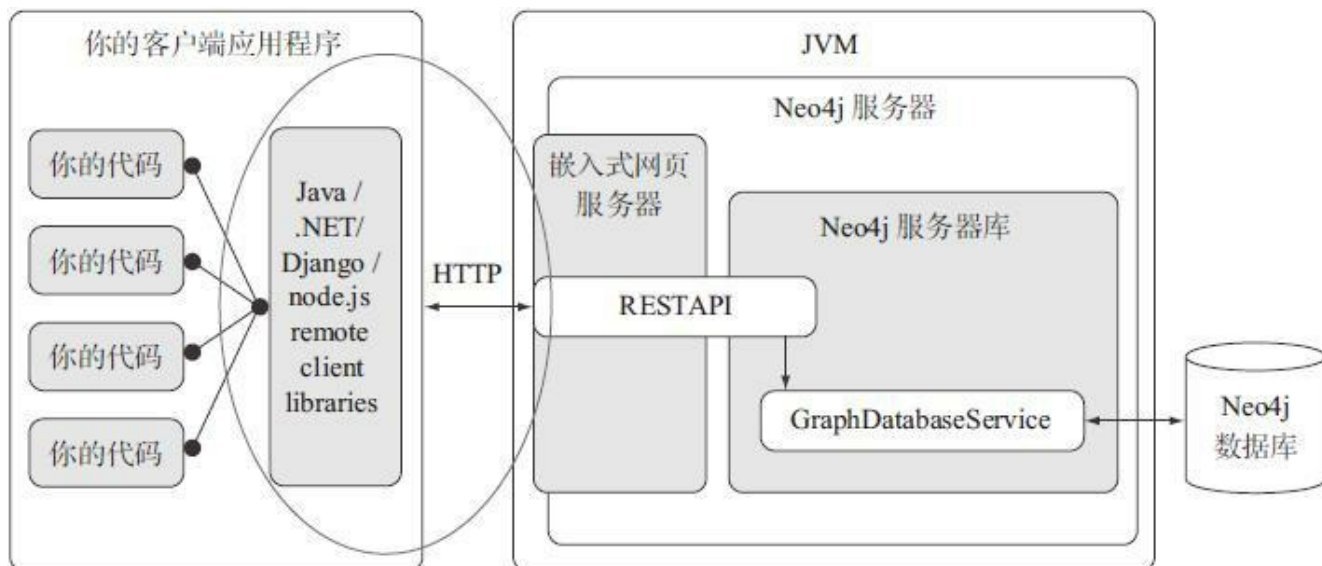


图10-5 使用远程REST客户端库的基于服务器的部署方法

【程序10-8】 Java REST客户端使用Java REST绑定库

```

...
GraphDatabaseService database = new RestGraphDatabase("http://localhost:7474/db/data");
Node adam = database.findNodesByLabelAndProperty(
    PERSON, "userId", "adam001")
    .iterator().next();
Iterable<Relationship> adamRelationships = adam.getRelationships();
for (Relationship rel: adamRelationships) {
    logger.info("Navigating Adams Relationship, found type " +
        rel.getType().name());
}

```

RestGraphDatabase 是一个 GraphDatabaseService 实施，它将所有的方法调用转换为场景后面的 REST 调用。

翻译成为一个在 `http://localhost:7474/db/data/node/0/relationships/all` 上的 HTTP GET 调用。

翻译成为一个在 `http://localhost:7474/db/data/label/Person/nodes?userId=%22adam001%22` 上的 HTTP GET 调用。

注意

neo4j-rest-graphdb绑定提供了两种方式与服务器进行交互。第一种方式是RestAPIFacade接口，它提供基本Neo4j REST API周围的单一封装。另一种方式（程序10-8中使用的）是使用一个GraphDatabaseService实施，它代表了场景背后适当的REST API调用。尽管第二种方法可以很方便地使用你已经熟悉并喜欢的GraphDatabaseService接口，但是必须强调的是你不能期待

获得与嵌入式模式同样的性能。涉及的额外网络调用将使Neo4j服务器调用比简单嵌入式模式调用具有更少的选择。当使用GraphDatabaseService时，你需要使用不同的策略，这样可以使GraphDatabaseService通过网络与服务器对话以获得可接受的性能。本章的后面将讨论这些选项。

10.3.5 服务器插件和非托管扩展

Neo4j提供两种主要机制避免基本REST API的冗长和繁琐的性质（Cypher REST端点除外）：服务器插件和非托管扩展，如图10-6所示。

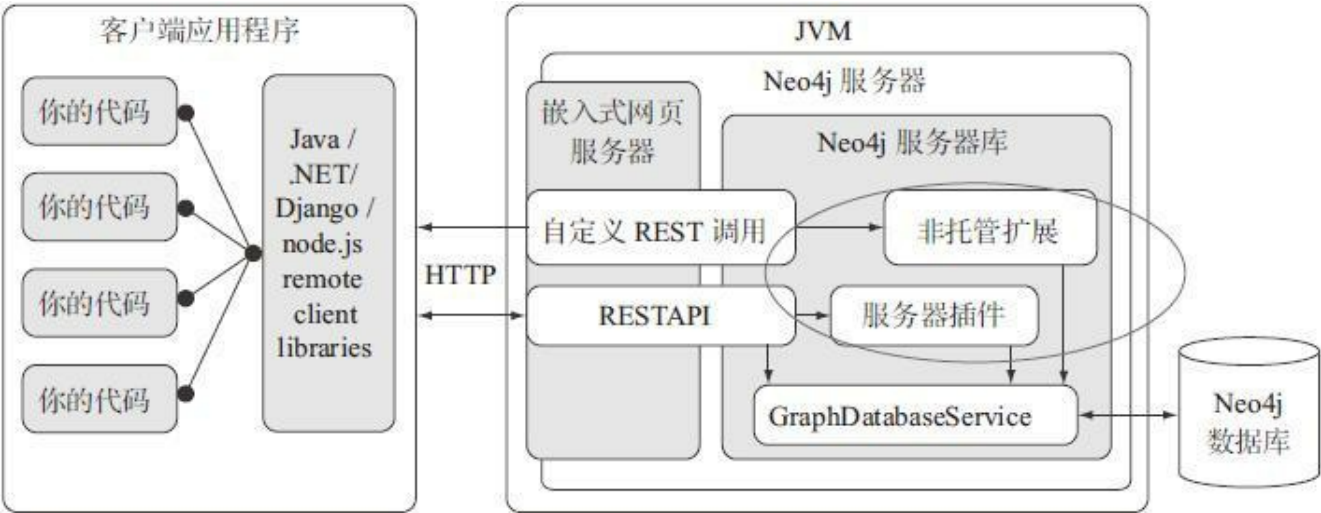


图10-6 通过服务器插件和非托管扩展访问Neo4j

这两个选项允许你写服务器端代码来补充或加强已存在的REST API提供工具，并且它们有时与关系数据库的已经保存的方法进行比较。他们试图通过提供一种卸载服务器端的一些繁重逻辑绕开一些服务器模式前后属性固有的性能限制，只将最后结果返回到客户端。特别地，非托管扩展提供了一个定义域更友好的REST API的机会。你将会在10.5.3和10.5.4节中看到如何写服务器插件和非托管扩展的例子，这两节将介绍如何设置服务器以便获得服务器的最好性能。在那之前，我们将对嵌入式模式和服务器模式进行比

较。

10.4 权衡选项

现在你已经对嵌入式模式和服务器模式的工作及它们之间的区别有了一个很深入的了解，现在我们将继续探讨每一种模式的优点和缺点。你将会看到选择一种模式而不选择另一种模式的道理，以及做出这种选择的可能影响。表10-1总结了讨论的主要方面。这里的大多数方面可以按结构和性能的考虑被广泛的分类，只有少数会分为另外的类别。我们将开始从这些观点攻克它们之间的比较。

表10-1 Neo4j的嵌入式模式与服务器模式的优点和缺点

服务类型	优 点	缺 点
嵌入式模式	速度 能够直接充分利用低层 APIs 能够操作 HA 设置	语言限制（仅支持 Java 和 JVM 语言） 可能的公共库与你的应用的冲突 应用程序流程与 Neo4j 的紧耦合 应用程序可能潜在的影响数据库的性能或数据库影响应用程序的性能 不能在应用程序中独立的定制 Neo4j
服务类型	优 点	缺 点
服务器模式	非耦合结构；你可以不依赖于应用程序定制和管理 Neo4j 支持大量的客户端平台（不仅仅是基于 JVM 的） 多客户端可以使用数据库 能够操作 HA 设置	尴尬和麻烦的细粒度 REST API （续） 较慢的速度，虽然使用 REST 流、Cypher 批处理、服务器插件和扩展可能给予一定的帮助 在现在的时间段，仅限于对本机的 REST API 处理 JSON 或 HTML 响应

Neo4j被成功地以嵌入式模式和服务器模式应用于刚开始的公司和大公司，例如，Adobe、Ebay和GameSys，因此，你可以放心的使用，这两种方式都经过了不同规模的公司的使用验证。Neo Technology公司是Neo4j的商业资助人，提供了一个具有各种不同设置成功使用Neo4j顾客清单（包括他们当中的一些实例学习）。更多的信息请参阅Neo Technology公司的网站：<http://www.neotechnology.com/customers>。

10.4.1 对架构的考虑

当你开始着手任何一个项目时，第一件需要考虑的事情是总体的架构需求是什么，包括需要支持哪些类型的客户端用户。

现在，我们将不考虑像是否需要HA的问题。这并不是说HA不重要，但是为了本节的目的，它不是等式的一部分，HA将在第11章中讨论。这足以说明嵌入式模式和服务器模式确实适合HA。

1.语言考虑

当涉及项目时，如果你有任何特定的语言限制，这自然是构成你做决策的主要因素之一。服务器模式相对于嵌入式模式能够适合大多数的客户端平台。嵌入式模式仅限于使用Java或其他基于JVM的语言之一；而服务器模式能够处理任何的能与HTTP对话的客户端。

2.关注点的分离：对应用（APP）关注VS对数据库（DB）关注

先不考虑客户端的选择，需要考虑的更基本事项之一是你需要什么样的规模和与Neo4j数据库分离的管理你的应用程序。

为了使这个讨论更具体，考虑图10-7中显示的前面章节提到的两种可以选择规划基于电影的社交网络应用的模式：嵌入式模式和服务器模式。让我们假设对一个网页应用有一个需求使普通的用户能够互相交互，以及一个管理部分或分离的应用，在那里被授权的管理员可以执行维护和日常事务，例如加载新电影、删除老用户等。

在嵌入式模式中，生命周期、内存和应用程序的处理能力（social-

movie.war) 紧紧地与嵌入式Neo4j数据库捆绑在一起；而对于服务器模式，具有分离的JVM管理应用程序和Neo4j数据库（JVM1和JVM2）。在服务器版本中，管理应用程序以自己基于PHP的网页客户端分离出来，甚至主程序部分和应用程序互相脱离。

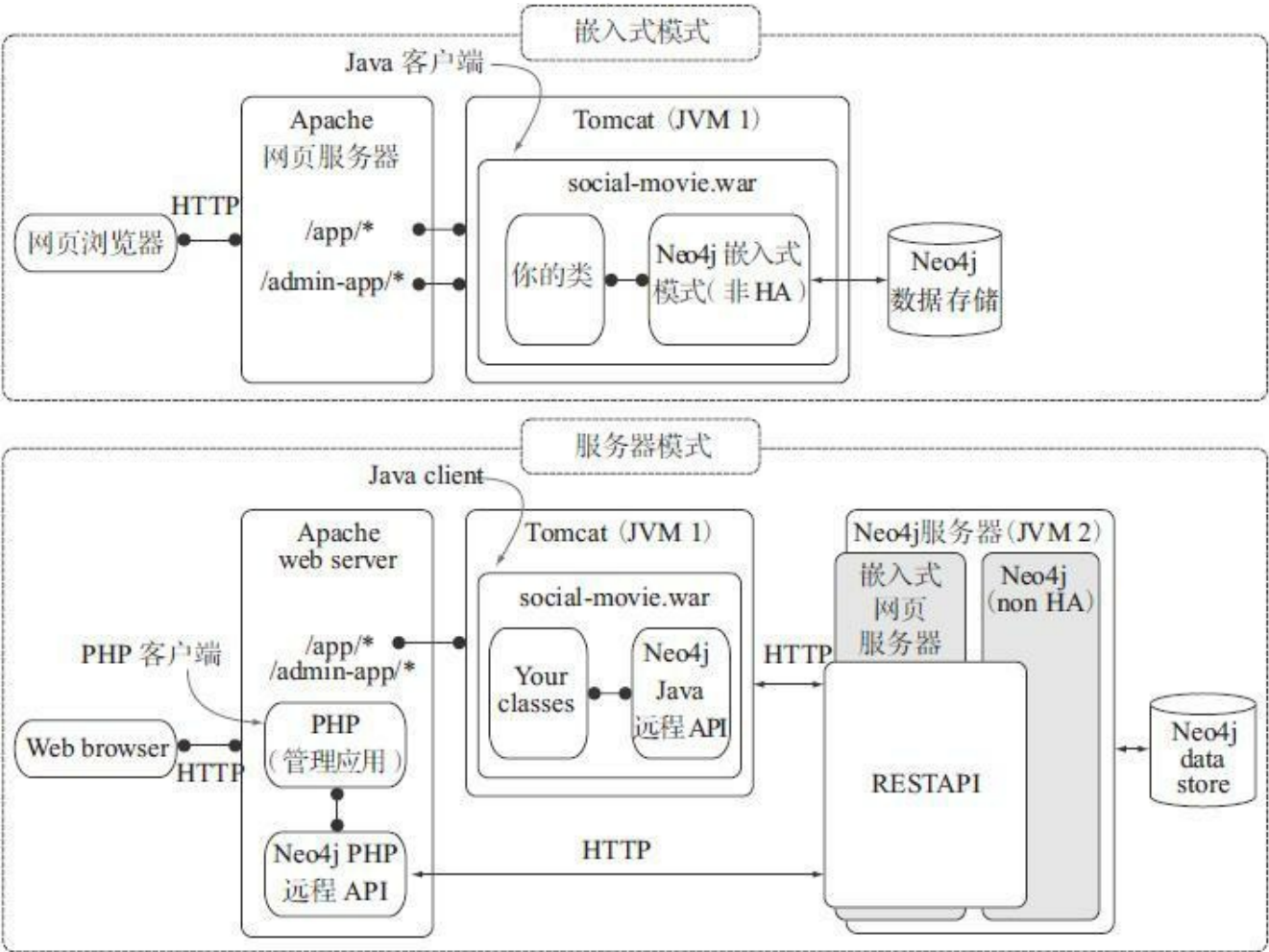


图10-7 社交网络应用程序的两种可能规划情况：嵌入式和服务器模式

在嵌入式模式中，你的应用程序与Neo4j共享同一个JVM，因此也共享相同的Java堆；它们经受同样的垃圾收集（GC）周期并且将从本质上一同生存和一起消失。如果你的应用程序引起或触发嵌入式模式的GC，将会影响到Neo4j的性能。当Neo4j反应慢的时候，可能仅仅是一个暂停在等待一个GC的完成。当你想从你的应用程序分离调试Neo4j JVM和GC参数时也会有这种情况发生，这是因为它们具有本质上不同的使用方式。不幸的是，在嵌

嵌入式模式中，这是不可能的。你需要寻找对数据库和应用程序结合需求的合理功能设置。就像前面已经说过的，许多公司具有成功的Neo4j嵌入式模式应用程序，但是很明显这一领域需要一些特殊的考虑。第11章将更深入详细地讨论如何调试和维护JVM以便以最佳的状态使用Neo4j。

另外，因为Neo4j是一个基于Java的应用程序，它将依赖于其他的一些公共库。Neo4j使用Lucene作为它的核心索引实现（见程序10-2）。尽管已经关注到了最小化这些依赖，如果你的应用程序也在使用任何的这些共享公共库，你将需要确保在JVM中任何的时间只包含和使用一个合适的版本，否则或者在你的应用程序中或者在数据库的行为上将会导致不可预期的结果，这一般不会是个大的问题，但是必须注意到这个问题。

3.对硬件的考虑

与前面关注的分离问题紧密相关的是对硬件的考虑。为了使Neo4j的操作和功能尽可能保持有效，理想的情况是需要一台强壮的电脑，具有许多内存和足够快的磁盘（详见第11章）。如果你的设置不可能实现，例如，如果你有一个已经存在的有限的应用程序服务器或者一组因各种原因不能升级的组件，那么在你的应用程序中使用嵌入式模式的Neo4j将肯定使你的硬件瘫痪。

在这种情况下，一种可能是换一台具有足够内存的新设备，使Neo4j服务器实例能够安装在上面，把应用程序服务器留给更符合的内存配置的其他程序用。如果你能够升级你的设备以适应Neo4j的额外需求，使用嵌入式的方式是一个可行的选择。

10.4.2 对性能的考虑

性能是嵌入式模式和服务器模式不同的关键问题。当通过本机的Java API相对于REST API做一组相同的操作直接比较运行的时间时，嵌入式模式的Neo4j总会胜过服务器模式的Neo4j。这是由于增加的延迟和相关的通过网络的调用。

通过原始比较的方式，结果如表10-2所示，显示的是以毫秒表示的嵌入式和服务器模式创建一百万个具有name属性的新用户节点所用的时间（括号里面的是每秒的节点数）。每一个新用户节点都是在自己的事务（TX）中（嵌入式模式使用本机的Java API，服务器模式使用REST API）创建的。

表10-2 当创建新节点时嵌入式模式相对于服务器模式的性能原始结果

场景	描述	嵌入式模式	服务器模式
1	每节点 1 个事务 (1 000 000 × 1)	168 815 毫秒 (5952 节点 / 秒)	2 380 140 毫秒 (420 节点 / 秒)

注：运行于苹果笔记本电脑上，具有16GB的内存、1TB的固态硬盘（SSD）的MacBook Pro（加密系统FileVault打开）。Neo4j服务器运行于同样的本地电脑上且同时运行单元测试，单元测试使用的neo4j-rest-图形数据库REST客户端库前面已详细讲过。

从表面上看，这些数字并不很好看，即使嵌入式模式也是。三分钟创建一百万个节点？不要感到绝望；这个例子是为了证明一个点。这个问题不只是一个讨论哪一种模式更快的简单的案例。相反，给定一种情况，做什么才能获得最佳的性能，并且是可接受的性能才是我们要考虑的重点。一些执行操作的调节方式能对性能造成重大的影响。

表10-3显示当你充分利用事务（嵌入式模式的本机事务和服务器模式的批处理）时，性能变得好很多。这一简单的变化对初始的数字具有很大影响：嵌入式模式快了高达10倍，服务器模式使用批处理快了16倍。

表10-3 当创建新节点时嵌入式模式相对于服务器模式的扩展结果

场景	描述	嵌入式模式	服务器模式
1	每节点 1 个事务 (1 000 000 × 1)	168 815 毫秒 (5952 节点 / 秒)	2 380 140 毫秒 (420 节点 / 秒)
2	所有节点 1 个事务 (1 × 1 000 000)	25 654 毫秒 (40 000 节点 / 秒)	时间太长，挂机了
3	批处理事务 (20 × 500 000)	16 081 毫秒 (62 500 节点 / 秒)	148 357 毫秒 (6756 节点 / 秒)

每当你看到性能的数据时，确保理解如何将性能比较的测试放到一起以及什么因素起作用或不起作用。为了直接说明这些情况，程序10-9和程序10-10给出了执行这些比较所使用的代码。

【程序10-9】 嵌入式模式性能测试比较所用代码

```

...

@Before
public static void setup() throws IOException {
    graphDbBuilder = new GraphDatabaseFactory()
        .newEmbeddedDatabaseBuilder("target/mytestdb");
    graphDB = graphDbBuilder.newGraphDatabase();
}

@Test
public void timeCreateNodesEmbeddedModeInSeparateTxs() {
    timeCreateNodes("EmbeddedModeInSeparateTxs", 1000000, 1);
}

@Test
public void timeCreateNodesEmbeddedModeInOneTx() {
    timeCreateNodes("EmbeddedModeInOneTx", 1, 1000000);
}

@Test
public void timeCreateNodesEmbeddedModeInMultiBatchedTx() {
    timeCreateNodes("EmbeddedModeInMultiBatchedTx", 20, 50000);
}

private void timeCreateNodes(
    String scenario, int numTx, int numNodes2CreatePerTx) {
    Stopwatch stopwatch = new Stopwatch(scenario);
    stopwatch.start();
    for (int a = 0; a < numTx; a++) {
        try (Transaction tx = graphDB.beginTx()) {
            for (int i = 0; i < numNodes2CreatePerTx; i++) {
                Node node = graphDB.createNode();
                node.setProperty("name", "User " + i);
            }
            tx.success();
        }
    }
    stopwatch.stop();
}

```

创建测试使用的新嵌入式数据库

嵌入式测试表 10-3 中第 1 行的对应结果

嵌入式测试表 10-3 中第 2 行的对应结果

嵌入式测试表 10-3 中第 3 行的对应结果

使用本机的图形 API 创建新节点

【程序10-10】 服务器模式性能测试（原始API）比较所用代码

```

@BeforeClass
public static void setup() throws IOException {
    restAPI = new RestAPIFacade("http://localhost:7474/db/data");
}

@Test

```

连接局域 Neo4j 服务器运行测试

```

public void timeServerModeRawRestAPIInSeparateBatchesStreamingOff() {
    timeCreateNodes("ServerModeRawRestAPIInSeparateBatchesStreamingOff",
        false, false, 1000000, 1);
}

@Test
public void timeServerModeRawRestAPIOneBatchStreamingOff () {
    timeCreateNodes("timeServerModeRawRestAPIOneBatchStreamingOff",
        false, true, 1, 1000000);
}

@Test
public void timeServerModeRawRestAPIInMultiBatchesStreamingOff() {
    timeCreateNodes("ServerModeRawRestAPIInMultiBatchesStreamingOff",
        false, true, 20, 50000);
}

private void timeCreateNodes(String scenario,
    boolean stream, boolean batchMode,
    int numBatches, int numNodes2CreatePerBatch) {

    System.setProperty(Config.CONFIG_STREAM,
        Boolean.toString(stream));
    System.setProperty(Config.CONFIG_BATCH_TRANSACTION,
        Boolean.toString(batchMode));

    Stopwatch stopWatch = new Stopwatch(scenario);
    stopWatch.start();
    for (int a = 0; a < numBatches; a++) {
        try (Transaction tx = restAPI.beginTx()) {
            for (int i = 0; i < numNodes2CreatePerBatch; i++) {
                Map<String, Object> props = new HashMap<String, Object>();
                props.put("name", "User " + i);

                Node node = restAPI.createNode(props);
                tx.success();
            }
        }
    }
    stopWatch.stop();
}

```

服务器模式测试表 10-3 中第 1 行的对应结果

服务器模式测试表 10-3 中第 2 行的对应结果

服务器模式测试表 10-3 中第 3 行的对应结果

用值“true”添加自定义 HTTP“X-Stream”头用于请求

如果启用则指示 REST API 使用 REST 批处理 API 实施 REST 事务

用 REST 批处理 API 实施 REST 事务

通过 REST API 发出请求创建新节点

当一个REST事物开始后（❷），所有后续的REST调用仅仅是收集起来并存在内存中直到这个事务标记为完成（直到具有Transaction的try模块完成）。这时，整个请求集通过网络作为单一的批处理请求发送。应该注意的是这个批处理行为默认是没启用的。要打开它，你需要设置系统的属性 `org.neo4j.rest.batch_transaction=true`，这是通过❶完成的。

作为一个一般的规律，对服务器模式，尤其是REST API，在性能方面会降低很多，这是由于基于网络的调用需求，但这并不意味着它应该被简单地排除在外。有相当多的可选方式和方法可以改进Neo4j在服务器模式的性能使其达到可接受的水平。10.5节将专门讨论这个问题。

10.4.3 其他需要考虑的事项

除了已经考虑过的结构和性能问题，还有几个额外的问题需要考虑，这也会影响到选择哪一种模式（嵌入式模式还是服务器模式）。

1.REST API: 支持的数据交换格式

REST API当前仅支持JSON和HTML作为数据交换格式。如果你喜欢用XML或其他一些格式，那你的运气就欠佳了。使用不同的数据交换格式的唯一方式是使用非托管扩展（参见10.5.4节），这需要定义你自己的REST接口访问数据库和与数据库交互。当你沿着这条路走下去时，你的确失掉了使用所有预先构建的REST API功能的能力。

2.事务

Neo4j是一个完全兼容ACID的数据库。当使用嵌入式模式时，这是一个相当简单的命题，但是对于服务器模式，有时会产生几个有趣的具有挑战性的问题。

在服务器模式中，默认情况下，每一个HTTP请求被当作单个事物。这意味着，例如，如果你的所有处置都是使用本机REST API，你将不能创建两个节点作为单个事物的部分行为（本机REST API需要两个分离的HTTP POST请求，这将作为两个分离的事务）。

在服务器模式中处理事务场景的方法包括以下几种：

- 使用Cypher端点。

- 使用REST事物端点。
- 使用服务器插件或非托管扩展。
- 使用批处理操作。

根据具体情况，你可以定义一个单一的Cypher声明，它能够在一个事务调用中执行多个操作。10.5.2节提供了更详细的使用Cypher端点的情况。另外，也有一个事务REST端点（<http://docs.neo4j.org/chunked/stable/rest-api-transactional.html>），允许你在事务的范围内，通过多个HTTP请求执行一系列Cypher声明。具有了这个选项，客户端显式地通过专用的REST端点发出提交或回滚的指令。

服务器插件和非托管扩展使你可以编写服务器端代码获取数据在单个HTTP请求中创建节点，然后在服务器端确保在单个事物中创建这些节点（参阅10.5.3节和10.5.4节的例子）。Neo4j也提供称为**批处理操作**（batch operation）的功能，允许你通过单个HTTP调用发送成组或成批的低级指令，当在服务器端执行时，所有的批处理指令被当作一个单个单位执行。更多关于批处理操作的信息可以阅读Neo4j手册的“事物的HTTP端点”一节（<http://neo4j.com/docs/stable/restapi-transactional.html>）。

10.5 充分利用服务器模式

最后一节强调在给定的一组环境情况下，从服务器模式中获取最好性能
的选项。就像我们前面讲过的，服务器模式会比嵌入式模式在性能方面经
受更多的影响，但是，通过使用这些技术和方法，服务器模式的性能可以
达到可接受的水平。

要完全理解和领会我们将给出的这些方法之间的区别，本节的剩余部分
将使用社交网络领域作为一个例子来看看这些不同方法如何导致不同的性能
指标。假设你有一个需求要查找和返回一个用户的直接朋友的所有名字，尤
其是那些名字以字母J开头的朋友。在你的系统中，早已经存在一个用户设
置，Adam拥有600个直接朋友，其中15个名字以字母J开始（Adam的userId
是adam001；这个用户已经被专门索引过，他的节点编号碰巧是0）。

在试验中使用两个不同的客户端：第一个是UNIX curl客户端，另一个
是java-rest绑定客户端（你能够找到前面我们用来做源代码的一部分curl脚
本和JUnix客户端测试类，附录B提供了运行这些代码的指导）。

表10-4提供了一个模板，当你按照步骤去做时可以填上你得到的结果。

表10-4 性能指标日志：模板

场景	描述	做服务器调用的数量	curl 客户端		Java REST 绑定	
			冷调用	热调用	冷调用	热调用
1	原始 REST API					
2	Cypher 调用					
3	服务器插件					
4	非托管扩展					

注：1.时间以毫秒为单位。

2.curl客户端的时间是使用bash time命令完成的。

3.Java REST绑定的时间是使用Java（Spring）StopWatch完成的。

4.冷调用=服务器首先停止然后启动后在第一个REST调用之前的调用。

5.热调用=第一个调用后没有做服务器重新启动的第二个调用。

冷调用与热调用的时间

对服务器的第一个调用（冷调用）包括了执行第一次引导、缓存、初始化过程需求的时间，这在随后（热调用）的调用中可能不存在。这意味着冷调用总会比较慢，并且也可能有一点比热调用更具有不可预测性。我们将为你提供这两种调用时间的完整图示。

10.5.1 避免细粒度操作

作为一个一般的规则，当通过网络执行任何类的操作时，应该尽量减少执行那个操作需要的网络跳跃数，并且这一相同的原理也适用于使用REST API方式。本机的、低层REST API操作是非常细粒度的，一般是在任意时间对单一节点或关系的操作。如果使用不当，它们能产生大量的不必要的网络流量。

除了只使用具有可发现能力的低层REST API来获取数据，你应该考虑下面可能导致非常少的网络调用和更好性能的选项。

- 使用Cypher REST端点。

- 使用遍历REST端点。
- 创建一个服务器插件或非托管扩展返回结果。

为了创建数据，考虑这些选项：

- 使用Cypher API中的变更功能。
- 使用REST批处理API。
- 创建一个服务器插件或非托管扩展执行任务。

如果你是在使用具有超媒体驱动方法的本机REST API获取在例题场景中需要的数据，你将需要一个总数为602个的网络调用以实现这个目标。假设Neo4j服务器在局域网上运行在端口7474，你需要做以下的事情：

- 在[http://localhost: 7474/db/data/node/0](http://localhost:7474/db/data/node/0)上做1次GET获取Adam的可用选项和数据的初始信息。

- 在[http://localhost: 7474/db/data/node/{nId}/relationships/all/IS_FRIEND_OF](http://localhost:7474/db/data/node/{nId}/relationships/all/IS_FRIEND_OF)上做1次GET获取Adam所有进和出的IS_FRIEND_OF关系列表（参见程序10-6的请求/响应（request/response）的详细情况）。

- 在[http://localhost: 7474/db/data/node/{nId}/properties](http://localhost:7474/db/data/node/{nId}/properties)上做600次GET获取第一次调用返回的每一个关系结构（将有600个条目）。然后你将使用最终的URI构建下一个调用，这将返回结束节点（朋友）的所有属性。在每一次的调用中返回的JSON可以被解析和用于仅仅返回那些以J开头的姓名。

更新了的性能指标如表10-5所示。

表10-5 场景1本机REST API后的性能指标日志

场景	描述	做服务器调用的数量	curl 客户端		Java REST 绑定	
			冷调用	热调用	冷调用	热调用
1	原始 REST API	602	5726 毫秒	5303 毫秒	1066 毫秒	917 毫秒
2	Cypher 调用					
3	服务器插件					
4	非托管扩展					

10.5.2 使用Cypher

Neo4j REST API为你提供了通过对指定的负责响应查询的REST端点发布适当的查询或声明和参数对服务器运行任意Cypher声明。假设你具有了所有的需要创建查询或声明的所有信息，这将只需要一次网络调用。

使用从请求服务根URL返回的响应，在10.3.2节中有详细的描述，相对于Cypher键定义的URL（见程序10-4）提供进入点用于发送Cypher查询和声明到服务器。下面的程序片段给出了程序10-4的服务根响应。

```
...
"batch"  : "http://localhost:7474/db/data/batch",
"cypher" : "http://localhost:7474/db/data/cypher",
...
```

程序10-11和程序10-12给出了满足我们例子场景的执行一个Cypher查询相应的请求和响应。

【程序10-11】 Cypher REST请求

```

HTTP Request
  POST http://localhost:7474/db/data/cypher
  Accept: application/json
  Content-type: application/json

{ "query" :
  "MATCH ( x:Person { userId: { lookupId } } )
    -[r:IS_FRIEND_OF]-(friend)
  WHERE friend.name =~ {name2find}
  RETURN friend.name ",
  "params" : {
    "lookupId" : "adam001",
    "name2find" : "J.*"
  }
}

```

← 执行 Cypher 声明的 REST 端点

① "query" 包含一个标准的 Cypher 查询

② 在查询中使用的 "params"

实际的Cypher查询是在JSON请求的query的键中指定的（①），并以params键（②）作为键-值对提供任意相关的参数。

【程序10-12】 Cypher REST响应

```

HTTP Response
  200: OK
  Content-Type: application/json

{
  "columns" : [ "friend.name" ],
  "data" : [ [ "NonJFriend 1", "NonJFriend 2",
    "NonJFriend 3", "JFriend 4",
    ... ] ]
}

```

① 排序的数组字段名字表示在数据键中要返回的数据

② 排序的返回数据数组与那些字段标识的数组

结果以定义了字段标题（①）和对应的数据值（②）的结构返回。可以看到字段名字匹配了在程序10-11的请求中指定的RETURN声明，并且所有匹配了这个字段标题的名字作为程序10-12中数据段的一个名字数组提供。更新了的指标日志如表10-6所示。

表10-6 场景2 Cypher调用后的性能指标日志

场景	描述	做服务器调用的数量	curl 客户端		Java REST 绑定	
			冷调用	热调用	冷调用	热调用
1	原始 REST API	602	5726 毫秒	5303 毫秒	1066 毫秒	917 毫秒
2	Cypher 调用	1	740 毫秒	45 毫秒	104 毫秒	32 毫秒
3	服务器插件					
4	非托管扩展					

这一方法已被证实对查询的性能具有非常大的改进。

10.5.3 服务器插件

服务器插件提供了一个卸载一些处理密集型逻辑到服务器的机制，而不是把所有的都放在客户端运行，那样就会在完成同样的事情时多次请求，使得数据来回流动。服务器插件有一些像关系数据库中存储的程序过程。

服务器插件专门设计用于扩展已有的REST API返回的节点、关系或全局图形数据库的选项。回忆一下当我们一个特定节点的详细情况时，你获取了返回的很多选项，包括一个extensions键。参见下面的程序片段作为一个概括。

```
{ ...
  "extensions" : { . . . },
  "property" : "http://localhost:7474/db/data/node/0/properties/{key}",
  "self" : "http://localhost:7474/db/data/node/0",
  "data" : { "name" : "Adam" }
  ... }
```

这代表了节点的扩展点列表（可用的服务器插件），关系和作为一个整体的图形数据库也会有相似的扩展点。

要写一个服务器插件，你需要首先决定目标是什么或要扩展什么（节点、关系或图形数据库选项），然后按照下面的步骤：

- 1) 写一个扩展ServerPlugin类的类。

- 2) 确保服务器插件类的名字完全合格并在文件org.neo4j.server.plugins.ServerPlugin列表中。

3) 确保插件类和文件是JAR文件，并且放在Neo4j服务器的类路径中。

4) 通过发现然后调用适当的REST URL访问这些函数。

下面的程序给出了我们创建的用于扩展功能的一个ServerPlugin类，用于查找名字以J开头的所有朋友的节点。

【程序10-13】 服务器插件类

```
public class JFriendNamesServerPlugin extends ServerPlugin
{
    @PluginTarget(Node.class)
    @Name( "get_friendnames_starting_with_j" )
    @Description( "Returns names of all friends start with the letter J" )
    public List<String> getFriendNamesStartingWithJ(
        @Source Node node )
    {
        GraphDatabaseService database = node.getGraphDatabase();
        try (Transaction tx = database.beginTx()) {
            return JFriendGraphHelper.extractJFriendNamesFromRawAPI( node );
        }
    }
}

public class JFriendGraphHelper {

    private RelationshipType IS_FRIEND_OF =
        DynamicRelationshipType.withName( "IS_FRIEND_OF" );

    public static List<String> extractJFriendNamesFromRawAPI( Node node ) {
        List<String> names = new ArrayList<String>();
        for (Relationship rel: node.getRelationships( IS_FRIEND_OF )) {
            Node friendNode = rel.getOtherNode( node );
            String friendName =
                (String) friendNode.getProperty( "name", "unknown" );
            if ( friendName.startsWith( "J" ) ) {
                names.add( friendName );
            }
        }
        return names;
    }
}
```

发现点类型 ②

需要拾取的必须扩展的服务器插件 ①

将在 REST API 中暴露的在客户端使用的名字 ③

对 ② 中指定的发现点类型的相应引用，这将在后面的方法中用于执行需求的无论是逻辑还是函数 ④

使用核心本机 API 执行实际逻辑的辅助方法

所有节点都是平等的

当一个插件的目标是一个节点时，它的目标也是所有的节点。即使逻辑上你在数据库中定义了不同类型的节点，例如user节点和movie节点，Neo4j

将使这个服务器插件对所有的节点都可用。应该谨慎定义能够用于所有节点的服务器插件，或者有一些机制确保服务器插件只在适当类型的节点上执行。

扩展ServerPlugin类（❶）将会确保这个类在服务器启动时作为一个服务器选项获取。对每一个需求的扩展点，应该创建一个指定（通过@PluginTarget注释（❷））发现点是什么类型的方法。这将是节点、关系或GraphDatabaseService之一。与@Name注释（❸）相结合，这将决定在哪里和在什么名字下，额外的REST端点将会在全部的REST API中出现。也会需要一个对方法本身（❹）中的节点、关系或GraphDatabaseService的参数引用，因此这个引用能够被用于执行任意的可能需求的功能或逻辑。

下面的程序给出了如何做HTTP请求以获取节点0（Adam）的详细信息，以及部分响应结果，包括可用扩展的一个列表。

【程序10-14】 获取Adam节点信息的HTTP响应的扩展程序片段

Logical HTTP Request

```
GET http://localhost:7474/db/data/node/0
Accept: application/json
```

HTTP Response

```
200: OK
Content-Type: application/json
```

```
{
  ...
  "self" : "http://localhost:7474/db/data/node/0",
  "extensions" : {
    "JFriendNamesServerPlugin" : {
      "get_friendnames_starting_with_j" :
        "http://localhost:7474/db/data/ext/JFriendNamesServerPlugin/
node/0/get_friendnames_starting_with_j"
    }
  },
  ...
}
```

服务器插件扩展列表
基于的JSON键

客户端可以用于查找URL
相对于这一特定节点执行
这个插件的名字（相应于
程序10-13中的❸）

就像在本节的开始列举编号的步骤，创建一个JAR文件需要包含插件类和org.neo4j.server.plugins.ServerPlugin文件，就像下面的程序片段所示（在META-INF/services目录中）。

```
com.manning.neo4jia.chapter10.serverplugin.JFriendNamesServerPlugin
```

这个JAR文件应该放在服务器的类路径。通常是通过把这个JAR文件放在服务器安装位置的插件目录实现的。

这个服务器插件执行的结果的详细情况如表10-7所示，这实际上是使用本机的嵌入式API。再一次，相对于普通的REST API，有了一个很大的改进。

表10-7 场景3服务器插件执行后的性能指标日志

场景	描述	做服务器调用的数量	curl 客户端		Java REST 绑定	
			冷调用	热调用	冷调用	热调用
1	原始 REST API	602	5726 毫秒	5303 毫秒	1066 毫秒	917 毫秒
2	Cypher 调用	1	740 毫秒	45 毫秒	104 毫秒	32 毫秒
3	服务器插件	1	147 毫秒	20 毫秒	76 毫秒	16 毫秒
4	非托管扩展					

10.5.4 非托管扩展

如果你需要完全控制你的服务器端的代码，那么非托管扩展可能正是你要找的。与服务器插件仅允许你在特定的点增强已有的REST API不同，非托管扩展实质上允许你定义自己的特定域REST API。不仅仅是处理节点和关系，假如你做出这样的选择，现在你能够处理用户和电影。

Neo4j通过允许部署任意的JAX-RS（专对REST的网页服务Java API）类到服务器使得这个成为可能。JAX-RS提供了一组API，这组API是为了使开

发者开发REST服务成为一件很简单的事而提出的。一般来说，你通过一组注释定义一个Java类，绑定这个类到Neo4j服务器内得到一个特定URL模式和安装点。当这个安装点被调用时，将控制传递给这个类，这可以完全访问Neo4j图形数据库，返回期望的任意格式的数据。尽管仍然需要通过HTTP协议，数据格式并不局限于JSON和HTML，就像使用REST API和服务器插件一样。



非托管扩展从本质上给你非限制性访问使用和支配Neo4j服务器资源。这是极其强大的功能。但是，你需要谨慎小心别出意外以伤害到自己。这可能由于执行某种类型的复杂遍历消耗了所有的JVM堆空间而出现。假设你理解你正在做什么，这可能是你工具箱里的强有力工具，但俗话说：权力越大责任越大！

下一个程序给出了一个基于他们的名字查找一个用户的非托管扩展的实现，后来返回了这个用户所有名字以J开头的直接朋友。

【程序10-15】 一个非托管扩展


```

@Path( "/example" )
public class JFriendNamesUnmanagedExt {

    private final GraphDatabaseService database;
    private static final Label PERSON = DynamicLabel.label("Person");

    public JFriendNamesUnmanagedExt(
        @Context GraphDatabaseService database ) {
        this.database = database;
    }

    @Path( "/user/{userid}/jfriends" )
    @GET
    @Produces( MediaType.APPLICATION_JSON )

    public Response getJFriends( @PathParam( "userid" ) String userId )
    {
        List<String> names = null;
        try (Transaction tx = database.beginTx()) {
            Node theUser = database.findNodesByLabelAndProperty(
                PERSON, "userid", userId)
                .iterator().next();
            names=JFriendGraphHelper.extractJFriendNamesFromRawAPI(theUser);
        }

        ObjectMapper mapper = new ObjectMapper();
        String jsonString = mapper.writeValueAsString(names);
        return Response.status( Status.OK ).entity(jsonString ).build();
    }
}

```

开始 URL 的一部分，在这其中这个功能将会起作用

@Context 注释 将 GraphDatabaseService 注入到类，提供对数据库的完全访问

在 URL 模式的剩余路径，方法中定义的函数将会显露

将会导致以 JSON 的内容类型响应为标题的生成内容

将仅响应 HTTP 的 GET 方法

绑定由 #C 定义的 userid 变量中指定的任何数值到 Java userid 属性以在方法内使用

排列一个名为 JSON

由于具有服务器插件，这个类需要是一个JAR文件并使其对Neo4j服务器可用。按惯例，这是通过把JAR文件放在Neo4j服务器的插件目录中实现的。

另外，你将会需要添加和映射在neo4jserver.properties文件中的任何非托管扩展到org.neo4j.server.thirdparty_jaxrs_classes键（通常可以在Neo4j服务器安装的conf目录中找到）。这个映射是由定义包含扩展类的Java包构成的。下面的程序片段是映射这个到基础安装点：

```

org.neo4j.server.thirdparty_jaxrs_classes=com.manning.neo4jia.chapter10.unmanagedext=/n4jia/unmanaged

```

这意味着执行这个非托管扩展并且获得Adam的名字以J开头的所有朋友，你需要针对<http://localhost:>

7474/n4jia/unmanaged/example/user/adam001/jfriends发出一个HTTP GET。

表10-8给出了程序10-15中的非托管扩展与其他到目前为止你已经看过的方法的对比。

表10-8 场景4非托管扩展的性能指标日志

场景	描述	做服务器调用的数量	curl 客户端		Java REST 绑定	
			冷调用	热调用	冷调用	热调用
1	原始 REST API	602	5726 毫秒	5303 毫秒	1066 毫秒	917 毫秒
2	Cypher 调用	1	740 毫秒	45 毫秒	104 毫秒	32 毫秒
3	服务器插件	1	147 毫秒	20 毫秒	76 毫秒	16 毫秒
4	非托管扩展	1	158 毫秒	20 毫秒	119 毫秒	18 毫秒

除提供另一个改进服务器性能的机制以外，非托管扩展还提供了另一个好处，允许定义一个域特定的REST API以及一个使用任何数据交换格式的能力，这些格式包括JSON、二进制、文本或者你选定的其他格式。然而，我们重申我们的警告，这一强有力的工具需要谨慎地进行管理，免得你不经意间打开了“潘多拉魔盒”。

10.5.5 流REST API

从Neo4j 1.8版本起，“流”的选项和对REST请求的JSON响应被介绍为另一种改进Neo4j REST API性能的方法。默认的情况下，流是关掉的。

目前，从客户端的角度，所有使结果流回的需求是提供一个扩展的标题（X-stream=true），如图10-8所示。

```
tmp -- bash -- ttys008 -- 98x43
04:52:08(/tmp)$
04:52:09(/tmp)$
04:52:09(/tmp)$ # ----- Current number of nodes in DB -----
04:52:09(/tmp)$ curl -X POST -d '{"query": "start n=node(*) return count(n)" }'
-H "Accept: application/json" -H "Content-type: application/json"
http://localhost:7474/db/data/cypher
{
  "columns": [ "count(n)" ],
  "data": [ [ 120602 ] ]
}04:52:09(/tmp)$
04:52:09(/tmp)$
04:52:11(/tmp)$
04:52:11(/tmp)$
04:52:12(/tmp)$ # ----- Returning ALL nodes (Streaming OFF) -----
04:52:19(/tmp)$ curl -X POST -d '{"query": "start n=node(*) return n " }'
-H "Accept: application/json" -H "Content-type: application/json"
--limit-rate 12m -H "X-Stream: false"
http://localhost:7474/db/data/cypher > /dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 168M 100 168M 100 42 9740k    2 0:00:21 0:00:17 0:00:04 11.5M
04:52:37(/tmp)$
04:52:37(/tmp)$
04:52:39(/tmp)$
04:52:44(/tmp)$
04:52:45(/tmp)$ # ----- Returning ALL nodes (Streaming ON) -----
04:52:46(/tmp)$ curl -X POST -d '{"query": "start n=node(*) return n " }'
-H "Accept: application/json" -H "Content-type: application/json"
--limit-rate 12m -H "X-Stream: true"
http://localhost:7474/db/data/cypher > /dev/null
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 149M  0 149M 100 42 11.9M    3 0:00:14 0:00:12 0:00:02 12.0M
04:52:59(/tmp)$
04:52:59(/tmp)$
04:53:06(/tmp)$
04:53:07(/tmp)$
04:53:08(/tmp)$
```

图10-8 打开或关闭流的结果

注意这仍然是一个新的功能，它可能会经历修改，因为清晰的使用模式会出现并且API在不断发展。现在，我们不会讨论得太深入。

为了演示通过使用流API获得的额外性能改进，我们执行一个Cypher查询，返回例子中的数据库的所有节点。我们添加了额外的120000个节点产生数据，共出现120602个节点，包括Adam和他在已有数据库中的朋友。流的结果大约是149MB的数据量用时14秒，而168MB的数据量用时21秒。

注意

有效载荷比较小是因为流的结果在送回时压缩了（空白的地方删除了）。流也能减少服务器执行这一工作需求的内存量。因为请求的数据在从数据库读出的时候直接流回了客户端，在这个过程中没有经过临时存储任何

节点或关系对象，从而使得上述情况成为可能。

10.6 本章小结

你已经看到了嵌入式模式和服务器模式这两种方式如何提供访问以及与Neo4j数据库交互的功能，但是完成其工作的方式是完全不同的。

对于嵌入式模式，你与Neo4j之间有一个相当友好的关系，这仅仅是对Java和其他几种基于JVM的语言是可以的。尽管你能够直接对所有的Neo4j低层API进行访问并且能够增强与这一模式相关获得的性能，你也要与Neo4j共享你的资源（内存等）。这可能潜在的在应用程序的管理中引进一些额外的开销和并发症，这些是需要加以考虑的。

在服务器模式中，Neo4j的流程是隔离的，并且能够与你的应用程序彻底的分离进行管理，这是在大型分布式结构中的一个很大的成功。然而，所有的交互是通过REST提取层实现的，尽管这会根据其支持的客户端涉及广泛的网络，但会对性能产生一些影响，也需要对其进行理解并进行适当的处理。

这些性能问题并不是不能解决的，具有像服务器插件的选项、扩展、流和适当的使用Cypher声明，都能够帮助服务器模式以更好的性能运行。

下一个逻辑查询是如何在实际应用环境中运行Neo4j数据库。现在你已经具备了所有的基本知识，但是，当你真正投入到实际生活中时，还要考虑哪些因素？第11章，也是最后一章，将要讨论这一问题。你将会学习一些涉及在真实的应用环境中使用Neo4j的应考虑的因素。

第11章 Neo4j的架构与应用

本章包括以下内容：

- 高层Neo4j架构概述
- 如何准备将Neo4j应用于实战中
- 如何扩展与配置Neo4j使其高可用
- 如何备份与恢复Neo4j数据库

最后一章涵盖了涉及在实际应用环境下运行Neo4j的一些更具操作性的问题。

这包括查看一些诸如以高可用性（High Availability）模式运行Neo4j，特别注意保证Neo4j能够以容错和可升级的方式运行。为了一些比较完善的应用情况以及提供如何配制重要内存和缓存设置的理解和指令，本章额外包含了如何备份和恢复数据库，这将在决定Neo4j运行情况中起一个很重要的作用。

为帮助你在本章中获得最大的收获，并确实能从本书中获得全面的知识，这里通过Neo4j结构并在某些地方探究了深层次的东西，开始一个高水平的旅程。这个旅程的主要目的是提供上下文的连接以及用于引进和讨论可操作性和可配置性问题的结构路径。它也帮助我们回顾和加强一些在前面几章讨论过的重点和重要概念。

因此，让我们开始吧！

11.1 高层Neo4j架构

与任何的数据库一样，Neo4j也有许多设置——为了指示或影响数据库如何操作和执行，可以使用把手和拉杆进行“推”或“拧”。我们可以提供一个可使用的设置和选项位置的简单列表，但是我们希望 you 通过本章获得更多的知识，而不是仅仅知道哪里可以改变。理想的情况是我们希望你获得一点点Neo4j的机制感觉——Neo4j是如何在内部工作的一种基本感觉。这将使你理解为什么进行某些设置将引起Neo4j以这样的方式工作，而不是仅仅知道某一种特定设置的存在。

机械和谐

机械和谐是一种隐喻，它在计算机领域获得了流行。通常使用它传递想法以达到最佳的利用工具或最适当的利用系统（在这里是Neo4j），理解它的工作机制将给予你帮助。然后你可以以最合适的方式使用这一知识。

原始隐喻的本身归因于方程式1赛车司机Jackie Stewart，他被公认曾经说过为了成为最伟大的赛车手，你必须与你的赛车具有机械和谐。Stewart相信最好的表现来自于赛车手与赛车完美和谐的工作结果。

当在描述LMAX团队如何实现高流量干扰模式的低延迟时，LMAX架构的Martin Thompson凭借是第一个在计算意义上推广使用这一隐喻的人而闻名（就我所知）。他对这一隐喻的使用可以在“LMAX-如何在小于1毫秒延迟做100K TPS”演讲稿中看到，该演讲稿可以在www.infoq.com/presentations/LMAX网站中找到。

高层Neo4j结构如图11-1所示。对这一结构的浏览将从图的底部开始并环绕到顶部。

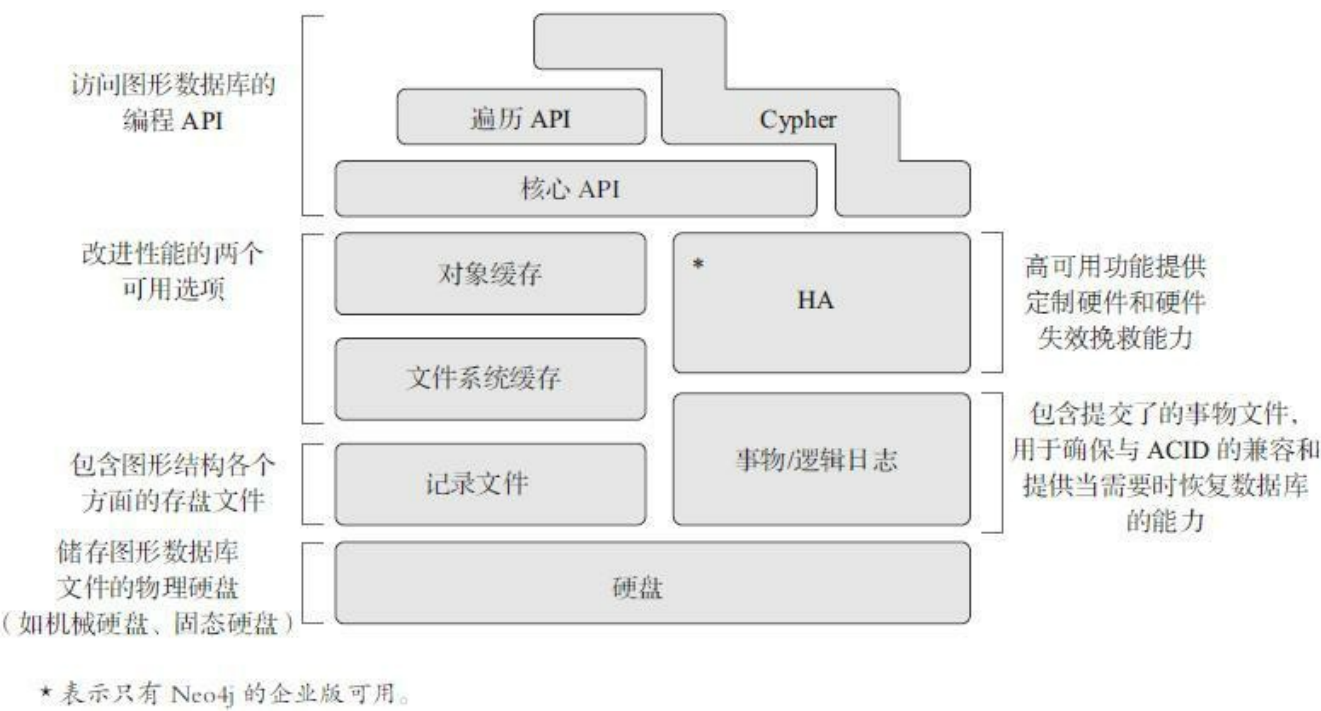


图11-1 高层Neo4j结构概况

因为这是一本入门书，我们将无法深入到Neo4j内部所有的方面。我们的目标是讨论深度足以帮助你欣赏和与Neo4j协调一致的工作，而不研究低层的细节。关于Neo4j内部如何工作课题的深层处理参见Jim Webber、Ian Robinson和Emil Eifrem的《Graph Database》（图形数据库）的第6章。尽管这是一本通用图形数据库著作，第6章使用Neo4j的结构作为一个例子解释如何建立和实施一个高性能的本地图形数据库。另外，Tobias Lindaaker已经做过Neo4j内部的多次演讲，其中之一的“Neo4j内部概述”可以在这里找到：www.slideshare.net/thobe/an-overviewof-neo4j-internals。

11.1.1 设置场景

现在到了应该将应用程序投入到实际工作中的时候了。从操作的角度看，涉及新型图形数据库时，通常会有三个主要的问题，或三类主要的问题。

- 物理存储——需要什么类型和多大的存储？

- 内存——支持高效率运行应用程序和数据库需要多少内存和几个CPU？

- 可扩展性和冗余——确保Neo4j能够扩展以适应业务的增长和高可用性（或足够可用），并且确保能够适当的备份并在出现故障事件后恢复需要做什么？

从后往前说，恢复和备份将在11.3节中详细介绍，HA将在11.2节中讲解，与内存和CPU相关的问题将在11.1.4节中介绍。首先，让我们解决存储和硬盘问题，这使我们恰好回到了Neo4j体系结构堆栈的底层。

11.1.2 硬盘

在Neo4j体系结构堆栈的底层是Neo4j实际存储图形数据库的物理硬盘。

1.应该使用什么类型的硬盘？

Neo4j并没有硬性规定使用什么类型的硬盘，但是不言而喻，在需要硬盘输入输出时，速度快的硬盘运行得快。使用低寻址时间的硬盘（例如固态硬盘，也称为SSDs）通常会比传统机械式硬盘提高10~20倍的读盘速度。

就像将在11.1.4节学习的，你应该尽可能多的将图形数据库保存在内存

中，这样会减少硬盘的输入和输出，并帮助Neo4j以最好的状态工作。然而，在某些情况下，你的应用程序或数据库将会需要到硬盘读取数据，至少，这会在开始启动数据库并访问图形数据库不常用部分时发生。这也会在你的图形数据库因为太大不足以调入内存中时发生（见11.2.4节缓存分区）。

避免硬盘输入输出是最大化Neo4j性能的关键因素之一，当不能避免硬盘输入输出时使用高速硬盘是完全不同的另一个因素。

2.图形数据库需要多大的空间？

需求的空间数量显然取决于你打算在Neo4j中存储多少数据和支持Neo4j运行操作需求的额外空间。

所有Neo4j的数据位于一个单一的、顶层的Neo4j目录中（默认的是基于服务器安装的data/graph.db）。这包括位于一组储存的文件中（在11.1.3节中讨论）的核心图形数据、事物日志（在11.1.5节中讨论）和在其他文件中的其他内容（包括索引和其他小文件）。

所有包含核心图形结构的主要存储文件在空间占用的数量上是一致的——它们有固定的记录大小，可以帮助你估算最终的文件大小。尽管这一单一因素并不总能准确地算出，这对计算核心数据库尺寸的需求是有极大帮助的。这有一部分原因是Neo4j本身有时创建额外的文件作为它的部分内部运行时间的优化，它不可以提前预期的。索引和标签也会占用较多的空间。

你可以基于对需求节点、关系和属性的估计做粗略的计算。一般的计算核心图形数据所需空间的公式是：

核心图形大小（字节）=（节点数×以字节记的节点存储大小）+（关系数×以字节记的关系存储大小）+（属性数×平均每属性的字节数）

固定记录大小将在下节详细讨论，但是，在本书的写作时，节点存储占用了14字节，而关系占用了33字节。此外还会有索引、事物日志等需要的额外硬盘占用，当在计算需要的总的硬盘占用时，这些也需要进行考虑。前面已经说过，通常计算Neo4j占用空间最大部分的核心图形数据尺寸相对比较简单。

Neo4j硬件参数计算器

Neo技术提供了一个在线硬件参数计算器用以估算最小的硬件配置需求，给予一组一定数据的输入，例如，期望的节点数、关系数、属性数、用户数等。这个计算器不仅处理硬盘容量，还有内存和一些设置推荐等。

应该强调的是这个计算器只是提供一个粗略的估计。如果你是在做一个至关重要的项目，建议你找合格的工程师寻求更精确的估算，考虑更多的其他因素。但是对于一个粗略的和准备开始的计算，这个计算器一般就足够了。

可以在Neo技术网站www.neotechnology.com/hardware-sizing找到硬件参数计算器。

11.1.3 存储文件

Neo4j存储图形数据库的不同部分在一组被称为存储文件（store file）中。这些存储文件通常通过记录类型分解——有节点、关系、属性、标签等

单独文件。这些文件中的结构和数据布局做了专门的设计和优化以提供高效的存储格式，Neo4j运行时引擎可以利用这些格式提供在图形数据库中高效查找和遍历。核心启用之一是Neo4j按照免索引邻近原则存储数据。

免索引邻近原则

免索引邻近原则（或属性）是一个幻想的术语，是指能够以非常不同的方式存储图形数据库的数据。尤其是，每一个节点在数据库中的存储有一个直接的链接或引用到它的临近节点（这些节点直接与它相连），并且必须不需要使用任何其他的辅助结构（例如索引）找到它们。

通过这种方式存储数据，当执行遍历时，Neo4j能够直接跟随指针连接节点和关系，相对于非免索引邻近存储，例如关系数据库，这种类型的访问数据速度显得非常快。当试图定位直接连接的节点时，在Neo4j中不会产生开销。非免索引邻近存储首先需要使用支持（索引）结构查找哪一个节点连接在第一个位置。只有那样它们才能够访问这些节点。免索引邻近原则允许查询时间与你想搜索多少图形数据成比例，而不是与整个图形尺寸成比例。

回忆一下第1章中的足球体育场模拟——“Neo4j的速度秘密在哪里？”

假设你本人要在一个当地的小足球场带领你的拉拉队为你的球队加油。如果有人问你坐在你身边15英尺以内有多少人，你会站起来，并以最快的速度数一数你身边的人。现在设想你在国家足球场看比赛，有很多观众，你要回答的是同一个问题，在你15英尺以内有多少人。假设这两个足球场的人数密度是一样的，那你会有大致相同的人数需要数，所用的时间也会非常接近。于是可以说，不管足球场能容纳多少人，你将以预计的时间把你身边的人数完，因为你只需要数坐在你身边15英尺以内的人，而不用去关心在球场

另一边包厢里的人。

这一类比提供了一个实际演示一个人如何自然地使用免索引邻近原则完成计数任务。

存储文件位于主图形数据库目录中（默认是服务器的data/graph.db）并以neostore作为前缀。表11-1给出了Neo4j使用的主存储文件以及它们的一些关键属性。

表11-1 使用的主存储文件及相关的属性

存储文件名	记录容量	内 容
neostore.nodestore.db	14 字节	节点
neostore.relationshipstore.db	33 字节	关系
neostore.propertystore.db	41 字节	节点和关系的简单属性（原始关联字符串）
neostore.propertystore.db.arraysneostore.	120+8 字节	数组属性的值（在块中*）
propertystore.db.strings	120+8 字节	字符串属性的值（在块中*）

注：*块尺寸可以使用string_block_size和array_block_size的参数进行设置。默认块的大小是120字节和用于管理的8个字节。

主存储文件具有一个固定的或统一的记录尺寸（14字节用于节点、33字节用于关系等）。除在快速查找和遍历中起重要作用，就像11.1.2节中提到的，固定的长度使得规划和计算需要多少硬盘容量和内存分配给你的图形数据库变得容易。

节点和关系存储文件仅仅存储指向其他节点、关系和属性记录的指针，因此可以正好存到固定的记录文件。另一方面，属性的处理有一点点困难，因为它们实际所代表的数据可能有不同的长度。尤其是字符串和数组将会有变化的数据长度，因为这个原因，Neo4j对它们进行特殊处理，将这种动态

变化类型的数据存到一个或多个字符串或数组属性块。更多的信息请参阅 Neo4j 手册（<http://docs.neo4j.org/chunked/stable/configuration-caches.html>）。细节会在手册的22.6、22.9和22.10节中介绍。

固定长度记录如何改进性能？

使用固定长度的记录意味着基于节点或关系编号的查找不需要通过存储文件本身的搜索。而是给定一个节点或关系编号，可以直接计算出文件中存储数据的起始点。节点存储文件内的所有节点记录都是14字节的长度（在写作本书时）。节点和关系编号是数值型的并在存储文件内直接与它们的位置相关。节点编号1将是第一个记录，节点编号1000将是第1000个。

如果你想查找与节点编号1000相关的数据，你将能够计算出这一数据将会在节点存储文件中从14000字节开始（14字节×节点编号1000）。计算这些数据起始位置的时间复杂度 $O(1)$ 远远小于搜索一个函数的时间复杂度 $O(\log n)$ 。如果大写字母O函数使你受到惊吓，不要怕，这里所有你需要理解的是通常计算一个起始点比搜索一个起始点快得多。当涉及大量数据时，这也会转变为显著的性能提高。

固定记录的大小在为计算一个文件中相邻记录块的存储位置提供一个快速机制中也扮演着重要角色，文件系统缓存中（见11.1.4节）也使用这一快速机制加载存储文件的部分到缓存中。

当涉及存储文件的结构时，有两个应该注意的重要特性。

第一个是这些固定记录大小可以用作帮助估计中心图形文件将占据的硬盘空间，因此，你的硬盘就需要有这么大（参见11.1.2节）。

第二个是这些存储记录在涉及文件系统缓存和它是如何工作时，具有1：1的映射。缓存将在下一节讲解，但是基本文件系统缓存负责加载这些存储文件的部分到可用的内存中以便快速访问。密切注意这些存储文件的大小是一个能够计算需要的内存和缓存需求的很重要的因素。

11.1.4 Neo4j缓存

即使具有超快的硬盘和Neo4j的高效存储结构，每当处理流程需要从CPU到硬盘时总会出现一个延时惩罚。如果需要访问的数据能够放在内存中时（某种RAM），这样就会减少甚至消除硬盘的输入和输出，这种延时惩罚就会大大减少。当数据在内存的缓存中时，Neo4j会进行优化操作。

因此，就Neo4j而言，在内存中是什么意思？Neo4j使用一个两层缓存策略：文件系统缓存和对象缓存。这些缓存，以及它们在单个服务器全部内存中的位置如图11-2所示。

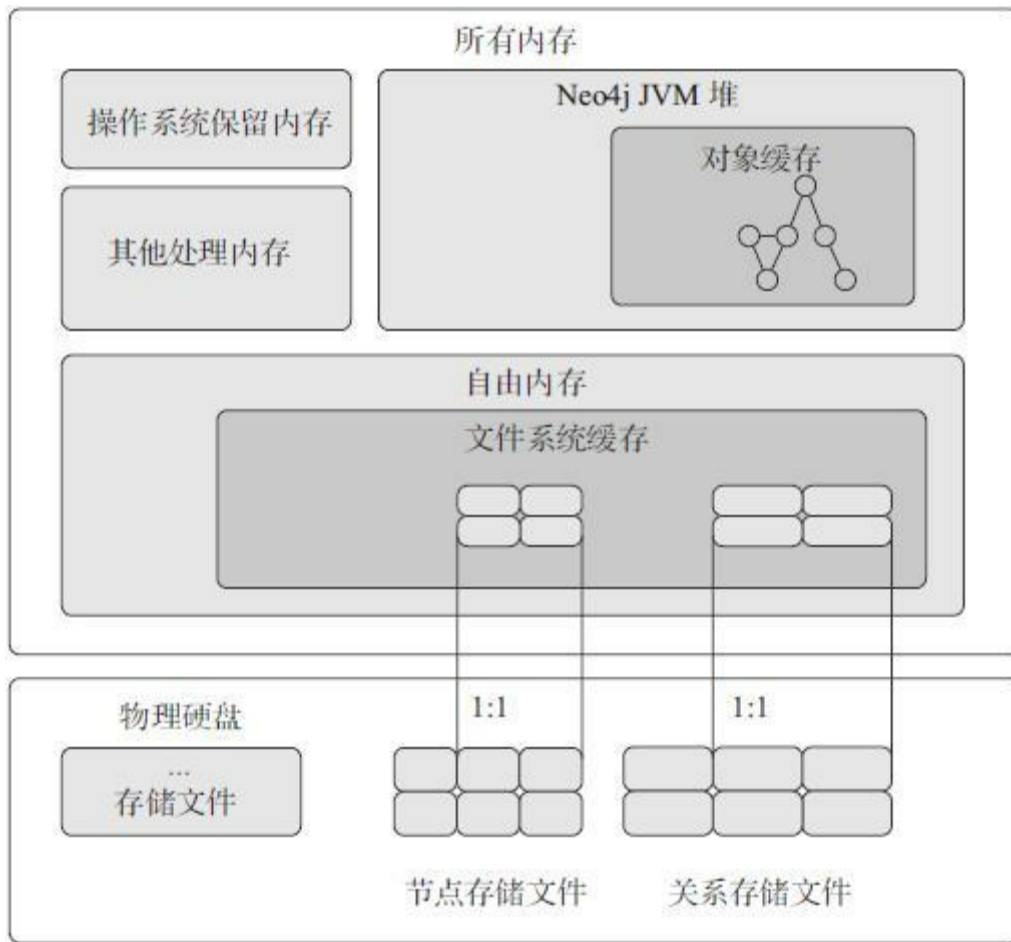


图11-2 Neo4j对内存缓存的使用

1.系统文件缓存

文件系统缓存是自由内存（还没有分配给任何处理流程中的内存）的一个区域，是操作系统保留的提高文件读写速度的内存。文件系统缓存使用一种称作内存映射输入输出的方法（memory-mapped IO）。每当一个处理请求一个文件时，文件系统缓存检查是否已经调入。如果没有调入，需要的文件（或部分需要的文件）从硬盘中读到内存区域。之后对同一文件（或部分的同一文件）的需求就可以直接从内存中访问，通过减少对物理硬盘输入输出的需求大大增加性能。数据的变化也会写到文件系统缓存，而不是直接写到物理硬盘。性能的提升是通过访问文件系统缓存获得的，而不是直接访问一个旋转的硬盘，这会大约快500倍。

操作系统负责管理这些内存，包括决定什么时间将在缓存中的数据变化直接写到物理硬盘中去。尽管操作系统管理这些数据什么时间读出和写入这个缓存区，处理过程也可以请求某些文件或部分的文件调入这一缓存区进行处理。

Neo4j利用操作系统的这一特点（通过Java NIO包）有效地从存储文件中加载、读出和写入。这提供了Neo4j中的第一层（有时称作低层）缓存功能。

如果系统崩溃并且数据只写入内存中而没有写入硬盘将会发生什么情况？

如果操作系统控制文件系统缓存中的数据写入硬盘的时间，在系统崩溃事件出现时将会发生什么情况？

简单来说，这一缓存区的数据就丢了。然而，不必害怕，因为Neo4j使用分离的、持久的事务日志来确保实际写入到硬盘文件每一次提交事务的实施。每当一个提交发生时，尽管存储的文件本身可能还没有实际更新，事物日志总会在硬盘上有数据。然后在系统崩溃后重启时可以使用事务日志（11.1.5节讨论）恢复数据。换句话说，这一事物日志可以用作重建存储文件，使之与上一次提交时完全一样，并且存储的文件也会与系统崩溃前把数据直接写入硬盘的文件一样。

2. 配制文件系统缓存

理想情况下，你的目标应该是加载尽可能多的持续图形数据（如存在硬盘中的物理存储文件）到内存中。你可以控制持续图形数据的哪一部分和多

少加载到内存中。在实际中，由于所有的数据都存在文件中，这通常涉及查找这些文件占据多少磁盘空间（或你希望它们成为多大的文件），并试图为每一个文件保留适当数量的内存。

假设你有一个顶层Neo4j数据库目录大约占据了2.2G的容量，有下列的存储文件和大小：

```
20M neostore.nodestore.db
60M neostore.propertystore.db
1.2G neostore.propertystore.db.strings
900M neostore.relationshipstore.db
```

下面的程序给出了将会确保整个图形数据库能够调入到内存放在文件系统缓存中的配置。这个配置文件的设置显示了大约2.5G的内存，允许数据大约有10%的增加。

```
neostore.nodestore.db.mapped_memory=22M
neostore.propertystore.db.mapped_memory=66M
neostore.propertystore.db.strings.mapped_memory=1.4G
neostore.relationshipstore.db.mapped_memory=1.0G
neostore.propertystore.db.arrays.mapped_memory=0M
```

当以服务器模式运行Neo4j时，这些设置是在conf目录中的neo4j.properties文件中指定的。

对嵌入式模式的设置，配置设置可以在数据库创建时传入，或者通过引用一个neo4j.properties文件，可以从classpath的某个地方访问该文件，或者直接通过映射在Java中引用。

你可能已经注意到了在硬盘的物理存储文件与配置设置具有1：1的映射。在更宽的文件系统缓存区域，Neo4j确保为每一个存储文件保留分离的单个文件缓冲缓存区，因此也允许基于每一个存储文件做缓存配置。这使得

配置每一个存储文件的多少含量应该调入内存成为可能。

不同的应用程序将会有不同的使用模式，并且独立调试这些缓存区使得你能确保应用程序运行的性能足够好，尤其是在没有足够的内存调入整个图形数据库到内存的时候。



当我们讨论HA时，我们也应该注意缓存分区概念，这个概念可以用于帮助加载图形数据的适当部分到多个计算机的内存中。

在官方的Neo4j手册的“内存映射输入输出设置”中提供了内存映射输入输出缓存的特定场景和配置例子，可以

在<http://docs.neo4j.org/chunked/stable/configuration-io-examples.html>中找到。

默认设置

在缺少用户的配置文件时，Neo4j将会试图根据系统自己的知识及给予它的可用内存数量确定怎样的配置是最好的。

3.对象缓存

文件系统经历了一个很长的时间通过减少硬盘的输入输出改进性能，但是Neo4j是一个基于JVM的应用程序，它能够处理节点和关系的概念（作为Java对象存储），而不是仅仅与原始文件交互。

Neo4j的对象缓存是JVM堆里面的一个区域，Neo4j在那里通过核心Neo4j API以一种便于用于遍历和快速恢复的优化形式存放Java对象的节点和关系版本。

前面给出了如何应用文件系统缓存相对于访问旋转机械硬盘上的同样数据提高500倍的速度。对象缓存是另一个大大超过文件系统缓存提高性能的策略。在对象缓存中访问数据（相对于到文件系统缓存）大约快5000倍。

4.配置对象缓存

配置对象缓存有两个方面。第一个涉及配置JVM，尤其是JVM堆，这可以通过在启动时给JVM提供适当的-Xmx??? m参数来实现。第二种方法是选择一种适当的缓存类型，这是通过在neo4j.properties文件中设置cache_type参数控制的。

我们首先解决JVM的配置，因为这将会对你的应用程序和数据库具有重大的影响。一般的经验是尺寸越大越好。因为这意味着大的缓存容纳和处理更多的对象，也会使应用程序获得更好的性能。这一观点在大多数情况下是对的，但是并不是总那么简单。

基于JVM的发展的一个吸引人的特点之一是作为一个开发者，你不必担心为你的对象分配和释放内存（如在其他的语言C++中需要的）。这是由JVM动态管理的，JVM负责对你的对象分配合适的内存，以及通过使用一个垃圾收集器清理没有引用的对象。垃圾收集器可以调整和配置，但是，一般情况下，JVM堆尺寸大于8GB时似乎会引起问题。

大的堆尺寸能够导致长GC暂停和抖动——这正是需要避免的浪费时间的类型。长GC暂停和抖动对性能具有可怕的影响，因为当JVM结束时将试图执行清理和释放对象的维护活动，这要比应用程序执行任何有用工作花费更多的时间。

因此，应该分配多少内存给JVM以便Neo4j运行的性能最佳？直到找到一个合适的利用JVM堆空间尽可能多而没有引起太多的与GC相关问题的协调点之前，这是你需要考虑的问题之一。利用以下的启动参数和数值提供一个好的开始点。

- 分配尽可能多的内存给JVM堆（考虑你的计算机和JMV特定的限制——对大多是人来说是6GB或更少），通过-Xmx??? m参数传递。

- 以-server标志启动JVM。

- 使用并发标记和清扫压缩垃圾收集器而不是默认配置的JVM，使用-XX: +UseConcMarkSweepGC参数。

关于在Neo4j中基于JVM和基于GC的性能优化的详细讨论请参阅官方手册<http://docs.neo4j.org/chunked/stable/configuration-jvm.html>。

创建一个早期的样本图形数据库以帮助决定内存（和硬盘）的需求

没有一个通用的简单和完全精确的方法准确计算JVM将会需要内存中多大的堆空间来容纳和处理图形数据库。许多人选择先建立一个小的，但是相对精确（如果有可能的话）的在开发过程中尽可能早的样本图形数据库。即使这一样本图形数据库将随着时间的进程发生变化，但是它也能帮助解决具有同样结构和访问模式的大型数据集可能需要的空间和内存。

这对解决文件系统缓存、对象缓存和通用存储需求的合适设置是有用的。

第二个设置是neo4j.properties文件中的cache_type参数设置。这将控制节点和关系对象是如何调入缓存的。为了简化，我们将列出在官方手册中具有简单描述的可用选项，如表11-2所示。更多信息请参阅Neo4j手册中的“Neo4j中的缓存”一节<http://docs.neo4j.org/chunked/stable/configuration-caches.html>。

表11-2 按照官方文档的对象缓存类型选项

缓存类型	描 述
none	不使用高层缓存，没有对象调入缓存
soft*	对可用的内存提供优化的使用。适用于高效的遍历。如果频繁访问的图形数据库部分没有放进缓存中时，在高强度的调入下，可能导致 GC 问题
weak	对缓存的对象提供短的生命周期。适应于高输入和高输出量的应用程序，这里图形数据库的较大部分比能够调入内存的图形访问的次数高

(续)

缓存类型	描 述
strong	保留调入到缓存的所有数据并且不再释放它。如果你的图形小到足以放进内存，提供优秀的性能
hpc**	提供了一种方法指定一定数量的内存作为加载的节点和关系的缓存。动作不大但能快速插入或查找。大多数情况下是最佳的选择。是一种高效缓存

注：*默认缓存类型

**仅在Neo4j的企业版中可用。（注：一些用户已经成功的使用这类缓存至堆尺寸多达200G）

随着JVM堆尺寸的不断增加，它几乎要占用了所有可用的内存，堆尺寸的设置和文件系统缓存的设置应该放在一起考虑。记住也要包括其他的因素，例如，操作系统对内存的需求和其他任意运行的流程。如果你是在以嵌入式模式运行，Neo4j数据库将会与你的主机应用程序共享JVM堆，因此，应确保也考虑这个因素。

5.缓存总结

缓存方式可以总结如下：

- 最低目标是设置将全部或尽可能多的图形调入文件系统缓存中。
- 然后，尽量多使用对象缓存。

11.1.5 事务日志及可恢复性

转到我们的高层结构图的右边（见图11-3）。我们将重点放在提供一个事务日志文件（有时也会指的是逻辑文件）如何用于确保Neo4j与ACID的兼容的简单回顾。但是本章更重要的是看看如何使用事务日志提升自身的修复和恢复。

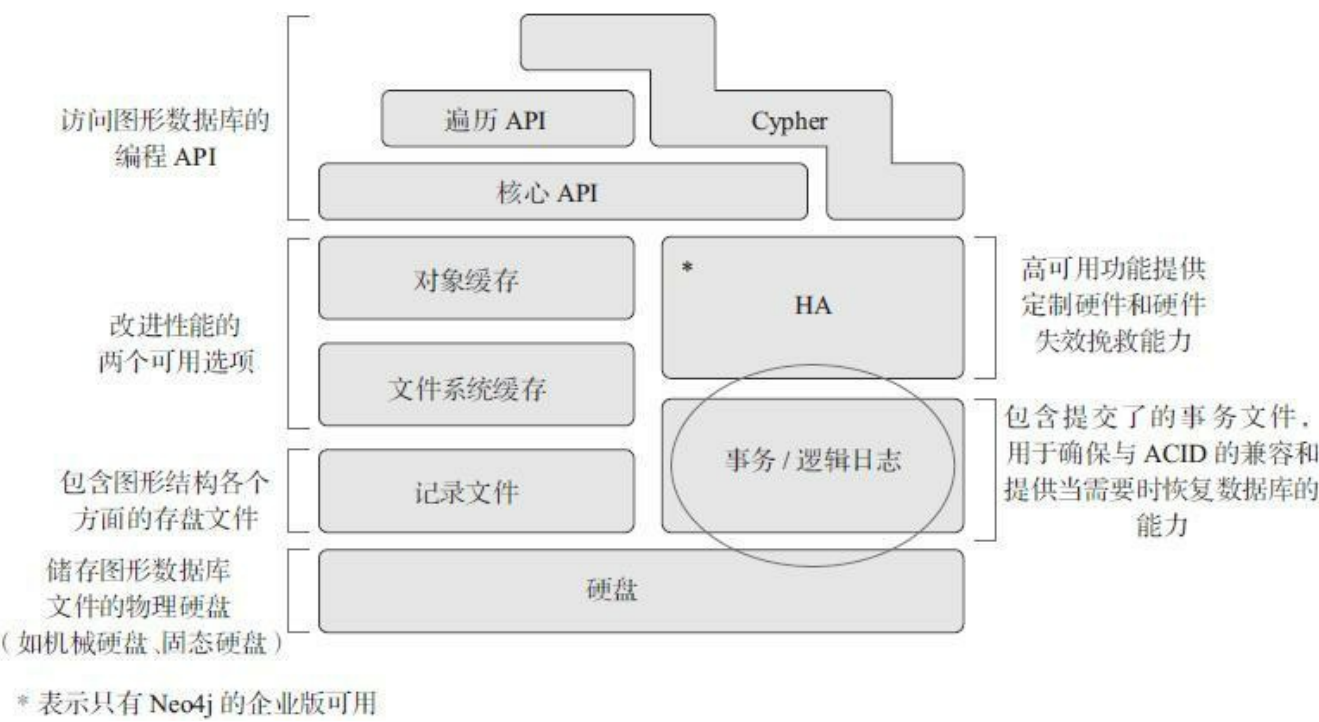


图11-3 事务日志在整个Neo4j结构中的位置回顾

在第7章，我们知道了Neo4j是一个完全兼容ACID（原子性、一致性、隔离性和持久性）的数据库。许多基于ACID的系统（包括Neo4j）使用称作

预写日志（WAL）作为机制提供ACID方面的原子性和持久性。预写日志的使用意味着每当提交一个事务，所有的数据变化在应用于存储文件本身之前写入（物理刷新）硬盘中激活的事务日志文件中。回忆一下使用文件系统缓存意味着写到存盘文件仍然可能也在内存中——操作系统负责决定什么时间将内存的这些区域写到硬盘中。但是即使系统崩溃并且提交了的变化还没有写到物理存盘文件，这些存盘文件也会使用这些事务日志文件进行恢复。

所有的事务日志文件可以在Neo4j数据库的顶层目录中找到，并且具有以下格式的文件名`nioneo_logical.log.*`。

非正常关闭的后台恢复

非正常关闭通常是指一个Neo4j实例的意外崩溃。也可能是指Neo4j的有意关闭，但是关键点是，不管是什么原因，Neo4j没有执行它的正常结束活动。当一个非正常关闭出现时，下面的警告信息会在下一次数据库启动时在日志中看到——`non clean shutdown detected`（检测到非正常的关闭）。

当Neo4j启动时，它要做的第一件事是查阅最近的事务日志，并且重做发现的所有针对实际存盘文件的所有事务。有可能这些事务已经应用到了存盘文件中（记住是操作系统控制什么时间文件系统缓存写入硬盘，这可能已经发生或没发生）。然而，重新再一次做这些事物不会出现问题，因为这些动作被看作为幂等的，即这些变化可以被多次应用而不会导致图形数据库中的不同结果。

一旦Neo4j实例已经启动，它的存盘文件将会完全恢复并且包含所有指向的事务和最后的提交，给你做好了继续工作的准备！

除了帮助事务和恢复性外，HA的功能也是基于事务日志建立的——以群设置方式运行Neo4j的能力。

11.1.6 编程API

在Neo4j结构的最顶层是三个主要的API（Cypher、遍历和核心），用于访问和处理Neo4j中的数据（见图11-1）。前面的章节已经深入讨论了这些API，因此这里我们不再讨论。

从技术上讲，这些API没有任何的操作问题，也没有像缓存一样的方式做特殊的设置来调节这些API。它们确实组成了你正在经历的整个Neo4j的结构，因此在这里特别提到它们。

只有一样东西在本节中我们要进行强调，尽管这些API中的每一个都能被独立地用于与图形数据库进行交互，从某种程度上它们是建立在相互的基础上的（有一点点像堆的方式）。更重要的是，它们每一个都有自己的适应点，具体取决于你想做的事情，即由你来选择合适的工具完成工作。图11-4给出了主要目标和每一个API使用的方法的概况。

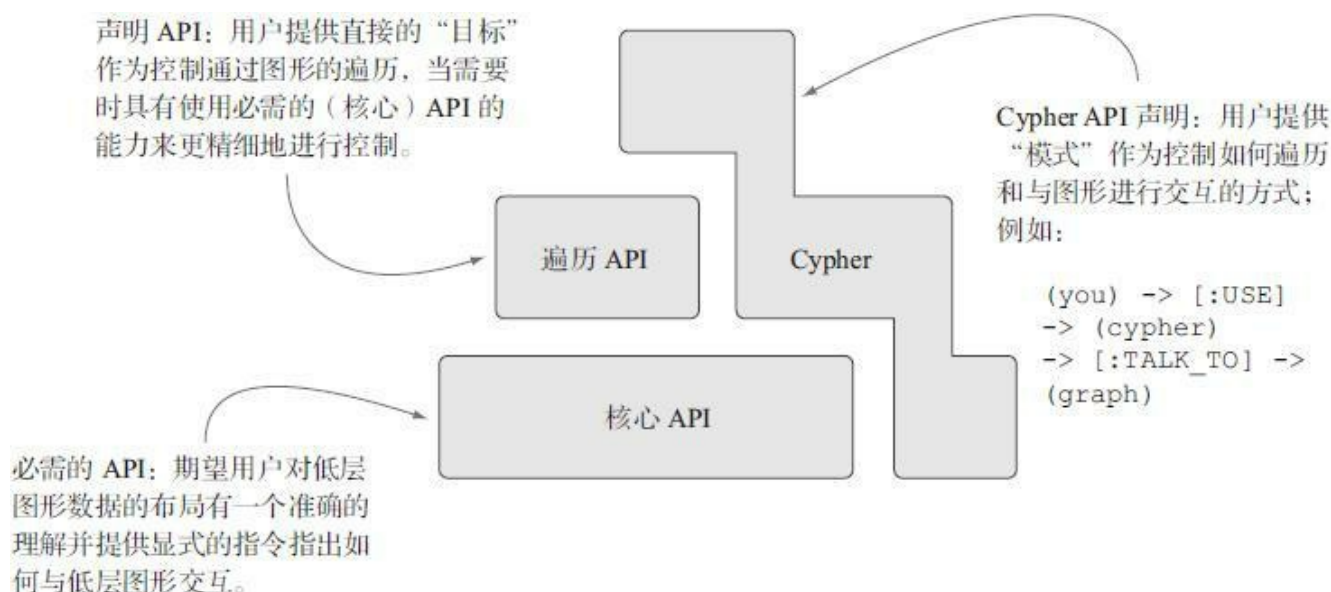


图11-4 编程API栈

如果性能是最重要的, 你或许会选择深入底层使用核心API, 因为这会提供最灵活的性能和对图形交互的控制。这是有代价的, 因为这需要对图形数据的布局有一个准确的把握, 以便与之交互, 并且这种交互可能是相当详细的。

遍历API是一个稍微友好的API, 在这里用户给出目标, 例如“做一个到深度3的深度优先搜索”, 允许Neo4j关注与核心API的交互以实现结果。尽管这个API对开发者而言是比较容易使用的, 但是略高一层的抽象意味着它可能没有像执行核心API一样达到最佳。

然后就是Cypher——声明的、人性化图形查询语言, 并且可以说是所有API中在使用方面最直观和最友好的。许多人会说Cypher代表了Neo4j的未来, 因此很多的开发努力都集中在了这一领域。其吸引力和易于使用与理解性(针对开发者和操作人员)使得它成为Neo4j产品中具有令人信服的优势。目前, 在大多数情况下, Cypher的性能不如核心API。在Neo4j 1.x中, Cypher绑定到核心和遍历API(因此, 它绝不能做的比那些API更好), 但是

在未来（尤其是从Neo4j 2.0往后），Cypher将会绑定到栈的较低层的一个相协调的API上，这将会支持优化查询和其他的运行效率。尽管目前Cypher在性能上还不能与核心API相比，但是没有理由认为它不会在将来与核心API看齐或者超过核心API——时间会说明一切。

11.2 Neo4j的高可用性

企业版的Neo4j具有一个Neo4j HA部件，这个部件为Neo4j提供运行群设置的能力，允许你将数据库分布在不同点的电脑中。Neo4j使用主从复制结构，广义地说，提供了支持两个关键特色的能力：在硬件故障时的应变能力 and 容错能力，扩展Neo4j读密集型数据场景的能力。

HA VS 集群

当在Neo4j中做多台电脑的设置时，你将会经常听到HA和集群这两个术语可以互换。在本质上，它们是指同样的概念，只是名字不同。

弹性和容错是指即使在网络中断或者硬件故障时，Neo4j继续为客户提供服务的能力。如果集群中的一台电脑坏了，或者网络连接断了，Neo4j应该能够继续提供服务，而不是完全失去服务的能力。

除了能够处理硬件故障之外，主从结构具有一个额外的好处，提供了一个横向扩展Neo4j的方法读密集型数据的场景。在这里，读密集型数据场景可以指当有需要或有期待时具有高吞吐量的应用程序，在多个从实例中横向扩展读取数据。它还支持缓存分区（将在11.2.4节中讨论），使得Neo4j能够处理比单个Neo4j实例更大的读载荷。

11.2.1 Neo4j集群概述

Neo4j HA集群涉及一系列Neo4j企业版实例，这些实例被配置到属于单一的逻辑单元（群）。在任何的给定时间，期望总有一个处于主导地位，根

据需要设置没有或有多多个从实例的集群。

Neo4j HA结构最近经历了大的改进，以简化设置和操作运行的需求。在Neo4j 1.9版本之前，Neo4j使用Apache ZooKeeper (<http://zookeeper.apache.org>) 提供集群协调功能。尽管ZooKeeper是一个优秀的产品，但是在Neo4j的使用中出现了很多问题，包括不得不处理操作的额外开销、设置和集成一个分离的协调部件。在自动配置其本身时也具有限制，在一些基于云的设置中证实了具有某些限制。

由于这些以及其他一些原因，Neo技术决定推出自己的集群协调机制，并且从Neo4j 1.9版本开始，包括Neo4j 2.0版，不再使用ZooKeeper。Neo4j的实施是基于Paxos协议，它主要处理主选择，但是Neo4j也负责处理一般的基于集群的管理任务。它不需要理解Paxos协议中的细节（这大大超出了本书的讨论范围），但是如果你感兴趣的话，Leslie Lamport的文章《Paxos化繁为简（2001）》提供了一个对这个主题的简单介绍：www.cs.utexas.edu/users/lorenzo/corsi/cs380d/past/03F/notes/paxos-simple.pdf。

图11-5给出了新的Neo4j集群协调机制基于Paxos协议的工作情况。

在一个Neo4j HA集群中，当每一个数据库实例遇到逻辑需求与群中的其他成员进行协调和通信时，其自身是能够自给自足的。当一个HA数据库实例启动时，要做的第一件事情是建立到它的配置集群的连接。`neo4j.properties`文件包含不同的属性来控制HA配置（都以`ha*`为前缀），包括属于本集群中成员的初始设置列表中的属性——`ha.initial_hosts`。

如果启动后不存在集群，意味着这是第一台试图连接集群的计算机，然

后这个过程将会建立开始的集群，并将这台计算机作为集群的群主。

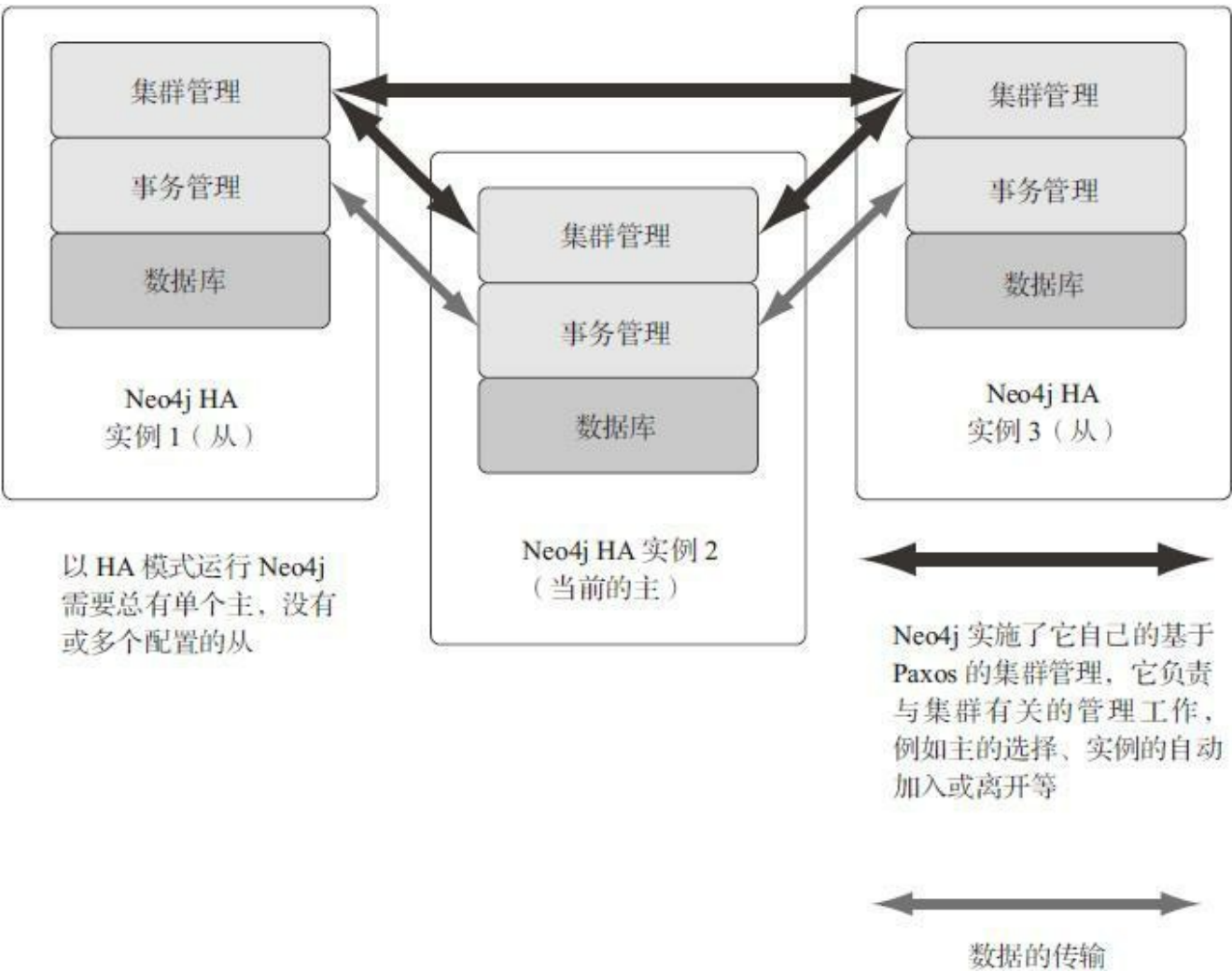


图11-5 在主1和从2中的Neo4j HA群设置样例

所有的写请求最终都是通过主计算机完成的，尽管也有可能把请求发送到从计算机去执行。然而，这样做要比直接由主计算机做慢得多。在这种情况下，从计算机在逻辑上内置了确保同步首先写到主计算机中，然后再写到其自身。

数据的修改通过一个可配置的基于推拉机制的从计算机传回主计算机。

什么是CAP和ACID? [1]

以集群设置运行对Neo4j提供的ACID保证具有一定影响。Neo4j的单机

一致性担保放宽了在一个集群中的[最终一致性](#)。这是什么意思？

一本NoSQL书，如果没有谈到或提到CAP定理就不算结束。Neo4j声称是ACID兼容的数据库，并且当在单机上运行时，这项保证是绝对的。然而，一旦你对数据库引进了在一个网络上分区的功能（通过主从设置），完美的ACID保证（至少是确保请求收到成功或失败的响应和数据的一致性方面）需要放松或从某一程度上重新定义。

有一个例子可以帮助你弄清楚这些事情。因此让我们假设你的数据库现在分布到（复制到）三个实例中，并且更新了其中一个（主机）的一个节点。对于从机将发生什么事情？当它们中的一个试图读这一数据时，它是否会马上读到这一更新的数据（是否所有的从机都能够在任何时间读到一致的数据）或是否会有任何时间数据会失效？如果有网络故障，你还能够通过任何的计算机，（包括从机）读和写吗？

为了这个讨论的目的，最初由Eric Brewer在2000年提出，并在12年后的回顾更新的CAP定理指出^[2]，当涉及分区时，不能在100%的时间中同时保证数据的一致性和确保请求收到成功或失败的响应——需要有一种权衡。

Neo4j是一个AP数据库，这意味着它的最终分区是不可用的（如果网络是极其分散的话，为了安全的原因，它只能是只读）。从机必须在经过一段时间推或者拉更新数据才能保证最终的数据一致性。这些值是可配置的，因此，数据不一致的窗口是可控制的，是由每一个应用程序或设置程序控制的。我们将在11.2.3节中讨论。

在我们更深入地探讨Neo4j内部某些HA特性是如何工作的更专业问题之前，先来讨论具有三台计算机集群的设置练习。这会帮助你锻炼动手能力，

从而能够设置并且能做一些设置选项的实验，就像我们在本章中讨论的这些设置选项一样。

从单机设置切换至集群设置

对当前使用已经存在的单机（嵌入式或服务器模式）的客户，切换至集群环境应该是一个相对简单并不会痛苦经历的事情。这可能是因为Neo4j HA的设计从开始就使这种转换尽可能的容易。这很明显需要涉及额外的配置以确保服务器能够将自身建立为集群成员（稍后将讨论），但是从客户端的角度，同服务器进行交互的可行操作和一般的方法应该大部分保持不变。

（1）嵌入式模式

对嵌入式模式的客户，如果还没有升级的话，Neo4j JAR依赖需要从共享（Community）版升级为企业（Enterprise）版。其后只需要修改GraphDatabaseService实例的建立。而不是使用GraphDatabaseFactory去创建GraphDatabaseService，应该使用HighlyAvailableGraphDatabaseFactory。由于这两个factories产生的实施使用同一个GraphDatabaseService接口，对每一个客户没有额外的变化需求。

下面的程序片段给出了实际中使用的代码，给出了加粗显示的HA特定变化。

```
graphDatabaseService = new HighlyAvailableGraphDatabaseFactory()  
    .newHighlyAvailableDatabaseBuilder("/machineA")  
    .loadPropertiesFromFile(machineAFilename)  
    .newGraphDatabase();
```

（2）服务器模式

当前没有运行企业版的单一服务器实例首先需要升级到企业版，然后将服务器配制成为一个合适的集群的一部分是一件很简单的事情。

11.2.2 设置Neo4j集群

让我们讨论一个具有三台计算机基于服务器集群的设置流程。Neo4j HA与以嵌入式模式实例工作类似。由于主要作为演示目的，我们将在同一台计算机上运行这三个实例；但是在实际中你将更可能在不同的计算机上运行不同的实例。



注意

这些指令是对*nix环境而言的（实际的例子是在OS X 10.9.2上做的），但是，对于Windows的设置可以仿照等效的过程。我们是在Neo4j 2.0.2版本上运行HA设置。

1.初始设置

按照这些步骤做初始设置：

1) 创建一个根目录，例如~/n4jia/clusterexample。

2) 下载Neo4j的企业版并解压zip文件（对Windows操作系统）或tar文件（对UNIX操作系统）到这一文件夹。对于Mac操作系统，这会有点像以下的样子：

```
~/n4jia/clusterexample/neo4j-enterprise-2.0.2
```

3) 重新命名这个文件夹为machine01，并且复制这个文件夹及其所有的内容到目录machine02和machine03。你的结果文件结构应该像这样：

```
~/n4jia/clusterexample/machine01
~/n4jia/clusterexample/machine02
~/n4jia/clusterexample/machine03
```

4) 在每一台计算机的根目录内，找到conf/neo4j-server.properties文件并确保以下的属性设置：

属性名	machine01	machine02	machine03
org.neo4j.server.webserver.port	7471	7472	7473
org.neo4j.server.webserver.https.port	7481	7482	7483
org.neo4j.server.database.mode	HA	HA	HA

5) 在每一台计算机的根目录内，找到conf/neo4j.properties文件并确保以下的属性设置：

属性名	machine01	machine02	machine03
ha.server_id	1	2	3
ha.server	127.0.0.1:6001	127.0.0.1:6002	127.0.0.1:6003
online_backup_server	127.0.0.1:6321	127.0.0.1:6322	127.0.0.1:6323
ha.cluster_server	127.0.0.1:5001	127.0.0.1:5002	127.0.0.1:5003
ha.initial_hosts	127.0.0.1:5001	127.0.0.1:5002	127.0.0.1:5003

2.启动与验证

按照以下步骤启动与验证你设置的服务器。

1) 在每一个目录中发出如下启动命令启动服务器：

```
~/n4jia/clusterexample/machine01)$ bin/neo4j start
~/n4jia/clusterexample/machine02)$ bin/neo4j start
~/n4jia/clusterexample/machine03)$ bin/neo4j start
```

2) 对每一个服务器用浏览器访问网页管理控制台（见附录A）一次，需要一定的时间启动以验证服务器正确启动并探测它提供的详细情况：

```

http://127.0.0.1:7471/webadmin/#/info/org.neo4j/High%20Availability/
http://127.0.0.1:7472/webadmin/#/info/org.neo4j/High%20Availability/
http://127.0.0.1:7473/webadmin/#/info/org.neo4j/High%20Availability/

```

图11-6和图11-7给出了machine01和machine02的网页管理页面。

The screenshot displays the Neo4j web administration interface. The browser address bar (1.1) shows the URL `127.0.0.1:7471/webadmin/#/info/org.neo4j/High%20Availability/`. The sidebar on the left contains a list of navigation links, including 'org.neo4j', 'Branched Store', 'Cache - NodeCache', 'Cache - RelationshipCache', 'Configuration', 'Diagnostics', 'High Availability', 'Kernel', 'Locking', 'Memory Mapping', 'Primitive count', 'Store file sizes', 'Transactions', 'XA Resources', 'com.sun.management', 'com.sun.management:type=HotSpotDiagnostic', 'java.lang', 'CMS Old Gen', 'CMS Perm Gen', 'Code Cache', 'CodeCacheManager', 'ConcurrentMarkSweep', 'Par Eden Space', 'Par Survivor Space', and 'ParNew'. The main content area is titled 'High Availability' and provides information about an instance participating in a HA cluster. The table below shows the cluster status:

High Availability	
Information about an instance participating in a HA cluster	
Alive Whether this instance is alive or not	true
LastUpdateTime The time when the data on this instance was last updated from the master	2014-06-14 04:03:23.070+0000
InstancesInCluster Information about all instances in this cluster	
org.neo4j.management.ClusterMemberInfo org.neo4j.management.ClusterMemberInfo	
alive alive	true
available available	true
----- screenshots merged above and below -----	
Role The role this instance has in the cluster	master (1.2)
InstanceId The identifier used to identify this server in the HA cluster	1 (1.2)
Available Whether this instance is available or not	true
LastCommittedTxId The latest transaction id present in this instance's store	3

Copyright © 2002-2014 Neo Technology. License: GPL v3 or AGPL for Open Source, and NTCU for Commercial. Icons by Glyphish & Tango.

图11-6 从machine01的视角看到的HA设置网页管理控制台的视图

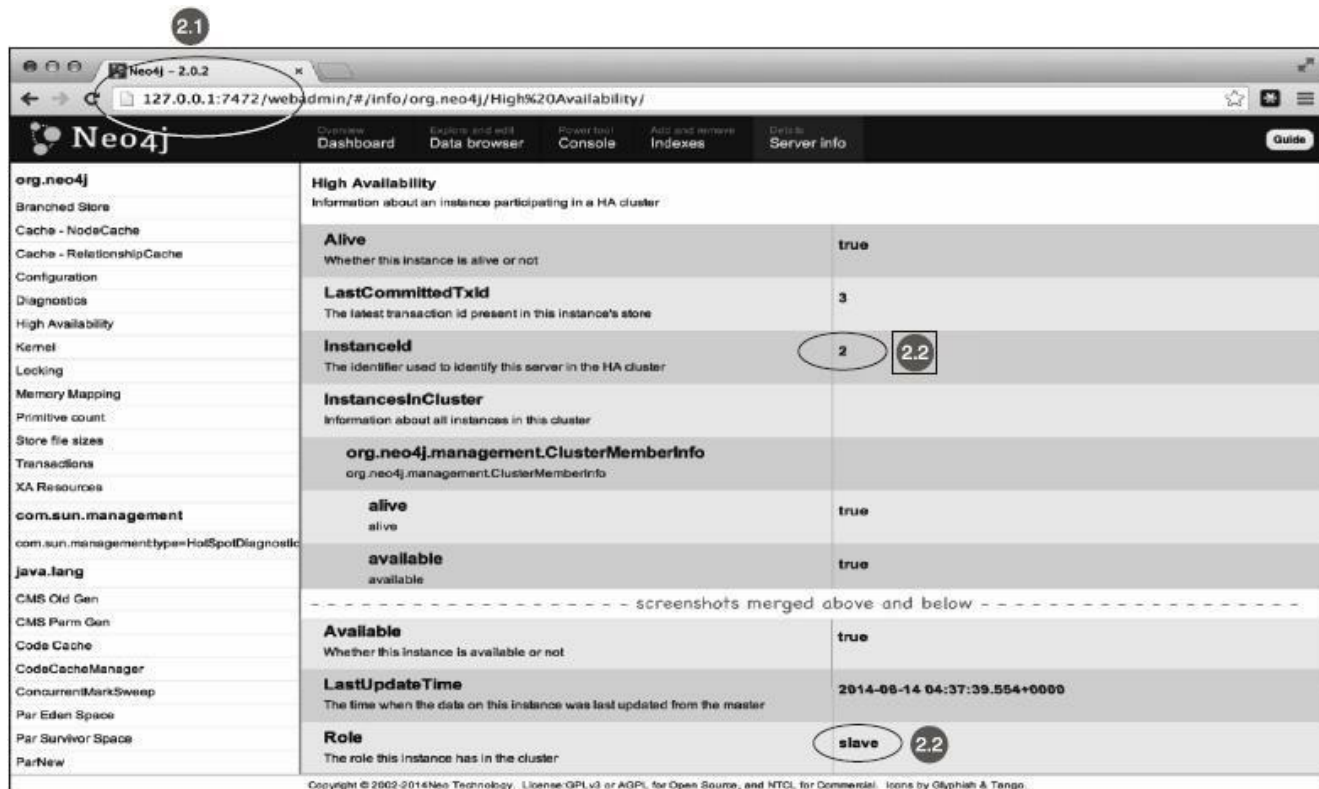


图11-7 从machine02的视角看到的HA设置网页管理控制台的视图

在这种特殊的启动场景中，machine01 (1.1) 首先启动，并且集群中还没有其他计算机启动，假设machine01是主机，就像你在图11-6中看到的 (1.2)。如果machine03已经首先启动，它将被假设为主机的角色。

网页管理控制台提供了一个关于HA集群的众多详细设置，包括集群中涉及的其他计算机。这些信息可以在InstancesInCluster (1.3) 中找到。这里显示的第一台计算机实际上是machine01，但是在页面的下面（图11-6中没给出），machine02和machine03也应该作为从计算机列出来，给出它们已经启动并且成功地加入了这个群。

在图11-7中，你可以看到machine02 (2.1) 在machine01启动后加入了这个集群，并已经参与了从计算机的角色 (2.2)。

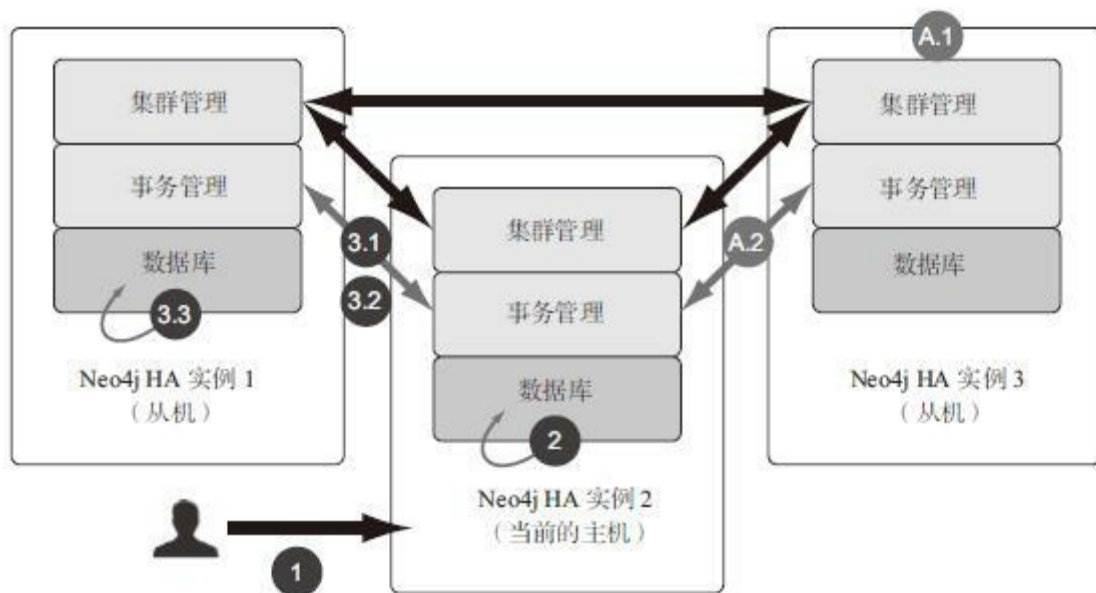
你现在应该已经设置好集群并且运行了。下面继续讨论一些HA的更详

细的特色。可以随意实验并试图做一些事情——这经常会是一种最好的学习方式！

11.2.3 复制——读和写的策略

与一般的所有写请求都通过主机而读请求通过从机的主-从复制设置不同，Neo4j能够处理来自于集群中任何实例的写请求。同样，Neo4j能够处理来自于任何主机、从机任何实例的读请求。

写请求的处理根据直接来自于主机还是通过从机而不同。图11-8给出了当一个写请求直接发送到主机的事件的处理流程。



客户发送写请求给主机

在后台

- 1 客户发出一个提交
- 2 主机获得锁定并且按照正常的对主机数据库实例的非事务规则应用到事务
- 3.1 主机参照 `ha.tx_push_factor` 设置并试图乐观的将变化推入到 `n` (默认的是 1) 个配置的从机——但是, 如果这个失败, (2) 仍然是成功的
- 3.2 从机确保首先数据是最新的, 如果不是, 从主机拉入所有的等待的事务
- 3.3 从机局域应用事务
- A.1 从机参照 `ha.pull_interval` 设置决定是否/什么时间从主机拉入更新 (默认是不定期拉入更新, 仅当在写入事务期间更新)
- A.2 当在间隔的范围内时, 任何的新事务都会被从主机拉入并且局域应用到从机

图11-8 当一个写请求直接发送到主机实例的事件处理流程

任何直接由主机收到的写请求将会以非常相同的方式作为一个标准的非HA事务处理；数据库需要一个锁定并执行一个局部事务。唯一的差别是在主机上提交以后，有一些配置属性（`ha.tx_push_factor`和`ha.tx_push_strategy`）额外地以乐观的方式把事务推入到`n`（默认为1）个从机。至于乐观的方式，是指主机尽最大的努力确保事物传送至配置的从机中去，但是如果这种传送不成功，不管是由于什么原因，主机上的原始事务仍不会失败。

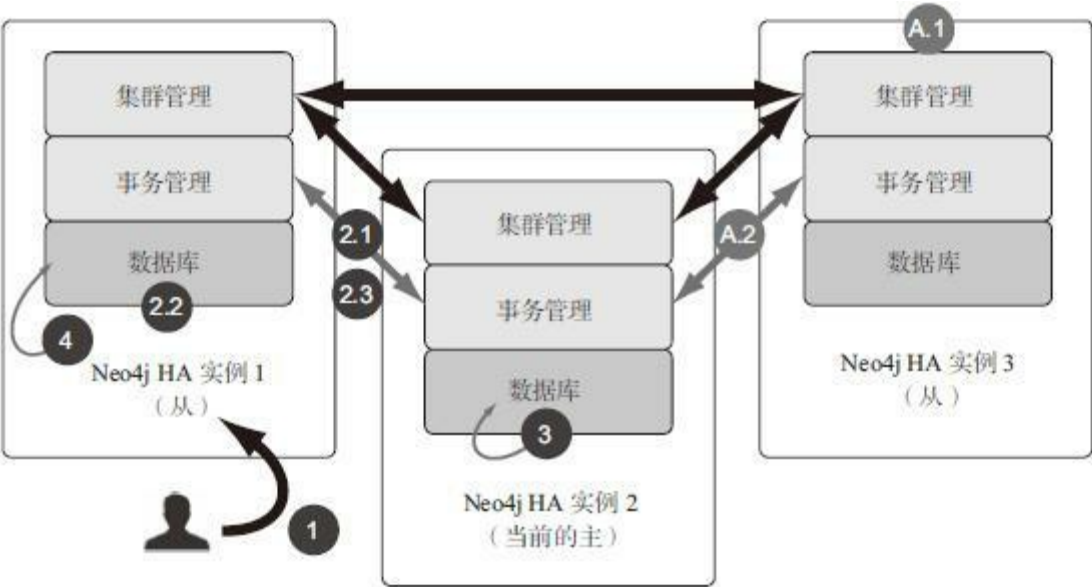
其他从机也能够周期性地从主机拉入做最新的更新。这是通过设置

ha.pull_interval属性为适当的值实现的，例如2s（每2秒）。这一推拉机制用于控制Neo4j在集群中最终具有一致性的速度。

 注意

这里乐观的使用并不意味着与传统数据库乐观锁概念相同，对于乐观锁，如果检测到数据库的冲突状态，整个事务（包括主机上的）将会回滚。

转到第二种情况，如果一个从机收到一个写请求，事件的不同顺序将会发生，如图11-9所示。



客户发送写请求给从机

- 1 客户发出一个提交
- 2.1 主机数据库获得锁
- 2.2 从机数据库获得锁
- 2.3 作为间隔协议的一部分，从机确保首先数据是最新的，如果不是，从主机拉入所有的等待的事务从机局域应用事务
- 3 更新提交到主机
- 4 假如主机的提交是成功的，局域从机事务也提交成功

在后台

- A.1 从机参照 ha.pull_interval 设置决定是否 / 什么时间从主机拉入更新 (默认是不定期拉入更新，仅当在写入事务期间更新)
- A.2 当在间隔的范围内时，任何的新事务都会被从主机拉入并且局域应用到从机

图11-9 当一个写请求发送到从机实例的事件处理流程

指向从机的写入依赖于可用的主机以便更新的实现。这是因为从机在继续执行前将会同时需要在主机和从机获得锁。Neo4j具有可靠的、内置的逻辑自动探测什么时间什么原因使实例（包括主机）潜在消失或不能访问。在这样的情况下，一个新的主机会被选出，并且如果需要，尽可能多的达到数据最新。如果无法选出一个新的主机，不管什么原因，这会在本质上停止所有对数据库的更新。

更新总是首先应用于主机，并且仅仅当主机更新成功后，更新才应用于从机。要确保全部的一致性，Neo4j需要从机在执行任何写操作前保持最新的数据，因此，作为内部交流协议的一部分，Neo4j将会确保从机在执行任何局域写操作之前将所有的最新更新应用到自身。

监视集群中更新的传送

为了观察集群中推拉更新的发生，可以使用不同的Neo4j HA设置，然后结合使用Neo4j网页管理控制台，创建节点和关系，并且监视它们在集群中的复制。

按照以下步骤尝试：

1) 确保所有服务器启动并且如11.2.2节中描述的那样运行。

2) 验证每一个服务器没有要开始的节点。这可以通过控制台选项卡发出以下的Cypher查询（它应该返回几行0）实现：

```
start n=node(*) return n;
```

3) 在主服务器上（在这一实例中是machine01）使用网页管理控制台使用以下Cypher声明创建两个节点和一个关系：

```
CREATE (n1) -[:LOVES] -> (n2);
```

4) 等到拉的时间间隔通过（默认是10秒，参见neo4j.properties文件中的ha.pull_interval设置），然后通过每一服务器上再次运行步骤2中的查询，验证这些值也存在于从机的实例中。现在你应该看到在每一个服务器上列出两个节点。

这就是实战中Neo4j的数据传送机制。

通过从机写入或不通过从机写入，这是一个问题！

能够发送写请求到主机或到一个或多个从机给部署选项提供了很大的灵活性，但是，如何选择需要慎重考虑。

- 通过一个从机写入要比通过一个专门的主机写入慢得多。当通过从机写入时，由于主机和从机的同步和数据活动的协调需求的额外网络流量，将会有一个很高的延迟。

- 通过一个从机写入能够增加持久性，不过这也能够通过配置主机一旦提交就复制变化实现。通过一个从机写入，可以确保总是至少有两个Neo4j实例保持最新的数据（从机本身，以及主机）。另外，或者除这个之外，能够通过配置一旦提交，写入到主机后复制出到指定的几个从机增加持久性。尽管这会发生在一个最优基础上的（通过ha.tx_push_factor属性），最后的结果是你的数据将会到你配置数量的电脑上。分布的持久性是重要的，由于这提供了更大程度上的自信，你的数据完整的保存到了多个地方，但是这也付

出了代价。

·需要某些种类的局域平衡器来保证写仅仅的、或主要的由主机处理。

Neo4j没有提供内置的加载平衡功能，而是依赖一个外部的某些种类的机制来提供这种功能。对客户端需求的这种功能，HAProxy (www.haproxy.org) 是一个流行的选择。什么时间从从机路由到主机的有关逻辑需要内置的加载平衡机制，这可能是一个棘手的问题。怎样才能知道什么是一个读请求或写请求？基于纯流量通常没有100%的准确方法来确定这个。这常常导致这样的情况，要求应用程序本身具有一些判断它的动作是读还是写的知识，以帮助对请求的合适路由。

你是通过主机还是从机写入不必全有或者全无决策。如果你的系统合理的话，也可以使用一个混合的方法（大多数情况是写入主机，读自从机）。要按实际情况做这种决策。现在提供给大多数客户的一般推荐是确保写入主要发送到主机，在可能的情况下，才使用`ha.tx_push_*`设置把数据推送至从机。这趋向于根据灵活性和性能提供最好的权衡。

11.2.4 缓存分区

有一些这样的实例，图形数据库太大以至于不能放入可用的内存中。现在的建议总是瞄准一个理想情况，但是，如果就是不可能放入怎么办？

在这种情况下，推荐考虑使用一种称为缓存分区的技术。缓存分区并不是传统的分区。

传统的分区（像其他数据存储采用的分区，如MongoDB）通常涉及保

证数据不同的部分存到不同的实例，常常是不同的物理服务器。如果你有一个大型的客户数据库，可以选择按客户组分区。例如，所有具有用户名A至D的用户存到一个实例中，E至F在另一个实例中，其他的再细分并存到其他区中。应用程序应该需要知道哪一个实例具有被查找的数据并确保到正确的实例中去得到数据。当然，许多激活分区的数据库提供自动的分区和访问合适的实例的方式，但是这里的重点是并不是所有的数据都存在每一个实例或服务器，而只是一个数据的子集存在每一个实例或服务器中。

对于Neo4j，每一个HA实例期待具有所有数据集的访问。缓存分区，不像传统分区，这是一种物理数据分区技术，是一种更像基于路由的模式。使用这种模式，作为一个集合单位通常访问的数据请求总是指向同一个Neo4j实例。通过一致性的路由这些请求到同一个实例，你增加了这些数据在访问时被放到内存中的可能性，从而允许这一实例使用Neo4j为这种情况提供的内存访问性能。

让我们再一次看看前面的客户数据库例子。对于Neo4j，所有的数据（从A至Z的客户）出现在每一个HA实例，但是通过使用某些种类的加载平衡机制，你能够确保所有与一组客户（具有用户名A的客户）相关的局域请求将总是路由到同一个HA实例，“B”到另一个，依此类推。图11-10给出了缓存分区对这种图形数据库的可能工作方式。

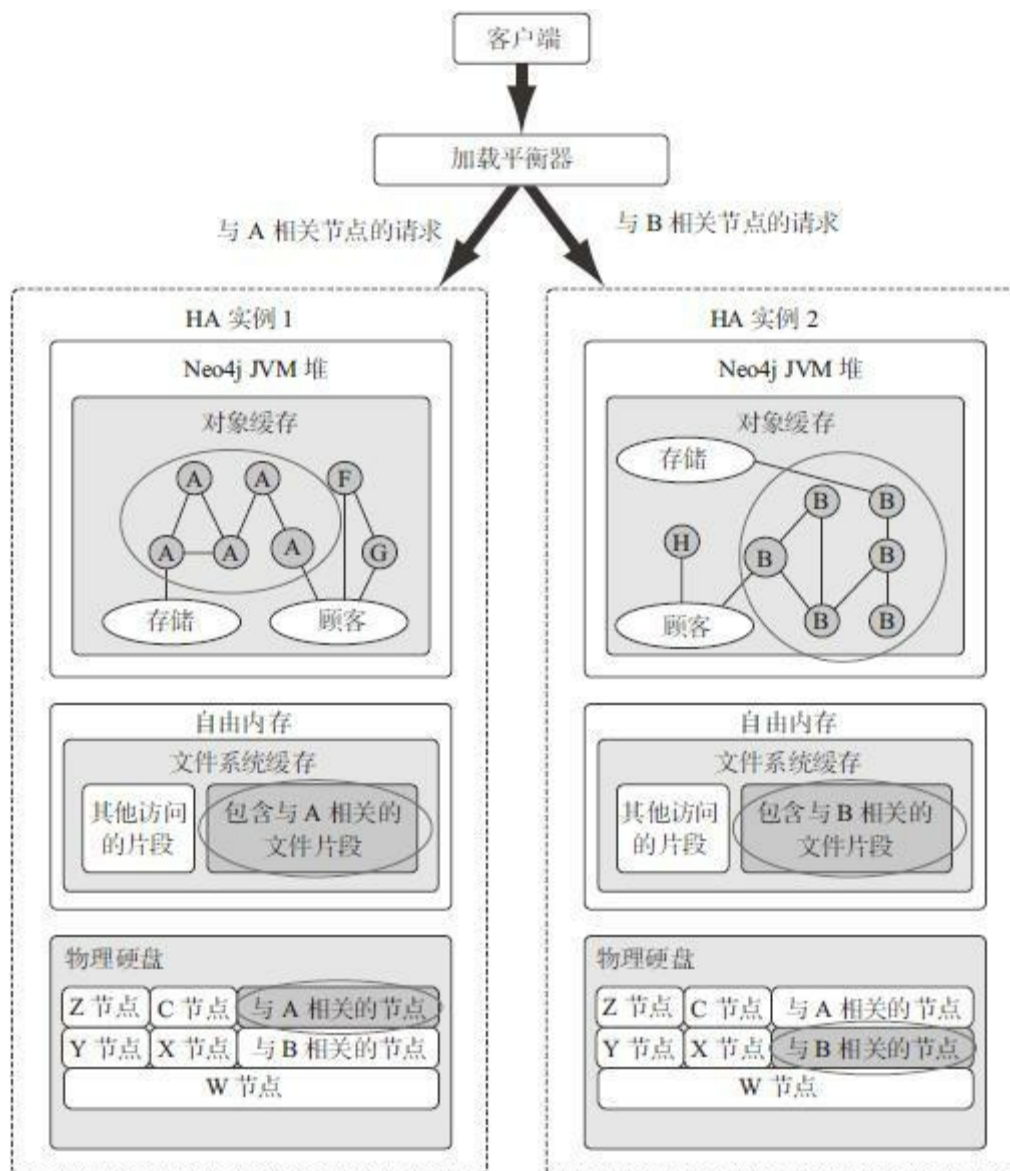


图11-10 缓存分区

为什么Neo4j不做传统的分区

简单地说，对图形数据库这是一个非常非常难以解决的问题。事实上，如此的难是这个问题属于一类“确实很难的数学问题”：“非确定性多项式时间难题”（<http://en.wikipedia.org/wiki/NP-hard>）。

传统的分区一般涉及分裂一个整的数据集使得不同的部分存到不同的实例中。假设某些条件为真，这种方式是一种扩展大型数据库的方法，而同时随着数据的增长保持一个可预测的性能水平。

前面叙述的关键点是“假设某些条件为真”。在大多数的激活分区的数据库中，关键是哪些数据在分区中是稳定和可预测的。另外，通常使用这样的访问模式，你可以在一个分区中查找你需要的数据（用一个关键字），也可以在另一个分区中查找，然后使应用程序在需要的时候将它们一起返回。

然而，图形数据库，尤其是Neo4j的最大优点在于局域的图形查询，即从给定的点开始并探索与它紧挨着周围的数据的连接模式。这可能涉及遍历非常不同的数据类型，这些数据类型会分布到不同的分区。假设你有一个这样的查询“对于顾客A，告诉我哪一个商店离她住的地方在1km以内，并且她的直接朋友在上一个月中在那个商店购物超过100英镑”。如果你决定分区数据使用户和朋友存到一个分区，零售商店在另一个单独的分区，购买历史在另一个分区，随着数据分散到不同的物理服务器，一个内置的图形查询将会结束，由于网络的交叉得到导航，会导致不可预测的和慢的数据检索时间。

你可能认为可以尝试基于我们描述的模式分区数据，这可能行也可能不行。然而，这可能潜在的否定你想要执行的其他遍历——在更广的数据集上从其他节点开始，而不是特定顾客A，并且从那里开始探索。

直到我们能够穿越网络边界以一种可接受和可预测的方式执行如此的遍历时，在此期间涉及Neo4j的这种情况的最好解决方案是使用缓存分区。Neo4j的分区仍然是一个正在探讨的问题，但是在近期还没有迹象显示能够解决这个问题。

路由策略

正如在11.2.3节中学到的，就客户端而言，HA设置中没有建立路由和加载平衡功能；Neo4j期待客户用像HAProxy（www.haproxy.org）的工具提供

或实施他们自己的路由和加载平衡功能，使用HAProxy是一个流行的选择。忽略物理机制的使用，每一个应用程序会决定数据路由的最合适策略。

这些选项包括：

- 基于一些数据特性的路由请求** 这是在我们的客户例子中使用的方法。
- 使用粘性会话** 这涉及确保来自于每一个客户端的请求总会到同一个服务器，不管请求什么。这可能合适也可能不合适，取决于不同的情况。
- 使用基于地理的路由** 这涉及确保所有与一个共同的地理位置相关或起源的请求总是路由到同一个实例。

11.2.5 HA小结

由于本书讨论了许多主题，我们仅仅能够涵盖实际可用的很小一部分，HA也不例外。对于一般的HA，参见Neo4j手册（第23章）的“高可用性”：<http://docs.neo4j.org/chunked/stable/ha.html>。

现在你已经学习了如何扩展和确保Neo4j能够继续操作并为客户端服务，即使在硬件失效时。现在我们将移到本章的最后一节，涉及数据的备份和当灾难确实发生时数据库的恢复。

[1] CAP，Consistency指数据的一致性，Availability是确保请求收到成功或失败的响应、Partition tolerance指由于网络的故障系统继续运行。——译者注

[2] Eric Brewer，《CAP十二年后：‘规则’是如何变化的》，

www.infoq.com/articles/captwelve-years-later-how-the-rules-have-changed。

11.3 备份

你的社交网站一直运行得很好，为你带来了很大的收入和声誉。然而，突然你的管理员通知你有一个硬件损坏，当前的数据丢失。

在这时，你的心脏要么咯噔一下并且你会说，“我真的应该想到把数据备份起来啊”，或者你会坐下并且想，“不必担心，我已经备份了”。

假设你选择了后一种，本节的目的是提供所有必需的信息以确保如果当这种灾难确实发生时，你已经有了准备，这并不意味着是世界的末日。

我们将通过查看做一个简单的离线备份所涉及的内容开始，一个涉及停机时间的流程，但是在任何版本的Neo4j数据库都能执行的流程，包括免费的社区版。对于那些你无法经受得起离线的情况，在线备份功能就会发挥作用，但是，这只在Neo4j的企业版可用。

11.3.1 离线备份

在一天结束时，包含Neo4j数据库的文件都在文件系统的一个目录中，这是最终需要备份的文件。就像在本节的介绍中叙述的，这一流程将在所有Neo4j数据库的版本中都有效，但是最可能仅应用于社区版（如果你是用的企业版，你很可能会使用在线备份功能）。

流程的本身相当简单，涉及以下步骤：

- 1) 关闭Neo4j实例。

2) 复制实际的Neo4j数据库文件到一个备份的位置。

3) 重新开始Neo4j实例。

让我们依次看看这些步骤：

1.关闭Neo4j实例

如果运行单一的服务器模式，你只需要用stop参数执行neo4j脚本。假设Neo4j是安装在/opt/neo4j/bin中，你可以通过发出以下命令停止服务器的运行：

```
/opt/neo4j/bin/neo4j stop
```

如果你在应用程序中运行的是嵌入式模式，那么除非你是显式提供了一个只关闭Neo4j数据库的机制，你应该用你选择的任何机制关闭你的应用程序。这就像发送一个SIGINT信号（例如，如果你的应用程序是以这种方式开始的，在一个终端窗口发出Ctrl-C）一样简单，或者可能更复杂一些，涉及一个专用的脚本语言或自定义的应用程序功能。

不管什么机制，当这一流程发生时，你应该确保Neo4j也关闭。在第10章的程序10-3中你会看到当以嵌入式模式运行时，需要什么确保这一情况的发生。要刷新你的内存，这里是注册一个关机点的代码片段：


```

private String DB_PATH = '/var/data/neo4jdb-private';
private GraphDatabaseService graphdb =
    new GraphDatabaseFactory().newEmbeddedDatabase( DB_PATH );
registerShutdownHook( graphDb );
...
private static void registerShutdownHook(
    final GraphDatabaseService graphDb )
{
    Runtime.getRuntime().addShutdownHook( new Thread()
    {
        @Override
        public void run()
        {
            graphDb.shutdown();
        }
    } );
}

```

当你的应用程序退出后确保Neo4j完全的关闭是很重要的；没有这样做将导致下一次启动时出现问题。当使用Neo4j服务器模式时，你不必担心是否是完全的关闭——服务器模式将会为你做好。

2.复制物理数据库文件

定位包含Neo4j物理数据库文件的目录。对于运行嵌入式模式的应用程序，这将是提供启动代码的路径。要刷新你的内存，以下的程序片段给出了启动嵌入式模式数据库需要的代码，这里的物理文件存在/var/data/neo4jdb目录中：

```

private GraphDatabaseService graphdb =
    new GraphDatabaseFactory().newEmbeddedDatabase("/var/data/neo4jdb" )

```

如果你是以服务器模式工作的，数据库文件将会位于neo4jserver.properties文件中以org.neo4j.server.database.location关键字指定的位置。这一设置将会有一点像以下的样子：

```

org.neo4j.server.database.location=/var/data/neo4jdb

```

复制这一目录和所有的子文件夹及其所有的文件到一个分离的备份位置。

3.重新启动数据库

对于Neo4j服务器模式，使用适当的Neo4j启动命令，并且对嵌入式应用程序，在合适的时间启动应用程序。



注意

Neo4j能够检测非完全的关闭并且试图恢复，但是这将导致恢复过程时更慢的启动，并且有时也会导致其他的一些问题。当一个非完全的关闭发生后，以下的警告信息通常会在下一次数据库重新启动时在日志中看到——`non clean shutdown detected`。你应该确认你的关闭流程，不管是强制的还是自愿的，尽最大的努力确保Neo4j完全的关闭。

11.3.2 在线备份

在线备份功能仅在Neo4j的企业版中可用，但是这是一个强大和可靠的备份你的单机或集群Neo4j环境而不需要任何停机的方式。

对在线备份你有两个选项：完全备份或增量备份。在两种情况下，对正在备份的Neo4j实例资源不需要锁定，允许它们在备份进行中继续发挥其功能。

1.完全备份

一个完全备份实质上涉及复制所有的核心数据库文件到一个分离的备份

目录。由于正在备份的数据库并没有锁定，很可能当备份开始后，一个事务正在运行。为了确保最后的备份保持一致性并数据最新，Neo4j将会确保这一事务和任何其他的在复制期间发生的，复制到最后的备份文件中。Neo4j通过记录当备份开始后任何进程中的事务编号实现上述目标。当复制核心文件结束后，然后备份工具会使用事务日志（在11.1.5节中讲过）重新做从记录的事务和包括的在复制期间发生的所有事务。在这一过程的最后，你将会有一个完全的备份和一致性的Neo4j实例。

2.增量备份

与完全备份不同，增量备份不复制所有的核心存储文件作为备份过程的一部分。相反，它假设至少已经做过一次完全备份，这次备份作为最初开始的点识别接下来发生的任何变化。第一次增量备份过程将仅通过从最初的完全备份发生的事务日志复制，因此使备份数据存储到一个新的上次标记的备份点。这一点是标记的，并且下一次增量备份将会使用这一点作为新的备份起始点，仅通过从最近一次增量备份点到当前的点发生的事务日志复制。这些日志在备份存储中被重新执行，使其成为最近一次的标记备份点。在此后的每次增量备份请求中都重复这一过程。增量备份是一个更高效的备份过程，由于它尽可能地最小化了通过线路传送的数据量以及实际做备份所花费的时间。

3.做备份的过程

无论是完全备份还是增量备份，激活和备份的过程是相同的。图11-11给出了一个单个服务器设置的备份场景的例子。

每当你的Neo4j实例开始时（嵌入式或服务器模式，单个或HA），它们

检查它们的online_backup_enabled配置参数是否设置为true。如果是true，一个分离的备份服务（来自于Neo4j发布时的一部分）也就开始了。默认的端口对单一服务器模式设置是6362，对HA设置是5001。这一服务为Neo4j备份工具的连接提供了访问点，以读取底层的存储和事务日志文件做备份。

Neo4j备份工具仅仅是一个能理解如何连接一个或多个Neo4j备份服务的面向Java应用的UNIX或Windows脚本语言（可以在bin文件夹中找到）。备份工具可以从网络上的任何计算机上运行，前提是这台计算机具有连接到备份服务的能力。UNIX系统的备份命令具有以下格式：

```
./neo4j-backup -from <source-uri> -to <backup-dir>
```

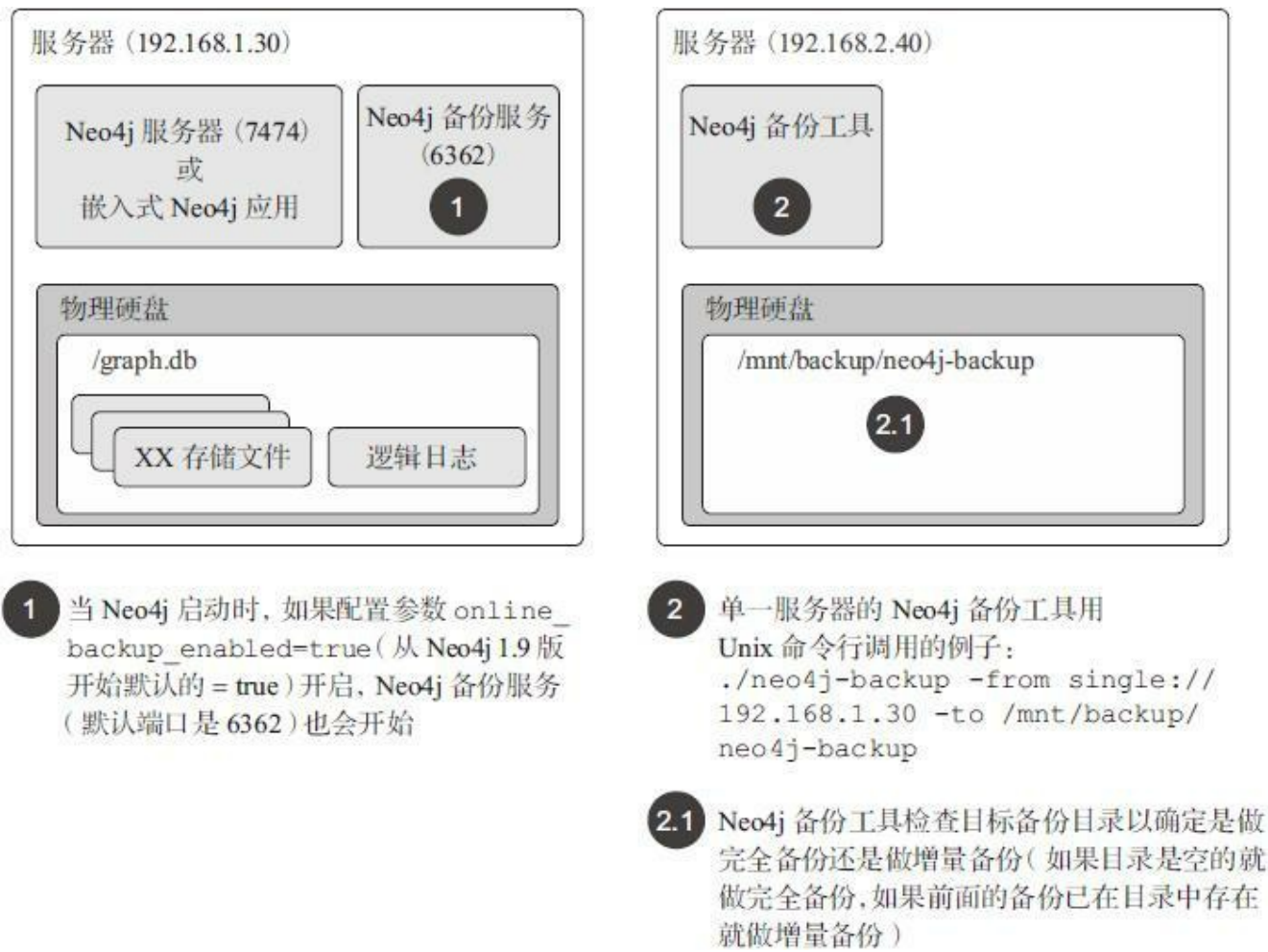


图11-11 单一服务器设置的备份场景例子

对Windows系统是这样的格式：

```
neo4j-backup -from <source-uri> -to <backup-dir>
```

参数如下：

·**backup-dir** 备份存储的目录。为了做完全备份，这个目录必须是空的。如果备份工具检测到一个早期版本的数据库在那里，它将试图做增量备份。

·**source-uri** 这给备份工具提供了需要连接到适当的备份服务的所有信息，并且需要以下面的格式指定（详细情况来自于人们的文档，可以在这里找到：<https://github.com/neo4j/neo4j/blob/2.0-maint/enterprise/backup/src/docs/man/neo4j-backup.1.asciidoc>）。

```
<running-mode>://<host>[:port]{,<host>[:port]*}

running-mode:
    'single' or 'ha'. 'ha' is for instances in High
    Availability mode, 'single' is for standalone databases.

host:
    In single mode, the host of a source database; in ha mode,
    the cluster address of a cluster member. Note that multiple
    hosts can be given when using High Availability mode.

port:
    In single mode, the port of a source database backup service;
    in ha mode, the port of a cluster instance. If not given, the
    default value 6362 will be used for single mode, 5001 for HA
```

如何安排备份？

Neo4j本身不提供任何日程功能。它期待你会使用一个外部的日程工具（如cron）来做这项工作。使用像cron类的东西，你可以设置每小时做一次增量备份，或者任何适合你的频次。

11.3.3 从备份中还原数据

从备份中还原Neo4j数据库，只需要按照下面的步骤：

- 1) 关闭要还原的指定Neo4j实例。

- 2) 删除包含旧的或损坏了的版本的Neo4j数据库并用备份版本的数据库替换它。

- 3) 重新启动Neo4j实例。

11.4 本书可能没有涵盖但你可能会关注的问题

安全与监控是两个我们没有涉及的主题，但是如果你在这些领域具有特殊的需要的话，这可能是值得研究的两个主题。

11.4.1 安全

确保能在生产环境中安全运行Neo4j应该是你的配置中的一个重要组成部分。取决于你是否正在使用Neo4j，有一些不同的方面需要考虑。

对于那些使用嵌入式模式的用户来说，没有提供随手就用的现成安全选项，Neo4j没有提供任何数据级的安全或加密措施。大多数随手就用的选项（以及一些辅助安全配置设置和方法）是面向使用单机模式Neo4j（Neo4j服务器模式）的。

官方的文档提供了对要查找的东西相对较好的涵盖，以及一些如何开始保护你的Neo4j系统的基本说明。更多的信息可以看Neo4j手册中的“对Neo4j服务器的安全访问”一节：<http://docs.neo4j.org/chunked/stable/security-server.html>。

11.4.2 监控

能够监控你的Neo4j系统的运行情况并能知道什么时间会遇到麻烦是一种确保系统稳定和长时间运行的好的方式。大多数Neo4j监控特性仅仅在Neo4j的企业版才有，并且一般是通过Java管理扩展（JMX）技术实现的

(<http://docs.oracle.com/javase/7/docs/technotes/guides/jmx/index.html>) 。

更多的关于什么是可用的和如何获得这些信息的内容，可以在Neo4j手册的第26章找到：<http://docs.neo4j.org/chunked/stable/operations-monitoring.html>。

11.5 本章小结

恭喜！你已经到达了本书Neo4j旅程的终点。在这最后的一章，你参与了一个通往高水平Neo4j结构的旅程，不时地潜入深处，获得一些有价值的深层理解和一些Neo4j内部的为什么，特别是在配置Neo4j的背景下在一个生产环境中最优运行Neo4j。

此外，你获得了粗略计算系统的硬盘和内存需求量的洞察内部参数能力。你也学会了有关Neo4j涉及文件系统缓存和对象缓存的两层缓存策略和如何使这些配置适合你的需要。

由于这是本书的最后一章，我们回顾了一些在本书的前面几章中遇到的基本概念。在对基本的Neo4j结构有一个坚实的理解基础上，你的最后任务涉及学习如何做关键的配置Neo4j的工作，以便以集群的高可用性设置运行Neo4j，还有如何备份和恢复数据库。

11.6 最后的设想

尽管本章标志着本书的结束，但是我们希望这标志着你的Neo4j生涯的开始。Neo4j具有太多的东西，我们无法全部涵盖。我们才只接触了一点皮毛的东西。我们确实希望我们已经提供了足够的基础知识和参考以帮助你上路，以便开始探索所有的Neo4j功能。我们很高兴编写本书并且愿意与你共享我们的经历，我们愿意听到你以新的和有趣的方式使用Neo4j的经历。

附录A 安装Neo4j服务器

这一附录中的详细说明是面向从压缩文件tar.gz（Mac/UNIX）或zip文件（Windows）安装Neo4j服务器的。

Windows用户足够幸运地成为二进制安装包的第一批接收者，而对于Mac/UNIX用户，二进制安装包会在后面的适当时候推出。如果你愿意使用二进制的Windows安装包，请按照Neo4j网站<http://neo4j.org/download>上的说明进行。

Neo4j的版本和正式文件引用

本书是针对Neo4j 2.0的；除非在特定的讨论中指定不同的版本，Neo4j 2.0是所有的代码运行和验证的版本。正像你能够体会到的，在产品的不断更新中想写一本保持最新版本的书具有很大的挑战性。假如没有非常大的变化，你应该能够替换成最新版的Neo4j 2.0.1。

为确保总能连接到最新的信息，我们一直在努力使用指向最新稳定版本的官方参考文件。在本书写作时，所有的Neo4j文件URL遵循一个共同的模式，即<http://docs.neo4j.org/chunked/<版本>>，这里的版本是你希望阅读文件的Neo4j版本，运行稳定常常是指最新发布的最稳定版本的别名。

对2.0.1的专门信息，请参阅<http://docs.neo4j.org/chunked/2.0.1>。对最新的稳定版本，请参阅<http://docs.neo4j.org/chunked/stable>。

A.1 安装和配置单一Neo4j服务器

按照这些步骤安装和配置单一Neo4j服务器：

1) 确保你已经安装了最新的Oracle 7 JDK，如果没有，可以在www.oracle.com/technetwork/java/javase/downloads/jdk7-downloads-1880260.html下载并安装。

2) 定位和下载合适版本的Neo4j安装文件（如前面所说的，本书的所有实例代码都是在2.0.1版本里运行并验证的，并且这里的所有说明是针对这一版本，随意尝试使用最新的版本，前提是假设没有引进重大的变化）。从<http://neo4j.com/download/other-releases/>选择适合Windows或Linux/Mac的文件：

–Linux/Mac: neo4j-community-2.0.1-unix.tar.gz

–Windows: neo4j-community-2.0.1-windows.zip

3) 解压缩文件到你的硬盘的Neo4j主目录位置（NEO4J_HOME），我们前面已经提到该目录。例如，一个像这样形式的目录：/users/xxxx/devstuff/neo4j-community-2.0.1。

对Linux/Mac，使用以下命令：

```
tar zxvf neo4j-community-2.0.1-unix.tar.gz
```

对Windows，使用一款合适的解压缩工具软件，例如7-zip或winzip。

4) 如果需要，修改控制Neo4j如何启动的配置文件。Neo4j服务器允许你通过位于conf目录中的两个文件配置Neo4j工作方式的许多方面。

–conf/neo4j-server.properties

`-conf/neo4j.properties`

要改变核心数据库文件的位置（默认是在data/graph.db目录中），如下更新neo4j-server.properties文件的org.neo4j.server.database.location属性。

```
org.neo4j.server.database.location=  
/users/xxxx/devstuff/neo4j-in-action/code-samples/chapter10/  
performance-demos-server/the-chapter10-server-db
```

关于选项的更多内容，可以参考Neo4j手册的第22

章：<http://docs.neo4j.org/chunked/stable/server-configuration.html>。

5) 在你的NEO4J_HOME目录中打开一个终端（Mac/Linux）或一个命令提示符（Windows），通过执行具有start命令的neo4j脚本语言启动数据库。

–Linux/Mac: `bin/neo4j start`

–Windows: `bin\neo4j.bat start`

6) 通过访问<http://localhost:7474>并检查Neo4j浏览器的窗口验证数据库的启动（参阅A.2节）。

7) 要关闭服务器，执行具有stop命令的neo4j脚本：

–Linux/Mac: `bin/neo4j stop`

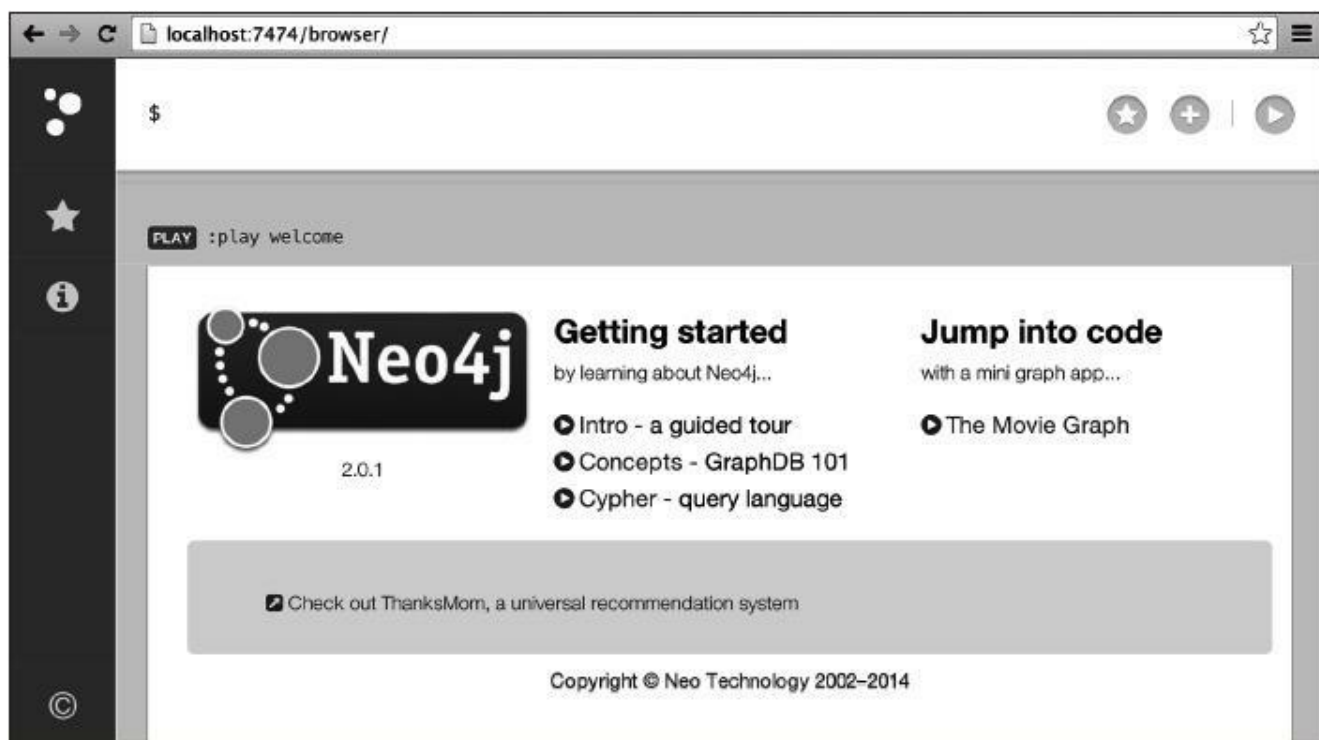
–Windows: `bin\neo4j.bat stop`

官方的安装过程也可以在Neo4j手册的第21章中找

到：<http://docs.neo4j.org/chunked/stable/serverinstallation.html>。

A.2 Neo4j浏览器

Neo4j服务器具有一个集成整洁的浏览器。在一个运行的服务器实例上访问[http://localhost: 7474/browser](http://localhost:7474/browser)。你遇到的第一个屏幕将会是一个启动页面，如图A-1所示。



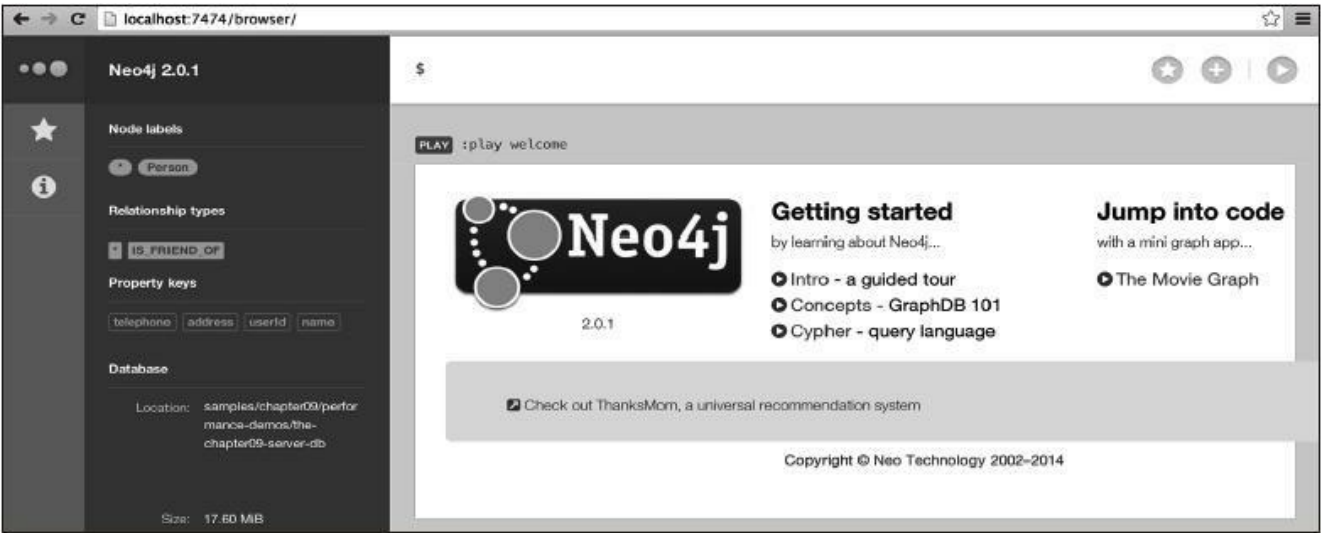
图A-1 Neo4j浏览器启动页面

你可以单击Intro按钮以得到一个浏览器的指导教程。然而，它的核心是浏览器允许你运行特定的查询并且很容易地以表格或图形的方式可视化查询的结果。

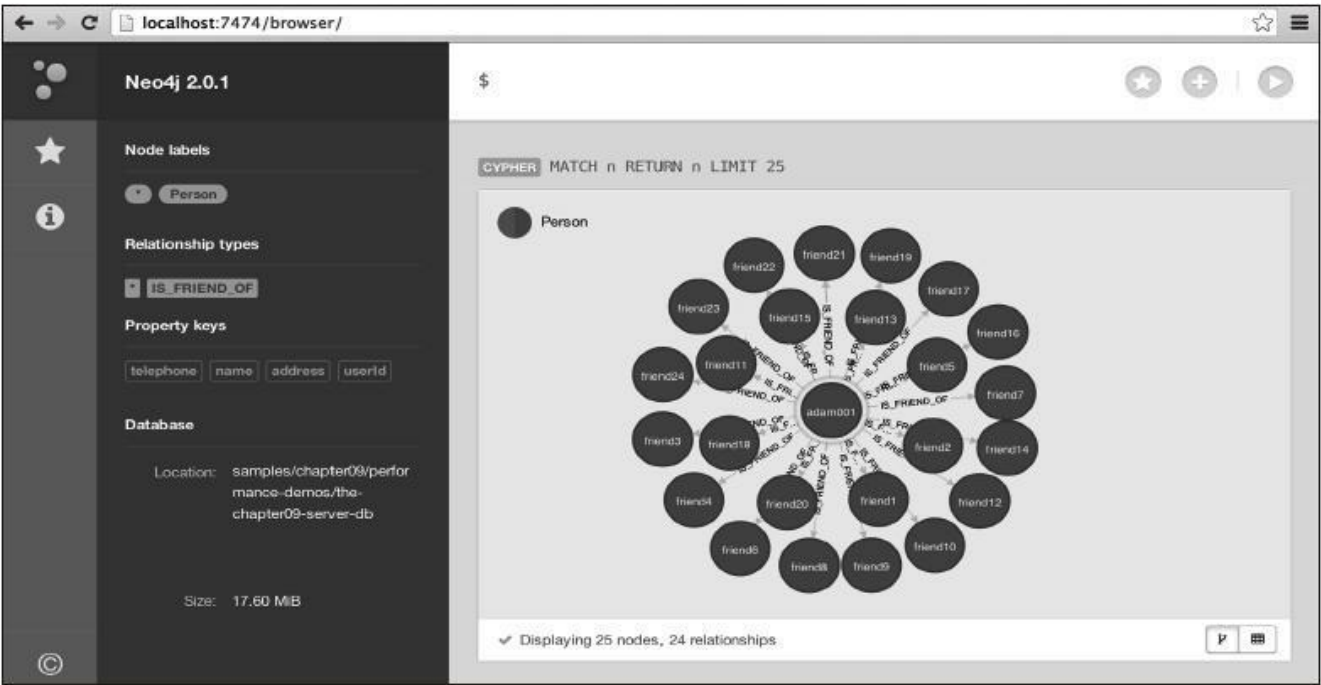
如果你单击页面左边的Neo4j与三个点的图标，你将会得到侧边栏给出一个系统中的主标签和关系的原始视图。你可以从这里开始导航，或者在页面的顶部输入一个特定的Cypher查询。图A-2描绘了第10章的一些样本数据库可用的选项。

单击节点标签下面的星号（*）执行默认的Cypher查询MATCH n RETURN n LIMIT 25，这会让你可视化图形数据库中开始的25个节点和它们的关系，如图A-3所示。

图A-3给出了第10章的样本演示数据的前25个节点。



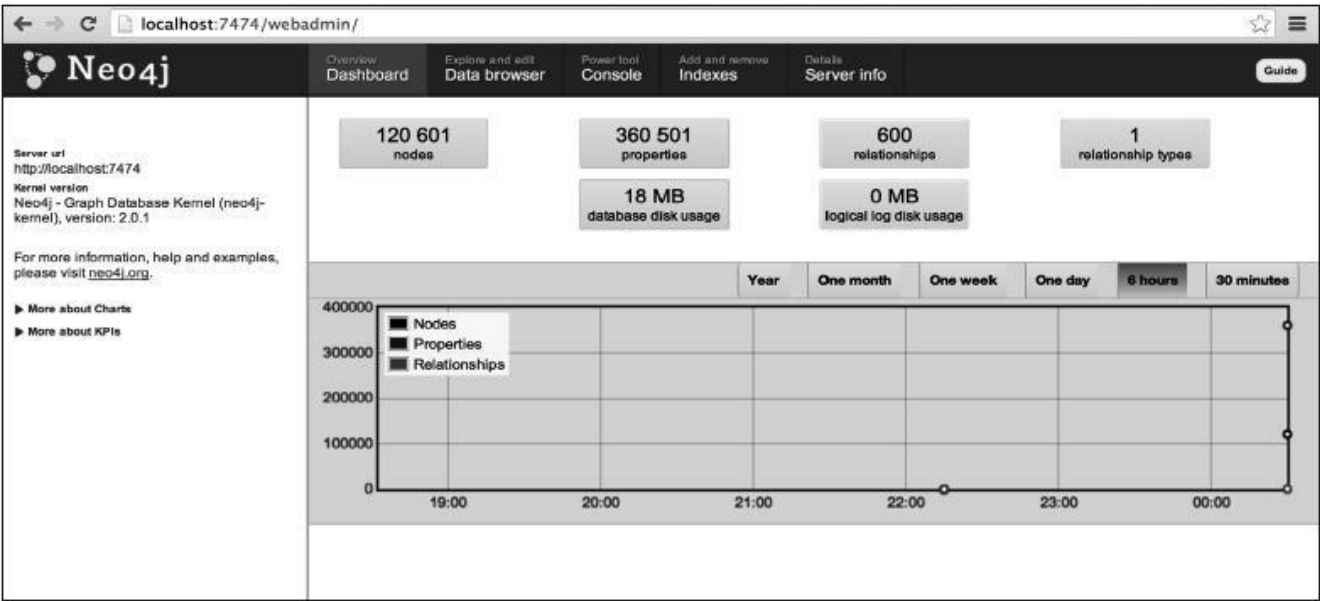
图A-2 具有扩展侧边栏的Neo4j浏览器启动页面



图A-3 Neo4j浏览器中系统节点的图形可视化

A.3 Neo4j网页管理控制台

Neo4j从2.0往后具有了Neo4j浏览器，并且它以图形用户界面（GUI）继续发展。Neo4j 2.0版本以前，用户需要使用网页管理控制台。网页管理控制台仍然可用，对于那些希望访问它的用户，可以从<http://localhost:7474/webadmin>获得，图A-4给出了网页管理控制台的界面。



图A-4 Neo4j网页管理控制台

附录B 设置和运行示例程序

本书使用的所有源代码都可以在www.manning.com/Neo4jinAction以及<https://github.com/opencredo/neo4j-in-action>下载。

所有的代码和演示都是使用Java，因此，你需要安装合适版本的Java。另外，我们使用Maven作为主要的框架提供代码的自动建立、编译和运行。

本书主要采用基于JUnit测试来演示特性，但是也有一些单机版本的类（这没有作为测试包的一部分参与运行），这些也可以执行以查看它们的运行结果。

本附录给出了设置系统运行这些例子和相关源代码需要的说明。

B.1 设置你的环境

本节描述如何准备你的电脑运行相关的例子和演示代码。

1. 下载样本代码

下载和解压样本代码到我们提及的目录，如N4JIA_SAMPLE_HOME。源代码以zip格式提供，并且能从www.manning.com/Neo4jinAction下载。另外，你也可以简单在在<https://github.com/opencredo/neo4j-in-action>复制github库。

2. 安装JDK（Oracle SE 7）

Neo4j需要Java 7作为最低要求的JDK（Java Development Kit，Java开发包）运行，并且已经针对Java 7做过测试。对那些想尝试Java 8的用户，尽管在理论上你可以这样做，但在现阶段还不能确保没有问题，因此我们还是推荐你现在坚持用Java 7，直到官方宣布支持Java 8后再用。Maven也是基于Java的工具，并且需要安装一个特定的JDK，而不仅是一个JRE（Java Runtime Environment，Java运行环境）。

你应该下载和在JAVA_HOME目录安装最新的Oracle JDK（在本书写作时是Java SE 7u60）。操作系统特定版可以在www.oracle.com/technetwork/java/javase/downloads/index.html下载，安装说明可以在<http://docs.oracle.com/javase/7/docs/webnotes/install/index.html>下载。

记住

按照JDK安装说明操作，而不是JRE的说明。

3.安装Maven（3.0.5+）

Maven（<http://maven.apache.org>）是一个基于Java的开发框架用于帮助管理代码的设置、创建、编译和依赖解决方案的许多方面，包括测试的运行。这些是所有的已知的和标准的流程，被称为Maven生命周期，由一个项目对象模型文件控制，也称为pom.xml文件。

由于其标准的结构和流程，Maven的优点之一是每当用户遇到一个新的Maven项目时，它们一般能够迅速启动并运行。所有的事情将会按期待的就位，并且会用通用的命令创建和执行代码。

使用Maven的另一个好处是它的依赖管理系统。你只需要告诉Maven你

需要什么样的库，它就会去获取这些库（以及任何的子依赖）——不需要手动下载。这意味着所有的源代码将会建立并且能够通过Maven的运行测试。

下面给出来自pom.xml文件依赖部分的代码片段，演示如何指定spring-data-neo4j库需要的版本号3.1.1.RELEASE。

```
<dependencies>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-neo4j</artifactId>
    <version>3.1.1.RELEASE</version>
  </dependency>
  ...
</dependencies>
```

可以在<http://maven.apache.org/download.cgi>找到合适的二进制文件以及操作系统特定版的安装说明。请确保你使用的是Maven 3.0.5或更高版本。Maven网站描述的安装过程将会帮助你建立Maven主目录（M2_HOME目录），这应该是一个相对简单的过程。

然而，如果是在受限制的环境运行（在一个防火墙后面或在一台有权限限制的计算机上），你可能需要多做一点配置。Maven网站的“用Maven建立项目”页为此提供了更详细的一组说明，包括它在受限的情况下的使用：<http://maven.apache.org/run-maven/index.html>。

B.2 运行演示和样例

本节提供运行本书附带的样例和演示代码的专用说明。这里假设你已经按照B.1节的详细安装说明做了。

1. 一般说明

这里的说明是为从命令行运行样例的，但是，你可以同样轻松地
从IDE（集成开发环境，如Eclipse或IntelliJ）运行它们，前提条件是有合适的
(Maven) 插件可用并且已做了基本的安装设置：

1) 如果你没有可用的插件，按照B.1节的说明安装Java和Maven。

2) 打开一个命令提示符（Windows）或终端（UNIX）窗口。

3) 改变目录到N4JIA_SAMPLE_HOME根目录。

4) 输入以下命令，这将会建立、编译和运行本书中详细介绍的所有主
代码样例的不同测试。

```
mvn clean install
```

5) 你将会在终端窗口看到许多输出；如果一切都成功的话，在最后你
会看到类似如下的输出：

```

[INFO] -----
[INFO] Reactor Summary:
[INFO]
[INFO] neo4j-in-action-reactor ..... SUCCESS [0.298s]
[INFO] chapter01 ..... SUCCESS [1.898s]
[INFO] chapter02 ..... SUCCESS [0.035s]
[INFO] chapter03 ..... SUCCESS [2.319s]
[INFO] chapter04 ..... SUCCESS [1.494s]
[INFO] chapter05 ..... SUCCESS [1.556s]
[INFO] chapter06 ..... SUCCESS [3.873s]
[INFO] chapter07-core ..... SUCCESS [5.450s]
[INFO] chapter07-spring ..... SUCCESS [2.420s]
[INFO] chapter08 ..... SUCCESS [0.290s]
[INFO] chapter09-current-simple ..... SUCCESS [7.704s]
[INFO] chapter09-current-advanced ..... SUCCESS [6.123s]
[INFO] chapter09-legacy-simple ..... SUCCESS [7.815s]
[INFO] chapter09-legacy-advanced ..... SUCCESS [5.268s]
[INFO] chapter10-embedded ..... SUCCESS [0.053s]
[INFO] chapter10-server ..... SUCCESS [0.201s]
[INFO] chapter10-performance-demos-server ..... SUCCESS [0.535s]
[INFO] chapter10-performance-demos-embedded ..... SUCCESS [0.122s]
[INFO] chapter11-community-embedded ..... SUCCESS [1.042s]
[INFO] chapter11-enterprise-embedded ..... SUCCESS [3.832s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 52.638s
[INFO] Finished at: Sat Jul 05 22:13:59 BST 2014
[INFO] Final Memory: 38M/305M
[INFO] -----

```

如果你只是想运行某一章的测试，可以改变目录到合适的章（确保这一目录有一个pom.xml文件）并且再次运行mvn clean install命令。另外，你也可以在N4JIA_SAMPLE_HOME目录中，使用以下命令（替换合适的目录，目录中需要有pom.xml文件）：

```
mvn clean install -f chapter04/pom.xml
```

2.第10章的说明

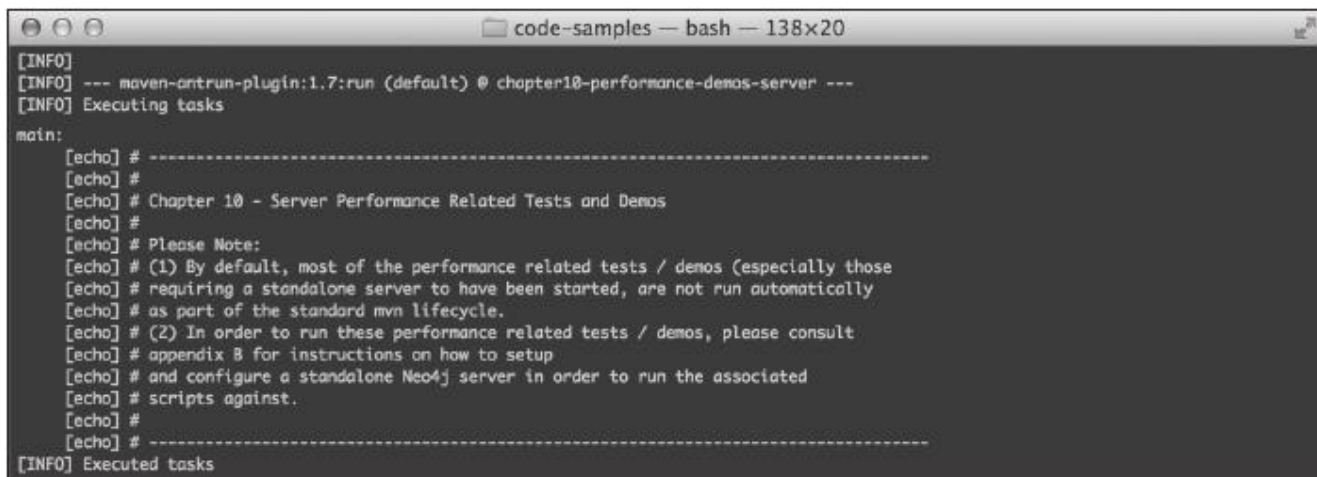
第10章的样例代码包括测试和演示都涉及Neo4j的服务器模式。由于这些测试的大部分是使用不同的选项测量与性能有关的指标，我们决定使服务器独立于其运行的测试启动，以得到尽可能精确的测量。这也使得你可以自

已设置你的服务器。

不同于其他的一些测试，直到目前还是使用基于Java JUnit的测试，第10章与性能有关的性能测试可以从命令行通过一系列bash^[1]脚本开始。

由于选定的执行一些性能测试的客户端之一仅仅是基于UNIX的curl (<http://curl.haxx.se>) 客户端，性能测试脚本仅能以bash脚本获得，它可以在*nix环境下运行。如果你用的是Windows操作系统，可以尝试安装Cygwin (www.cygwin.com)，它提供了一个仿真Linux，可以在那里运行脚本。

1) 默认情况下，当运行第10章与性能有关的测试的标准命令`mvn clean install`时，它们并不运行。如果看控制台上的输出，当到第10章时，你会看到一些如图B-1所示的输出。



```
code-samples — bash — 138x20
[INFO] --- maven-antrun-plugin:1.7:run (default) @ chapter10-performance-demos-server ---
[INFO] Executing tasks
main:
[echo] # -----
[echo] #
[echo] # Chapter 10 - Server Performance Related Tests and Demos
[echo] #
[echo] # Please Note:
[echo] # (1) By default, most of the performance related tests / demos (especially those
[echo] # requiring a standalone server to have been started, are not run automatically
[echo] # as part of the standard mvn lifecycle.
[echo] # (2) In order to run these performance related tests / demos, please consult
[echo] # appendix B for instructions on how to setup
[echo] # and configure a standalone Neo4j server in order to run the associated
[echo] # scripts against.
[echo] #
[echo] # -----
[INFO] Executed tasks
```

图B-1 默认时第10章的Maven输出

这是值得期待的，这样是为了确保基本测试能够在没有事先安装Neo4j服务器时都能运行，否则要等很长时间才能运行这些性能测试。接下来的说明给出了额外的详细步骤运行与性能相关的演示和测试。

2) 要运行一些嵌入式的与性能相关的测试，以Maven配置文件运行include-embedded-perfdemos:

```
mvn clean test -f chapter10/performance-demos-embedded/pom.xml -P
include-embedded-perfdemos
```

3) 按照附录A的详细安装步骤安装单一的Neo4j服务器（样例使用版本2.0.1）。我们将参考Neo4j服务器的安装目录<<NEO4J_HOME>>。

4) 在<<NEO4J_HOME>>/conf/neo4j-server.properties的文件中，

a) 确保通过修改如下所示的属性值将Neo4j服务器数据库设置到性能测试所期待的位置。

```
org.neo4j.server.database.location=<<N4JIA_SAMPLE_HOME>>/chapter10/
performance-demos-server/the-chapter10-server-db
```

b) 确保非托管扩展（在接下来的步骤中将会安装）已经注册。这可以按如下所示取消和修改下列的属性值实现。

```
org.neo4j.server.thirdparty_jaxrs_classes=com.manning.neo4jia.chapter
10.unmanagedext=/n4jia/unmanaged
```

5) 假设你已经成功运行了一个普通的mvn clean install，那么一个包含服务器插件和扩展的jar文件就会在这里产生。

```
<<N4JIA_SAMPLE_HOME>>/chapter10/server/target/chapter10-server-1.0-
SNAPSHOT.jar. Copy this file to the server's plugin directory located
here: <<NEO4J_HOME>>/plugins.
```

6) 在终端窗口创建并传出一个NEO4J_HOME变量，由于这是脚本所期待的。通过执行以下的命令完成这项工作。

```
export NEO4J_HOME=<<NEO4J_HOME>>
```

7) 转到根目录，bash脚本会在根目录运行：

```
<<N4JIA_SAMPLE_HOME>>/chapter10/performance-demos-  
server/src/test/scripts/bash。
```

8) 通过在bash目录中执行下面的脚本产生性能数据库：

```
./seed_performance_db.sh
```

你现在已经准备好运行bash脚本了。

9) 确保你还在根目录下，在这里bash脚本可以运行：

```
<<N4JIA_SAMPLE_HOME>>/chapter10/performance-demos-server/src/test/  
scripts/bash.
```

10) 运行基于bash的测试：

a) 运行基于curl的演示/性能测试脚本，使用这条命令：

```
./curl_adams_friends.sh
```

b) 运行基于Java REST绑定的演示/性能测试脚本，使用这条命令：

```
./javarestbinding_adams_friends.sh
```



由于这是一个Java REST客户端，这些脚本实质上称为某些基于Java的演示/JUnit后台测试。

这将会引出相当多的信息。如果你感兴趣想了解stopwatch类的详细情况，建议以下面的方式调用脚本：


```
./javarestbinding_adams_friends.sh | grep Stopwatch
```

[1] bash是Bourne-Again Shell的缩写，它是一种UNIX Shell。——译者注

附录C 设置使用SDN的项目环境

本附录将带着你浏览完成运行SDN所需设置的说明，尤其是那些针对SDN 3.2.1发布版本的说明，这一版本是官方发布的与Neo4j 2.1.5版相兼容的。SDN 3.2.1发布版本是在专门介绍SDN的章节（第10章）提到的版本，也是本书附带代码所使用的版本。

配置你的项目以使用SDN最容易的方法是通过一个Maven构建（build），随带的源代码也是使用它实现的。安装和设置Maven的通用说明在附录B中给出。如果你愿意，SDN也能够使用Gradle或Ant/Ivy构建；这样做的说明可以在<http://docs.spring.io/spring-data/data-neo4j/docs/current/reference/html/#setup>找到。

本附录也会提供使合适SDN功能所需要的最低Spring配置的详细情况。

根据是否使用简单或高级的映射模式，设置会有一点不同。第9章的源代码同时为简单和高级映射模式提供了一个配置设置。

最新的一些说明，以及在这里没有指定的选项，可以在Michael Hunger的《Good Relationships》中找到：<http://docs.spring.io/spring-data/data-neo4j/docs/current/reference/html/#setup>。

SDN版本和官方文件参考

本书是针对SDN的3.2.1版本的，与Neo4j 2.1.5版本兼容。就像你能理解的，写一本书并试图保持最新版本的公开源码一直以来具有很大的挑战性。

为了确保总是接触到最新的信息，我们一直在努力使用指向最新稳定版本的官方参考文件。在本书写作时，所有的SDN文件URL遵循一个共同的模式，即<http://docs.spring.io/springdata/data-neo4j/docs/<version>/reference/html>，这里的版本<version>是你希望阅读的SDN版本，当前是一个总指向最新、最稳定发布版本的别名。

对3.2.1版本的专门信息，请参阅<http://docs.spring.io/spring-data/data-neo4j/docs/3.2.1.RELEASE/reference/html>。对最新的稳定版本，请参阅<http://docs.spring.io/spring-data/data-neo4j/docs/current/reference/html>。

C.1 Maven配置

如果你正在使用Maven，你需要确保下面的Maven依赖组之一（根据情况或者C.1或者C.2）包含在你的pom.xml文件中，选择取决于你所使用的映射方式。

下面的程序给出了简单映射模式的依赖：

【程序C-1】 简单映射模式Maven需要的依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-neo4j</artifactId>
    <version>3.2.1.RELEASE</version>
  </dependency>

  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-validator</artifactId>
    <version>4.2.0.Final</version>
  </dependency>
</dependencies>
```

SDN 主依赖

JSR-303 Bean Validation 专用
API 的 Hibernate 实施

从理论上讲，有了核心spring-data-neo4j组件就已经足够了，但是在现实中，当前还需要包括一个其他的库，以便SDN正确发挥功能，这就是JSR-303 Validation专用API。在我们的样例代码中，我们选择了Hibernate实现。SDN库本身也使用Validation API，因此用户需要在运行的时间提供一个实际的实现。

SDN库的简单和高级版本都使用Maven的传递依赖管理系统以确保开发需要的支持库自动下载。

下面的程序给出了高级映射的依赖。

【程序C-2】 高级映射模式Maven需要的依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.data</groupId>
    <artifactId>spring-data-neo4j-aspects</artifactId>
    <version>3.2.1.RELEASE </version>
  </dependency>

  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-validator</artifactId>
    <version>4.2.0.Final</version>
  </dependency>
</dependencies>
```

SDN 主依赖（高级映射模式）

JSR-303 Bean Validation 专用
API 的 Hibernate 实施

对于简单映射模式，你将会需要显式包括对一个具体的JSR 303实现库依赖。我们在这里再一次使用了Hibernate实现。

除这些依赖本身，高级映射模式需要配置Maven构建以使用AspectJ在构建时编排入口。下面的程序给出了配置的部分代码，这些代码需要添加到你的pom.xml文件中以完成这项工作。

【程序C.3】 高级映射模式的AspectJ构建配置

```

<build>
<plugins>
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>aspectj-maven-plugin</artifactId>
    <version>1.4</version>
    <dependencies>
      <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjrt</artifactId>
        <version>1.7.4</version>
      </dependency>
      <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjtools</artifactId>
        <version>1.7.4</version>
      </dependency>
    </dependencies>
    <executions>
      <execution>
        <goals>
          <goal>compile</goal>
          <goal>test-compile</goal>
        </goals>
      </execution>
    </executions>
    <configuration>
      <outxml>true</outxml>
      <aspectLibraries>
        <aspectLibrary>
          <groupId>org.springframework</groupId>
          <artifactId>spring-aspects</artifactId>
        </aspectLibrary>
        <aspectLibrary>
          <groupId>org.springframework.data</groupId>
          <artifactId>spring-data-neo4j-aspects</artifactId>
        </aspectLibrary>
      </aspectLibraries>
      <source>1.7</source>
      <target>1.7</target>
    </configuration>
  </plugin>
</plugins>
</build>

```

C.2 Spring配置

本节讨论在应用程序中安装和配置Spring都需要什么。安装相对来说很简单，因此，即使你不熟悉Spring，也不会有什么問題。

Spring本身的设置可以通过一个XML文件或通过Spring Java bean^[1]配置（使用@Configuration注释）实现。由于在写书时大部分的SDN文件使用

XML提供例子，我们也这样做，但是，你可以在Michael Hunger的《Good Relationships》的第21章查找如何使用Java Bean方

法：<http://docs.spring.io/spring-data/data-neo4j/docs/current/reference/html/#setup>。

1.核心XML配置

你需要告诉Spring需要创建和示例什么样的组件。用XML方法，通过提供一个有如下所示的具有初始化内容的XML文件实现。

【程序C-4】 使用存储目录配置XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:neo4j="http://www.springframework.org/schema/data/neo4j"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/data/neo4j
    http://www.springframework.org/schema/data/neo4j/spring-neo4j.xsd">
  <context:annotation-config/>
  <neo4j:config
    storeDirectory="target/config-test"
    base-package="com.manning.neo4jia.chapter08.simple.domain"/>
</beans>
```

← 激活在 bean 类中检测到的各种注释

← 使用 Neo4j 名字空间简洁定义基本 SDN 配置

Neo4j专用XML名字空间单元neo4j: config提供了一种简洁的配置SDN的方法。它创建或给出核心图形数据库类以及其他帮助类，例如Neo4jTemplate。

neo4j: config单元本身具有四个可配置的属性（storeDirectory、graph-DatabaseService、entityManagerFactory和

typeRepresentationStrategyFactory)。为了方便，有一个storeDirectory属性，当将指针设置到一个特定的目录时，如果没有EmbeddedGraphDatabase文件（如果已经有了，就使用它），将会创建一个新的EmbeddedGraphDatabase文件。这是样本代码中使用的方法。

你可以使用graphDatabaseService属性指向一个特定的先前配置的可使用的Neo4j实例。在那种情况下，你也要定义一个factory bean创建一个适当的Neo4j实例，如下面的程序片段所示：

```
<neo4j:config graphDatabaseService="graphDatabaseService" />
<bean id="graphDbFactory"
      class="org.neo4j.graphdb.factory.GraphDatabaseFactory"/>
<bean id="graphDatabaseService"
      scope="singleton"
      destroy-method="shutdown"
      factory-bean="graphDbFactory"
      factory-method="newEmbeddedDatabase">
  <constructor-arg value="target/config-test"/>
</bean>
```

从SDN 3.0起，base-package XML单元是强制性的，并且期待你提供所有包的列表以便SDN能够搜索找到你的注释域实体。如果要使用交叉存储持久性，你也需要提供entityManagerFactory属性值（这里没有给出示例）。

如果你想要使用一类描述策略而不是默认的策略（SDN 3.x的默认策略是基于标签；SDN 2.x的默认策略是基于索引延迟），你可以有选择的定义一个名字为

org.springframework.data.neo4j.support.typerepresentation.TypeRepresentationStrategyFactory bean类，如下面的程序片段所示，为第二个构建器的参数选择一个合适的值。

```
<bean id="typeRepresentationStrategyFactory"
class="org.springframework.data.neo4j.support.typerepresentation.TypeRepresent
ationStrategyFactory">
    <constructor-arg index="0" ref="graphDatabase"/>
    <constructor-arg index="1" value="Indexed"/>
</bean>
```

更多关于可用选项的内容能够在Michael Hunger的《Good Relationships》的第20章的“实体类型描述”一节找到：http://docs.spring.io/spring-data/data-neo4j/docs/current/reference/html/#reference_programming_model_typerepresentationstrategy。

更多配置选项请参阅官方网站。

2.库配置

在XML的配置方法中，能够使用<neo4j: repositories>单元配置SDN库。下面的程序片段给出了一个如何指导Spring在com.manning.neo4jia.chapter09.simple.repository Java包内查找需要使其作为Spring beans对SDN可用库类的例子。

```
<neo4j:repositories base-package=
    "com.manning.neo4jia.chapter09.simple.repository"/>
```

你应该用你的特定库实现所在的位置替换上述包的名字。包单元定义了搜索查找库的根包（或多个包）。

额外的SPRING数据公共配置选项

SDN建在Spring Data Commons项目提供的基础结构和库的上部。通过SD Commons获得的额外配置选项在这里也应该工作。

Spring如何知道到哪里查找XML配置文件？

简单来说这需要你告诉它，但是通常取决于你正在创建什么样的应用程序。

如果你在建立一个网页应用程序，Spring网页框架提供了一个定制的Context LoaderListener类，你可以在你的web.xml文件中使用下面的程序片段所示的方法创建容器。注意在这种情况下，XML配置文件（applicationContext.xml）从它在WEB-INF目录中的位置被引用：

```
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/applicationContext.xml</param-value>
</context-param>
<listener>
    <listener-class>
        org.springframework.web.context.ContextLoaderListener
    </listener-class>
</listener>
```

另外，如果你在建立单机Java应用程序，可以如下面的程序片段所示使用Class PathXmlApplicationContext类：

```
ClassPathXmlApplicationContext ctx
= new ClassPathXmlApplicationContext(
    "classpath:applicationContext.xml");
UserRepository userRepository = ctx.getBean(UserRepository.class);
...
```

用配置提供给Spring的其他方法的更多选务器的安装目ring框架参考文件的“Spring IoC容器和beans导论”一

节：<http://docs.spring.io/spring/docs/4.0.x/springframework-reference/htmlsingle/#beans-introduction>。

[1] Java组件。——译者注

附录D 获得更多的帮助

当你的Neo4j邻居发生了一些奇怪的事情时，你将找谁？

当然是社区！

因为这是一个源码公开项目，有一个宽泛和多样的Neo4j社区，他们愿意帮助你解决问题，如果你选择参与其中并也为这个项目奉献，他们也是很愿意的。下面的资源是你可以去寻求帮助的，并且你也可以为其添砖加瓦。

带着你的问题寻找答案，或找到开发伙伴：

<http://stackoverflow.com/questions/tagged/neo4j>

与你的图形数据库伙伴分享你的经验与专业知识：

<http://groups.google.com/group/neo4j>。

发现一个Neo4j的问题？在这里提出来：

<https://github.com/neo4j/neo4j/issues>。

Neo4j聚会遍布全世界。连接或自己开始一个新组：

<http://neo4j.meetup.com>。

